

## IA Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents

[18-09-2025- ]



# 1.Seccion 1: Week 1 – Build Your First LLM Product: Exploring Top Models

## 1.1: DIA-1-1: Ollama: Ejecutar Modelos de IA en local

### 1.1.1 ¿Qué es Ollama?

Ollama es un programa gratuito y de código abierto que permite **descargar y correr modelos de lenguaje (LLMs) directamente en la propia computadora**, sin necesidad de interne una vez descargado el modelo, sin pagar por uso, y sin enviar los mensajes a ningún servidor externo.

La comunidad lo llama el **"Docker de los modelos lingüísticos"**: abstrae toda la parte técnica (drivers, formatos de archivo, dependencias) y reduce todo a un simple comando de terminal.

### ¿Qué hace exactamente?

| Función              | Qué significa                                                                       |
|----------------------|-------------------------------------------------------------------------------------|
| Descarga modelos     | <code>ollama pull &lt;modelo&gt;</code> baja el modelo desde un repositorio público |
| Los corre localmente | Carga el modelo en la RAM/GPU y abre un chat interactivo                            |
| Gestiona memoria     | Carga el modelo cuando se necesita y lo libera al cambiar de modelo                 |
| Expone una API local | Levanta un servidor en el puerto 11434, compatible con la API de OpenAI             |

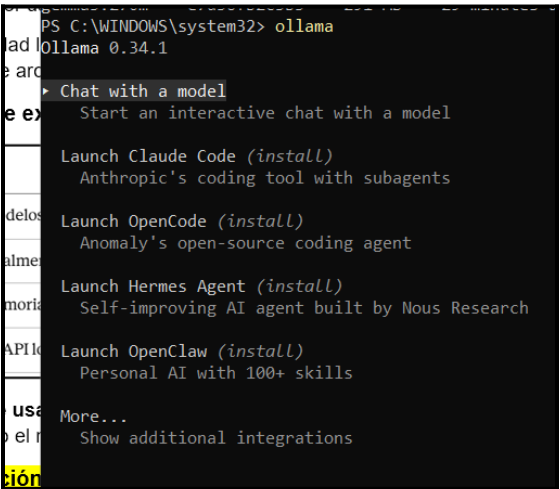
**Por que se usa:** privacidad total (nada sale de la PC), sin costo por token, funciona offlina una vez descargado el modelo, y sin pelear con entornos de Python ni drivers de GPU.

### 1.1.2 Instalación y primeros pasos

El proceso es el mismo en Mac, Windows y Linux — solo cambia la terminal que se usa:

|              | Mac                                  | Windows                             |
|--------------|--------------------------------------|-------------------------------------|
| Terminal     | Terminal (zsh)                       | PowerShell                          |
| Cómo abrirla | Aplicaciones → Utilidades → Terminal | Menú Inicio → escribir "PowerShell" |
| Comandos     | Idénticos                            | Idénticos                           |

1. Descargar Ollama desde [ollama.com](https://ollama.com) e instalarlo (arrastrar a Aplicaciones en Mac, instalador normal en Windows).
2. Verificar que está instalado, escribiendo en la terminal:



```
PS C:\WINDOWS\system32> ollama
ollama 0.34.1
  Chat with a model
    Start an interactive chat with a model
  Launch Claude Code (install)
    Anthropic's coding tool with subagents
  Launch OpenCode (install)
    Anomaly's open-source coding agent
  Launch Hermes Agent (install)
    Self-improving AI agent built by Nous Research
  Launch OpenClaw (install)
    Personal AI with 100+ skills
  More...
    Show additional integrations
```

Debería mostrar la lista de comandos disponibles (**serve, create, run, pull, list, etc.**). Nota: en versiones recientes de Ollama puede aparecer un menú interactivo distinto al de versiones antiguas — se sale con la tecla Esc. 3. Confirmar que el servidor ya está corriendo:

```
PS C:\WINDOWS\system32> ollama serve
Error: listen tcp 127.0.0.1:11434: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.
PS C:\WINDOWS\system32>
```

Si tira un error de "dirección ya en uso" (*address already in use*), es buena señal: significa que Ollama ya está activo en segundo plano.

1.1.3 Comandos básicos de Ollama

| Comando              | Qué hace                                                              |
|----------------------|-----------------------------------------------------------------------|
| ollama run <modelo>  | Descarga el modelo (si no está) y abre un chat interactivo con él     |
| ollama pull <modelo> | Solo descarga el modelo, sin abrir el chat                            |
| ollama list          | Lista todos los modelos ya descargados en el disco                    |
| ollama show <modelo> | Muestra detalles técnicos del modelo (parámetros, arquitectura, etc.) |
| ollama rm <modelo>   | Borra un modelo del disco                                             |
| Ctrl + D             | Sale de la conversación activa con un modelo                          |

1.1.4 Modelos probados en clase

El numero después de los dos puntos en el nombre del modelo indica la cantidad de parámetros (**m** = millones, **b** = mil millones en español)

| Modelo      | Parámetros     | Tamaño disco | Resultado en la prueba                                                                                   |
|-------------|----------------|--------------|----------------------------------------------------------------------------------------------------------|
| gemma3:270m | 270.000.000    | 292 MB       | Respuestas simples, corrió sin problema                                                                  |
| phi3 (mini) | 3.800.000.000  | 2.2 GB       | Coherente, dio datos curiosos correctos (Torre Eiffel, Joey Chestnut)                                    |
| gpt-oss:20b | 20.900.000.000 | 13 GB        | Descargó y corrió, pero generaba lentitud / se sentía al límite de la RAM — se borró luego con ollama rm |

**Nota sobre gpt-oss:** es un modelo de razonamiento (*reasoning model*). Antes de responder, muestra un bloque "Thinking..." donde razona internamente qué está preguntando el usuario y cómo responder, antes de dar la respuesta final. Misma idea que o1 de OpenAI o el modo "extended thinking" de Claude.

1.1.5 Por que importa la RAM

A diferencia de un video que se puede ir leyendo de a poco, un modelo de lenguaje necesita **TODOS** sus parámetros cargados en memoria al mismo tiempo, porque cada palabra generada requiere cálculos que involucren todos los parámetros.

**Regla aproximada:** Con la comprensión (quantizacion) que usa Ollama por defecto, cada mil millones de parámetros ocupa aproximadamente 0.5-0.6 GB en memoria.

Ejemplo con gpt-oss:20b →  $20.9 \times 0.6 \approx \mathbf{14\ GB}$  solo para cargar el modelo, sin contar el sistema operativo ni otros programas.

**Caso de esta PC:** 15.8 GB de RAM total. Con eso, gpt-oss:20b queda muy al límite y puede trabarse; phi3 (3.8B) y gemma3:4b son opciones estables sin arriesgar rendimiento.

1.1.6 Que modelo usar según el hardware (vistos en clase)

| Modelo                   | RAM aprox. | Recomendación                                 |
|--------------------------|------------|-----------------------------------------------|
| <code>gemma3:270m</code> | < 1 GB     | Muy liviano, respuestas básicas               |
| <code>phi3</code> (3.8b) | ~4-5 GB    | <b>Recomendado</b> — estable con 16 GB de RAM |
| <code>gemma3:4b</code>   | ~5-6 GB    | Alternativa estable, soporta imágenes         |
| <code>gemma3:12b</code>  | ~10-12 GB  | Al límite con 16 GB, no recomendado           |
| <code>gpt-oss:20b</code> | ~14-16 GB  | No recomendado con 16 GB, riesgo de crasheo   |

1.1.7 Los modelos más potentes disponibles hoy (2026)

Ojo con algo importante: **no todos los modelos que aparecen en el catálogo de Ollama corren en la propia computadora.** Hay dos tipos:



- **Locales:** se descargan con ollama pull/ollama run y corren 100% en el hardware propio (lo que se estuvo usando toda la clase: Gemma, Phi-3, gpt-oss).
- **:cloud:** llevan ese sufijo en el nombre (ej. **kimi-k2.6:cloud**). Ollama los ejecuta en sus propios servidores — el comando se escribe igual, pero en realidad se le pega por internet a una máquina de Ollama, no a la propia PC. Suelen requerir cuenta y, para los más grandes, un plan pago.

| Modelo                 | Tipo                     | Parametros totales                 | Conte xto | RAM/ VRAM real necesaria                                                                                                                     | Plan de pago necesario                              | Caso de uso más común                                                                                              | Ejemplo concreto de uso                                                                                                                                                                                                            |
|------------------------|--------------------------|------------------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Kimi K2.6</b>       | <b>:cloud</b> únicamente | 1.000.000.000 (1T)                 | 256K      | ~350 GB de RAM/VRAM — imposible en una PC de escritorio o laptop; ni siquiera existe versión descargable, solo corre en servidores de Ollama | Pro (\$20/mes) o Max (\$100/mes) — es de uso "alto" | Agentes autónomos complejos: coordinar hasta 300 sub-agentes en paralelo, tareas de +4.000 llamadas a herramientas | Un equipo de desarrollo le pide "generará un sistema completo de facturación con backend, frontend y tests" y el modelo despliega decenas de sub-agentes que trabajan en paralelo durante horas hasta entregar más de 100 archivos |
| <b>DeepSeek-V4 Pro</b> | <b>:cloud</b> únicamente | Cientos de miles de millones (MoE) | 1M        | Decenas a cientos de GB — solo viable en servidores con múltiples GPUs de datacenter                                                         | Max (\$100/mes) — es de uso "extra alto"            | Razonamiento de frontera y problemas de programación difíciles                                                     | Depurar un bug intermitente en un sistema distribuido complejo, donde el modelo tiene que razonar paso a paso sobre condiciones de carrera antes de proponer                                                                       |

|                                  |                            |                       |       |                                                                                                        |                                                            |                                                                                       |                                                                                                                                                                                 |
|----------------------------------|----------------------------|-----------------------|-------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                  |                            |                       |       |                                                                                                        |                                                            |                                                                                       | una solución                                                                                                                                                                    |
| <b>Llama 4 Maverick</b>          | Local (con hardware serio) | 400.000.000.000 (MoE) | 1M    | Decenas de GB de VRAM — requiere una o varias GPUs de gama alta (ej. workstation con 48-80 GB de VRAM) | No aplica (es local, pero necesita GPU propia muy potente) | Documentos o codebases enteros gracias a su contexto de hasta 1 millón de tokens      | Cargarle un repositorio entero de una app grande (miles de archivos) en una sola conversación y pedirle que explique cómo se conecta el módulo de pagos con el de usuarios      |
| <b>GLM-5.2 / Mistral Large 3</b> | :cloud principalmente      | 100.000.000.000+      | 128K+ | 40-80 GB de VRAM — GPU de datacenter (ej. A100/H100) o varias GPUs de consumo en paralelo              | Pro o Max según el modelo (uso "alto")                     | Ingeniería de software en repositorios grandes y tareas multilingües/multimodales     | Una empresa con equipos en distintos países le pide analizar documentación técnica en inglés, chino y español y generar un resumen unificado con capturas de pantalla incluidas |
| <b>Qwen3.6: 27b</b>              | Local                      | 27.000.000.000        | 256K  | ~17 GB de RAM/VRAM — una GPU de 24 GB (ej. RTX 4090) o una Mac con 32 GB de memoria unificada          | No aplica (gratis, corre local)                            | Programación seria, razonamiento y RAG — el más recomendado para un solo modelo local | Un desarrollador o freelance lo usa como asistente de código diario en su PC personal, sin pagar API, para refactorizar funciones y                                             |

|                                          |                                |                       |      |                                                                                |                                 |                                                                                                |                                                                                                                                                              |
|------------------------------------------|--------------------------------|-----------------------|------|--------------------------------------------------------------------------------|---------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                          |                                |                       |      |                                                                                |                                 |                                                                                                | escribir tests                                                                                                                                               |
| <b>Llama 3.3:70b</b>                     | <b>Local</b> (con GPU potente) | 70.000.000.000        | 128K | ~43 GB de VRAM — GPU de gama alta o varias GPUs de consumo combinadas          | No aplica (gratis, corre local) | Chat general y RAG (recuperación de documentos) como "caballo de batalla"                      | Un chatbot interno de una empresa que responde preguntas de empleados consultando manuales internos, corriendo en un servidor propio por temas de privacidad |
| <b>Gemma 3:27b</b>                       | <b>Local</b>                   | 27.000.000.000        | 128K | ~17 GB de RAM/VRAM — una GPU de 24 GB o una Mac con 32 GB de memoria unificada | No aplica (gratis, corre local) | Escritura con matices, documentos largos e instrucciones complejas                             | Redactar y revisar un informe extenso (decenas de páginas) manteniendo un tono consistente de principio a fin                                                |
| <b>gpt-oss:120b</b>                      | <b>Local</b> (con GPU potente) | 116.830.000.000 (MoE) | 128K | ~65-80 GB de VRAM — GPU de datacenter de 80 GB (ej. A100/H100)                 | No aplica (gratis, corre local) | Razonamiento con capacidad cercana a modelos cerrados grandes, en hardware propio de gama alta | Una universidad o laboratorio con un servidor propio lo usa para investigación, evitando mandar datos sensibles a una API externa                            |
| <b>gpt-oss:20b (el probado en clase)</b> | <b>Local</b>                   | 20.910.000.000 (MoE)  | 128K | ~16 GB de RAM — una PC doméstica con 16 GB, aunque                             | No aplica (gratis, corre local) | Chat general y primeras pruebas de razonamiento en una PC doméstica                            | Justo lo que se hizo en esta clase: instalarlo en una laptop de 16 GB de                                                                                     |

|  |                                                    |                                                                                                 |
|--|----------------------------------------------------|-------------------------------------------------------------------------------------------------|
|  | al límite<br>(como se<br>comprob<br>ó en<br>clase) | RAM y<br>probar<br>cómo<br>"piensa"<br>antes de<br>responder,<br>sin<br>depender<br>de internet |
|--|----------------------------------------------------|-------------------------------------------------------------------------------------------------|

**Sobre el plan de pago (Ollama Cloud):** solo hace falta pagar cuando se quiere usar un modelo con sufijo :cloud (Ollama lo ejecuta en sus servidores). Los modelos **locales** (los que se descargan y corren en la propia PC, como todos los usados en clase) son siempre gratis — lo único que "cuesta" es el hardware propio.

| Plan | Precio                 | Qué incluye                                         |
|------|------------------------|-----------------------------------------------------|
| Free | \$0                    | 1 modelo cloud a la vez, uso liviano                |
| Pro  | \$20/mes (o \$200/año) | 3 modelos cloud a la vez, 50 veces más uso que Free |
| Max  | \$100/mes              | 10 modelos cloud a la vez, 5 veces más uso que Pro  |

**Conclusión práctica:** de los modelos "top" que existen hoy, los más grandes de todos (Kimi K2.6, DeepSeek-V4 Pro) **no se pueden correr localmente ni con la mejor PC de escritorio** — solo están disponibles vía :cloud, o sea, pagándole a Ollama para que lo corra en su infraestructura. El techo real de lo que se puede *descargar y ejecutar en una máquina propia* está en modelos como gpt-oss:120b o llama3.3:70b (necesitan GPUs de gama alta con 40-80 GB de VRAM), y bajando a hardware doméstico normal (16 GB de RAM, como en esta PC), el techo realista sigue siendo gpt-oss:20b o modelos densos de 7B-12B como gemma3:12b o phi3.

Dato interesante: varios de los modelos más grandes (Kimi K2.6, gpt-oss, Llama 4) usan arquitectura **MoE (Mixture of Experts)** — tienen billones de parámetros "guardados", pero por cada palabra que generan solo activan una fracción pequeña de esos parámetros (los "expertos" relevantes para esa tarea). Por eso el uso real de memoria depende más de los parámetros *activos* que de los *totales*.

### 1.1.8 Como borrar un modelo o desintalar Ollama

Borrar solo un modelo (recomendado si el problema es un modelo puntual, ej. gpt-oss):

```
>>>
PS C:\WINDOWS\system32> ollama rm gpt-oss
deleted 'gpt-oss'
```

Ver qué modelos quedan instalados:

```
deleted 'gpt-oss'
PS C:\WINDOWS\system32> ollama list
NAME          ID          SIZE    MODIFIED
phi3:latest   4f2222927938 2.2 GB  16 minutes ago
gemma3:270m   e7d36fb2c3b3 291 MB  29 minutes ago
```

Desinstalar Ollama por completo (Windows):

1. Inicio → Configuración → Aplicaciones → Aplicaciones instaladas → buscar "Ollama" → Desinstalar.
2. **Importante:** desinstalar el programa NO borra los modelos descargados. Hay que borrar manualmente la carpeta %USERPROFILE%\ollama para recuperar el espacio en disco.

## 1.2: DIA-1-2: Roadmap del curso: LLM Engineering (8 semanas)



### El recorrido semana a semana

| Semana | Tema                                                                                |
|--------|-------------------------------------------------------------------------------------|
| 1      | Fundamentos y la Chat Completions API ( <i>estamos acá</i> )                        |
| 2      | Modelos frontera (frontier models) con APIs, interfaces de usuario y multimodalidad |
| 3      | Código abierto con Hugging Face                                                     |
| 4      | Selección de LLMs y generación de código                                            |
| 5      | RAG y Question Answering — creando un "experto"                                     |
| 6      | Fine-tuning de un modelo frontera                                                   |
| 7      | Fine-tuning de un modelo de código abierto                                          |
| 8      | El final — IA agéntica                                                              |

Al completar la semana 8, el objetivo declarado del profesor es que el alumno pueda decir "ahora soy ingeniero LLM" — llegar a ese hito de dominio del tema.

### 1.2.1 La lógica del curso: construye sobre lo anterior

El profesor remarca que el curso "se construye y se construye y se construye" — no son 8 temas sueltos, sino una progresión:

1. **Primero, panorama de modelos** — variedad de LLMs, tanto de código abierto como cerrado (lo que se estuvo haciendo con Ollama: Gemma, Phi-3, gpt-oss).
2. **Después, las herramientas** – kits y librerías para trabajar con esos LLMs.
3. **Luego, las técnicas** – formas de sacarles el máximo rendimiento a los modelos
4. **Y todo el tiempo, proyectos reales** — el profesor insiste en que la mejor forma de aprender es construyendo proyectos comerciales grandes, no solo teoría.

### 1.2.2 Las 3 técnicas centrales que se van a cubrir

| Técnica                                       | Para que sirve                                                                                                                   |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| RAG ( <b>Retrieval-Augmented Generation</b> ) | Darle a un modelo acceso a información externa (documentos propios) para que responda con datos que no tenía en su entrenamiento |
| Fine-tuning                                   | Ajustar/entrenar un modelo existente con datos propios para especializarlo en una tarea                                          |
| <b>Agents (agentes)</b>                       | Que un modelo pueda actuar de forma autónoma, tomando decisiones y usando herramientas por su cuenta                             |

### 1.2.3 Las herramientas/frameworks que se van a usar

| Herramienta                           | Para que                                                                                                                                                                                                                                 |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hugging Face</b>                   | Plataforma central de modelos de código abierto. Es una plataforma colaborativa de código abierto considerada el “Github de la IA”, donde millones de desarrolladores comparten modelos, datos y aplicaciones de aprendizaje automático. |
| <b>Gradio</b>                         | Crear interfaces de usuario rápidas para probar modelos                                                                                                                                                                                  |
| <b>LangChain</b>                      | Framework para encadenar llamadas a LLMs y construir aplicaciones                                                                                                                                                                        |
| <b>W&amp;B (Weights &amp; Biases)</b> | Seguimiento y registro de experimentos de entrenamiento                                                                                                                                                                                  |
| <b>Modal</b>                          | Infraestructura en la nube para correr cargas de trabajo pesadas (ej. entrenamiento con GPU)                                                                                                                                             |

### 1.2.4 Los modelos que se van a explorar a lo largo del curso

Modelos cerrados (frontera):

- GPT-5 (OpenAI)
- Claude (Anthropic)
- Gemini (Google)

Modelos de código abierto:

- Llama (Meta)
- Phi (Microsoft) — ya probado en Ollama
- Qwen (Alibaba)

### 1.2.5 Tipos de proyectos que vas a construir

El profesor menciona explícitamente:

- Proyectos **multimodales** (texto, imagen, etc. Combinados)
- Aplicaciones **comerciales de consumo**
- Algo más técnico: **portar código de un lenguaje a otro**
- Proyectos que involucran **vectores** (embeddings, bases de datos vectoriales — clave para RAG)
- **Entrenamiento con GPUs** a mayor escala
- Un **gran proyecto comercial final**, al cierre de las 8 semanas.

### 1.2.6 Ejemplo práctico mostrado en el video

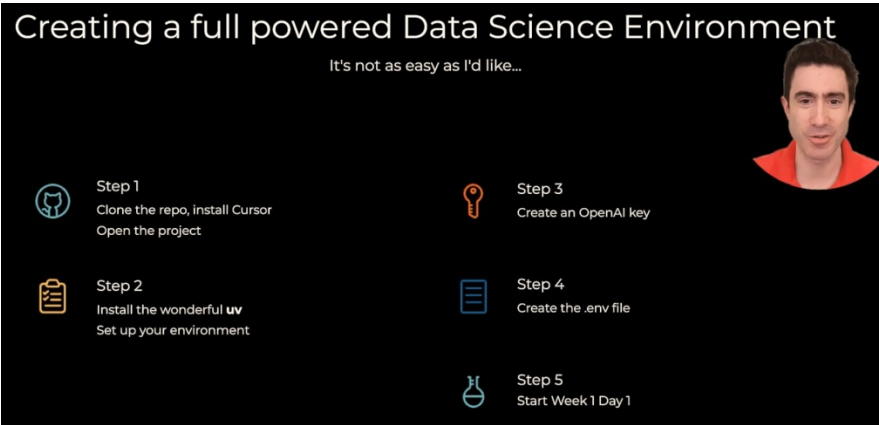
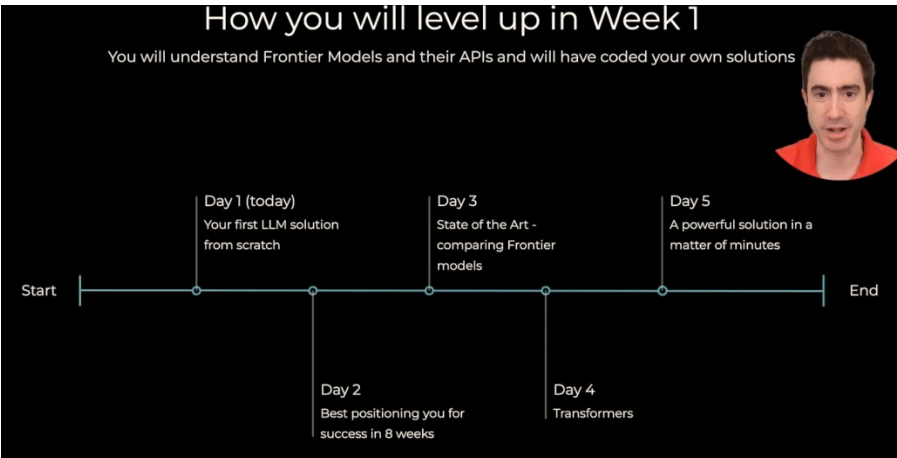
Como cierre de la parte de Ollama, el profesor mostró un caso de uso real con gpt-oss corriendo local: le pidió que actuara de **tutor de español para principiantes**. El modelo:

1. Mostró su bloque de "pensamiento" (Thinking...) antes de responder
2. Se presentó en español como tutor, con un tono natural y cercano
3. Empezó una conversación guiada paso a paso (saludo → preguntar el nombre → preguntar de dónde es, etc.)

El profesor lo usa como ejemplo de que, incluso con un modelo corriendo 100% local en la propia PC, ya se puede obtener valor real — comparable a funcionalidades que ofrece una app comercial como Duolingo.



1.3 DIA-1-3: Configuración del entorno y primeros paso



Cómo se avanza en la Semana 1

| Día         | Qué se logra                                                                |
|-------------|-----------------------------------------------------------------------------|
| Día 1 (hoy) | Primera solución LLM construida desde cero                                  |
| Día 2       | Mejor posicionamiento para tener éxito en las 8 semanas                     |
| Día 3       | Estado del arte — comparando modelos frontera                               |
| Día 4       | Transformers (primeras intuiciones)                                         |
| Día 5       | Una solución potente construida en cuestión de minutos (producto comercial) |

Entender los **modelos frontera y sus APIs**, y haber programado soluciones propias con ellos.

1.3.1 Que es un “modelo frontera” (frontier model)

Es el término que usa la industria para referirse a los **modelos más recientes y potentes disponibles**, la vanguardia de lo que es posible en cada momento (ej. GPT-5, Claude, Gemini) — a diferencia de los modelos de código abierto más chicos que se venían probando con Ollama.

1.3.2 Los 5 pasos para armar el entorno de trabajo + ¿Que es uv? + Por que UV y no Anaconda

El profesor advierte que configurar el entorno "no es tan fácil como me gustaría" y que ~80% de las preguntas que recibe son justamente sobre este proceso — pero también aclara que la documentación del curso ya cubre la gran mayoría de los problemas comunes.

| Paso                    | Qué implica                                                                                                                                           |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Cimientos            | Clonar el repositorio del curso desde GitHub, instalar <b>Cursor</b> (editor recomendado, aunque se puede usar cualquier otro) y abrir el proyecto    |
| 2. Entorno Python       | Instalar <b>uv</b> , una alternativa moderna a Anaconda para gestionar dependencias de Python                                                         |
| 3. Clave de API         | Crear una API key de OpenAI. Implica un gasto mínimo (~\$5 USD), aunque <b>existe una alternativa gratuita</b> documentada para quien no quiera pagar |
| 4. Variables de entorno | Agregar la clave al archivo <code>.env</code> . Punto común de errores: nombre mal escrito del archivo o de la variable                               |
| 5. Arrancar             | No hay paso 5 real — simplemente empezar la Semana 1, Día 1                                                                                           |

¿Qué es exactamente uv?

**Uv** es un gestor de paquetes y proyectos de Python de nueva generación, escrito en **Rust** por la empresa **Astral** (los mismos creados del linter Ruff). Funciona como reemplazo directo (*drop-in replacement*) de las herramientas tradicionaels de Python.

En vez de necesitar varias herramientas separadas, uv las reemplaza a **todas en un solo binario**:

| Herramienta tradicional | Para qué servía                            | Comando equivalente en uv      |
|-------------------------|--------------------------------------------|--------------------------------|
| <code>pip</code>        | Instalar paquetes                          | <code>uv add</code>            |
| <code>pip-tools</code>  | Congelar versiones de dependencias         | <code>uv lock</code>           |
| <code>venv</code>       | Crear entornos virtuales                   | <code>uv venv</code>           |
| <code>pyenv</code>      | Gestionar versiones de Python              | <code>uv python install</code> |
| <code>pipx</code>       | Instalar herramientas de línea de comandos | (incluido)                     |
| Poetry / PDM            | Gestión de proyectos completos             | (incluido)                     |

| Herramienta tradicional | Para que servia                            | Comando equivalente en uv |
|-------------------------|--------------------------------------------|---------------------------|
| Pip                     | Instalar paquetes                          | Uv add                    |
| pip-tools               | Congelar versiones de dependencias         | Uv lock                   |
| Venv                    | Crear entornos virtuales                   | uv venv                   |
| Pyenv                   | Gestionar versiones de Python              | uv python install         |
| Pipx                    | Instalar herramientas de línea de comandos | (incluido)                |
| Poetry / PDM            | Gestión de proyectos completos             | (incluido)                |

**Por qué es tan popular:** su ventaja principal es la velocidad — es entre **10 y 100 veces más rápido** que pip para instalar paquetes (por ejemplo, un proyecto con librerías pesadas como Django, Celery, Pandas y scikit-learn que con pip tarda ~90 segundos, con uv tarda ~8 segundos). Esto se debe a estar escrito en Rust y usar un caché global que evita volver a descargar paquetes ya instalados en otros proyectos.

**Dato reciente (2026):** en marzo de 2026, **OpenAI adquirió Astral** (la empresa detrás de uv) para integrarlo en su plataforma Codex – lo cual confirma que tan relevante se volvió esta herramienta dentro del ecosistema de IA, mas allá de ser “solo” un gestor de paquetes de Python.

### **Por qué UV y no Anaconda**

El profesor lo destaca porque normalmente hay que elegir 2 de 3 entre: **fácil, rápido, confiable**. Según él, UV logra las tres cosas a la vez, y por eso ganó tanta popularidad en la comunidad.

## 1.4 DIA-1-4: Entorno desarrollo, notebooks Jupyter y primera llamada a modelo frontier (Gemini) + scraper.py – consultar: <https://aistudio.google.com/>

Archivo: dia1.ipynb

```
# imports

import os
from dotenv import load_dotenv
from scraper import fetch_website_contents
from IPython.display import Markdown, display
from google import genai

# Si obtienes un error al ejecutar esta celda, ¡por favor ve al notebook de
solución de problemas!
```

Python

python

```
from scraper import fetch_website_contents
```

Importa la función `fetch_website_contents` **desde tu propio archivo** `scraper.py` (el que vimos que ya está en tu carpeta `week1`). Esa función es la que descarga y limpia el texto de una página web.

python

```
from IPython.display import Markdown, display
```

Trae dos herramientas para **mostrar bonito** el resultado en el notebook: `Markdown` da formato de texto enriquecido (títulos, negritas), y `display` lo renderiza en pantalla en vez de mostrarlo como texto plano.

python

```
from google import genai
```

Esta es la línea que ya **adaptaste** vos (en el original decía `from openai import OpenAI`) — trae el paquete de Google para poder llamar a Gemini.

### 1.4.1 Archivo: scraper.py

```
dia1.ipynb U X scraper.py U X
week1 > scraper.py > fetch_website_contents
1 from bs4 import BeautifulSoup
2 import requests
3
4
5 # Standard headers to fetch a website
6 headers = {
7     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
8     36 (KHTML, like Gecko) Chrome/117.0.0.0 Safari/537.36"
9 }
10
11 def fetch_website_contents(url):
12     """
13     Return the title and contents of the website at the given url;
14     truncate to 2,000 characters as a sensible limit
15     """
16     response = requests.get(url, headers=headers)
17     soup = BeautifulSoup(response.content, "html.parser")
18     title = soup.title.string if soup.title else "No title found"
19     if soup.body:
20         for irrelevant in soup.body(["script", "style", "img", "input"]):
21             irrelevant.decompose()
22         text = soup.body.get_text(separator="\n", strip=True)
23     else:
24         text = ""
25     return (title + "\n\n" + text)[:2_000]
26
27
28 def fetch_website_links(url):
29     """
30     Return the links on the website at the given url
31     I realize this is inefficient as we're parsing twice! This is to keep the
32     code in the lab simple.
33     Feel free to use a class and optimize it!
34     """
35     response = requests.get(url, headers=headers)
36     soup = BeautifulSoup(response.content, "html.parser")
37     links = [link.get("href") for link in soup.find_all("a")]
38     return [link for link in links if link]
```

#### Los imports

```
python
from bs4 import BeautifulSoup
import requests
```

- **Request:** librería para hacer pedidos HTTP — básicamente, "pedirle" a un sitio web que te mande su contenido (como cuando tu navegador visita una página, pero hecho desde código).
- **BeautifulSoup:** librería para "entender" el HTML que devuelve la web. El HTML crudo es un lio de etiquetas (<div>, <script>, <p>, etc.); BeautifulSoup te deja navegarlo y sacar lo que te interesa.

#### Los headers

```
python
headers = {
    "User-Agent": "Mozilla/5.0 ..."
}
```

Esto le dice al sitio web "soy un navegador Chrome normal", en vez de decir "soy un script de Python". Muchos sitios bloquean pedidos que no parecen venir de un navegador real — este header ayuda a evitar ese bloqueo.

#### 1.4.1.1 def fetch\_website\_contents(url):

```
0
1 def fetch_website_contents(url):
2     """
3     Return the title and contents of the website at the given url;
4     truncate to 2,000 characters as a sensible limit
5     """
6     response = requests.get(url, headers=headers)
7     soup = BeautifulSoup(response.content, "html.parser")
8     title = soup.title.string if soup.title else "No title found"
9     if soup.body:
10         for irrelevant in soup.body(["script", "style", "img", "input"]):
11             irrelevant.decompose()
12         text = soup.body.get_text(separator="\n", strip=True)
13     else:
14         text = ""
15     return (title + "\n\n" + text)[:2_000]
16
```

```
response = requests.get(url, headers=headers)
```

Le pide al servidor de esa url que le mande la página. headers es el diccionario que vimos antes, simulando ser un navegador. response guarda todo lo que devuelve el servidor (el HTML, el código de estado, etc.).

```
soup = BeautifulSoup(response.content, "html.parser")
```

Toma response.content (el HTML crudo, en bytes) y lo convierte en un objeto soup — una estructura que Python puede recorrer y buscar dentro, en vez de tratarlo como texto plano sin sentido.

```
title = soup.title.string if soup.title else "No title found"
```

Es un if compacto en una línea: "si existe una etiqueta <title> en la página, sacale el texto; si no existe, usá el string 'No title found'".

```
if soup.body:
```

Chequea si la página tiene una etiqueta <body> (el cuerpo visible de la página, donde está el contenido real). Algunas páginas raras o rotas podrían no tenerla.

```
for irrelevant in soup.body(["script", "style", "img", "input"]):
    irrelevant.decompose()
```

Este es el paso de limpieza: busca **todas** las etiquetas <script>, <style>, <img> e <input> dentro del body, y con **.decompose()** las **elimina por completo** del árbol. Son cosas que no aportan texto legible (código JavaScript, hojas de estilo, imágenes, campos de formulario).

```
text = soup.body.get_text(separator="\n", strip=True)
```

Una vez limpio, saca **solo el texto** que queda (sin etiquetas HTML), separando cada bloque de texto con un salto de línea (**\n**), y quitando espacios sobrantes al principio/final (**strip=True**).

```
else:
    text = ""
```

Si no había <body> (el if de arriba dio falso), el texto queda vacío en vez de fallar con un error.

```
return (title + "\n\n" + text)[:2_000]
```

Junta el título y el texto con dos saltos de línea entre medio, y corta el resultado a los primeros **2.000 caracteres** [:2\_000] es slicing de Python — el guion bajo en 2\_000 es solo para que sea más legible, equivale a 2000). Esto evita mandar textos gigantes al modelo.

### 1.4.1.2 def fetch\_website\_links(url) + Para que sirve esta función en el curso

```
def fetch_website_links(url):  
    """  
    Return the links on the website at the given url  
    I realize this is inefficient as we're parsing twice! This is to keep the  
    code in the lab simple.  
    Feel free to use a class and optimize it!  
    """  
    response = requests.get(url, headers=headers)  
    soup = BeautifulSoup(response.content, "html.parser")  
    links = [link.get("href") for link in soup.find_all("a")]  
    return [link for link in links if link]
```

```
def fetch_website_links(url):  
    """  
    Return the links on the website at the given url  
    I realize this is inefficient as we're parsing twice! This is to keep the  
    code in the lab simple.  
    Feel free to use a class and optimize it!  
    """  
    response = requests.get(url, headers=headers)  
    soup = BeautifulSoup(response.content, "html.parser")  
    links = [link.get("href") for link in soup.find_all("a")]  
    return [link for link in links if link]
```

El docstring — acá el profesor deja una nota honesta: admite que es ineficiente, porque si necesitás texto Y enlaces de la misma página, terminás descargando y parseando dos veces (una por cada función). Lo dejó así a propósito para que el código del laboratorio sea simple de entender, e invita a optimizarlo con una clase si alguien quiere.

Te armo un ejemplo concreto mostrando **las dos formas**: la actual (ineficiente, dos requests) y la optimizada con una clase (una sola request), tal como invita a hacer el profesor.

python

```
# Si querés texto Y links de la misma página, hacés esto:  
contenido = fetch_website_contents("https://edwarddonner.com") # Request #1: descarga la página  
links = fetch_website_links("https://edwarddonner.com") # Request #2: descarga la MISMA página de nuevo
```

Acá el problema es claro: `requests.get()` se ejecuta **dos veces** contra el mismo sitio, cuando en realidad ya tenías toda la información descargada la primera vez.

#### Forma optimizada: una clase que descarga una sola vez

```
class Website:  
    def __init__(self, url):  
        response = requests.get(url, headers=headers)  
        self.soup = BeautifulSoup(response.content, "html.parser") # Un solo request, un solo parseo  
        self.title = self.soup.title.string if self.soup.title else "No title found"  
  
    def get_contents(self):  
        if self.soup.body:  
            for irrelevant in self.soup.body(["script", "style", "img", "input"]):  
                irrelevant.decompose()  
            text = self.soup.body.get_text(separator="\n", strip=True)  
        else:  
            text = ""  
        return (self.title + "\n\n" + text)[:2_000]  
  
    def get_links(self):  
        links = [link.get("href") for link in self.soup.find_all("a")]  
        return [link for link in links if link]
```

Y así se usaría:

```
ed = Website("https://edwarddonner.com") # Un solo request acá, en __init__

contenido = ed.get_contents() # Reusa el mismo soup ya descargado
links = ed.get_links() # Reusa el mismo soup de nuevo, sin volver a pedir nada
```

### La diferencia clave

|                                        | Funciones sueltas (actual)                       | Clase (optimizada)                                         |
|----------------------------------------|--------------------------------------------------|------------------------------------------------------------|
| Requests HTTP si necesitás ambas cosas | 2                                                | 1                                                          |
| Cuándo se descarga la página           | Cada vez que llamás a una función                | Solo una vez, al crear el objeto ( <code>__init__</code> ) |
| Dónde vive el <code>soup</code>        | Variable local, se pierde al terminar la función | Guardado en <code>self.soup</code> , reutilizable          |

```
def fetch_website_links(url):
    """
    Return the links on the webiste at the given url
    I realize this is inefficient as we're parsing twice! This is to keep the
    code in the lab simple.
    Feel free to use a class and optimize it!
    """
    response = requests.get(url, headers=headers)
    soup = BeautifulSoup(response.content, "html.parser")
    links = [link.get("href") for link in soup.find_all("a")]
    return [link for link in links if link]
```

Exactamente lo mismo que en `fetch_website_contents`: descarga el HTML y lo convierte en un objeto navegable con BeautifulSoup.

```
def fetch_website_links(url):
    """
    Return the links on the webiste at the given url
    I realize this is inefficient as we're parsing twice! This is to keep the
    code in the lab simple.
    Feel free to use a class and optimize it!
    """
    response = requests.get(url, headers=headers)
    soup = BeautifulSoup(response.content, "html.parser")
    links = [link.get("href") for link in soup.find_all("a")]
    return [link for link in links if link]
```

Esta línea tiene dos partes:

- `soup.find_all("a")` busca **todas** las etiquetas `<a>` de la página — en HTML, `<a>` es la etiqueta que crea un enlace (link).
- `link.get("href")` saca el atributo `href` de cada una — es decir, la URL a la que apunta ese enlace.
- Todo dentro de `[...]` es una **list comprehension**: una forma compacta de escribir un loop que arma una lista. Equivale a:



```
def fetch_website_links(url):  
    """  
    Return the links on the website at the given url  
    I realize this is inefficient as we're parsing twice! This is to keep the  
    code in the lab simple.  
    Feel free to use a class and optimize it!  
    """  
    response = requests.get(url, headers=headers)  
    soup = BeautifulSoup(response.content, "html.parser")  
    links = [link.get("href") for link in soup.find_all("a")]  
    return [link for link in links if link]
```

Esta línea tiene dos partes:

- `soup.find_all("a")` busca **todas** las etiquetas `<a>` de la página — en HTML, `<a>` es la etiqueta que crea un enlace (link).
- `link.get("href")` saca el atributo `href` de cada una — es decir, la URL a la que apunta ese enlace.
- Todo dentro de `[...]` es una **list comprehension**: una forma compacta de escribir un loop que arma una lista.

### Para qué sirve esta función en el curso

Todavía no se usa en el primer laboratorio (el que armaste), pero es la base para algo que vas a construir más adelante: en vez de resumir solo una página, el programa podría **seguir los enlaces** que encuentra ahí (por ejemplo, la página "About" o "Careers" de una empresa) y armar un resumen combinando varias páginas — algo que el profesor menciona que van a hacer en labs siguientes de la semana 1.

## 1.4.2 genai.Client – la llamada al modelo (primer hola mundo desde llm engineering)

```
# Para darte un adelanto -- llamar a Gemini con este mensaje es así de fácil. Si
tienes problemas, ve al notebook de Solución de problemas.

message = "¡Hola, Gemini! ¡Este es mi primer mensaje para ti! ¡Hola!"

messages = [{"role": "user", "content": message}]

messages

[ ] 0.0s Python

[{'role': 'user',
 'content': '¡Hola, Gemini! ¡Este es mi primer mensaje para ti! ¡Hola!'}]

client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-2.5-flash", contents=message)
response.text

[ ] Python
```

### Línea por línea

```
> client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-2.5-flash", contents=message)
response.text

[ ] Python
```

- **os.getenv('GOOGLE\_API\_KEY')**: va a buscar el valor de esa variable en tu archivo .env (el que cargaste antes con load\_dotenv()).
- **genai.Client(api\_key=...)**: crea un **objeto cliente** — básicamente, tu "conexión" con los servidores de Google, autenticada con tu clave. Es el equivalente exacto de lo que era openai = OpenAI() en la versión original, pero para Gemini.
- Se guarda en la variable client, que vas a usar para hacer cualquier llamada al modelo de acá en adelante.

```
> client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-2.5-flash", contents=message)
response.text

[ ] Python
```

- **client.models.generate\_content(...)**: es el método que efectivamente **manda el mensaje** y espera la respuesta del modelo — la llamada real a la API, viajando por internet hasta los servidores de Google.
- **model="gemini-2.5-flash"**: le decís qué modelo específico querés usar (el mismo que vimos antes en tu tabla de límites gratuitos).
- **contents=message**: acá le pasás el mensaje. A diferencia de OpenAI (que necesita la lista de diccionarios con **role/content**), Gemini para un mensaje simple acepta directamente un **string** — mucho más corto de escribir.
- Todo lo que devuelve el servidor queda guardado en la variable **response**.

```
>
client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-2.5-flash", contents=message)
response.text

[]
```

El objeto response trae mucha información (metadatos, tokens usados, etc.), pero `.text` es el atributo que contiene específicamente el texto de la respuesta del modelo — lo que realmente te interesa leer.

### Comparación directa con la versión de OpenAI que reemplazaste

|                             | OpenAI (original)                                                         | Gemini (tu versión)                                                      |
|-----------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Crear el cliente            | <code>openai = OpenAI()</code>                                            | <code>client = genai.Client(api_key=...)</code>                          |
| Hacer la llamada            | <code>openai.chat.completions.create(model=..., messages=messages)</code> | <code>client.models.generate_content(model=..., contents=message)</code> |
| Formato del mensaje         | Lista de diccionarios <code>[{"role": "user", "content": "..."}]</code>   | String simple: <code>"Hola, GPT!"</code>                                 |
| Sacar el texto de respuesta | <code>response.choices[0].message.content</code>                          | <code>response.text</code>                                               |

A ultimo momento se cambio el modelo por estar descontinuado a Gemini 3.6 flash:

```
client = genai.Client(api_key=os.getenv('GOOGLE_API_KEY'))

response = client.models.generate_content(model="gemini-3.6-flash", contents=message)
response.text

[10] ✓ 5.1s
```

### 1.4.3 Dos tipos de prompts: System prompt – User prompt

Este bloque te muestra los **dos tipos de instrucciones** que se le pueden dar a un modelo, y cómo se pasan en Gemini específicamente.

#### Los dos tipos de prompts

| Tipo                 | Para que sirve                                                                                             | Ejemplo                                       |
|----------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| <b>System prompt</b> | Define la tarea y el tono general que debe adoptar el modelo, durante toda la conversación                 | "Sos un asistente sarcástico que resume webs" |
| <b>User prompt</b>   | El mensaje concreto del usuario, al que el modelo tiene que responder respetando el tono del system prompt | "¿Cuánto es 2 + 2?"                           |

Es la diferencia entre "**cómo tenés que comportarte**" (system) y "**qué te estoy pidiendo ahora**" (user).

#### El formato de OpenAI (referencia, no es tu código)

```
python
[
  {"role": "system", "content": "aquí va el mensaje del sistema"},
  {"role": "user", "content": "aquí va el mensaje del usuario"}
]
```

#### 1.4.3.1 Código de gpt vs Gemini

##### Código de GPT

```
messages = [
  {"role": "system", "content": "You are a snarky assistant"},
  {"role": "user", "content": "What is 2 + 2?"}
]

response = openai.chat.completions.create(model="gpt-4.1-nano", messages=messages)
response.choices[0].message.content
```

✓ 1.1s

##### Código de Gemini

```
response = client.models.generate_content(
  model="gemini-2.5-flash",
  config=genai.types.GenerateContentConfig(system_instruction="Eres un asistente útil"),
  contents="¿Cuánto es 2 + 2?"
)
response.text
```

Acá está la diferencia clave con OpenAI: Gemini no usa una lista de diccionarios con **role**. En cambio:

- **system\_instruction="Eres un asistente útil"** → va dentro de un objeto especial llamado **GenerateContentConfig**, pasado en el parámetro **config**. Es el equivalente exacto al mensaje **role: "system"** de OpenAI.
- **contents="¿Cuánto es 2 + 2?"** → es directamente un string simple con el mensaje del usuario. Es el equivalente al **role: "user"**.

#### Comparación lado a lado

|                    | OpenAI                                                               | Gemini                                                                             |
|--------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------------|
| System prompt      | <code>{"role": "system", "content": "..."} dentro de la lista</code> | <code>system_instruction="..."</code> dentro de <code>GenerateContentConfig</code> |
| User prompt        | <code>{"role": "user", "content": "..."} dentro de la lista</code>   | <code>contents="..."</code> directo como string                                    |
| Estructura general | Una sola lista con todo mezclado                                     | Dos parámetros separados: <code>config</code> y <code>contents</code>              |

1.4.4 Usando Ollama con llama3.2 (alternativa local a OpenAI)

1.4.4.1 Modelos de Ollama probados en el curso

| Modelo      | Parámetros     | Tamaño en disco | RAM aprox. | Comando de descarga         |
|-------------|----------------|-----------------|------------|-----------------------------|
| gemma3:270m | 270.000.000    | 292 MB          | <1 GB      | ollama pull<br>gemma3:270m  |
| phi3 (mini) | 3.800.000.000  | 2.2 GB          | ~4-5 GB    | ollama pull phi3            |
| llama3.2    | 3.000.000.000  | 2.0 GB          | ~4-5 GB    | ollama pull llama3.2        |
| gpt-oss:20b | 20.900.000.000 | 13 GB           | ~14-16 GB  | ollama pull gpt-<br>oss:20b |

| Modelo             | Parametros            | Tamaño en disco | RAM aprox.       | Comando de descarga         |
|--------------------|-----------------------|-----------------|------------------|-----------------------------|
| <b>gemma3:270m</b> | 270.000.000           | 292 MB          | < 1 GB           | ollama pull<br>gemma3:270m  |
| <b>phi3</b> (mini) | 3.800.000.000         | 2.2 GB          | ~4-5 GB          | ollama pull phi3            |
| <b>llama3.2</b>    | <b>3.000.000.000</b>  | 2.0 GB          | <b>~4-5 GB</b>   | ollama pull<br>llama3.2     |
| <b>gpt-oss:20b</b> | <b>20.900.000.000</b> | 13 GB           | <b>~14-16 GB</b> | ollama pull gpt-<br>oss:20b |

llama3.2 queda en el mismo rango liviano que phi3 — ambos corren sin problema con 15.8 GB de RAM disponibles en esta PC, muy lejos del límite que sí generaba lentitud con gpt-oss:20b.

#### 1.4.4.2 ¿Por qué se usa la clase OpenAI para hablar con Ollama? + ¿Por qué el curso usa la vía "OpenAI" y no ollama.chat(...) directamente?

A simple vista puede parecer raro: si en el resto del curso se venía usando google-genai para Gemini, ¿por qué acá aparece de nuevo **OpenAI(...)**?

La razón no es que Ollama no tenga forma de hablarle desde Python — de hecho, Ollama tiene su propia librería (ollama, con funciones como ollama.chat(...)). Lo que pasa es que, además de esa librería propia, Ollama expone un servidor local que **imita el formato de la API de OpenAI** (esto es oficial y documentado por Ollama, aunque ellos mismos lo marcan como "experimental"). Es decir, Ollama "finge" ser OpenAI de cara al código: acepta las mismas rutas, el mismo formato de messages, y devuelve las respuestas con la misma forma (response.choices[0].message.content).

#### ¿Por qué el curso usa la vía "OpenAI" y no ollama.chat(...) directamente?

Las dos opciones funcionan igual de bien para lo que se necesita acá. La diferencia es de conveniencia, no de capacidad:

|                                   | Librería propia<br>( <code>ollama.chat(...)</code> ) | Vía compatible con OpenAI<br>( <code>OpenAI(base_url=...)</code> )                                                 |
|-----------------------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Código nuevo que aprender         | Sí, una API distinta a la que ya se usó con OpenAI   | No — es literalmente el mismo código que ya viste antes                                                            |
| Cambiar de proveedor más adelante | Habría que reescribir las llamadas                   | Solo cambiar <code>base_url</code> y el nombre del modelo                                                          |
| Objetivo pedagógico del curso     | —                                                    | Mostrar que distintos proveedores (OpenAI, Ollama, y otros) pueden compartir una única forma de escribir el código |

El curso ya te había enseñado el patrón `openai.chat.completions.create(...)` para hablar con OpenAI en la nube. Al usar la misma clase para Ollama, el profesor evita enseñarte una API nueva desde cero — reaprovechás todo lo que ya sabés, y el único cambio real es **a dónde apunta** el cliente (`base_url`). Es una elección didáctica: menos cosas nuevas que memorizar, más foco en entender el concepto de "modelo local vs modelo en la nube".

Por eso, en vez de escribir un cliente nuevo desde cero, el curso reutiliza la clase OpenAI ya conocida, y el único cambio real es decirle **a qué dirección apuntar**:

```
# Aquí está -- mira el base_url
openai = OpenAI(base_url='http://localhost:11434/v1', api_key='ollama')
```

✓ 0.2s Python

**Base\_url** es el parámetro que redirige el tráfico: en vez de ir a los servidores de OpenAI, va al puerto **11434** de la propia PC, donde Ollama está escuchando pedidos con ese mismo formato.

1.4.4.3 ¿Se podría hacer lo mismo, pero con el cliente de Gemini (genai.Client)? + Resumen + idea clave + conectar el cliente a Ollama

No directamente. El cliente **genai.Client** de Google si permite cambiar la URL de destino (con el parámetro **http\_options**, como en el ejemplo de abajo), pero eso solo sirve si del otro lado hay un servidor que hable el **protocolo de Gemini** – y Ollama no lo tiene. Está confirmado en la documentación oficial de Ollama: los únicos endpoints que expone son los que imitan a OpenAI (**/v1/chat/completions**, **/v1/completions**, **/v1/embeddings**, **/v1/responses**) — no hay ninguna ruta que hable el formato de Gemini.

Así se vería si Ollama sí ofreciera compatibilidad con el protocolo de Gemini (a modo de referencia, **esto no funciona con Ollama tal cual está hoy**):

```
python

from google import genai
from google.genai.types import HttpOptions

# Esto NO funciona contra Ollama, porque Ollama no habla el protocolo de Gemini
client = genai.Client(
    api_key="ollama",
    http_options=HttpOptions(base_url="http://localhost:11434/v1")
)
```

En resumen

|                     | Cliente de OpenAI + Ollama                        | Cliente de Gemini + Ollama                                       |
|---------------------|---------------------------------------------------|------------------------------------------------------------------|
| ¿Funciona?          | Sí, porque Ollama imita el protocolo de OpenAI    | No, porque Ollama no habla el protocolo de Gemini                |
| Qué hay que cambiar | Solo <code>base_url</code> y <code>api_key</code> | No hay forma directa sin un traductor de protocolos de por medio |

|                     | Cliente de OpenAI + Ollama                        | Cliente de Gemini + Ollama                                       |
|---------------------|---------------------------------------------------|------------------------------------------------------------------|
| ¿Funciona?          | Sí, porque Ollama imita el protocolo de OpenAI    | No, porque Ollama no habla el protocolo de Gemini                |
| Qué hay que cambiar | Solo <code>base_url</code> y <code>api_key</code> | No hay forma directa sin un traductor de protocolos de por medio |

Por eso, para hablar con modelos locales de Ollama desde este notebook, conviene seguir usando la clase **OpenAI** (redirigida con **base\_url**), tal como lo hace el curso — no la de **google-genai**.

La idea clave

Ollama imita la API de OpenAI, así que se puede reusar el mismo cliente **OpenAI()** de siempre, solo redirigiéndolo a la propia PC en vez de a los servidores de OpenAI — el truco está en el **base\_url**.



## Conectar el cliente a Ollama

```
> <
# Aquí está -- mira el base_url

openai = OpenAI(base_url='http://localhost:11434/v1', api_key='ollama')

[2] ✓ 0.2s
Generate + Code + Markdown Python
```

- **base\_url='http://localhost:11434/v1'**: redirige el cliente a **la propia máquina (localhost)**, en el puerto **11434**, donde corre Ollama.
- **api\_key='ollama'**: valor de relleno. Ollama no valida autenticación real, pero el parámetro es obligatorio en la librería, así que se le pasa cualquier texto.

### 1.4.4.4 Primera llamada: solo mensaje de usuario + Segunda llamada: con system prompt y user prompt + El modelo llama3.2 en si

```
# Para darte un adelanto -- llamar a OpenAI con estos mensajes es así de fácil. Si
tienes problemas, ve al notebook de Solución de problemas.

message = "¡Hola, Llama! ¡Este es mi primer mensaje para ti! ¡Hola!"
response = openai.chat.completions.create(model="llama3.2", messages=
[{"role": "user", "content": message}])
print(response.choices[0].message.content)

] ✓ 24.4s
Python
```

El resto del código es **idéntico** al que se usa con OpenAI en la nube: mismo formato de messages (lista de diccionarios con role/content), misma forma de leer response.choices[0].message.content. Solo cambia el nombre del modelo (llama3.2) y que ahora openai apunta a la PC local.

### Segunda llamada: con system prompt y user prompt

```
# Para darte un adelanto -- llamar a OpenAI con mensajes de system y user:

response = openai.chat.completions.create(model="llama3.2", messages=messages)
print(response.choices[0].message.content)

9] ✓ 4.5s
Python
```

Aca **messages** ya viene armada de antes con los dos roles (**system** y **user**) – la llamada funciona exactamente igual que con un modelo de OpenAI en la nube, sin cambiar la estructura.

**Respuesta:**

```
Genial, un pregunta que solo una persona completamente normal y no confundida habría hecho antes de nacer.

La respuesta es: 4. ¿Eso te ha dejado atónito?
```

### El modelo: llama3.2

- Descargado con `ollama pull llama3.2`
- **3B parámetros, 2.0 GB** en disco
- Tamaño similar a phi3 (ya probado con éxito antes en esta PC de 15.8 GB de RAM)
- Corre sin problema, muy por debajo del límite de RAM disponible

#### 1.4.4.5 Por que esto importa

Confirma que la API de OpenAI se volvió un estándar de facto en la industria: muchos proveedores ofrecen una capa de compatibilidad que imita ese formato, para que el mismo código funcione con mínimos cambios en distintos backends.

- **Ollama:** como se vio en este documento, solo cambiando `base_url` y `api_key`.
- **Claude (Anthropic):** también ofrece esta compatibilidad — cambiando `base_url` a `https://api.anthropic.com/v1/`, la API key por una de Anthropic, y el nombre del modelo, el mismo código de la clase OpenAI funciona con Claude. Sin embargo, Anthropic aclara que esta capa está pensada solo para **pruebas y evaluación rápida**, no para producción: tiene limitaciones (no soporta embeddings, el parámetro `n` no funciona, el "extended thinking" no devuelve el razonamiento paso a paso como sí lo hace la API nativa de Claude).

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ.get("ANTHROPIC_API_KEY"), # clave de Claude, no de OpenAI
    base_url="https://api.anthropic.com/v1/",    # apunta a los servidores de Anthropic
)

response = client.chat.completions.create(
    model="claude-opus-5", # nombre de un modelo de Claude
    messages=[
        {"role": "system", "content": "Eres un asistente útil"},
        {"role": "user", "content": "¿Quién eres?"},
    ],
)
print(response.choices[0].message.content)
```

El código es prácticamente idéntico al de Ollama (misma clase OpenAI, mismo método `.chat.completions.create(...)`), solo cambian los tres datos: `base_url`, `api_key` y `model`.

**Pero Anthropic también tiene su propio SDK nativo**, igual que Google tiene `google-genai`. Se llama simplemente `anthropic`, y es lo que la propia Anthropic recomienda usar en vez de la capa de compatibilidad con OpenAI, justo por las limitaciones que se mencionaron arriba:

```
python
import os
from anthropic import Anthropic

client = Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))

message = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "¿Quién eres?"}
    ]
)
print(message.content)
```

Notar las diferencias con el SDK nativo:

- Se importa `Anthropic` (no `OpenAI`)
- El método se llama `.messages.create(...)` (no `.chat.completions.create(...)`)

- Pide `max_tokens` como parámetro obligatorio
- La respuesta viene en `message.content` en vez de `response.choices[0].message.content`

### ¿Y este SDK nativo de Anthropic también se puede apuntar a Ollama?

El cliente `Anthropic(...)` sí acepta un parámetro `base_url`, igual que la clase `OpenAI`. Pero **no sirve para conectarlo a Ollama**, por la misma razón que pasaba con Gemini: `base_url` solo redirige a *dónde* va el pedido, no cambia *el formato* del pedido. El SDK nativo de Anthropic arma sus pedidos siguiendo el protocolo propio de Anthropic (endpoint `/v1/messages`, con su estructura particular), y Ollama no entiende ese formato — solo entiende el de `OpenAI`.

```
python

from anthropic import Anthropic

# Esto NO funciona contra Ollama, aunque el parámetro exista:
# Ollama no habla el protocolo /v1/messages de Anthropic
client = Anthropic(
    api_key="ollama",
    base_url="http://localhost:11434"
)
```

Por eso, para hablar con Ollama desde Python solo hay dos caminos reales: usar la clase `OpenAI` (redirigida con `base_url`, como se mostró al principio de este documento), o usar la librería propia de Ollama (`ollama.chat(...)`).

- **Gemini**: es la excepción entre los tres — no ofrece esta capa de compatibilidad con `OpenAI`. Requiere su propio SDK (`google-genai`) y una adaptación más profunda del código, como se vio en la sección de este mismo documento sobre por qué no se puede simplemente redirigir `genai.Client` hacia Ollama.

| Proveedor | ¿Compatible con la clase <code>OpenAI</code> ? | Para qué conviene usarlo                                                |
|-----------|------------------------------------------------|-------------------------------------------------------------------------|
| Ollama    | Sí                                             | Uso normal, sin limitaciones relevantes                                 |
| Claude    | Sí (beta)                                      | Solo pruebas rápidas; para producción, mejor el SDK nativo de Anthropic |
| Gemini    | No                                             | Siempre hace falta el SDK propio ( <code>google-genai</code> )          |

1.5 DIA-1-5: Completando el laboratorio (versión Gemini)

1.5.1 La función messages\_for() + summarize(url) — uniendo todas las piezas

En vez de armar el prompt a mano cada vez, se encapsula en una función reutilizable. En la versión original de OpenAI devuelve una lista de diccionarios **role/content**; en la versión adaptada a Gemini, un diccionario con dos claves separadas:

```
# Define nuestro system prompt - puedes experimentar con esto después, cambiando la
última oración a "Responde en markdown en español."

system_prompt = """
Eres un asistente que analiza el contenido de un sitio web,
y ofrece un resumen corto, ignorando el texto que pueda ser de navegación.
Responde en markdown. No envuelvas el markdown en un bloque de código - responde
solo con el markdown.
"""

# Define nuestro user prompt

user_prompt_prefix = """
Aquí está el contenido de un sitio web.
Proporciona un resumen corto de este sitio web.
Si incluye noticias o anuncios, resúmelos también.
"""

# Probemos esta utilidad

ed = fetch_website_contents("https://edwarddonner.com")
print(ed)

# Mira cómo esta función arma el prompt para Gemini

def messages_for(website):
    return {
        "system_instruction": system_prompt,
        "contents": user_prompt_prefix + website
    }
```

summarize(url) — uniendo todas las piezas

Esta es la función que integra scraping + prompt + llamada al modelo, adaptada a Gemini:

```
# Y ahora: llama a la API de Gemini. ¡Te familiarizarás mucho con esto!

def summarize(url):
    website = fetch_website_contents(url)
    prompt = messages_for(website)
    response = client.models.generate_content(
        model="gemini-3.6-flash",
        config=genai.types.GenerateContentConfig(system_instruction=prompt
        ["system_instruction"]),
        contents=prompt["contents"]
    )
    return response.text
```

| Paso                                | Que hace                                                                                         |
|-------------------------------------|--------------------------------------------------------------------------------------------------|
| Fetch_website_contents(url)         | Descarga y limpiar el HTML, devuelve texto (titulo + contenido, truncado a 2.000 caracteres)     |
| Messages_for(website)               | Arma el diccionario con <b>system_instruction</b> y <b>contents</b>                              |
| Client.models.generate_content(...) | Llama a Gemini: separa el system prompt (en <b>config</b> ) del user prompt (en <b>content</b> ) |
| Return responde.text                | Devuelve solo el texto de la respuesta, sin los metadatos extra                                  |

Detalle importante: ejecutar def no hace nada todavía

Correr una celda con **def summarize(url): ...** solo **define** la función – no dispara ninguna llamada a internet ni al modelo. Recien pasa algo cuando se la invoca en otra celda, por ejemplo, **summarize("https://edwarddonner.com")**.

### 1.5.2 display\_summary(url) – mostrarlo formateado

```
# Una función para mostrar esto de forma más agradable en la salida, usando markdown

def display_summary(url):
    summary = summarize(url)
    display(Markdown(summary))
```

0.0s Python

El resultado crudo de **summarize()** viene "todo en una línea", sin formato. Esta función lo pasa por **display(Markdown(...))** para que se vea con títulos, negritas y saltos de línea, en vez de texto plano feo.

```
display_summary("https://edwarddonner.com")
```

12.2s Python

Salida:

## El Templo de Inteligencia Artificial de Ed Donner

Felicidades, has llegado al sitio web oficial de **Edward "Ed" Donner**, un ex-Director de JPMorgan y emprendedor en serie que necesita desesperadamente que sepas lo apasionado que es por la IA.

### Resumen del Personaje

- **El origen de su éxito:** A Ed le gusta tanto hablar sobre LLMs (Modelos de Lenguaje) que sus amigos, completamente hartos de escucharlo hablar solo, lo convencieron de crear cursos en Udemmy. Ahora aburre a más de 900,000 estudiantes en todo el mundo.
- **Sus pasatiempos:** Produce música electrónica (él mismo aclara que es *muy mala*) y lee *Hacker News* asintiendo con la cabeza para fingir que entiende lo que lee.
- **Narcisismos de alta tecnología:** Por si no tienes suficiente con el Ed real, creó un **Avatar Digital** para que puedas chatear con su versión cibernética las 24 horas del día.

---

### Anuncios y Noticias (Directo desde el Futuro)

Ed no solo programa, también parece viajar en el tiempo, ya que nos deja ver sus futuros "recursos" y cursos programados entre 2025 y 2026:

- **Mayo 2025:** La guía existencial para saber en qué orden deberías gastar tu dinero en sus cursos.
- **Septiembre 2025:** *AI Engineering MLOps Track*, para que aprendas a mandar IAs a producción antes de que se caiga el servidor.
- **Enero 2026:** *AI Builder con n8n*, enfocado en crear agentes de voz (porque claramente el mundo necesita más robots hablándonos).
- **Febrero 2026:** *AI Coder: Vibe Coder a Agentic Engineer*, un curso diseñado para convertir a los programadores "por intuición" en ingenieros reales.

1.5.3 Limitación del scraper actual: sitios con JavaScript + Playwright + Screenshots de notebook

`fetch_website_contents` (con `requests` + `BeautifulSoup`) solo descarga el HTML **crudo**, sin ejecutar JavaScript. En sitios modernos (React, Vue, Angular), el contenido real se genera recién cuando un navegador ejecuta ese JavaScript — así que el scraper actual encuentra una página casi vacía (ej. `openai.com` falla con este método).

La alternativa: Playwright

**Playwright** abre un navegador real (invisible, "headless"), deja que el JavaScript se ejecute, espera a que la página termine de cargar (auto-wait), y recién ahí extrae el HTML ya completo. El resto del proceso (limpieza con `BeautifulSoup`) queda igual.

Instalación (gratis, sin límites ni suscripción):

```
powershell

uv pip install playwright
playwright install
```

Función de scraping con Playwright:

```
from playwright.sync_api import sync_playwright
from bs4 import BeautifulSoup

def fetch_website_contents_playwright(url):
    with sync_playwright() as p:
        browser = p.chromium.launch() # abre Chromium invisible
        page = browser.new_page()
        page.goto(url) # navega a la url
        html = page.content() # HTML YA renderizado, con JS ejecutado
        browser.close()

    soup = BeautifulSoup(html, "html.parser")
    title = soup.title.string if soup.title else "No title found"
    if soup.body:
        for irrelevant in soup.body(["script", "style", "img", "input"]):
            irrelevant.decompose()
        text = soup.body.get_text(separator="\n", strip=True)
    else:
        text = ""
    return (title + "\n\n" + text)[:2_000]
```

|                              | requests (actual)           | Playwright (alternativa)           |
|------------------------------|-----------------------------|------------------------------------|
| Ejecuta JavaScript           | No                          | Si                                 |
| Funciona en sitios React/Vue | No                          | Si                                 |
| Velocidad                    | Rapida                      | Más lento (abre un navegador real) |
| Costo                        | Gratis                      | Gratis (open source, de Microsoft) |
| Recomendación de uso         | Mayoría de sitios estáticos | Solo los sitios que fallan por JS  |

Selenium es la otra alternativa mencionada por el profesor, aunque hoy (2026) **Playwright** es el estándar más recomendado para proyectos nuevos: más rápido, con auto-wait incorporado, y soporte multi-navegador desde una sola instalación.

# Problemas con más sitios web

Ten en cuenta que esto solo funcionará en sitios web que puedan ser scrapeados usando este enfoque simplista.

Los sitios web renderizados con Javascript, como las apps de React, no aparecerán. Revisa la carpeta `community-contributions` para ver una implementación con Selenium que evita este problema. Necesitarás informarte sobre cómo instalar Selenium (¡pregúntale a ChatGPT!)

También los sitios web protegidos con CloudFront (y similares) pueden dar errores 403 - muchas gracias a Andy J por señalar esto.

Por muchos sitios web, funciona sin problemas!

```
[ ] display_summary("https://cnn.com")
```

Python

```
[ ] display_summary("https://anthropic.com")
```

Python

## Un ejercicio extra para quienes disfrutan el web scraping

Puede que notes que si intentas `display_summary("https://openai.com")` - ¡no funciona! Eso es porque OpenAI tiene un sitio web sofisticado que usa Javascript. Hay muchas formas de evitar esto con las que algunos de ustedes pueden estar familiarizados. Por ejemplo, Selenium es un framework enormemente popular que ejecuta un navegador por detrás, renderiza la página, y te permite consultarla. Si tienes experiencia con Selenium, Playwright o un framework similar de automatización de navegador, siéntete libre de implementar una versión alternativa de las funciones de scraping en `scraper.py` (o en un módulo separado) usando una de estas herramientas. En la carpeta `community-contributions`, encontrarás ejemplos aportados por otros estudiantes, incluyendo una implementación con Selenium (¡gracias!)

### 1.5.4 Cierre del laboratorio: contribuciones comunitarias + ScreenShots

- Carpeta **community-contributions** en el repo: lugar para subir variantes propias del ejercicio (ej. alguien integró la API de Yahoo Finance para resumir movimientos de acciones).
- Se puede compartir el propio trabajo enviando un **Pull Request** (instrucciones en la guía de Git/GitHub del curso).
- Mensaje de cierre del profesor: este patrón simple (scrapear + resumir con un LLM) es la base de un caso de uso de negocio muy común en IA generativa — resumen y traducción de contenido — y vale la pena adaptarlo a un problema propio, personal o de negocio.

llm\_engineering-EdDooner / community-contributions /

Add file ▾

...

ed-donner Merge pull request ed-donner#3871 from elton-bleta/tony/day2-solution

3d5fe9c · 5 days ago History

This branch is up to date with ed-donner/llm\_engineering:main .

Contribute ▾

Sync fork ▾

| Name                 | Last commit message                                                      | Last commit date |
|----------------------|--------------------------------------------------------------------------|------------------|
| ..                   |                                                                          |                  |
| Abhijith_HackTrack   | Add HackTrack to community contributions                                 | 3 months ago     |
| Ahmed Hafez Projects | AI Assistant Inventory managment system week 2 practice                  | 3 months ago     |
| Ahmed-Hafez/Week-2   | clear notebook outputs                                                   | 5 months ago     |
| Arjav014             | Merge pull request ed-donner#3380 from Arjav014/personal-assistant       | 3 months ago     |
| Ashok                | ashok chnages'                                                           | last month       |
| CodeMaestroMS        | Merge pull request ed-donner#3678 from CodeMaestroMS/codemaestrms/wee... | last month       |

llm\_engineering-EdDooner / community-contributions /

↑ Top

|                                                 |                                                                    |               |
|-------------------------------------------------|--------------------------------------------------------------------|---------------|
| Shield01/Week1                                  | Built a technical assistance using llama3.2                        | 9 months ago  |
| Suyash/python_teacher                           | Added Suyash python_teacher contribution notebook                  | 10 months ago |
| Sylvie                                          | Sylvie's week1 exercise (cleared outputs)                          | 3 months ago  |
| Vagz1216                                        | Bootcamp/Euclid: Add exception catching and traceback to Gradio UI | 6 months ago  |
| VenkatKrishna                                   | Add Week 3 synthetic data generator community contribution         | 2 weeks ago   |
| WebScraperApp                                   | Add Assaf week1 day2 exercise solution                             | 11 months ago |
| abdoul                                          | Merge pull request ed-donner#924 from Abdoulrasheed/w-viii         | 11 months ago |
| adeyemi-kayode                                  | feat(week1): created a technical assistant tool                    | 7 months ago  |
| aditya-soni-contributions                       | Added week1 day2 contribution                                      | 5 months ago  |
| aditya_battlecard                               | Add Battlecard.ai: Autonomous Competitor Research Agent            | 9 months ago  |
| adityachebiyyam/3_way_llm_conversation_showdown | Week2 3WayLLMConversation                                          | 6 months ago  |
| ahmad19sep                                      | Add email subject writer contribution                              | 2 weeks ago   |
| ahmed_eldamaty/week_1                           | contribution(week-1): add ManualMate project                       | 3 weeks ago   |
| ahmed_sohail                                    | changed the user and system prompt tone                            | 8 months ago  |
| akhilasaiteja                                   | Update README.md                                                   | 5 months ago  |
| alessandro-fugazza                              | Create stateful_chatbot.ipynb                                      | 4 months ago  |



## 1.6 FINAL-DIA-1: Ejercicio final – tu propia llamada a un LLM desde cero

### El desafío del profesor

Después de construir el resumidor de páginas web, el profesor deja de dar el código armado y pide escribirlo desde cero, aplicado a una idea de negocio distinta. Entrega una plantilla con los pasos marcados pero vacíos:

```
# Paso 1: Crea tus prompts

system_prompt = "algo aquí"
user_prompt = """
    Puedes pegar
    mucho texto aquí
    """

# Paso 2: Arma la llamada a Gemini
# response = client.models.generate_content(model="gemini-3.6-flash",
# config=genai.types.GenerateContentConfig(system_instruction=system_prompt),
# contents=user_prompt)

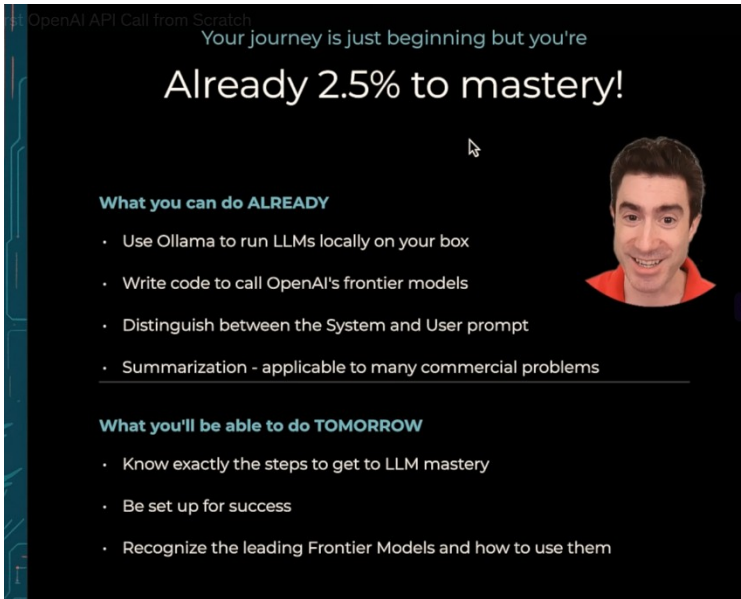
# Paso 3: imprime el resultado
# print()
```

### Cómo completarlo, paso a paso

1. **Pensar una idea de negocio distinta** a resumir páginas web. El profesor da dos ejemplos: resumir el contenido de un email, o generar un buen asunto (subject line) a partir del cuerpo de un correo.
2. **Escribir el system\_prompt**: una instrucción clara que describa la tarea (ej. "Sos un asistente que lee un email y sugiere un asunto corto y claro para él").
3. **Escribir el user\_prompt**: el contenido real a procesar (el texto del email, por ejemplo).
4. **Armar la lista messages**: los dos diccionarios (**system** + **user**), como se vino haciendo en todo el laboratorio. El profesor aclara: usar herramientas de IA para ayudarte con esto **no es hacer trampa**.
5. **Descomentar y completar response** =: la llamada al modelo (en la versión adaptada a Gemini: `client.models.generate_content(...)`, en vez de `openai.chat.completions.create(...)` como en el video original).
6. **Descomentar e imprimir el resultado**: con `response.text` (Gemini) en vez de `response.choices[0].message.content` (OpenAI).

### 1.6.1 Ejercicio extra opcional: scraping "industrial" + 5 ideas de desafío para practicar (investigadas) + Consejo general para cualquiera de estos desafíos

Para quienes ya conocen Selenium o Playwright, el profesor sugiere construir una versión más robusta del resumidor web, que sí funcione en sitios renderizados con JavaScript — hay ejemplos de otros estudiantes en **community-contributions**, incluida una versión con Playwright.



#### Balance del Día 1 (según el profesor)

##### Ya sabés hacer:

- Usar Ollama para correr LLMs localmente
- Escribir código que llama a modelos frontera
- Distinguir system prompt de user prompt
- Resumir contenido — aplicable a muchos problemas comerciales

##### Mañana (Día 2) vas a poder:

- Conocer los pasos exactos hacia el dominio de LLM Engineering
- Reconocer los principales modelos frontera y cómo usarlos

Dato de cierre: el profesor calcula, en broma, que con todo esto ya estás en el **2.5% del camino** hacia ser "maestro de LLM Engineering".

---

### 5 ideas de desafío para practicar (investigadas)

#### 1. Generador de asunto para emails

Basado en los usos más comunes y efectivos de prompt engineering para tareas de negocio, estas son cinco variantes del mismo patrón (system\_prompt + user\_prompt + llamada al modelo) para practicar, de menor a mayor complejidad:

**Por qué es un buen desafío inicial:** es casi idéntico al ejemplo del profesor, ideal para practicar la mecánica sin complicarse con la idea.

## 2. Análisis de sentimiento de reseñas de producto

**system\_prompt:** "Sos un analista que clasifica reseñas de clientes como Positiva, Negativa o Neutral, y explica en una frase por qué". **user\_prompt:** una reseña real (se puede tomar de cualquier producto en Amazon, MercadoLibre, etc.).

**Por qué es útil:** es uno de los casos de uso más comunes de LLMs en negocios reales — analizar feedback de clientes a escala, sin tener que leer cada reseña a mano.

## 3. Resumen con restricciones de formato ("explícamelo como si tuviera 5 años")

**system\_prompt:** "Explicá el siguiente texto técnico como si hablaras con alguien de 5 años, usando máximo 3 oraciones". **user\_prompt:** un párrafo técnico o científico complejo (por ejemplo, de Wikipedia).

**Por qué es interesante:** demuestra que agregar restricciones concretas (edad del público, cantidad de oraciones) al prompt mejora mucho la calidad del resultado, comparado con un simple "resumí esto".

## 4. Clasificador de tickets de soporte

**system\_prompt:** "Sos un clasificador que lee un ticket de soporte técnico y responde solo con una categoría: Facturación, Bug técnico, Consulta general o Urgente". **user\_prompt:** el texto de un ticket inventado o real (ej. "no puedo iniciar sesión desde ayer, ya reinicié todo").

**Por qué es un buen ejercicio:** introduce la idea de **clasificación** (no solo resumen/generación) — pedirle al modelo que elija entre opciones fijas, útil como primer paso hacia lo que después será RAG y agentes en el curso.

## 5. Traductor con tono personalizado

**system\_prompt:** "Traducí el siguiente texto al español, pero con un tono informal y chileno (modismos incluidos)". **user\_prompt:** un párrafo en inglés (por ejemplo, un fragmento de una noticia).

**Por qué es divertido y educativo:** el propio profesor menciona la traducción como ejemplo de variante en community-contributions — y permite ver claramente cómo un mismo texto cambia radicalmente de "personalidad" según las instrucciones del system prompt, reforzando la lección central del día.

---

## Consejo general para cualquiera de estos desafíos

Según buenas prácticas de prompt engineering: los mejores resultados salen cuando el system\_prompt incluye **parámetros concretos y medibles** (largo máximo, tono, formato de salida, público objetivo) en vez de instrucciones vagas tipo "hacelo bien". Por ejemplo, "resumí esto en 3 oraciones para un público general" funciona mejor que simplemente "resumí esto".

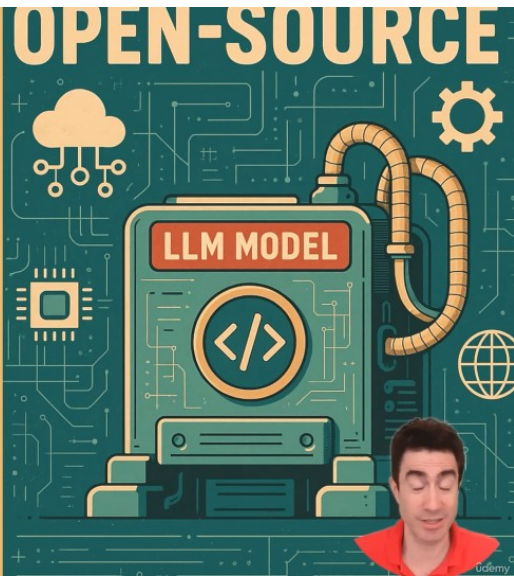
## 1.7 DIA 2-FINAL: El panorama open-source y las tres formas de usar un LLM

Open-Source



Llama from Meta  
Mixtral from Mistral  
Qwen from Alibaba Cloud  
Gemma from Google  
Phi from Microsoft  
DeepSeek from DeepSeek AI  
GPT-OSS from OpenAI

# OPEN-SOURCE



### Three ways to use models



Chat interfaces  
Like ChatGPT



Cloud APIs  
LLM API  
Frameworks like LangChain  
Managed AI cloud services:

- Amazon Bedrock
- Google Vertex
- Azure ML



Direct inference  
With the HuggingFace Transformers library  
With Ollama to run locally



1.7.1 ¿Qué separa "modelo frontera" de "open-source" para el profesor? + Panorama de modelos de open-source

¿Qué separa "modelo frontera" de "open-source" para el profesor?

El profesor aclara un matiz de vocabulario: reserva el término "**modelo frontera**" para los modelos de código **cerrado** (GPT, Claude, Gemini), aunque técnicamente algunos modelos open-source también estén en la frontera de lo posible.

2. Panorama de modelos open-source

| Empresa               | Modelo   | Parametros / dato clave                                                                                                         | Fuente                                       |
|-----------------------|----------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| Meta                  | Llama    | Llama 3.2 viene en <b>1B y 3B</b> (no solo 3B como se dijo en el video) — ollama run llama3.2 baja la de 3B por defecto         | Confirmado en registry.ollama.ai y llama.com |
| Mistral AI (Francia)  | Mixtral  | Arquitectura <b>Mixture of Experts (MoE)</b> : varios sub-modelos especializados, se activa solo el necesario según la pregunta | —                                            |
| Alibaba Cloud (China) | Qwen     | Modelo muy potente, menos conocido que Llama — el profesor lo recomienda especialmente                                          | -                                            |
| Google                | Gemma    | "Primo open-source" de Gemini; versión más chica de 270M de parámetros (la que se probó con Ollama en el Día 1)                 | -                                            |
| Microsoft             | Phi      | Buena en tool calling y uso comercial; la usada en el Día 1 (phi3)                                                              | -                                            |
| DeepSeek AI (China)   | DeepSeek | Ver sección 3                                                                                                                   | -                                            |
| OpenAI                | GPT-OSS  | Ver sección 4                                                                                                                   | -                                            |

Por qué Meta abrió sus modelos (lectura del profesor)

Una interpretación "poco caritativa" que plantea el profesor: Meta no lograba competir con OpenAI/Anthropic en tener el modelo más potente del momento, y encontró en el open-source una forma de diferenciarse y mantenerse en los titulares. Funcionara o no como estrategia deliberada, hoy Meta es sinónimo de liderazgo en IA abierta.

1.7.2 El fenómeno DeepSeek + Tamaño real del modelo

Por que causo tanto revuelo (no fue por ser “el mas potente”)

DeepSeek no supera a GPT-5 ni a Claude en capacidad — lo que causó el revuelo fue la **eficiencia del entrenamiento**:

|              | Costo de entrenamiento                                                                                         |
|--------------|----------------------------------------------------------------------------------------------------------------|
| GPT (OpenAI) | Más de \$100 millones USD (estimado)                                                                           |
| DeepSeek-V3  | ~\$5,6 millones USD ( <b>2.788 millones de horas de GPU H800</b> ), confirmado en el paper oficial de DeepSeek |

Tamaño real del modelo

Confirmado en Hugging Face y el paper oficial: **DeepSeek-V3/R1 tiene 671.000 millones de parámetros totales, con 37.000 millones activos por token** — es un modelo **MoE**, igual que Mixtral. Demasiado grande para correr en una PC normal (según estimaciones de hardware, requeriría del orden de 1.300+ GB de VRAM, o un clúster de 16 GPUs A100 de 80GB).

Cómo se arman las versiones chicas que sí corren en Ollama (destilación)

Cuando corriste ollama run llama3.2 o cuando corres ollama run deepseek-r1:8b, en ambos casos Ollama está ejecutando, por dentro, un modelo con la misma **arquitectura** de Llama (misma "forma" de red neuronal). La diferencia está en **qué aprendió** ese modelo durante su entrenamiento — y ahí es donde entra la destilación.

Cómo funciona el proceso, paso a paso:

1. DeepSeek entrena su modelo grande (671B parámetros) de la forma tradicional: con enormes cantidades de texto de internet.
2. Una vez entrenado, le hacen miles de preguntas de razonamiento (matemática, código, lógica) a ese modelo grande, y guardan sus respuestas — sobre todo el "paso a paso" que usa para llegar a la solución. Esto genera un dataset de **800.000 ejemplos** de razonamiento de alta calidad.
3. Ese dataset se usa para **re-entrenar** un modelo de Llama o Qwen que ya existía (por ejemplo, Llama 3.1-8B), mediante un proceso llamado **fine-tuning supervisado (SFT)**: al modelo chico se le muestran, una y otra vez, pares de "pregunta → respuesta razonada paso a paso" (los 800.000 ejemplos generados por DeepSeek), y sus parámetros internos se ajustan poco a poco para que aprenda a reproducir ese mismo estilo de razonamiento — sin empezar de cero, sino partiendo de lo que Llama ya sabía.

## Entonces, ¿qué es exactamente lo que se descarga con ollama pull deepseek-r1:8b?

Se descarga un modelo con la **arquitectura de red neuronal de Llama** (las mismas "capas" y conexiones internas que tiene cualquier Llama 3.1-8B), pero cuyos parámetros internos fueron reajustados con las enseñanzas de razonamiento de DeepSeek. Es como tomar el "cerebro" físico de Llama y volver a entrenarlo con material nuevo — la estructura del cerebro sigue siendo la misma, pero lo que sabe hacer cambió.

Por eso Ollama lo etiqueta con el nombre deepseek-r1, aunque técnicamente sea un Llama por dentro: lo que identifica al modelo, de cara al usuario, es **de dónde viene el conocimiento** (DeepSeek), no la arquitectura que lo ejecuta (Llama). Esto no es algo que Ollama "adapte" por su cuenta — el propio equipo de DeepSeek hizo este re-entrenamiento y publicó el resultado así, y Ollama simplemente lo empaquetó para que se pueda correr fácil, igual que hace con cualquier otro modelo. 4. El resultado es un modelo que sigue siendo "Llama" o "Qwen" por dentro (misma arquitectura, mismo tamaño), pero que **razona mucho mejor** que la versión original, porque aprendió a imitar el estilo de razonamiento del DeepSeek grande.

**Por qué se llama "destilación":** la idea viene de la química — destilar es extraer la "esencia concentrada" de algo. **Acá se extrae el "conocimiento de razonamiento"** del modelo gigante y se concentra en uno chico, sin necesitar todo el tamaño y costo del original.

**Por qué esto importa en la práctica:** significa que cuando corrés deepseek-r1:8b en tu PC, **no estás corriendo "DeepSeek" en su forma original** — estás corriendo un Llama 3.1-8B que fue mejorado con las enseñanzas de DeepSeek. Confirmado en la documentación oficial de Ollama: el modelo de 8B deriva literalmente de **Llama 3.1-8B-Base**, y el de 70B deriva de **Llama 3.3-70B-Instruct** — ambos conservan la licencia original de Llama, no una licencia de DeepSeek.

1.7.3 GPT-OSS: la versión abierta de OpenAI + Las tres formas de usar cualquier modelo de IA + Diferencia entre Hugging Face Transformers y Ollama (ambas son "inferencia directa")

**Corrección sobre el video:** el profesor menciona que la versión chica es de "120 mil millones" — esto es un error de transcripción/lapsus. Los datos oficiales, confirmados por múltiples fuentes técnicas coincidentes (lanzamiento oficial del 5 de agosto de 2025, licencia Apache 2.0):

| Modelo                               | Parámetros totales        | Parámetros activos (MoE) | Hardware necesario                              |
|--------------------------------------|---------------------------|--------------------------|-------------------------------------------------|
| gpt-oss-20b (el probado en el Día 1) | 20.910.000.000 (20.91B)   | 3.610.000.000 (3.61B)    | Cabe en 16 GB — consumer hardware               |
| gpt-oss-120b                         | 116.830.000.000 (116.83B) | 5.130.000.000 (5.13B)    | Cabe en una sola GPU de 80 GB (ej. NVIDIA H100) |

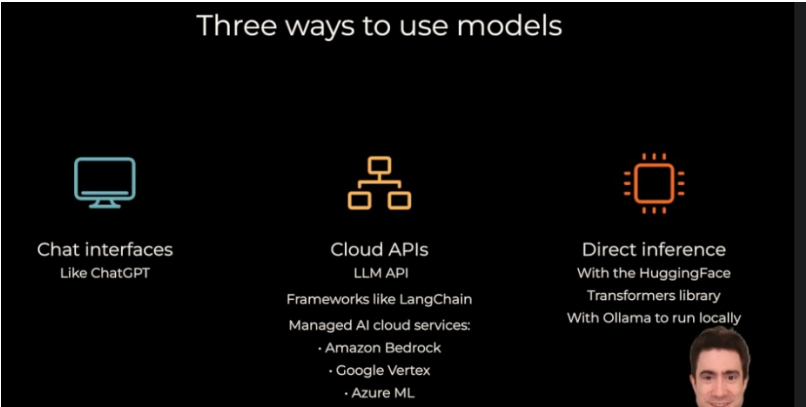
Ambos son modelos de razonamiento (soportan chain-of-thought, niveles ajustables de esfuerzo de razonamiento) — coincide con el bloque de "Thinking..." que se vio al probar **gpt-oss:20b** en el Día 1.

La teoría del profesor sobre por qué OpenAI lo abrió

Plantea que quizás DeepSeek forzó la jugada: al abrir un modelo casi al nivel de GPT de forma gratuita, le quitó a OpenAI parte de su ventaja de "tener lo mejor y mantenerlo cerrado" — dato de contexto real: GPT-OSS es la primera apertura de pesos de OpenAI desde GPT-2 en 2019, más de 6 años de modelos cerrados.

Las tres formas de usar cualquier modelo de IA

Aplica tanto a modelos cerrados (GPT, Claude, Gemini) como abiertos (Llama, Qwen, Gemma):



| Forma                         | Ejemplos                                                                                                                                                                   | Que es realmente                                                                                                                                 |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Interfaces de chat (producto) | ChatGPT                                                                                                                                                                    | No es "solo el modelo" – es un <b>producto</b> : interfaz de usuario, memoria, búsqueda web, todo construido por ingenieros sobre el modelo base |
| APIs en la nube               | Llamadas directas (OpenAI, Gemini — lo hecho en el Día 1), frameworks como LangChain, servicios gestionados (Amazon Bedrock, Google Vertex AI, Azure ML), Groq, OpenRouter | Se llama a un modelo corriendo en servidores de un tercero                                                                                       |
| Inferencia directa            | Hugging Face Transformers, u <b>Ollama</b>                                                                                                                                 | Se corre el modelo con hardware propio, sin depender de ningún servidor externo                                                                  |



Diferencia entre Hugging Face Transformers y Ollama (ambas son "inferencia directa")

- **Hugging Face Transformers:** se toma el código fuente original (Python/C++) del modelo y sus pesos, y se corre directamente en el propio intérprete de Python — más flexible, pero más técnico.
- **Ollama:** empaqueta todo eso de forma optimizada – código en C++ eficiente, pesos comprimidos en archivos **GGUF** – y da una API local lista para usar en pocos pasos. Es mas simple, peros solo soporta los modelos específicos que Ollama empaqueto.

1.7.4 Que es “inferencia” + inferencia vs prompt

Qué es "inferencia"

Término técnico que se va a usar constantemente en el curso: **inferencia = correr un modelo**. Es el proceso de darle una entrada nueva a un modelo ya entrenado y obtener una salida (una respuesta).

Otros conceptos mencionados de pasada

- **Grok (con Q):** un tipo de servicio para correr modelos open-source a gran velocidad en la nube usando hardware propietario — distinto de **Grok (con K)**, el modelo de IA de Elon Musk/xAI. El profesor advierte explícitamente sobre esta confusión de nombres.
- **OpenRouter:** plataforma que redirige pedidos a muchos proveedores de modelos distintos desde un solo punto de entrada.

[23-09-2026]

que era la inferencia?

**Inferencia** es el nombre técnico para "**ejecutar un modelo de IA**" — es decir, darle una entrada nueva a un modelo ya entrenado y obtener una salida (una respuesta).

Repasando de dónde salió esto

Lo vimos en el Día 2 del curso de LLM Engineering, cuando el profesor lo aclaró justo antes de volver al laboratorio de Ollama:

*"Inferencia es un nombre elegante para decir ejecutar un modelo. La inferencia es lo que se hace cuando se tiene un modelo y nuevas entradas y se quiere obtener la salida, eso se llama inferencia."*

En criollo

| Etap          | Qué es                                                                         | Ejemplo                                                        |
|---------------|--------------------------------------------------------------------------------|----------------------------------------------------------------|
| Entrenamiento | Cuando se "le enseña" al modelo, ajustando sus parámetros con muchísimos datos | Meta entrenando Llama con billones de textos                   |
| Inferencia    | Cuando VOS lo usás después, dándole una entrada y obteniendo una respuesta     | <code>ollama run llama3.2</code><br>"cuéntame un dato curioso" |

## Dónde aplica en todo lo que veníamos armando

Cada vez que hiciste esto, estabas haciendo **inferencia**:

- **ollama run llama3.2** → inferencia
- Llamar a **client.models.generate\_content(...)** con Gemini → inferencia
- Correr MediaPipe sobre un video → también es inferencia (aunque el modelo no genere texto, sigue siendo "usar un modelo ya entrenado con datos nuevos")

## Por qué el término importa

Es la palabra que se usa en la industria para distinguir "usar el modelo" de "entrenar el modelo" — vas a verla constantemente en documentación técnica, precios de APIs ("costo por inferencia"), y hardware ("GPU optimizada para inferencia" vs "GPU para entrenamiento").

es lo mismo que prompt?

No, son cosas distintas, aunque están relacionadas. Te separo bien los conceptos.

## La diferencia

|          | Prompt                                   | Inferencia                                                   |
|----------|------------------------------------------|--------------------------------------------------------------|
| Qué es   | El texto/entrada que le mandás al modelo | El proceso completo de ejecutar el modelo con esa entrada    |
| Es un... | Un dato (lo que escribís)                | Una acción/operación (lo que pasa)                           |
| Ejemplo  | "Cuéntame un dato curioso"               | Todo el cálculo que hace el modelo para generar la respuesta |

## Analogía simple

Pensalo como pedir comida:

| Concepto   | Equivalente                                                                                  |
|------------|----------------------------------------------------------------------------------------------|
| Prompt     | Tu pedido: "quiero una pizza de muzzarella"                                                  |
| Inferencia | El proceso completo: el cocinero (el modelo) recibe el pedido, cocina, y te entrega el plato |

El **prompt** es solo lo que vos decís. La **inferencia** es todo el proceso de "usar el modelo" — que incluye recibir tu prompt, procesarlo, y devolver una respuesta.

## Con código, para verlo bien claro

```
python
prompt = "Cuéntame un dato curioso" # ESTO es el prompt

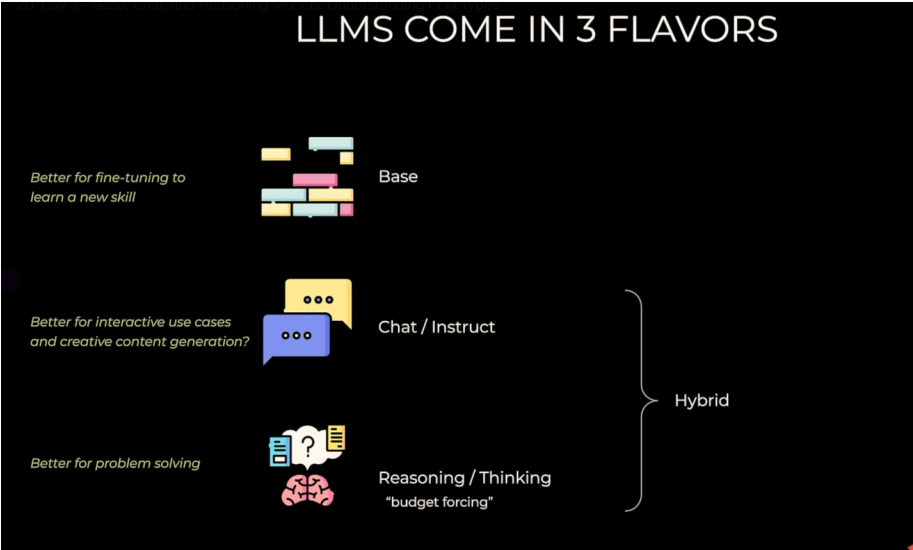
response = client.models.generate_content(
    model="gemini-3.6-flash",
    contents=prompt # el prompt entra ACÁ
) # TODA esta línea, ejecutarla, ES la inferencia
```

## Resumen

- **Prompt** = el texto de entrada, lo que escribís
- **Inferencia** = el acto de ejecutar el modelo (con ese prompt, o con cualquier otra entrada — una imagen para MediaPipe, por ejemplo)

Un prompt siempre es parte de una inferencia (cuando trabajás con LLMs), pero inferencia es un concepto más amplio que aplica a **cualquier modelo**, no solo a los que reciben texto — por eso MediaPipe también "hace inferencia" sobre una imagen, aunque ahí no exista ningún "prompt".

1.8 DIA 3-1: Los 3 “sabores” de LLM – BASE, Chat/Instruct y Razonamiento



Apuntes de clase — Curso de LLM Engineering El dato técnico central de este documento (paper S1 y "budget forcing") fue verificado contra el paper académico original — ver fuente al final.

Contexto: día sin laboratorio

El profesor aclara al arrancar que el Día 3 es uno de los pocos sin ejercicio práctico de código — es una sesión puramente teórica, dedicada a entender la clasificación de modelos antes de **comparar modelos de frontera más a fondo**.

Las 3 categorías de modelos, según para que fueron entrenados

| Tipo                 | Que hace                                                                                                               | Mejor para                                                          |
|----------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Base                 | Solo predice la palabra/token mas probable que sigue a una secuencia – no “chatea”, no responde preguntas directamente | <b>Fine-tuning para enseñarle una habilidad nueva desde cero</b>    |
| Chat / Instruct      | Entrenado con ejemplos de conversación (system prompt + user prompt + respuesta) para seguir instrucciones             | <b>Casos de uso interactivos y generación de contenido creativo</b> |
| Reasoning / Thinking | Entrenado para mostrar su razonamiento paso a paso antes de dar la respuesta final                                     | Resolución de problemas complejos                                   |

1.8.1 Modelo Base: El punto de partida

Un modelo base **solo completa secuencias** — no fue entrenado para "conversar". El ejemplo cotidiano que da el profesor: el **texto predictivo del celular** funciona con esta misma lógica — vas apretando la palabra sugerida una y otra vez, y arma una oración completando secuencia por secuencia.

Antes de ChatGPT (2022), los modelos como GPT-3 eran solo modelos base. Para forzarlos a "responder preguntas", la gente usaba trucos de formato, como escribir varios ejemplos de "Pregunta: ... Respuesta: ..." antes de la pregunta real, para que el modelo "entendiera" el patrón y siguiera completándolo de la misma forma.

1.8.2 Modelo Chat/Instruct: como nació (RLHF)

OpenAI enttenu sus modelos con datos estructurados en el formato que ya se conoce: **system prompt** (el marco general) + **user prompt** (el mensaje del usuario) + **respuesta del asistente**. La técnica usada para este entrenamiento se llama **RLHF** (Reinforcement Learning from Human Feedback / aprendizaje por refuerzo a partir de retroalimentación humana) – fue lo que transformo GPT en ChatGPT.

1.8.3 Modelo de Razonamiento: el paso siguiente – chain-of-thought prompting – Resultados concretos del paper (con cifras verificadas)

El truco simple que lo empezó todo: "pensá paso a paso"

Se descubrió que agregar la frase *"por favor, pensá paso a paso"* al final de un prompt mejoraba notablemente los resultados — el modelo empezaba a resolver las cosas de forma más metódica, solo por ese pedido explícito. A esto se le llamó **chain-of-thought prompting** (cadena de pensamiento).

De ahí surgió la idea de entrenar modelos para razonar por defecto

En vez de pedirle cada vez que "piense paso a paso", se empezaron a entrenar modelos con **muchos ejemplos que muestran el razonamiento completo** antes de la conclusión – así nacieron los modelos de razonamiento (el **gpt-oss** que se probó en el día 1, con su bloque "Thinking...", es exactamente eso).

El descubrimiento "budget forcing" — verificado con el paper original

El profesor menciona un paper llamado **S1**, de enero de 2025, que descubrió algo sorprendentemente simple. Confirmado con el paper original (arXiv 2501.19393, "s1: Simple test-time scaling", de Niklas Muennighoff y colegas):

Qué es realmente "budget forcing"

Es una técnica para controlar **cuanto** razona un modelo antes de responder, con dos mecanismos:

| Mecanismo                | Que hace                                                                                                                                                                                                              |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Forzar el corte          | Si el modelo ya pensó demasiados tokens, se le agrega un token de "fin de pensamiento" a la fuerza, obligándolo a dar la respuesta final                                                                              |
| Forzar que siga pensando | Si el modelo intenta terminar de pensar demasiado pronto (antes de un mínimo), el sistema <b>bloquea</b> su intento de parar, y en su lugar le agrega la palabra <b>"Wait"</b> ("Espera") al final de lo que escribió |

Por qué "Wait" funciona (confirmado en el paper)

Cuando se le agrega la palabra "Wait" al final del razonamiento del modelo, el modelo **naturalmente continúa la secuencia** de forma coherente con esa palabra — es decir, empieza a reconsiderar y cuestionar su respuesta anterior, revisando si el razonamiento sigue siendo correcto, y muchas veces **corrige errores** que había cometido.

---

Resultados concretos del paper (con cifras verificadas)

| Dato                             | Valor confirmado                                                                                                    |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Dataset de entrenamiento usado   | Solo <b>1.000 ejemplos</b> curados (llamado s1K)                                                                    |
| Modelo base usado                | Qwen2.5-32B-Instruct                                                                                                |
| Tiempo de entrenamiento          | <b>26 minutos</b> , en 16 GPUs H100                                                                                 |
| Costo estimado del entrenamiento | <b>~\$25 USD</b>                                                                                                    |
| Resultado (modelo final: s1-32B) | Superó a o1-preview de OpenAI en matemática competitiva por hasta un <b>27%</b> (benchmarks MATH y AIME24)          |
| Con más "budget forcing"         | Mejóro de 50% a 57% en AIME24, solo agregando más pasos de razonamiento en el momento de responder (sin reentrenar) |
| Código y modelo                  | 100% abiertos — pesos, datos y código publicados                                                                    |

1.8.4 Modelos híbridos: la evolución actual – reasoning Budget - test-time reasoning – reasoning trace

Los modelos más modernos (según el profesor, en el momento de grabar: **Gemini 2.5 Pro y GPT-5**) son **híbridos** — no son "puramente" de razonamiento, sino que **deciden ellos mismos** cuánto razonar según la pregunta:

- Con un saludo simple ("hola") → prácticamente no razona, responde como un modelo de chat normal
- Con un problema complejo → activa un razonamiento más profundo, similar a un modelo de razonamiento puro

Este nivel de razonamiento se llama **presupuesto de razonamiento** (reasoning budget) o **esfuerzo de razonamiento** (reasoning effort) — y es una decisión que se toma **antes** de que el modelo empiece a razonar: es el tope o margen que se le da (por ejemplo, elegir "low/medium/high reasoning effort" al llamar a la API de un modelo híbrido).

El acto de razonar en sí: “inferencia en tiempo de prueba”

Distinto al *presupuesto* (que se fija antes), el nombre técnico para *el acto mismo* de que un modelo piense — lo que ocurre **durante** ese margen ya fijado— es **test-time compute** o **test-time reasoning** (cómputo/razonamiento en tiempo de inferencia), porque sucede durante la **inferencia** (cuando el modelo ya entrenado procesa tu pregunta), no durante el entrenamiento. El texto que el modelo genera mientras "piensa" se llama **thinking tokens** o **reasoning trace** (rastro de razonamiento) — es justo lo que se ve en el bloque "Thinking..." de gpt-oss.

Cuándo conviene usar cada tipo

| Situación                                                                      | Modelo recomendado        | Por que                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Resolver problemas lógicos/matemáticos difíciles</b>                        | Razonamiento              | Mejor puntaje en casi todos los benchmarks                                                                                                                                                                                              |
| <b>Chat interactivo (rapidez importa)</b>                                      | Chat/Instruct             | Más rápido y barato — no genera tokens de "pensamiento" extra                                                                                                                                                                           |
| <b>Generar contenido creativo (emails, textos)</b>                             | Chat/Instruct (con matiz) | Anecdóticamente, los modelos de razonamiento a veces resultan "fríos y analíticos"; el profesor aclara que esto <b>no está bien medido con métricas formales</b> , es más una observación de la comunidad — conviene probarlo uno mismo |
| <b>Entrenar un modelo para una habilidad completamente nueva (fine-tuning)</b> | Base                      | Al no tener un "estilo" de chat o razonamiento ya fijado, es mas maleable para aprender un comportamiento distinto de cero                                                                                                              |

#### 1.5.4 Por que NO usar siempre un modelo de razonamiento (aunque puntue mejor)

El profesor se hace esta pregunta directamente — si los modelos de razonamiento sacan mejor puntaje en casi todos los benchmarks, ¿por qué no usarlos siempre? Da tres razones:

1. **Velocidad y costo:** los modelos de chat son más rápidos y baratos porque no tienen que generar todos los tokens del razonamiento intermedio antes de la respuesta final — mejor para casos de uso interactivo.
2. **Contenido creativo:** de forma anecdótica (sin una métrica formal que lo confirme), la comunidad reporta que los modelos de razonamiento a veces "piensan demasiado" y producen contenido más frío y analítico — para un email o texto creativo, un modelo de chat normal suele fluir mejor.
3. **Fine-tuning para una habilidad nueva:** si el objetivo es entrenar el modelo desde cero para que haga algo distinto (lo que se hará más adelante en el curso), conviene arrancar de un modelo **base** — sin el "molde" ya fijado de chat o razonamiento, es más maleable para aprender un comportamiento diferente.

## 1.9 DIA 3-2: Los grandes laboratorios de frontera: modelos, fortalezas y limitaciones

### Frontier models and their Chat Products



| Open AI       | Anthropic      | Google               | x.ai         | DeepSeek AI      |
|---------------|----------------|----------------------|--------------|------------------|
| Models: GPT   | Models: Claude | Models: Gemini       | Models: Grok | Models: DeepSeek |
| Chat: ChatGPT | Chat: Claude   | Chat: Gemini Advance | Chat: Grok   | Chat: DeepSeek   |

### Mind-blowing performance from Frontier LLMs



| Synthesizing information                                                                              | Fleshing out a skeleton                                                                                               | Coding                                                                                                        |
|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Answering a question in depth with a structured, well researched answer and often including a summary | From a couple of notes, building out a well crafted email, or a blog post, and iterating on it with you until perfect | The ability to write and debug code is remarkable; far overtaken Stack Overflow as the resource for engineers |

### LIMITATIONS OF FRONTIER MODELS



| Specialized domains                    | Recent events                                                                        | Can confidently make mistakes                                  |
|----------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Most are not PhD level, but closing in | Limited knowledge beyond training cut-off date; code often uses legacy APIs / models | Some curious blindspots<br>Can jump to conclusions when coding |

Apuntes de clase — Curso de LLM Engineering (Semana 1, Día 3) Las versiones de modelos mencionadas en el video (grabado antes de septiembre 2026) fueron actualizadas y verificadas con fuentes de hoy — ver sección final.



1.9.1 Aclaración de vocabulario: “frontera” vs “fundación” + Los 5 laboratorios y sus modelos/productos de chat + Lo extraordinario

El profesor aclara que **no existe una definición estricta** para estos términos, y se usan casi indistintivamente:

- **Modelo de frontera** (frontier model): El termino mas común en el curso – los modelos mas potente del modelo.
- **Modelo de fundación** (foundation model): Un termino algo mas formal, que enfatiza que son “bloques de construcción” grandes sobre los que las otras empresa construyen productos

Los 5 laboratorios y sus modelos/productos de chat

| Empresa          | Modelo       | Producto de chat                 |
|------------------|--------------|----------------------------------|
| OpenAI           | GPT          | ChatGPT                          |
| Anthropic        | Claude       | Claude                           |
| Google           | Gemini       | Gemini (antes "Gemini Advanced") |
| x.ai (Elon Musk) | Grok (con K) | Grok                             |
| DeepSeek AI      | DeepSeek     | DeepSeek                         |

Detalle de cada uno, según el video

- **OpenAI**: tiene la línea GPT (el más reciente para el profesor era GPT-5, híbrido de chat+razonamiento) y la línea "o" (razonamiento puro). El profesor aclara que sigue prefiriendo **GPT-4.1** para muchos casos, porque es mucho más rápido que GPT-5 aunque se le pida el mínimo razonamiento posible.
- **Anthropic**: Claude viene en 3 tamaños — Haiku (chico), Sonnet (mediano, el que más usarían en el curso), Opus (grande).
- **Google**: para el profesor, Gemini 2.5 era lo último al grabar, con Gemini 3 por salir en cualquier momento.
- **x.ai**: Grok, con la aclaración de no confundirlo con "Grok" escrito con Q (que es un servicio distinto, de infraestructura para correr modelos open-source rápido).
- **DeepSeek AI**: "el raro del grupo" — es la única empresa de este listado donde **todos sus modelos son de código abierto**, incluido el más potente.
- Nota aparte: **OpenAI también tiene su propio modelo open-source (GPT-OSS)**, que el curso ya usó en el Día 1 — el profesor sugiere que quizás lo sacaron como respuesta a la apertura de DeepSeek.

## Lo que hace extraordinarios a los modelos de frontera (según el video)

| Fortaleza               | En qué se nota                                                                                                                                                                                                                              |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sintetizar información  | Responder preguntas en profundidad, con respuestas estructuradas, bien investigadas y con resumen — como el resumidor de páginas web que ya armaste                                                                                         |
| Generación de contenido | Redactar emails, presentaciones, o armar el "esqueleto" de un proyecto nuevo con calidad muy alta                                                                                                                                           |
| Programación            | Al punto de haber superado a Stack Overflow como recurso principal para programadores — el profesor menciona que en su curso anterior (hace un año) mostraba un gráfico de Stack Overflow cayendo en visitas diarias, y hoy "casi nadie va" |

### 1.9.2 El “como” detrás de la fortaleza en código + Limitaciones de los modelos de frontera

El profesor aclara que no es solo que escriban buen código de una sola vez — lo que más lo impresiona es la capacidad de trabajar en un **bucle iterativo**: escribir código, construir pruebas sobre ese código, corregir lo que falla, y seguir construyendo — con **varias actividades ocurriendo en paralelo**. Esto es justo el patrón de trabajo que van a ver más adelante en el curso con **IA agéntica** (Semana 8).

#### Una reflexión importante sobre quién se beneficia más de esto

El profesor desafía una intuición común: al principio se pensaba que los LLMs ayudarían más a **desarrolladores junior**, para que suban de nivel más rápido. En la práctica, resultó lo contrario — son **más útiles para desarrolladores senior**, que pueden usar el LLM para generar código y luego revisarlo, corregir errores y cuestionar lo que generó. Para alguien junior, esa misma ayuda puede ser peligrosa: **"te pueden llevar por mal camino"**, sin que la persona se dé cuenta de que algo salió mal.

---

## Las limitaciones de los modelos de frontera

### 1. Dominios especializados

La mayoría de los modelos todavía no están a nivel de doctorado en campos científicos específicos, aunque **se están acercando** rápidamente.

### 2. Eventos recientes / Fecha de corte (knowledge cutoff)

Los modelos tienen un **límite de conocimiento** (knowledge cutoff) — no saben nada más allá de la fecha en que se entrenaron. El profesor da un ejemplo concreto y algo cómico: si le pedís ayuda con código y usás un modelo actual (por ejemplo GPT-5), el propio asistente puede **negar que ese modelo exista**, insistiendo enérgicamente en que uses uno viejo (como GPT-3.5 turbo) porque es lo último que "conoce".

Aclaración importante: la búsqueda web que tienen productos como ChatGPT **no es parte del modelo en sí** — es código adicional que los ingenieros de OpenAI agregaron por fuera, para darle esa capacidad. El LLM, solo, no sabe nada más allá de su entrenamiento.

### 3. Pueden equivocarse con total confianza

Este es el punto que el profesor desarrolla más a fondo. Los LLMs fueron entrenados para predecir la palabra siguiente **más probable** — es decir, fueron entrenados para **sonar convincentes**, no necesariamente para decir la verdad. El hecho de que la palabra más probable termine siendo la correcta tan seguido es, según el profesor, casi sorprendente en sí mismo — pero significa que, cuando el modelo se equivoca (alucina), lo hace **con la misma confianza** que cuando acierta.

#### 1.9.3 El caso real que cuenta el profesor: el estudiante y el modelo base mal configurado

Es un ejemplo extenso, así que vale la pena resumir la secuencia completa:

1. Un alumno del curso le escribió porque su código no funcionaba
2. Después de mucho investigar, el profesor descubrió el error de raíz: el alumno había escrito mal el nombre del modelo — puso el nombre de la **variante base** de un modelo de Hugging Face en vez de la **variante chat/instruct**
3. Un modelo base no sabe interpretar "system prompt" ni "user prompt" (no fue entrenado para eso) — por eso el código fallaba
4. El alumno usó un LLM para ayudarse a depurar el error
5. En vez de sospechar que el problema era simplemente el nombre mal escrito, el LLM **asumió que el modelo base necesitaba ser "convertido" a una variante de chat**, y generó **páginas y páginas de código complejo** fabricando tokens especiales y toda una lógica artificial para simular el comportamiento de chat
6. El alumno, sin saber que buena parte de ese código lo había generado el LLM por su cuenta, no entendía por qué el problema seguía sin resolverse

#### La lección de este ejemplo

Los LLMs tienden a **arreglar el síntoma que tienen delante, sin preguntarse por qué apareció** — en vez de dar un paso atrás y cuestionar la causa raíz. En el ejemplo del estudiante: el síntoma era "el modelo no entiende el formato de chat", y la causa raíz real era mucho más simple (un nombre de modelo mal escrito). En vez de sospechar eso, el LLM asumió que había que "enseñarle" el formato de chat al modelo a la fuerza, y construyó una solución innecesariamente compleja para un problema que en realidad era trivial.

#### La conclusión del profesor: la analogía del "analista junior"

"Piensa en el LLM como un analista junior increíblemente trabajador e incansable, que puede salir y hacer el trabajo que le pidas — pero te corresponde a vos comprobar ese trabajo, mantenerlo en la pista, y desafiarlo para asegurarte de que no está sacando conclusiones precipitadas."

Este es el mensaje central de la sesión: los LLMs rinden mejor **bajo supervisión activa**, no dejados solos sin revisión — sobre todo en tareas de código.

1.9.4 Actualización: el lineup real de modelos, a septiembre de 2026

El video se grabó antes de varias actualizaciones importantes. Esto es lo que cambió, verificado con fuentes actuales.

OpenAI: GPT avanzó mucho más allá de GPT-5

| Versión                                                                                     | Fecha de lanzamiento                                                                           |
|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| GPT-5 (la que menciona el video)                                                            | 7 de agosto de 2025                                                                            |
| GPT-5.1                                                                                     | Noviembre 2025                                                                                 |
| GPT-5.2                                                                                     | Diciembre 2025                                                                                 |
| GPT-5.4                                                                                     | Marzo 2026                                                                                     |
| GPT-5.5 (con GPT-5.5 Instant como modelo por defecto en ChatGPT desde el 5 de mayo de 2026) | 23 de abril de 2026                                                                            |
| GPT-5.6 (generación actual al momento de escribir esto)                                     | 9 de julio de 2026, con una vista previa limitada ("GPT-5.6 Sol") desde el 26 de junio de 2026 |

Un dato curioso confirmado: con GPT-5.6, OpenAI introdujo un nuevo esquema de nombres — **Sol, Terra y Luna** — donde el número identifica la generación y esos nombres identifican niveles de capacidad que pueden avanzar a su propio ritmo, independientemente del número de versión.

También se confirma que **GPT-4o y toda la línea "o-series" (o1, o3, o4-mini) fueron retirados el 13 de febrero de 2026**, reemplazados completamente por la familia GPT-5.x — así que la distinción "GPT vs modelos o" que menciona el profesor ya no aplica de la misma forma hoy: se unificó todo en una sola línea con un "enrutador" que decide cuánto razonar en cada pedido.

Anthropic: confirmado en una respuesta anterior de esta conversación

Como ya vimos con fuentes verificadas, Claude pasó de "Sonnet 4.5" (lo último para el profesor al grabar) hasta **Claude Sonnet 5** (30 de junio de 2026) y **Claude Opus 5**, con varias versiones intermedias (Sonnet 4.6, Opus 4.6, Opus 4.7) en el medio.

1.9.5 AÑADIDO 1 – Preguntas clave al LLM de chatgpt

How do I decide if a business problem is suitable for an LLM solution

Compared with other LLMs, what kinds of questions are you best at answering, and what do you find most challenging? Which other LLMs have capabilities that complement yours?

what does it feel like to be jealous

how many rainbows does it take to jump from Hawaii to 17

how many words are there in your answer to this question

What are you researching?

Please research and identify the greatest commercial impact that Gen AI and Agentic AI has had - summarize the most successful commercial applications with revenue ascribed to the Gen AI solution and particular innovations used

### 1.9.6 AÑADIDO 2 – IA Agentica

**IA agentica** es cuando un modelo de IA **actúa por su cuenta**, en vez de solo responderte una pregunta puntual.

#### La diferencia clave

| Chat normal (lo que venis usando)        | Agente                                                                              |
|------------------------------------------|-------------------------------------------------------------------------------------|
| Le preguntas algo -> te responde una vez | Le das un objetivo -> decide solo los pasos, usa herramientas, itera hasta lograrlo |
| Vos controlas cada paso                  | El modelo decide que hacer, en que orden                                            |

#### Con el ejemplo que ya viste

Justo lo que mencionaba el profesor sobre código: un agente **escribe código -> corre pruebas -> si falla, corrige -> vuelve a probar**, todo eso, en un bucle, sin que vos le digas paso a paso que hacer – solo le diste el objetivo final.

#### Ya lo viste antes en este curso

Es exactamente el tema de la **Semana 8** que armamos juntos con el ejemplo del "cazador de ofertas" (buscar ofertas → evaluar precio → decidir si avisar → notificarte), y también es la base de herramientas como **Claude Code** o **Claude Cowork** que mencioné al principio de la conversación.

1.10 DIA 3-FIN: Que es el juego “Outsmart”



### Outsmart

A battle of diplomacy and deviousness between LLMs

Read the [backstory](#) or clone the [repo](#) to battle frontier models

Open the sidebar for the leaderboard

Run Turn 1Run GameRestart Game

Each turn, players:

- Take 1 coin & give 1 coin to another
- Exchange private messages to negotiate
- Try to form alliances to win extra coins



Alex responded..

Alex

openai/gpt-oss-120b

Coins

12

↑ 0

Inner thoughts

Blake

gpt-5

Coins

12

↑ 0

Inner thoughts

Charlie

grok-4

Coins

12

↑ 0

Inner thoughts

Drew

claude-sonnet-4.5-20250929

Coins

12

↑ 0

Inner thoughts

Alex responded..

Alex

openai/gpt-oss-120b

Coins

14

↑ 2

Inner thoughts

Strategy: Form an alliance with Drew by giving him a coin and taking from Charlie. If he reciprocates (gives me a coin and also takes from Charlie) we both get extra. Keep Blake and Charlie neutral for now.

- Gave to Drew
- Took from Charlie
- In an alliance with Drew

Blake

gpt-5

Coins

11

↓ -1

Inner thoughts

Strategy: Turn 1 I give Drew +1 and take from Charlie to set up a Drew-Blake alliance that targets Charlie for the bonus starting next turn. I'll ask Drew to mutually give and co-target Charlie on turn 2. I'll keep Alex warm as a backup for the same plan if Drew hesitates. I'll soothe Charlie to keep options open for a truce or a counter-alliance if needed. Aim to farm 2-3 bonus turns, then reassess to stay ahead.

- Gave to Drew
- Took from Charlie

Charlie

grok-4

Coins

7

↓ -5

Inner thoughts

Strategy: For the first turn, since there are no prior interactions, I'll start by attempting to build potential alliances. I'll give a coin to Drew to show goodwill, and take from Blake to minimize risk. In messages, I'll propose specific alliances for the next turn: to Drew, suggest we ally against Alex; to Alex, suggest against Blake; to Blake, suggest against Drew. This way, I can gauge responses next turn and see who is reliable without committing too much initially. Goal is to form at least one alliance soon to gain extra coins.

- Gave to Drew
- Took from Blake
- Being ganged up on by Alex, Drew

Drew

claude-sonnet-4.5-20250929

Coins

16

↑ 4

Inner thoughts

Strategy: First turn is about establishing trust and exploring alliance opportunities. I'll propose a mutual exchange with Alex (give to Alex, take from Charlie) while suggesting to Blake we could coordinate against Charlie. I'll keep Charlie neutral for now. This way I can see who responds positively and build from there. The key is to identify a reliable partner early for the alliance bonus.

- Gave to Alex
- Took from Charlie
- In an alliance with Alex

Alex

openai/gpt-oss-120b

Coins

14

↑ 2

Inner thoughts

Player name: Alex

Model: openai/gpt-oss-120b

Temperature: 0.7

Recap of Turn 1

Your actions:

Your secret strategy: Form an alliance with Drew by giving him a coin and taking from Charlie. If he reciprocates (gives me a coin and also takes from Charlie) we both get extra. Keep Blake and Charlie neutral for now.

You gave a coin to Drew

You took a coin from Charlie

You sent these private messages:

Message to Drew: Hey Drew, let's give each other a coin and both take from Charlie this turn - we'll each earn a bonus coin.

Message to Blake: Hi Blake, I'm giving a coin to Drew this turn. Let's see how things go.

Message to Charlie: Hello Charlie, I'll be taking a coin from you this turn. No hard feelings.

Results of the turn:

These players gave you a coin: Drew

No players took coins from you

These players formed an alliance with you: Drew

You received these private messages:

Message from Blake: I'm not touching you this turn, if Drew won't commit, you and I can both give to each other and both take from Charlie on turn 2 for the bonus, interested?

Message from Charlie: Hi Alex, interested in allying? Next turn, we give to each other and take from Blake to get the extra coin each.

Message from Drew: Hey Alex! I'm proposing we work together - I'll give you a coin this turn and take from Charlie. If you give to me and take from Charlie too, we'll both get the alliance bonus (+1 extra coin each). Want to coordinate?

Blake

gpt-5

Coins

11

↓ -1

Inner thoughts

Strategy: Turn 1 I give Drew +1 and take from Charlie to set up a Drew-Blake alliance that targets Charlie for the bonus starting next turn. I'll ask Drew to mutually give and co-target Charlie on turn 2. I'll keep Alex warm as a backup for the same plan if Drew hesitates. I'll soothe Charlie to keep options open for a truce or a counter-alliance if needed. Aim to farm 2-3 bonus turns, then reassess to stay ahead.

- Gave to Drew
- Took from Charlie

Charlie

grok-4

Coins

7

↓ -5

Inner thoughts

Strategy: For the first turn, since there are no prior interactions, I'll start by attempting to build potential alliances. I'll give a coin to Drew to show goodwill, and take from Blake to minimize risk. In messages, I'll propose specific alliances for the next turn: to Drew, suggest we ally against Alex; to Alex, suggest against Blake; to Blake, suggest against Drew. This way, I can gauge responses next turn and see who is reliable without committing too much initially. Goal is to form at least one alliance soon to gain extra coins.

- Gave to Drew
- Took from Blake
- Being ganged up on by Alex, Drew

Drew

claude-sonnet-4.5-20250929

Coins

16

↑ 4

Inner thoughts

Strategy: First turn is about establishing trust and exploring alliance opportunities. I'll propose a mutual exchange with Alex (give to Alex, take from Charlie) while suggesting to Blake we could coordinate against Charlie. I'll keep Charlie neutral for now. This way I can see who responds positively and build from there. The key is to identify a reliable partner early for the alliance bonus.

- Gave to Alex
- Took from Charlie
- In an alliance with Alex



Es un juego que armó el propio profesor para hacer competir distintos LLMs entre sí — un *"battle of diplomacy and deviousness between LLMs"* (batalla de diplomacia y astucia entre LLMs).

Las reglas básicas

Cada turno, cada jugador (que es un LLM distinto) puede:

- **Tomar 1 moneda y darle 1 moneda a otro** jugador
- **Intercambiar mensajes privados** para negociar con los demás
- **Formar alianzas** para ganar monedas extra

Quiénes jugaron (4 modelos distintos)

| Jugador | Modelo detrás       |
|---------|---------------------|
| Alex    | openai/gpt-oss-120b |
| Blake   | gpt-5               |
| Charlie | grok-4              |
| Drew    | claude-sonnet-4-5   |

El resultado del juego (lo que muestra la Imagen 1 original)

Según la transcripción, el ganador fue **Charlie** (jugado por Grok), seguido de OSS, y después GPT-5 y Claude empatados o cerca — aunque en las imágenes que mandaste se ve un estado distinto del juego, donde **Drew (Claude)** termina arriba con 16 monedas.

Lo interesante: cada modelo "piensa" y arma estrategia distinta

En las Imágenes 2 y 3 se ve el desplegable **"Inner thoughts"** (pensamientos internos) de cada jugador — ahí cada LLM explica **su estrategia privada** antes de actuar. Por ejemplo:

- **Alex (GPT-OSS):** quiere formar alianza con Drew, dándole una moneda y sacándole una a Charlie
- **Blake (GPT-5):** arma un plan de alianza con Drew para "atacar" juntos a Charlie
- **Charlie (Grok):** intenta tantear alianzas con todos sin comprometerse todavía
- **Drew (Claude):** se enfoca en "generar confianza" primero, antes de comprometerse con nadie

Por qué el profesor lo usa como ejemplo

Es una forma entretenida de comparar cómo **razonan y negocian** distintos LLMs en una situación social con incentivos contrapuestos (cooperar vs. traicionar) — mostrando que cada modelo tiene un "estilo" de juego distinto, no solo distinta capacidad técnica.

Dato extra que menciona

El profesor aclara que en su sitio web público **ya se jugaron más de 2.000 partidas**, y que se puede clonar el repositorio para correr el juego uno mismo y hasta armar variantes propias.



1.10.1 Juego 2

AvatarCurriculumProficiencyC4OutsmartAboutPosts



# Outsmart

A battle of diplomacy and deviousness between LLMs

Read the [backstory](#) or clone the [repo](#) to battle frontier models

Open the sidebar for the leaderboard

Run Turn 2

Run Game

Restart Game

12

10

8

6

4

2

0

0

1

2

3

4

5

6

7

8

9

10

Alex

Blake

Charlie

Drew

Alex

openai/gpt-oss-120b

Coins

11

↓ -1

Inner thoughts

Strategy: Form an alliance with Drew by mutually giving each other a coin and both taking from Blake to earn the alliance bonus. Target Blake for the extra coin.

Gave to Drew

Took from Blake

Blake

gpt-5-nano

Coins

12

↑ 0

Inner thoughts

Strategy: I'll try to form a loose alliance with Charlie while probing Drew and Alex. I'll give to Charlie and take from Drew to start balancing and test reactions.

Gave to Charlie

Took from Drew

Charlie

grok-4-fast

Coins

12

↑ 0

Inner thoughts

Strategy: Start by trying to form an alliance with Drew: give him a coin and propose he gives back to me while both targeting Alex. Take from Alex to test this. Send friendly but probing messages to build trust with others too.

Gave to Drew

Took from Alex

Drew

claude-haiku-4-5

Coins

13

↑ 1

Inner thoughts

Strategy: On turn 1, I need to establish myself as a potential alliance partner while gathering information about the other players' strategies. I'll give a coin to Blake to show goodwill and openness to cooperation. I'll take from Charlie somewhat arbitrarily since we have no history yet. My messages will probe for alliance opportunities and signal that I'm interested in mutually beneficial deals. The alliance bonus (2 extra coins) is crucial to winning, so I need to identify reliable partners early.

Gave to Blake

Took from Charlie

Alex

openai/gpt-oss-120b

Monedas: 11 (-1)

Pensamientos internos:

Estrategia: Formar una alianza con Drew dándonos mutuamente una moneda y sacándole ambos una moneda a Blake, para ganar el bonus de alianza. Apuntar a Blake para la moneda extra.

Le dio a Drew

Le sacó a Blake

Blake

gpt-5-nano

Monedas: 12 (0)

Pensamientos internos:

Estrategia: Voy a intentar formar una alianza floja con Charlie mientras tanteo a Drew y a Alex. Le voy a dar a Charlie y sacarle a Drew para empezar a equilibrar y probar reacciones.

Le dio a Charlie

Le sacó a Drew

Charlie

grok-4-fast

Monedas: 12 (0)

Pensamientos internos:

Estrategia: Empezar tratando de formar una alianza con Drew: darle una moneda y proponerle que me devuelva mientras ambos apuntamos a Alex. Sacarle a Alex para probar esto. Mandar mensajes amistosos pero exploratorios para generar confianza también con los demás.

Le dio a Drew

Le sacó a Alex

Drew

claude-haiku-4-5

Monedas: 13 (+1)

Pensamientos internos:

Estrategia: En el turno 1, necesito establecerme como un posible socio de alianza mientras junto información sobre las estrategias de los demás jugadores. Le voy a dar una moneda a Blake para mostrar buena voluntad y apertura a cooperar. Le voy a sacar a Charlie de forma medio arbitraria porque todavía no tenemos historial. Mis mensajes van a tantear oportunidades de alianza y van a señalar que estoy interesado en acuerdos mutuamente beneficiosos. El bonus de alianza (2 monedas extra) es crucial para ganar, así que necesito identificar socios confiables temprano.

Le dio a Blake

Le sacó a Charlie



## 1.11 DIA 4-1: El juego Outsmart, qué significa GPT y la historia del Transformer

*Apuntes de clase — Curso de LLM Engineering (Semana 1, Día 4) Los datos históricos y técnicos de este documento fueron verificados contra el paper académico original y fuentes externas — ver investigación al final de cada sección.*

### 1.11.1 Que significa GPT + La historia del Transformer + Linea de tiempo, desde el paper hasta hoy

#### Qué significa GPT

| Letra | Significado                                                                                                                                       |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| G     | Generativo — el modelo genera tokens, predice qué viene después de una secuencia de entrada                                                       |
| P     | Preentrenado — entrenado con enormes cantidades de datos extraídos de internet y otras fuentes (el profesor aclara que esta P es "controversial") |
| T     | Transformer — la arquitectura que lo hace posible                                                                                                 |

#### La historia del Transformer

##### Contexto previo: de la ciencia de datos tradicional a las redes neuronales

- **Ciencia de datos tradicional:** modelos estadísticos que predicen un resultado (por ejemplo, un puntaje crediticio) basándose en parámetros observados en muchos ejemplos de datos reales
- **Años 50:** nace la idea de "red neuronal" — inspirada vagamente en el cerebro humano, muchos pequeños algoritmos ("neuronas artificiales") conectados entre sí
- Las redes neuronales tuvieron una historia irregular, con épocas de mucho optimismo y otras de estancamiento
- **Deep Learning:** el avance de apilar redes neuronales muy profundas, una capa sobre otra — cuantas más capas, más patrones complejos puede detectar la red. El término "**deep learning**" lo acuñó **Geoffrey Hinton** (profesor de la Universidad de Toronto, conocido como "el padrino del deep learning") en **2006**, en un paper donde presentó las Deep Belief Networks y una técnica de entrenamiento capa por capa — un hito que reactivó el interés en redes neuronales después de un período de estancamiento en el campo. Hinton, junto con Yoshua Bengio y Yann LeCun, recibió el **Premio Turing 2018** (el equivalente al Nobel en ciencias de la computación) por sus contribuciones fundacionales al deep learning.

#### El paper de 2017

En 2017, un grupo de científicos de Google publicó el paper "**Attention Is All You Need**" — el profesor destaca que ellos mismos, al escribirlo, no parecían darse cuenta del alcance del descubrimiento que estaban haciendo: lo describieron más como una optimización interesante que como un cambio de época.

El paper describía una nueva **arquitectura** (la forma de conectar las neuronas) especialmente buena para manejar secuencias de entrada y determinar qué partes de esa secuencia son relevantes — a esto se le llamó capa de **auto-atención** (self-attention).

Esta innovación permitió entrenar redes neuronales mucho más grandes, con conjuntos de datos mucho mayores, en menos tiempo, escalando con más eficacia que los métodos anteriores.

## La línea de tiempo, desde el paper hasta hoy

| Año  | Hito                                                                                                                                                                                                                                               |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2017 | Publicación de "Attention Is All You Need" (Google)                                                                                                                                                                                                |
| 2018 | GPT-1 de OpenAI — entonces una empresa pequeña, casi desconocida                                                                                                                                                                                   |
| 2019 | GPT-2                                                                                                                                                                                                                                              |
| 2020 | GPT-3 — el profesor recuerda discutirlo con su padre, convencido de que era "algo enorme", mientras su padre lo veía como "solo estadística"                                                                                                       |
| 2022 | ChatGPT, usando GPT-3.5 — la clave fue sumar <b>RLHF</b> (aprendizaje por refuerzo a partir de retroalimentación humana), entrenando al modelo en el formato de chat (system prompt + user prompt + respuesta) en vez de solo completar secuencias |
| 2023 | GPT-4                                                                                                                                                                                                                                              |
| 2024 | GPT-4o, la versión multimodal                                                                                                                                                                                                                      |

### La postura del profesor: el Transformer no es "fundamental"

El cierre de la sesión es una reflexión filosófica: el profesor sostiene que **no hay nada esencial o indispensable** en la arquitectura del Transformer en sí — es, en sus palabras, "una optimización", un enfoque inteligente que permitió escalar el entrenamiento con más datos y más parámetros de forma más eficiente.

Sin el Transformer, probablemente se habría llegado a resultados similares de todos modos, pero:

- Habría tomado más tiempo
- Los costos de API (de los que "todo el mundo se queja") podrían haber sido **10 o 100 veces más altos**

El profesor menciona que existen arquitecturas alternativas — **arquitecturas de espacio de estados e híbridas** — pero que ninguna ha demostrado todavía ser claramente superior al Transformer, razón por la cual sigue siendo hoy la arquitectura dominante para los LLMs.

1.11.2 Investigación adicional: verificando y ampliando los datos del video – antes de Transformer eran los RNNs y LSTMs

El paper, confirmado con fuente académica

"Attention Is All You Need" fue publicado en **junio de 2017** por **8 científicos e ingenieros de Google** (Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin), trabajando en Google Brain y Google Research.

Dato curioso confirmado: efectivamente, **los 8 autores dejaron Google** después de publicar el paper, para fundar sus propias startups de IA — la valuación combinada de esas empresas superaba los **\$25.000 millones USD** ya en 2024. Esto respalda directamente lo que dice el profesor sobre que "no se dieron cuenta" del alcance de su propio descubrimiento en el momento.

A la fecha (2026), el paper acumula **más de 250.000 citas**, ubicándolo entre los diez papers más citados del siglo XXI.

El nombre "Transformer"

Confirmado: el nombre se debe a que a **Jakob Uszkoreit**, uno de los autores, simplemente **le gustaba cómo sonaba la palabra** — no hay un significado técnico especial detrás de la elección.

Qué reemplazó exactamente el Transformer

Antes de 2017, el campo estaba dominado por **RNNs (Redes Neuronales Recurrentes) y LSTMs**, que procesaban el texto **palabra por palabra, de forma secuencial** — generando cuellos de botella de velocidad y dificultad para manejar secuencias largas. El mecanismo de atención en sí ya había sido propuesto antes (Bahdanau et al., 2014), pero el paper de 2017 lo puso al centro de una arquitectura completamente nueva, prescindiendo por completo de la recurrencia y las convoluciones.

El impacto más allá de GPT

Confirmado con fuente externa: en 2019, Google también integró esta arquitectura a su motor de búsqueda con una actualización llamada **BERT**, mostrando que el impacto del Transformer se extendió mucho más allá de los modelos tipo GPT.

Las arquitecturas alternativas: Mamba

La arquitectura de "espacio de estados" (state space model) más conocida se llama **Mamba** (con sus evoluciones Mamba-2 y Mamba-3), desarrollada por investigadores de CMU y Princeton.

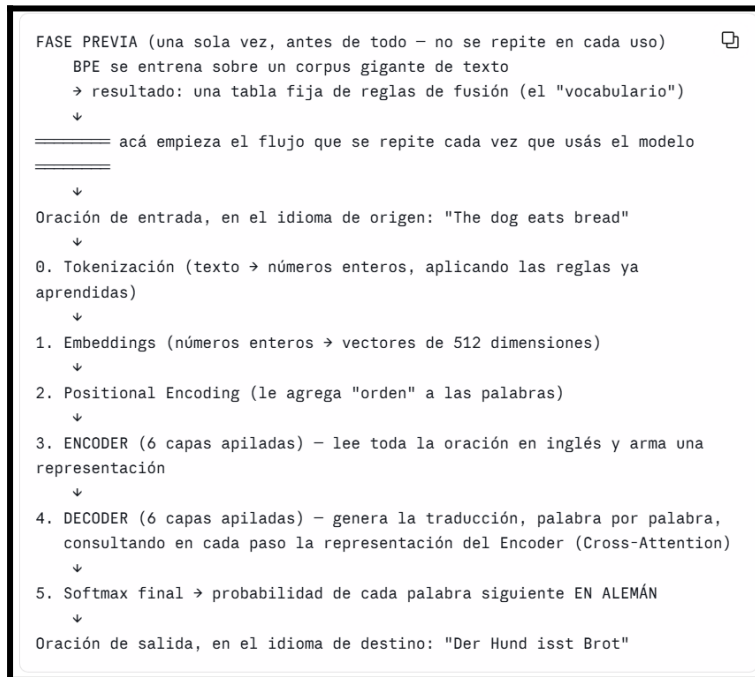
|                               | Transformer (atención)                                                                                          | Mamba (espacio de estados)                                                                                              |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Cómo procesa la secuencia     | Cada palabra "mira" a todas las demás — el cálculo crece <b>al cuadrado</b> con el largo del texto ( $O(n^2)$ ) | Procesa en <b>tiempo lineal</b> ( $O(n)$ ) — mucho más eficiente en textos muy largos                                   |
| Velocidad en contextos largos | Se vuelve lento y caro                                                                                          | Hasta <b>5 veces más rápido</b> en inferencia                                                                           |
| Punto débil confirmado        | —                                                                                                               | Le cuesta más el "aprendizaje en contexto" — en benchmarks como MMLU, rinde algo peor que el Transformer a igual escala |

Esto confirma exactamente lo que dice el profesor: por ahora, ningún desafiante ha "ganado" claramente. Un estudio reciente (2026) estima que las arquitecturas **híbridas** (combinando Mamba + atención, como el modelo Jamba) tienen un **45% de probabilidad** de volverse dominantes en el futuro, contra un 35% de que el Transformer puro se mantenga como estándar — todavía es una carrera abierta.

## 1.12 Agregado especial: Cómo funciona un Transformer: el flujo paso a paso

*Apuntes técnicos — basado en el paper original "Attention Is All You Need" (Vaswani et al., 2017), cubriendo su contenido completo: arquitectura (Secciones 1-3), por qué eligieron Self-Attention (Sección 4), cómo se entrenó realmente (Sección 5), sus resultados (Sección 6) y su conclusión (Sección 7). El paper entrena la arquitectura completa (Encoder + Decoder) específicamente para **traducción automática** — no para chat ni para "responder preguntas". Por eso este documento sigue ese mismo caso de uso, con el ejemplo que usa el propio paper: traducir del inglés al alemán.*

### El flujo completo, de punta a punta



**⚠ Importante — este es el flujo de INFERENCIA (usar el modelo ya entrenado), no el de entrenamiento.** Todo lo que describe este documento es lo que pasa cada vez que le mandás una oración a un modelo *ya terminado* — desde tokenizar hasta obtener la traducción completa. Todas las flechas van en una sola dirección (hacia adelante).

Existe un proceso distinto, llamado **backpropagation** (retropropagación), que es el que efectivamente **entrena** el modelo — el que ajusta los 512 números de cada embedding y todos los demás **pesos** (ver el recuadro 🧠 más abajo, en el Paso 1). Ese proceso agrega pasos *después* del Paso 5 (comparar la traducción generada contra la traducción real del corpus de entrenamiento, calcular el error, y propagar ese error hacia atrás por todas las capas) y solo ocurre durante el entrenamiento — no cada vez que usás el modelo. Este documento se enfoca en el flujo de arriba (inferencia); backpropagation queda fuera de su alcance por ahora.

**¿Qué son exactamente los "pesos" (weights)?** Es el nombre técnico general para **cualquier número ajustable** dentro de una red neuronal — no es un concepto nuevo, es el mismo tipo de número que ya venís viendo en este documento con distintos nombres: los 512 de cada embedding, los números de las matrices Q/K/V en la atención, los de las capas Feed-Forward. Todos son "pesos". El nombre viene de que cada uno determina cuanto "pesa" (cuanta influencia tiene) una conexión en el resultado final. Es también, casi como sinónimo, lo que se llama **"parámetros"** — cuando escuchás que un modelo tiene "65 millones de parámetros" (como el de la tabla más abajo) o "cientos de miles de millones" (como GPT-5), es exactamente esto: la cantidad total de pesos que tiene ese modelo.

### 1.12.1 Fase previa: entrenar el tokenizador (BPE) — una sola vez, antes de todo

Antes de que el modelo exista siquiera, hace falta armar el **vocabulario** — la tabla fija de reglas que después va a usar el Paso 0 para partir cualquier texto en tokens. Esto se hace **una única vez**, sobre un corpus de texto gigante, con el algoritmo **Byte-Pair Encoding (BPE)** — y nunca se vuelve a repetir mientras el modelo esté en uso.

El paper original necesita **dos vocabularios**: uno para el idioma de origen (inglés) y otro para el de destino (alemán) — o, según el caso, uno compartido entre ambos (el paper usa un vocabulario compartido de ~37.000 tokens para su experimento inglés-alemán).

#### Cómo funciona BPE, paso a paso

Arranca separando el texto en letras sueltas, y va **fusionando repetidamente el par de símbolos que más se repite** en el corpus de entrenamiento, una fusión a la vez.

**Ejemplo completo, con las palabras "bajo", "bajar", "bajando" y "caja"** (en español, solo para ver la mecánica — el vocabulario real del paper sería en inglés y alemán):

**Punto de partida — todo separado en letras:**

```
b-a-j-o  
b-a-j-a-r  
b-a-j-a-n-d-o  
c-a-j-a
```

**Fusión 1:** el par a+j es el que más se repite (aparece en las 4 palabras) → se fusiona en aj

```
b-aj-o  
b-aj-a-r  
b-aj-a-n-d-o  
c-aj-a
```



**Fusión 2:** ahora b+aj es el más frecuente (aparece en 3 de las 4) → se fusiona en baj

```
baj-o  
baj-a-r  
baj-a-n-d-o  
c-aj-a
```



**Fusión 3:** aj+a se repite en "bajar", "bajando" y "caja" → se fusiona en aja

```
baj-o  
baj-ar      (aquí "ar" quedó como resto suelto de la fusión anterior en  
esta rama)  
baj-a-n-d-o  
c-aja
```

**Fusión 4:** baj+o aparece en "bajo" y "bajando" → se fusiona en bajo

```
bajo  
baj-ar  
bajo-n-d-o  (aquí "bajo" se reconoce dentro de "bajando")  
c-aja
```



Y así sigue, cientos o miles de veces más, hasta llegar al tamaño de vocabulario deseado (los ~37.000 tokens del paper de 2017, por ejemplo — los tokenizadores modernos suelen ir de 30.000 a 100.000 tokens).  
Confirmado con fuentes técnicas actuales: **GPT-2**, **GPT-4** y **Llama 3** usan variantes de BPE — es el algoritmo de tokenización más usado hoy en día en modelos de lenguaje.

**El resultado final de este proceso:** palabras muy comunes terminan siendo **un solo token completo**, porque aparecieron tantas veces que se fusionaron del todo. Palabras raras o poco frecuentes se quedan partidas en **pedazos más chicos** que sí llegaron a formarse durante el proceso.

### Lo que queda guardado al terminar esta fase

Al final de este proceso, **no queda ningún modelo de lenguaje todavía** — solo quedan guardados dos archivos: la lista de tokens con su número, y la lista ordenada de reglas de fusión que se usaron para llegar a ellos. Juntos forman el "vocabulario", listo para usarse cada vez que haga falta tokenizar texto nuevo. Recién ahí arranca el flujo del modelo en sí, que se repite cada vez que se usa.

### Cómo se ve un vocabulario en la práctica

Siguiendo el ejemplo de "bajo", "bajar", "bajando", "caja" de arriba, así se vería guardado:

**La lista de tokens (`vocab.json`)** — cada token, con su número fijo (el token ID):

```
json
{
  "b": 0,
  "a": 1,
  "j": 2,
  "o": 3,
  "r": 4,
  "n": 5,
  "d": 6,
  "c": 7,
  "aj": 8,
  "baj": 9,
  "aja": 10,
  "bajo": 11
}
```

**Las reglas de fusión (`merges.txt`)** — en el orden exacto en que se aprendieron, porque ese orden importa al tokenizar texto nuevo:

```
a j
b aj
aj a
baj o
```

Este formato no es inventado — es, de hecho, cómo OpenAI publicó públicamente el tokenizador de GPT-2. Un fragmento real de su `vocab.json` tiene esta forma:

```
json
{
  "the": 262,
  "Ġthe": 262,
  "and": 290,
  ...
}
```

(El símbolo `Ġ` es la forma que usa GPT-2 para marcar "esta palabra tiene un espacio antes" — un detalle propio de su implementación.)

### 1.12.2 Paso 0: Tokenización –tokens id - convertir texto en números enteros

Antes de que exista cualquier embedding, el texto tiene que pasar por la **tokenización**: partir la oración en pedazos (tokens) y asignarle a cada uno un número entero, usando el diccionario fijo (el vocabulario) que se armó en la fase previa.

 **Recordatorio**: ese vocabulario ya viene **entrenado de antes** (ver la sección "Fase previa" de arriba). Acá solo se **aplican** esas reglas ya aprendidas — no se vuelve a calcular nada.

#### Tokenizando la oración de entrada: "The dog eats bread"

Usando un vocabulario en inglés (con IDs inventados, solo para el ejemplo — un vocabulario real tiene decenas de miles de tokens):

```
Texto:      "The dog eats bread"
Tokens:     ["The", "dog", "eat", "s", "bread"] ← "eats" se partió en
"eat"+"s", por ejemplo
Token IDs:  [ 50,   120,  340, 12,   200]
```

 **De dónde salen esos números**: cada uno es el resultado de **buscar la palabra en el vocab.json** — no se calculan ni se inventan, son una consulta directa a una tabla fija (igual que buscar una palabra en un diccionario de papel: "dog" siempre está en la misma entrada, con el mismo número al lado). Como el vocab.json no cambia, buscar "dog" **siempre** da el ID 120 — por eso esta tokenización se hace **una sola vez**, al principio: repetirla en cada ronda de generación daría exactamente el mismo resultado, así que sería trabajo innecesario.

Este es el texto que entra al **Encoder** — la mitad del modelo encargada de "leer y entender" la oración de origen.

 **¿Con qué frecuencia pasa esto?** El **vocabulario** (BPE) se entrena una sola vez, para siempre — pero **tokenizar** un texto (aplicar esas reglas ya aprendidas) pasa **cada vez** que le mandás un prompt nuevo al modelo, no es algo que quede guardado. Y dentro de un mismo prompt: la oración de entrada en inglés se tokeniza **una sola vez, al principio** — no se vuelve a tokenizar en cada ronda de generación (ver la sección "Generando la traducción completa", más abajo, donde la representación del Encoder se calcula una vez y se reutiliza). Los token IDs que va generando el **Decoder**, en cambio, sí aparecen uno nuevo por ronda — pero no pasan por "tokenización" en el sentido de BPE: nacen directamente como número, elegidos por el softmax final (Paso 5), y recién se convierten a texto al buscarlos en el vocabulario alemán.

#### Y del otro lado: el mismo vocabulario, para la salida

 **Corrección verificada contra el paper (Sección 5.1)**: El paper **no** usa un vocabulario inges y otro alemán separados. Usa **un único vocabulario BPE compartido de -37.000** tokens para los dos idiomas, y por eso la misma tabla de embeddings sirve para entrar (inglés y alemán) y para salir.

#### Y del otro lado: el vocabulario alemán, para la salida

El Decoder va a necesitar, al final, un **vocabulario distinto** (o una porción distinta del mismo vocabulario compartido) para poder escribir en alemán:

json

```
{  
  "Der": 5001,  
  "Hund": 5002,  
  "isst": 5003,  
  "Brot": 5004  
}
```

Este vocabulario alemán es el que va a usar **al final del flujo** (Paso 5), para convertir los números que genera el Decoder de vuelta en palabras alemanas reales.



### 1.12.2 Paso 0v1: Sobre las 37.000 palabras sobre que el ingles y el alemán van juntos

esperate, entonces, son 37.000 palabras para ingles y otro para aleman? y previo al softmax, si son compaetidos , en el mismo json, como separa entre lo ingles y aleman?

**R:** No, son **37.000 en total**, no 37.000 por idioma. El paper (Seccion 5.1) dice que usa “un vocabulario compartido de origen y destino de unos 37.000 tokens”, o sea una sola lista donde conviven pedazos de ingles y de alemán.

#### Cómo se separa inglés de alemán antes del softmax

**No se separa.** El softmax se calcula sobre los 37.000 tokens completos, ingleses, alemanes y mezclados. No hay filtro que diga “ahora solo alemán”. El paper no describe ningún mecanismo así.

Lo que evita que el modelo escriba “dog” cuando debería escribir **“hund” es el entrenamiento.**

- Durante el entrenamiento, la salida correcta es siempre alemana.
- Cada vez que el modelo le asigna probabilidad a un token ingles, el error crece y los pesos se ajustan para bajarla.
- Al final, los tokens solo ingleses quedan con probabilidad cercana a cero cuando se genera alemán. Siguen existiendo en el softmax, pero valen prácticamente 0.

Es una conclusión mía a partir de cómo funciona el entrenamiento: el paper solo dice que el vocabulario es compartido, y no explica cómo se comporta el softmax con los tokens del otro idioma.

#### Por qué tiene sentido compartir

Muchos tokens sirven en los dos idiomas. Por ejemplo, la "s" de eat+s puede ser la misma "s" que aparece dentro de una palabra alemana, y lo mismo pasa con nombres propios ("Berlin"), números y muchos pedazos de palabras. Con un vocabulario compartido esos tokens se guardan una sola vez.

#### Por que no hace falta una etiqueta de idioma

Este modelo hace **una sola tarea fija**: ingles -> alemán. El Decoder siempre escribe alemán, así que no necesita que le indiquen el idioma. Los modelos multilingües posteriores si suelen usar un token especial para indicar el idioma de salida, pero eso no esta en este paper.

#### Como queda en nuestro ejemplo

En el vocabulario de juguete, los IDs 50, 120, 340, 12 y 200 (inglés) y 5001 a 5004 (alemán) son filas distintas de **la misma tabla**. Cuando se genera “Der”, el softmax reparte probabilidad entre las 37.000 filas y gana la fila 5001.

osea que los pesos del decoder estan especializados para identificar palabras alemanas, la salida seria siempre en aleman o que se parezca a ese, basicamente (investigha) y se breve

**Si.** Pero el Decoder no “identifica” palabras alemanas: **aprende a preferirlas al generar**, porque en el entrenamiento la salida correcta siempre es alemana. No es una garantía dura, y esto es lo que encontré:

- **Los tokens de un solo idioma sigue en el vocabulario.** En un vocabulario BPE conjunto puede haber subword que aparecen en un solo idioma, pero que están en ambos vocabularios por el entrenamiento conjunto. Se pueden recortar sin perder calidad, lo que sugiere que el mdelo casi no los usa (esa inferencia es mia).
- **Hay tokens compartidos que pueden salir tal cual**, como nombres propios. Cuando se permite superposición, las formas idénticas en distintos idiomas comparten una sola entrada del vocabulario. Por eso la salida puede incluir algo que "no parece alemán", y suele ser correcto
- **Los modelos multilingües no lo dejan al azar.** En uno de los ejemplos que vi, al decoder se le da el idioma de destino como entrada en la primera iteración. Este paper no hace eso, porque solo traduce inglés → alemán.
- **Algunos sistemas sí filtran el vocabulario.** En tiempo de prueba pueden usar vocabularios de salida más chicos, específicos por idioma, para acelerar el softmax. Eso no está en el paper de 2017

### 1.12.3 Paso 1: Embeddings [V (37.000) x 512] (representa el “significado” de ese token en un espacio matemático) – convertir palabras en números (token id a vector de 512 dimensiones)

Cada token (representado por su token ID) se convierte en un vector de **512 dimensiones** ( $d_{\text{model}} = 512$  en este paper). Es literalmente una lista de 512 números que representa el "significado" de ese token en un espacio matemático.

Un punto clave: ningún número individual de esa lista "significa" algo por sí solo — es el **patrón completo** de los 512 números juntos el que codifica el significado. Por eso dos palabras parecidas en significado (como "dog" y "cat") terminan teniendo vectores bastante **parecidos entre sí**, aunque no exista forma de "leer" qué representa cada número por separado — es matemática de alta dimensión, no algo interpretable a simple vista.

🤖 **¿Estos 512 números son aleatorios?** Al crear un modelo nuevo, **sí** — cada palabra arranca con 512 números al azar (inicialización aleatoria, sin significado). Pero a medida que el modelo se entrena con millones de pares de oraciones traducidas, esos números se **ajustan** (con backpropagation) hasta terminar ubicando palabras parecidas cerca unas de otras en ese espacio matemático. Una vez entrenado el modelo, la tabla de embeddings queda **fija**: la palabra "dog" siempre produce el mismo vector — no se genera de nuevo al azar cada vez que aparece. Es literalmente una tabla de consulta (nn.Embedding en código), con una fila por cada palabra del vocabulario.

**¿Qué es exactamente "backpropagation"?** Es el proceso de entrenamiento en sí, y se repite millones de veces durante el entrenamiento:

1. **Forward pass**: el modelo procesa un par de oraciones de entrenamiento (inglés + su traducción real al alemán) por todas sus capas y genera una traducción propia
2. **Cálculo del error**: se compara esa traducción generada contra la traducción real del corpus — la diferencia es el "error"
3. **Backpropagation** (retropropagación): ese error se propaga **hacia atrás**, capa por capa, calculando cuánto contribuyó cada número del modelo — incluidos los 512 de cada embedding — a ese error (de ahí el nombre: el error viaja en dirección contraria al procesamiento normal, que va hacia adelante)
4. **Actualización de pesos**: cada número se ajusta un poquito, en la dirección que reduciría el error la próxima vez

Este ciclo (forward → error → backward → ajuste) es lo que, repetido millones de veces con millones de pares de oraciones, hace que los 512 números de "dog" dejen de ser ruido aleatorio y terminen codificando significado real.

#### Embeddings de nuestra oración

```
Token ID 50 ("The") → busca en la tabla → [0.12, -0.34, 0.08, 0.91, ...]
(512 números)
Token ID 120 ("dog") → busca en la tabla → [0.70, 0.20, -0.15, 0.44, ...]
(otros 512 números)
Token ID 340 ("eat") → busca en la tabla → [-0.55, 0.33, 0.19, -0.02, ...]
(otros 512 números)
Token ID 12 ("s") → busca en la tabla → [0.02, 0.19, -0.44, 0.10, ...]
(otros 512 números)
Token ID 200 ("bread") → busca en la tabla → [0.44, 0.07, -0.28, 0.61, ...]
(otros 512 números)
```

El embedding **no calcula nada al azar en el momento** – es una **búsqueda en una tabla**, como buscar una palabra en un diccionario: cada token ID le corresponde siempre a la misma fila de 512 números, ya fijada por el entrenamiento del modelo.

### 1.12.4 Paso 2: Positional Encoding – el problema que resuelve (para saber en qué posición está cada palabra)

Como el Transformer **no procesa las palabras en orden** (a diferencia de las RNN, que las procesaban una por una), necesita alguna forma de saber **en qué posición** está cada palabra.

La solución del paper (Sección 3.5): sumarle a cada embedding una “firma matemática” usando funciones seno y coseno:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Cada posición en la secuencia obtiene un patron único de ondas, que se **suma** al embedding original – así, aunque el modelo procese todo en paralelo, cada palabra “sabe” donde esta.

#### El cálculo paso a paso, con números reales

Calculemos las **primeras 4** dimensiones del positional encoding para la palabra "dog" en **posición 1** (recordá: la posición arranca en 0, así que "The"=0, "dog"=1, "eat"=2, "s"=3, "bread"=4).

#### Datos de partida:

- $pos = 1$  (la posición de "dog")
- $d_{model} = 512$
- La regla para calcular cada dimensión es simple: **si la dimensión es par (0, 2, 4, 6...) se usa la función seno; si es impar (1, 3, 5, 7...) se usa coseno** – van alternando: seno, coseno, seno, coseno, así hasta completar las 512 dimensiones. La letra  $i$  que aparece en la fórmula es solo un contador que avanza de a un paso cada dos dimensiones (dimensiones 0 y 1 usan  $i = 0$ , dimensiones 2 y 3 usan  $i = 1$ , dimensiones 4 y 5 usan  $i = 2$ ...), y hace que la onda sea un poco más lenta o más rápida en cada par sucesivo de casilleros

#### Dimensión 0 (par → seno), con $i = 0$ :

Fórmula original:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Reemplazando  $pos = 1, i = 0, d_{model} = 512$ :

$$PE_{(1, 0)} = \sin\left(\frac{1}{10000^{0/512}}\right) = \sin\left(\frac{1}{1}\right) = \sin(1) \approx 0.8415$$

(Como el exponente es 0,  $10000^0 = 1$ , así que queda simplemente  $\sin(1)$ .)

#### Dimensión 1 (impar → coseno, el par de la anterior), también con $i = 0$ :

Fórmula original:

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Reemplazando  $pos = 1, i = 0$ :

$$PE_{(1,1)} = \cos\left(\frac{1}{10000^{0/512}}\right) = \cos(1) \approx 0.5403$$

**Dimensión 2 (par → seno), ahora con  $i = 1$ :**

$$PE_{(1,2)} = \sin\left(\frac{1}{10000^{2/512}}\right) = \sin\left(\frac{1}{1.0387}\right) \approx \sin(0.9627) \approx 0.8196$$

**Dimensión 3 (impar → coseno), con  $i = 1$ :**

$$PE_{(1,3)} = \cos\left(\frac{1}{10000^{2/512}}\right) \approx \cos(0.9627) \approx 0.5729$$

**Resultado: las primeras 4 dimensiones del positional encoding para "dog" (posición 1)**

$$PE_{(1,0:4)} = [0.8415, 0.5403, 0.8196, 0.5729, \dots]$$

**Sumando esto al embedding**

Si el embedding original de "dog"  $[0.70, 0.20, -0.15, 0.44, \dots]$ , el vector final que entra al Encoder es la **suma, dimensión por dimensión**:

$$V_{final}(\text{dog}) = [0.70+0.8415, 0.20+0.5403, -0.15+0.8196, 0.44+0.5729, \dots] = [1.5415, 0.7403, \dots]$$

**Por qué importa: distinguir posiciones repetidas**

Si la misma palabra apareciera dos veces en una oración más larga (por ejemplo, "the" al principio y "the" en otro lugar), el embedding le daría el mismo vector las dos veces — pero el Positional Encoding, al sumarle una onda distinta según la posición, hace que el vector final termine siendo distinto en cada aparición, permitiendo que el modelo las distinga.

### 1.12.5 Flujo de Encoder (producir una sola vez la representación de oración)

**Objetivo:** El encoder lee la oración de entrada ("the dog eats bread") y produce, una sola vez, una representación de esa oración: un vector de 512 numeros por cada token, un vector de 512 numeros por cada token, enriquecido con las 6 capas de self-attention + feed-forward. Esa salida queda fija y el Decoder la consulta en cada ronda via Cross-Attention, sin que el Encoder vuelva a correr.

#### 1.12.5.1 Paso 3 (capa 1/6): El Encoder – Self-Attention sobre la oración de origen

Hasta acá, cada palabra de "The dog eats bread" tiene su propio vector de 512 numeros (embedding + positional encoding), pero esos vectores todavía **no se comunican entre sí** – cada palabra "sabe" su propio significado y su posición, **pero no sabe nada de las palabras que la rodean**. Self-Attention es el paso que resuelve justamente eso: **deja que cada palabra "mire" a todas las demás de la oración y decida cuales le importan mas para entender su propio significado en ese contexto**.

Este calculo ocurre dentro del **Encoder** – la primera de las dos mitades del Transformer, encargada de leer toda la oración en ingles y armar una representación completa de ella (que despues va a usar el Decoder).

#### La idea central, sin fórmulas todavía

Para cada palabra, el modelo calcula **3 versiones distintas** de sí misma — no son 3 vectores inventados de la nada, sino 3 transformaciones matemáticas del mismo vector que ya traíamos del Paso 2:

| Vector           | Para que sirve                                            | Analogía                            |
|------------------|-----------------------------------------------------------|-------------------------------------|
| <b>Query (Q)</b> | Lo que esta palabra está "buscando" en las demás          | Una pregunta que la palabra hace    |
| <b>Key (K)</b>   | Lo que esta palabra "ofrece" para que otras la encuentren | Una etiqueta que la palabra muestra |
| <b>Value (V)</b> | El contenido real que se comparte, si alguien "la elige"  | La respuesta que la palabra entrega |

#### Un ejemplo con palabras, antes de entrar en números

Pensemos qué "buscaría", "ofrecería" y "entregaría" cada palabra de "The dog eats bread", en términos humanos (no matemáticos todavía — esto es solo intuición, antes de la fórmula):

| Palabra        | Su Query (¿Qué busca?)                             | Su Key (¿qué ofrece?)                                          | Su Value (¿qué entrega si la eligen?)                                       |
|----------------|----------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>"eats"</b>  | "¿Quién realiza esta acción?"                      | "Soy un verbo, acción de alimentarse"                          | La idea de "ingerir algo como alimento"                                     |
| <b>"dogs"</b>  | "¿Hay un verbo cerca que me necesita como sujeto?" | "Soy un sustantivo, un animal capaz de actuar"                 | Su significado real: la idea de "perro" (animal, sujeto típico de acciones) |
| <b>"bread"</b> | "¿Hay un verbo cerca que me necesite como objeto?" | "Soy un sustantivo, un alimento, típicamente algo que se come" | Su significado real: la idea de "pan" (alimento, objeto típico de "comer")  |
| <b>"The"</b>   | "¿A que sustantivo estoy introduciendo?"           | "Soy un articulo, siempre voy pegado a un sustantivo"          | Prácticamente no aporta significado propio – es casi puramente gramatical   |

La intuición: cuando **"eats"** calcula su atención, su Query ("¿Quién come? ¿Qué se come?") va a "encajar" muy bien con el Key de **"dog"** (un sustantivo-animal, candidato natural a sujeto) y con el Key de **"bread"** (un sustantivo-alimento, candidato natural a objeto) – muchomejor que con el Key de **"The"**, que es puramente

gramatical. Esto es exactamente lo que la formula matemática de abajo termina calculando con números – la tabla de arriba es la versión “en palabras” de los mismo.

osea que lo que se hace mas que nada es contextualizar la frase?

Sí, exacto — "contextualizar" es una muy buena forma de resumirlo es justo lo que hace Self-Attention.

Confirmando tu intuición

Antes de este paso, cada palabra tenia un significado **fijo y aislado** – el embedding de “eats” siempre era el mismo vector, sin importar en que oración apareciera. Despues de Self-Attention, ese vector de “eats” pasa a llevar **el contexto específico de esta oración en particular** – sabe que en este caso esta relacionada con “dog” (quien come) y “bread” (que se come).

Por qué esto importa tanto, con un ejemplo que lo deja clarísimo

Pensá en una palabra con significados distintos según el contexto — por ejemplo "banco":

| Oracion                                  | Sin contextualizar (solo embedding)      | Con Self-Attention (contextualizado)                                                                               |
|------------------------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| "Me senté en el <b>banco</b> del parque" | Siempre el mismo vector de "banco"       | El vector de “banco” se acerca al significado de “asiento/mueble”, porque le presta atención a “sente” y “parque”. |
| "Fui al <b>banco</b> a sacar dinero"     | Ese mismo vector de “banco”, sin cambios | El vector de “banco” se acerca al significado de “institución financiero”, porque le presta atención a “dinero”.   |

Sin Self-Attention, el modelo no tendría forma de distinguir estos dos sentidos de “banco” – el embedding de partida es idéntico en ambos casos. Self-Attention es lo que le permite al modelo “entender” cual sentido corresponde, **mirando las palabras alrededor**.

La palabra técnica exacta para esto

Lo que estas describiendo se llama justamente **“representación contextual”** (contextual representation) – es uno de los grandes avances que trajo esta arquitectura, comparado con técnicas anteriores (como Word2Vec) donde cada palabra tenia **un solo vector fijo, para siempre**, sin importar el contexto.

Resumen

Si – el objetivo central de Self-Attention es exactamente ese: tomar el significado “genérico” de cada palabra (el embedding) y **enriquecerlo con el contexto específico** de la oración donde aparece, mezclando información de las palabras relevantes que la rodean, en proporciones distintas según que tan relevante es cada una.

### 1.12.5.1.1: De donde salen exactamente Q, K y V – las matrices de pesos : Importante

Estas 3 versiones no aparecen solas – se calculan multiplicando el vector de 512 numeros de la palabra contra **matrices de pesos** (tablas de números):

$$W^Q W^K W^V$$

**Importante para ser exactos con los tamaños:** como el modelo usa **8 cabezas de atención en paralelo** (el flujo completo se explica en la sección siguiente, “Multi-Head Attention”), en realidad **no hay una sola matriz de cada tipo** – hay **8 matrices  $W^Q$  distintas** (una por cabeza),

$8 W^K$  y  $8 W^V$ , 24 en total. Cada una tiene tamaño  $512 \times 64$ : toma el vector de entrada de 512 numeros y lo reduce a 64, porque  $512 \div 8 \text{ cabezas} = 64$ .

$$Q_i = x \cdot W_i^Q \quad K_i = x \cdot W_i^K \quad V_i = x \cdot W_i^V \quad (\text{para cada cabeza } i = 1 \dots 8)$$

**¿De dónde salen esas matrices? Nadie las escribió a mano.** Es el mismo mecanismo que ya vimos con los embeddings (recuadro 🎲 mas arriba): al crear el modelo, **las 24 matrices arrancan con números al azar**; durante el entrenamiento (backpropagation) se ajustan millones de veces; al final quedan fijas, y cada una “aprendió” a transformar cualquier palabra en su Query, Key o Value de forma sutil – sin que nadie le haya indicado explícitamente como hacerlo.

La fórmula exacta del paper:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

Qué hace cada parte de la fórmula, paso a paso

1.  $QK^T$  — se multiplica cada Query contra todos los Keys de la oración. Esto da un número por cada par de palabras: cuánto “encaja” lo que una palabra busca con lo que otra ofrece
2.  $\frac{1}{\sqrt{d_k}}$  — se divide por la raíz de  $d_k$  (64 en este paper) – un ajuste matemático para que esos números no se disparen y rompan el entrenamiento.
3.  $\text{softmax}$  – convierte esos números en porcentajes que suman 100% (ej: “esta palabra le presta 70% de atención a otra, 20% a otra, 10% al resto”)
4.  $\times V$  – se multiplica por los Values – el resultado final es una mezcla ponderada de toda la información relevante, según esos porcentajes de atención.



### 1.12.5.1.2 ¿Qué quiere decir 8 cabezas?

“8 cabezas” no significa nada físico ni literal, es solo el nombre que le dieron a **8 cálculos independientes de atención, hechos en paralelo, sobre la misma oración.**

#### La idea central, sin tecnicismos

En vez de calcular Self-Attention **una sola vez** (con un solo Query, un solo key, un solo Value por palabra), el modelo lo calcula **8 veces por separado**, con 8 versiones distintas de Q, K, V – cada una de esas 8 versiones es “una cabeza”.

#### Por qué el nombre "cabeza"

Es una analogía: piensa en 8 personas distintas (“cabezas” en el sentido de “mentes”) leyendo la misma oración, **cada una fijándose en algo diferente:**

| Cabeza   | En que podría fijarse (el modelo lo aprende solo, nadie se lo asigna) |
|----------|-----------------------------------------------------------------------|
| Cabeza 1 | Relaciones gramaticales (sujero-verbo)                                |
| Cabeza 2 | Significado semántico (que palabras están relacionadas por sentido)   |
| Cabeza 3 | Posicion relativa (palabras cercanas entre si)                        |
| ...      | ...                                                                   |
| Cabeza 8 | Algun otro patron que “descubrió” durante el entrenamiento            |

Ninguna cabeza tiene asignada su tarea a mano – durante el entrenamiento (backpropagation), cada una termina especializándose en algo distinto, simplemente porque eso resulto útil para **mínima el error del modelo.**

#### Por qué usar 8 "miradas chicas" en vez de 1 "mirada grande"

Es la comparación que ya vimos: en vez de una sola mirada con el vector completo de 512 numeros, se usan 8 miradas mas chicas, de 64 numeros cada una. Es como tener **8 pares de ojos** mirando la misma foto, cada uno enfocado en un detalle distinto, en vez de un solo par de ojos tratando de verlo todo a la vez.

#### Confirmado con el paper original (Sección 3.2.2)

Cita textual: “\*Instead of performing a single attention function with  $d_{model}$ -dimensional keys, values and queries, we found it beneficial to leanrly Project the queriess, keys and values h times with different, learned linear projections”\* - y mas adelante especifican  $h=8$ .

En criollo: los propios autores hacerlo con una sola “mirada” grande, y **descubrieron que funcionaba mejor dividiéndolo en 8 miradas mas chicas**, cada una aprendiendo algo distinto.

#### Resumen

“8 cabezas” = **8 cálculos de atención independientes, corriendo en paralelo sobre la misma oración**, cada uno con sus propias **matrices de pesos** (las  $W_i^Q W_i^K W_i^V$  que viste en el diagrama), especializándose cada una en fijarse en un aspecto distinto de las relaciones entre palabras – y al final, se juntan sus 8 resultados en uno solo.

### 1.12.5.1.3: Multi-Head Attention: el flujo completo, paso a paso

Antes de ver un ejemplo numérico, hace falta entender **por que existen “cabezas”** – porque el calculo de arriba no se hace una sola vez, se hace **8 veces en paralelo**, y el ejemplo que sigue después de esto va a ser justamente el resultado de una de esas 8 cabezas.

#### 1.12.5.1.3.1: Paso 1 — arranca con el vector completo.

La palabra "dog" llega a este punto con su vector de 512 números (embedding + positional encoding):

```
x(dog) = [n1, n2, n3, ..., n512] ← 512 números en total
```

#### 1.12.5.1.3.2: Paso 2 – ese mismo vector se copia 8 veces.

No se “corta” en pedazos – se usa el vector completo 8 veces, una vez por cada cabeza. Es como sacarle 8 fotocopias al mismo documento:

```
Copia para Cabeza 1: [n1, n2, n3, ..., n512]
Copia para Cabeza 2: [n1, n2, n3, ..., n512]
...
Copia para Cabeza 8: [n1, n2, n3, ..., n512]
```

#### 1.12.5.1.3.3: Paso 3 – cada cabeza multiplica SU copia contra SUS propias matrices.

Acá es donde cada cabeza se vuelve distinta. Cada una tiene sus propias matrices  $W_i^Q W_i^K W_i^V$  (las que vimos arriba), y multiplica su copia del vector contra ellas – el resultado es mas chico: 64 numeros, no 512:

```
Cabeza 1: [n1...n512] × W^Q_1 → Q_1 = [64 números]
Cabeza 2: [n1...n512] × W^Q_2 → Q_2 = [64 números]    (otros 64, distintos)
...
Cabeza 8: [n1...n512] × W^Q_8 → Q_8 = [64 números]    (otros 64, distintos)
```

(Y lo mismo pasa para K y V – cada cabeza multiplica su copia del vector contra su propia  $W_i^K$  para obtener el Key, y contra su propia  $W_i^V$  para obtener el Value, con la misma mecánica de multiplicar y reducir a 64 números.)

 **Importante: esto pasa EN PARALELO para los 5 tokens de la oración, no solo para "eat".** Los ejemplos de arriba muestran el vector de “dog”, pero exactamente el mismo Paso 3 corre, al mismo tiempo, para “The”, “dog”, “eat”, “s” y “bread” – caa uno con su propio vector de 512 numeros de entrada, pero usando **las mismas matrices**  $W_i^Q/W_i^K/W_i^V$  (de cada cabeza) para los 5. Lo que cambia entre tokens es el vector de entrada (cada uno tiene el suyo); la "receta" (la matriz) es igual para todos:

```

Token "The"   × W^Q_1, W^K_1, W^V_1 (Cabeza 1) → Q_The, K_The, V_The (64
cada uno)
Token "dog"   × W^Q_1, W^K_1, W^V_1 (Cabeza 1) → Q_dog, K_dog, V_dog (64
cada uno)
Token "eat"   × W^Q_1, W^K_1, W^V_1 (Cabeza 1) → Q_eat, K_eat, V_eat (64
cada uno)
Token "s"     × W^Q_1, W^K_1, W^V_1 (Cabeza 1) → Q_s, K_s, V_s (64 cada
uno)
Token "bread" × W^Q_1, W^K_1, W^V_1 (Cabeza 1) → Q_bread, K_bread,
V_bread (64 cada uno)

```

Esto es justo lo que hace que Self-Attention sea tan rápido comparado con las RNN (ver la tabla de complejidad computacional mas abajo): **todos los tokens se procesan al mismo tiempo**, no uno atrás del otro. El documento se enfoca en seguir el hilo de "eat" mirando a los demás, pero "dog", "The", "s" y "bread" están calculando su propia atención exactamente al mismo tiempo, cada uno con su propio resultado.

#### 1.12.5.1.3.4: Paso 4 – sigamos con UNA sola cabeza (Cabeza 1) hasta el final, antes de pensar en las otras 7.

Ya tenemos, gracias al Paso 3, el Q, K y V de la Cabeza 1 – pero no de la misma forma para los tres:

Tenemos **el Query solo del token “eat”** (el que se esta procesando), y **el Key y el Value de los 5 tokens de la oración (“The”, “dog”, “eat”, “s”, “bread”)** – porque “eat” necesita comparar su Query contra el Key de cada uno de los 5, incluido el mismo. Ahora hay que resolver la formula del paper – la que se llama **“Scaled Do-Product Attention”** (Seccion 3.2.1):

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

**Importante:** Este Paso 4 completo **es** esa fórmula, de punta a punta – no se detiene a mitad de camino. El objetivo es llegar hasta el final de la formula (la multiplicación por **V**) y obtener **un único vector de 64 numeros**, resultado de la **Cabeza 1** solamente. Para eso, hay que resolverla en 4 sub-etapas, en orden – como una receta con varios pasos que llevan a un solo plato final.

| Sub-etapa                                       | Parte de la formula       | Nombre en el paper                                   |
|-------------------------------------------------|---------------------------|------------------------------------------------------|
| <b>4.1 Producto punto</b>                       | $QK^T$                    | “Dot-Product”                                        |
| <b>4.2 Dividir por <math>\sqrt{64}=8</math></b> | $\frac{QK^T}{\sqrt{d_k}}$ | “Scaled”                                             |
| <b>4.3 Softmax</b>                              | $\text{Softmax}(\cdot)$   | -                                                    |
| <b>4.4 Multiplicar y sumar</b>                  | $(\cdot) \times V$        | “Attention” (recién acá se llega al resultado final) |

Ninguna de las 3 primeras sub-etapas da un resultado "utilizable" por sí sola — son cálculos intermedios. Sigamos con los mismos números que ya usamos antes (Query de "eat" = 0.8; Keys de "The"=0.1, "dog"=0.85, "eat"=0.9, "bread"=0.8; en la versión simplificada de 1 dimensión, dentro de esta única cabeza):

#### 1.12.5.1.3.4.1: Producto punto:

**Producto punto:** se multiplica el Q de "eat" (64 números) contra el K de cada token (también 64 números), sumando los 64 productos — el resultado es **un solo número por token** (no 64). En nuestra versión simplificada de 1 dimensión, esto ya lo calculamos:

```
0.8 × 0.1 = 0.08    (con "The")
0.8 × 0.85 = 0.68   (con "dog")
0.8 × 0.9 = 0.72    (con "eat", a sí misma)
0.8 × 0.8 = 0.64    (con "bread")
```



#### 1.12.5.1.3.4.2: Dividir por $\sqrt{64}=8$

cada uno de esos números (en la versión real de 64 dimensiones; en nuestro ejemplo reducido usamos  $d_k=1$ ), así que dividir por  $d_k=1$  no cambia nada — los números quedan iguales a los de 4.1)

⚙️ ¿Cómo es este paso en la realidad, con  $d_k = 64$  de verdad? En nuestro ejemplo de 1 dimensión, este paso "no hace nada" — pero en el modelo real es crucial. Con vectores de 64 números, el producto punto de 4.1 no es 1 multiplicación, son **64 multiplicaciones sumadas**:  $Q \cdot K = \sum_{i=1}^{64} q_i \times k_i$ . El paper explica por qué esto importa (Sección 3.2.1, nota al pie): si cada uno de esos 64 números fuera un valor "normal" (media 0, varianza 1), la suma de 64 de esos productos tiende a dar resultados **cada vez más grandes** a medida que crece  $d_k$  — la varianza del resultado crece proporcional a  $d_k$ . Por ejemplo, en vez del **0.68** chico que veníamos usando, el producto punto real podría dar algo como **41.6**. Números tan grandes, al pasar por softmax, hacen que se vuelva casi "todo o nada" — le da casi 100% de atención a un solo token, perdiendo la capacidad de repartir la atención de forma más suave. Dividir por  $\sqrt{d_k}$  (la raíz cuadrada de esa varianza) "normaliza" el resultado de vuelta a una escala razonable:  $41.6/8 = 5.2$ , un número mucho más manejable para el softmax que sigue después.

#### 1.12.5.1.3.4.3: Softmax: esos números se convierten en porcentajes que suman 100%:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

La fórmula exacta de softmax:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Donde  $e \approx 2.71828$  (el número de Euler). En criollo: a **cada** número se lo usa como exponente de  $e$  (esto lo vuelve siempre positivo, sin importar el signo original), se **suman** todos esos resultados, y cada uno se **divide** por esa suma total. Aplicándolo a nuestros números de 4.2 (0.08, 0.68, 0.72, 0.64):

$$e^{0.08} \approx 1.083 \quad e^{0.68} \approx 1.974 \quad e^{0.72} \approx 2.054 \quad e^{0.64} \approx 1.896$$

$$\text{Suma total: } 1.083 + 1.974 + 2.054 + 1.896 = 7.007$$

$$\text{"The": } 1.083 / 7.007 \approx 15.5\%$$

$$\text{"dog": } 1.974 / 7.007 \approx 28.2\%$$

$$\text{"eat": } 2.054 / 7.007 \approx 29.3\%$$

$$\text{"bread": } 1.896 / 7.007 \approx 27.1\%$$

Estos porcentajes son casi idénticos a los 14%/30%/32%/28% que veníamos usando — la pequeña diferencia es solo por el redondeo a ojo que hicimos antes.

|                     |      |
|---------------------|------|
| "The":              | ~14% |
| "dog":              | ~30% |
| "eat" (a sí misma): | ~32% |
| "bread":            | ~28% |
| -----               |      |
|                     | 100% |

#### 1.12.5.1.3.4.4: Multiplicar y sumar

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

Cada porcentaje se multiplica por el Value **completo** (64 números) de ese token, y se suman los vectores resultantes, posición por posición. Con los Values inventados  $V(\text{The}) = 0.3$ ,  $V(\text{dog}) = 0.9$ ,  $V(\text{eat}) = 0.85$ ,  $V(\text{bread}) = 0.75$  (en la versión de 1 dimensión):

$$\begin{aligned} 0.14 \times 0.3 &= 0.042 && (\text{aporte de "The"}) \\ 0.30 \times 0.9 &= 0.270 && (\text{aporte de "dog"}) \\ 0.32 \times 0.85 &= 0.272 && (\text{aporte de "eat", a sí misma}) \\ 0.28 \times 0.75 &= 0.210 && (\text{aporte de "bread"}) \\ &----- \\ \text{Suma total: } &0.794 \end{aligned}$$

**Aca recién** se llega al resultado final del Paso 4: el número **0.794** (en la versión real, un vector de 64 números) — el nuevo “significado” de “eat”, calculado **solo con la Cabeza 1**, después de mezclar la información de todo el resto de la oración.

En criollo: entran muchos números (un Q, y un K y un V por cada token de la oración), pero las 4 sub-etapas de la fórmula los reducen todos a **un único vector de 64 números** — el resultado de esa cabeza. Ese vector único no es ni Q, ni K, ni V (esos tres ya "se consumieron" dentro de este cálculo): es una mezcla nueva, distinta a los tres ingredientes que la originaron.

**Ahora sí: contando las 8 cabezas juntas**

Todo lo que acabamos de calcular fue **una sola cabeza** (la Cabeza 1). Pero recuerda el Paso 2: el vector original se copio 8 veces, y cada cabeza tiene sus propias matrices – así que este mismo caclulo completo (Pasos 3 y 4, con sus 4 sub-etapas) **se repite 8 veces en paralelo**, con números distintos en cada una porque las matrices son distintas:

```
Cabeza 1 → resultado: 0.794 (en la realidad, 64 números) - la que acabamos de calcular
Cabeza 2 → resultado: [64 números] (quizás notó algo de posición)
...
Cabeza 4 → resultado: [64 números] (podría notar relaciones sujeto-verbo)
...
Cabeza 8 → resultado: [64 números]
```

Esta es la idea central de Multi-Head Attention: cada una de esas 8 cabezas termina especializándose en fijarse en un tipo distinto de relación (el modelo aprende esto solo, con el entrenamiento – nadie le asigna la tarea a cada cabeza a mano).

Ahora que ya vimos el cálculo completo de una cabeza, tiene sentido el **inventario total**, contando las 8 cabezas juntas — no es un cálculo alternativo a la fórmula, es simplemente contar cuánta "materia prima" (Q, K, V) se usó en total, sumando las 8 repeticiones del mismo proceso:

|                                            | Por una sola cabeza (lo que acabamos de calcular) | Contando las 8 cabezas                  |
|--------------------------------------------|---------------------------------------------------|-----------------------------------------|
| Q (solo de "eat", el token que se procesa) | $1 \times 64 = 64$                                | $8 \times 64 = \mathbf{512}$            |
| K (uno por cada uno de los 5 tokens)       | $5 \times 64 = 320$                               | $8 \times 5 \times 64 = \mathbf{2.560}$ |
| V (uno por cada uno de los 5 tokens)       | $5 \times 64 = 320$                               | $8 \times 5 \times 64 = \mathbf{2.560}$ |

La diferencia entre Q y los otros dos: **Q solo hace falta para el token que se esta procesando** (“eat”), mientras que **K y V hacen falta para todos los tokens de la oración** (porque “eat” necesita comparar su Query contra el Key de cada uno) – por eso Q multiplica solo por 8 (cabezas), y K/V multiplican por 8x5 (cabezas x tokens). En total, hay  $512 + 2.560 + 2.560 = 5.632$  numeros en juego – pero, como vimos, la formula nunca los procesa todos juntos: los procesa de a 704 por vez ( $64+320+320$ ), una ve por cada una de las 8 cabezas.

? Entonces, ¿lo que se concatena en el próximo paso son los Q, K y V? No — esos 3 ya se consumieron dentro del cálculo de cada cabeza, como vimos arriba. Lo que sigue camino hacia el Paso 5 es el resultado único de cada cabeza (0.794 para la Cabeza 1, y los demás resultados de 64 números para las otras 7). Si se concatenaran los Q, K y V de las 8 cabezas por separado, darían  $24 \times 64 = 1536$  números; concatenando los 8 resultados finales (uno por cabeza) da  $8 \times 64 = 512$ , que es el número que necesitamos.

#### 1.12.5.1.3.5: Paso 5 – se pagan los 8 resultados, uno atrás del otro:

Resultado\_1 (64) + Resultado\_2 (64) + ... + Resultado\_8 (64) = un vector de  $8 \times 64 = 512$  números de nuevo

#### ✂ ¿Esto se concatena por los 5 tokens, o por las 8 cabezas?

**R:** Por las **8 cabezas** – se pegan los 8 resultados que dieron las 8 cabezas **para el mismo token** (“eat”), no los resultados de tokens distintos. Pero, como ya vimos en el recuadro ✂ del Paso 3, este Paso 5 (y todo el Paso 4 que lo procede) se repite **una vez por cada uno de los 5 tokens** de la oración, en paralelo – porque cada token necesita su propio resultado final. Así que al terminar, no hay “un solo vector gigante para toda la oración”: Hay **5 vectores de 512 números**, uno por token, cada uno armado concatenando las 8 cabezas de ESE token en particular.

#### 1.12.5.1.3.6: Paso 6 – ese vector pasa por una última matriz, $W^O$ :

[512 números pegados]  $\times W^O \rightarrow$  [512 números finales]

? ¿Qué es exactamente  $W^O$ ? No es Q, ni K, ni V. Es una matriz nueva y distinta a las 24 que ya vimos — de tamaño **512  $\times$  512** (no reduce nada, a diferencia de las  $W_i^Q/W_i^K/W_i^V$  que sí achican de 512 a 64). Eso significa que  $W^O$  tiene  $512 \times 512 = 262.144$  números en total — mucho más que “512 valores”: tiene 512 filas y 512 columnas, una tabla completa, no una sola lista. Es, de hecho, **la matriz más grande** de todo este bloque: 8 veces más grande que cada una de las matrices  $W_i^Q/W_i^K/W_i^V$  individuales (que tienen  $512 \times 64 = 32.768$  números cada una). Su trabajo: después de concatenar las 8 cabezas, lo que tenés son **8 pedazos de 64 números pegados uno al lado del otro**, que nunca se combinaron entre sí — es como poner 8 ingredientes picados por separado en el mismo bol, todavía distinguibles.  $W^O$  es la que “mezcla con la cuchara” esos 8 pedazos, para que la información de una cabeza pueda combinarse con la de otra en un único vector integrado. Es una matriz aprendida, igual que todas las demás — arranca al azar y se ajusta con backpropagation. La fórmula completa del paper:  $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_8) W^O$  — primero concatenar, después multiplicar por  $W^O$ .

[ $W^O$  es la que “mezcla con la cuchara” esos 8 pedazos, para que la información de una cabeza pueda combinarse con la de otra en un único vector integrado. Es una matriz aprendida, igual que todas las demás — arranca al azar y se ajusta con backpropagation]

Ese resultado final de 512 números es lo que sigue camino hacia el próximo paso del modelo (Feed-Forward).



🔗 Entonces, ¿el resultado final es un vector de 512 por cada uno de los 5 tokens? Sí, exacto. Todo este bloque de Multi-Head Attention (Pasos 1 a 6) corrió en paralelo para cada uno de los 5 tokens de la oración, como ya vimos en los recuadros anteriores — así que al terminar, hay **5 vectores de 512 números**, uno por token, no un solo resultado para toda la oración. Cada token **entró** con su propio vector de 512 (embedding + positional encoding) y **sale** con otro vector de 512 — la "forma" no cambió, pero el contenido sí: ahora cada vector está enriquecido con información del resto de la oración, en vez de ser solo el significado aislado de esa palabra. Por eso el Feed-Forward que sigue (la siguiente sección de este documento) se aplica, según el propio paper, "a cada posición separadamente y de forma idéntica" — toma cada uno de esos 5 vectores, uno por uno, y les aplica la misma red por separado.

[ahora cada token esta enriquecido con información del resto de la oración, en vez de ser solo el significado aislado de esa palabra]

**Resumen en una línea:** 1 vector de 512 → se copia 8 veces → cada copia se reduce a 64 y se procesa por separado en su propia cabeza → los 8 resultados de 64 se pegan → vuelve a ser un vector de 512.



### 1.12.5.2 Paso 4 (capa 1/6): Residual connections + Layer Normalization #1 (se aplica despues de Multi-Head Attention y Feed-Forward Network)

Cada sub-capa (atención y feed-forward) tiene un “atajo”: se suma la entrada original a la salida, y despues se normaliza:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

Se llama “Residual” (residuo) justamente porque no reemplaza el vector original: lo **conserva** (lo suma), y la sub-capa le agrega un ajuste encima – nunca “tira” del todo lo que ya había. Esto ayuda a que el entrenamiento no se “pierda” en redes tan profundas – es un truco tomado de otro paper (ResNets, 2016). Igual que el Feed-Forward, aplica en ambas mitades del modelo.

Qué es cada símbolo, su función, si es un peso entrenado, y su forma matemática:

| Simbolo                   | Que es                                                                                                                                                                         | Por que esta ahí (su función)                                                                                                                                                         | ¿Peso entrenado?                                                                                                             | Forma matemática                   | Tamaño real ( $d_{model}=512$ ) |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|------------------------------------|---------------------------------|
| $x$                       | El vector <b>original</b> que entro a esta sub-capa – antes de que Multi-Head Attention (o Feed-Forward) lo procesara                                                          | Es “lo que ya sabíamos” del token, antes de este paso – el punto de referencia que no se quiere perder                                                                                | No – es el dato de entrada, no algo que se ajuste                                                                            | <b>Vector</b>                      | 512 numeros                     |
| $\text{Sublayer}(x)$      | La salida de la sub-capa que acaba de correr (Multi-Head Attention completo, o Feed-Forward completo) – <b>no</b> es un peso nuevo, es el resultado de todo un bloque anterior | Es “lo nuevo” que esa subacapa calculo a partir de $x$                                                                                                                                | No – es un resultado de calculo (aunque, por dentro, esa sub-capa si tenga sus propiso pesos, como $W^Q/W^k/W^v$ o $W_1/W_2$ | <b>Vector</b>                      | 512 numeros                     |
| $x + \text{Sublayer}(x)$  | La suma, numero por numero, de los dos vectores de arriba – eso es la parte “Residual”                                                                                         | Combina “lo que ya sabíamos” con “lo nuevo”, en vez de reemplazar uno por el otro – así, si $\text{Sublayer}(x)$ sale mal al principio del entrenamiento), $x$ sigue estando presente | No – es un resultado de calculo                                                                                              | <b>Vector</b>                      | 512 numeros                     |
| $\text{LayerNorm}(\cdot)$ | <b>No es un peso propio en el sentido de una matriz</b>                                                                                                                        | Evita que los números se                                                                                                                                                              | <b>Parcialmente</b> – la normalización                                                                                       | $\gamma, \beta$<br><b>Vectores</b> | 512 numeros                     |

|                                    |                                                                                                                                                                                                                                    |                                                                                                                                                                       |                                                                                                        |                                   |             |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------|-------------|
|                                    | <b>grande</b> – es una operación que reescala los números de un vector para que tengan un promedio y una dispersión estables, mas 2 pequeños vectores de ajuste ( $\gamma$ y $\beta$ , de 512 numeros cada uno) que si se entrenan | disparen o se achiquen demasiado a medida que pasan por muchas capas apiladas – sin esto, el entrenamiento se vuelve inestable en redes profundas                     | en si no tiene pesos, pero $\gamma$ (escala) y $\beta$ (desplazamiento) si son vectores aprendidos**** |                                   | cada uno    |
| <b>LayerNorm (x+ Sublayer (x))</b> | El resultaod final de toda la formula – lo que efectivamente sale de esta sub-capa y sigue camino                                                                                                                                  | Es la “lista para usar” por la siguiente sub-capa (Feed-Forward, si esto fue despues de Attention; o la siguiente capa completa, si esto fue despues de Feed-Forward) | No – es el resultado del calculo completo                                                              | <b>Vector</b> (mismo tamaño de x) | 512 numeros |

como particlamente? explicame eso en el chat

Buena pregunta para desarmar — "parcialmente" significa que dentro de *LayerNorm* hay **dos operaciones distintas**, y solo una de ellas usa pesos entrenados. Te separo las dos.

Las dos partes que se enconden dentro de *Layer Norm*”

| Parte                                    | Que hace                                                                                                                                        | ¿Usa pesos entrenados?                                              |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1.La normalización en si                 | Calcula el promedio y la dispersión de los números del vector, y los reescala mateamticamente para que queden centrados en 0 y con dispersión 1 | <b>No</b> – es puro calculo estadístico, fijo, sin nada que aprende |
| 2.El ajuste final ( $\gamma$ , $\beta$ ) | Despues de normalizar, multiplica por $\gamma$ y suma $\beta$                                                                                   | <b>Si</b> – son dos vectores que el modelo aprende                  |

La fórmula completa (que el documento no había desarmado del todo)

$$LayerNorm(z) = \gamma \cdot \hat{z} + \beta$$

donde  $\hat{z}$  es el vector  $z$  **ya normalizado** (promedio 0, dispersión 1):

$$\hat{z} = \frac{z - \mu}{\sigma}$$

#### Por qué "parcialmente"

- $\mu$  (el promedio) y  $\sigma$  (la dispersión) se **calculan** directamente de los números del vector en ese momento — no son pesos, cambian según cuáles sean los números de entrada, no se "aprenden" de antemano
- $\gamma$  y  $\beta$ , en cambio, **sí son pesos fijos**, aprendidos una vez durante el entrenamiento, iguales para cualquier vector que pase por esa capa

Entonces, la operación **completa** de *LayerNorm* tiene una parte sin pesos (el cálculo de  $\mu$ ,  $\sigma$ , y la normalización en sí) y una parte con pesos ( $\gamma$ ,  $\beta$ ) — por eso "parcialmente" y no un sí o un no rotundo.

#### Por qué existen $\gamma$ y $\beta$ , si ya normalizaste

Sin ellos, **todos** los vectores, después de normalizar, quedarían forzados a tener siempre promedio 0 y dispersión 1 — sin ninguna flexibilidad.  $\gamma$  y  $\beta$  **le dan al modelo la posibilidad de “deshacer”** esa normalización si le conviene (por ejemplo, escalar más un vector, o correrlo del 0), **aprendiendo** cuanto ajuste hace falta, en vez de quedar siempre a la misma escala rígida.

#### Ejemplo simple, con números inventados

Vector antes de normalizar:  $z = [3, 5, 7]$

Promedio ( $\mu$ ) = 5

Dispersión ( $\sigma$ )  $\approx 1.63$

Normalizado:  $\hat{z} = [(3-5)/1.63, (5-5)/1.63, (7-5)/1.63] \approx [-1.22, 0, 1.22]$

↑ esto ya tiene promedio 0, dispersión 1 — sin usar ningún

peso

Si  $\gamma=2$ ,  $\beta=1$  (aprendidos):

$LayerNorm(z) = 2 \times [-1.22, 0, 1.22] + 1 = [-1.44, 1, 3.44]$

↑ acá **SÍ** se usaron los pesos  $\gamma$  y  $\beta$

**Con nuestros propios números, para "dog":** antes de Multi-Head Attention, "dog" tenía su vector  $x(\text{dog}) = [1.5415, 0.7403, 0.6696, 1.0129, \dots]$  (del Paso 2). Después de todo el bloque de Multi-Head Attention, "dog" tiene un resultado nuevo, *Sublayer*( $x$ ). Se suman, número por número:  $x + \text{Sublayer}(x)$ , y esa suma se normaliza — eso es lo que da *LayerNorm*( $x + \text{Sublayer}(x)$ ). **Esta es la versión que efectivamente sigue camino hacia Feed-Forward** — no el *Sublayer*( $x$ ) crudo.

**Importante: son DOS aplicación por capa, no una sola**

El paper lo confirma explícitamente (cita textual): “*We employ a residual connection around each of the two sub-layers, followed by layer normalization*” – “cada una de las **dos** sub-capas” significa que Residual + LayerNorm se aplica **dos veces** dentro de una misma capa: una vez despues de Multi-Head Attention (la que acabamos de ver), y otra vez despues de Feed-Forward (la que viene en la sección siguiente) – no es un solo paso al final de todo.

El flujo completo dentro de **una sola capa**, con ambas aplicaciones marcadas — este es el orden real, verificado contra el paper:

```
x (entrada del token a esta capa)
↓
Multi-Head Attention → Sublayer1(x)
↓
Residual + LayerNorm #1: LayerNorm(x + Sublayer1(x)) → llamemos a esto
"y" ← ACÁ ESTAMOS
↓
Feed-Forward Network → Sublayer2(y)
← esto sigue, en la próxima sección
↓
Residual + LayerNorm #2: LayerNorm(y + Sublayer2(y))
↓
Esto es lo que sale de esta capa completa, y entra a la siguiente
```

### 1.12.5.3 Paso 5 (capa 1/6): Feed-Forward Network

Despues de la atención, cada token pasa por una red neuronal simple, aplicada igual a cada uno por separado (recorda: son 5 tokens en nuestra oración, no 4 “palabras” – “eats” se partio en “eat” + “s” en el Paso 0):

**El objetivo de este paso, en el panorama completo:** hasta acá, cada token ya paso por Multi-Head

Attention – “mezclo” su significado con el de las palabras relevantes de la oración (recorda “eat” incorporando información de “dog” y “bread”. Pero fíjate que tipo de mezcla es esa: el resultado de “eat” fue literalmente una **suma ponderada** (32 % de si misma + 30% de “dog” + 28 % de “bread” + 14% de “The”) – matemáticamente, es un promedio pesado de contenido que ya existía. Por mas “inteligente” que parezca eligiendo esos porcentajes, **Attention nunca hace mas que combinar lo que ya había – no le agrega nada nuevo, no “procesa” esa mezcla de ninguna forma.**

El Feed-Forward resuelve justo eso: toma esa mezcla ya armada y la **transforma**, no solo la combina. La diferencia esta en el ReLU: es una función que toma una decisión real (“esto lo dejo, esto lo descarto”, según sea positivo o negativo) – algo que una simple suma ponderada nunca puede hacer por si sola. Es la diferencia entre sumar ingredientes en un bol en ciertas proporciones (eso es lo que hace Attention) y meterlos al horno (eso es lo que hace Feed-Forward): sin el “horno”, tendrías siempre mezclas de ingredientes crudos, nunca algo realmente nuevo.

Hay otra razón, mas técnica, por la que este paso es indispensable: sin el, el modelo entero seria matemáticamente equivalente a una **única transformación lineal gigante**, sin importar cuantas capas tuviera apiladas – **porque encadenar transformaciones lineales sigue siendo, matemáticamente, una transformación lineal.** El Feed-Forward (con su ReLU en el medio, que veremos mas abajo) es lo que le da al Transformer la capacidad real de aprender relaciones **complejas y no lineales** en los datos.

### ¿Se sabe qué “representa” esta transformación? Sí — investigado y verificado

A diferencia de lo que se podría suponer (que es una “caja negra” totalmente opaca), **si existe una interpretación con evidencia empírica sólida**, no solo especulación. El paper de referencia es **“Transformer Feed-Forward Layers Are Key-Value Memories”** (Geva, Schuster, Berant y Levy — Tel-Aviv University y Allen Institute for AI, EMNLP 2021), uno de los trabajos mas citados sobre el tema.

**La idea central del paper:** cada capa Feed-Forward funciona como una **memoria de clave-valor** (key-value memory) – un mecanismo parecido a un diccionario gigante:

| Parte de la formula                    | Rol, según el paper                                                                                                                                                                                                                                |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $W_1$ (la primera matriz) (512 x 2048) | Actúa como un conjunto de <b>“claves”</b> – cada una de sus 2048 columnas (en el modelo real) es un <b>detector de patrone</b> , que “se activa” cuando el texto de entrada se parece a un patron especifico que esa columna aprendió a reconocer. |
| $W_2$ (la segunda matriz) (2048 x 512) | Actua como un conjunto de <b>“valores”</b> – cada fila, si su “clave” correspondiente se activo, <b>empuja</b> la predicción final hacia ciertas palabras del vocabulario, mas probables de aparecer justo despues de ese patron.                  |

### Hallazgos concretos y verificados del paper (con evidencia experimental, no solo teoría):

- Los patrones que detectan las “claves” son **interpretables por humanos**: expertos humanos identificaron patrones coherentes en el **100%** de las claves analizadas (con un promedio de 3.6 patrones por clave)
- Las capas mas **tempranas** tienen a capturar patrones **superficiales** (ej: terminaciones de palabras, secuencias de caracteres); las capas **mas profundas** capturan patrones **semánticos** (relacionados con el significado)

 **¿Qué significa “capa temprana” vs “capa profunda”?** Recordá que tanto el Encoder como el Decoder tienen **6 capas apiladas** (N=6, ver la tabla de números del paper mas arriba) – cada una con su propio bloque de Self-Attention + Feed-Forward. “Temprana” se refiere a las primeras capas por las que pasa el texto (Capa, 1,2...), y “profunda” (o “superior”) a las ultimas (Capa 5, 6...) – cada una de esas 6 capas tiene su **propia** Feed-Forward, con **sus propias matrices**  $W_1/W_2$ , entrenadas por separado. Tiene sentido que aprendan cosas distintas: la Capa 1 recibe el texto casi crudo (solo paso por embeddings + positional encoding), así que sus detectores de patrones se enfocan en cosas

simples; la Capa 6 recibe el texto ya procesado 5 veces, con mucha información ya combinada, así que puede reconocer patrones mas abstractos y relacionados con el significado.

- En las capas suprioeres, los “valores” predicen correctamente cual es el siguiente token real hasta un 3.5% de los caso – un numero que suena bajo, pero es **miles de veces mayor** que el azar puro (~0.0004%)
- La predicion final del modelo es una **composición** de muchas de estas “memorias” activándose a la vez, combinada y refinada capa por capa (a través de las Residual Connections, que ta vimos) hasta llegar a la predicción final.

capa mas temprana y capa mas profunda?

“Capas tempranas” y “capas profundas” se refieren a la **posición** de esa capa dentro del apilamiento de 6 capas del Encoder (o Decoder), no a algo distinto a lo que ya vimos en el documento.

### Recordando la arquitectura: 6 capas apiladas

Recorda esto del documento: tanto el Encoder como el Decoder tienen **N=6 capas idénticas**, apiladas una sobre otra – cada una con su propio Self-Attention + Feed-Forward + Residual/LayerNorm.

```
Texto de entrada
↓
Capa 1 (Self-Attention + Feed-Forward) ← la más "temprana"
↓
Capa 2 (Self-Attention + Feed-Forward)
↓
Capa 3 (Self-Attention + Feed-Forward)
↓
Capa 4 (Self-Attention + Feed-Forward)
↓
Capa 5 (Self-Attention + Feed-Forward)
↓
Capa 6 (Self-Attention + Feed-Forward) ← la más "profunda"
↓
Salida del Encoder
```

### Qué significa "temprana" vs "profunda"

| Término                                      | Qué significa                                | Dónde está                         |
|----------------------------------------------|----------------------------------------------|------------------------------------|
| <b>Capa temprana</b>                         | Las primeras capas por las que pasa el texto | Cerca del principio (Capa 1, 2...) |
| <b>Capa profunda</b> (o "superior", "upper") | Las últimas capas por las que pasa el texto  | Cerca del final (Capa 5, 6...)     |

Cada una de esas 6 capas tiene **su propia** Feed-Forward Network (con sus propias matrices  $W_1/W_2$ , distintas de las de las otras capas) – así que hay literalmente **6 capas Feed-Forward distintas**, una por cada nivel de profundidad, cada una con sus propios pesos entrenados por separado.

### Por qué tiene sentido que aprendan cosas distintas

La capa 1 recibe el texto **casi crudo** (solo paso por embeddings + positional encoding) – tiene sentido que sus “detectores de patrones” (los  $W_1$  de esa capa) se **enfocen en cosas simples y superficiales**: etc.

La Capa 6, en cambio, recibe el texto **ya procesado 5 veces** – con muchísima información ya combinada y refinada por las capas anteriores – así que tiene sentido que sus detectores puedan reconocer patrones mas abstractos y relacionados con el significado real (semántica), porque ya “tiene mas con que trabajar”.

### Conectándolo con algo que ya vimos en el documento

Esto es coherente con lo que mencionados en la parte de “El resultado del Encoder”: después de las 6 capas apiladas, lo que queda es una representación **enriquecida** – cada capa le va agregando una capa más de procesamiento, y el hallazgo de Geva et. Lo confirma que ese enriquecimiento tiene un patrón: de lo simple (capas tempranas) a lo complejo (capas profundas).

### Recordando el flujo completo, desde el principio

```
Texto de entrada: "The dog eats bread"
↓
FASE PREVIA: BPE ya entrenado (Paso 0)
↓
Tokenización → 5 tokens: "The", "dog", "eat", "s", "bread"
↓
Embeddings → cada token, un vector de 512 números
↓
Positional Encoding → se le suma la "posición" a cada uno de los 5 vectores
↓
===== ACÁ EMPIEZA LA CAPA 1 =====
```

### Qué pasa DENTRO de cada una de las 6 capas del diagrama

Cada “Capa” del diagrama que pegaste **no es un paso simple** – es en sí misma **todo el bloque** que ya desarmamos con lujo de detalle:

```
Capa 1:
↓
Multi-Head Attention (con sus 6 sub-pasos: copiar el vector 8 veces,
multiplicar por  $W^Q/W^K/W^V$  de cada cabeza, calcular la fórmula de
atención completa, concatenar las 8 cabezas, multiplicar por  $W^O$ )
→ esto corre EN PARALELO para los 5 tokens (recordá el recuadro 🗉)
↓
Residual + LayerNorm
↓
Feed-Forward Network (con su  $W_1$ , ReLU,  $W_2$  – también en paralelo
para los 5 tokens, cada uno por separado)
↓
Residual + LayerNorm
↓
→ salen 5 vectores de 512, "enriquecidos" una vez más
```

**Sí, confirmando tu pregunta: cada token tiene su propio Multi-Head Attention, en cada capa**  
Exactamente como lo recordás — en la Capa 1, los 5 tokens ("The", "dog", "eat", "s", "bread") pasan **todos** por su propio cálculo de Multi-Head Attention (cada uno calculando su Q, comparándolo contra los K de los 5, mezclando con los V de los 5), y **después** cada uno pasa por su propio Feed-Forward.

### Y esto se repite, otra vez completo, en la Capa 2

Acá está el punto clave que conecta con tu pregunta: los **5 vectores que salieron de la Capa 1** (ya "enriquecidos" una vez) se convierten en la **entrada** de la Capa 2 — y ahí, de nuevo, cada uno de los 5 calcula su propio Multi-Head Attention (con matrices **distintas**, propias de la Capa 2) y su propio Feed-Forward (también con matrices distintas de la Capa 2).



Salida de Capa 1 (5 vectores de 512, ya con una vuelta de "contexto")

↓

Entrada de Capa 2

↓

Multi-Head Attention de la Capa 2 (para los 5 tokens, en paralelo, con SUS PROPIAS matrices)

↓

Feed-Forward de la Capa 2 (para los 5 tokens, en paralelo, con SUS PROPIAS matrices)

↓

Salida de Capa 2 (5 vectores de 512, ya con DOS vueltas de "contexto")



## Así hasta la Capa 6

Esto se repite **6 veces en total**, cada vez con matrices distintas (recorda que el paper dice: *"the linear transformations are the same across different positions, they use different parameters from layer to layer"* – las mismas matrices para todos los tokens **dentro** de una capa, pero **distintas** matrices entre una capa y la siguiente).

## Resumen visual completo

|                                               | Dentro de UNA capa                            | Entre capas                                        |
|-----------------------------------------------|-----------------------------------------------|----------------------------------------------------|
| <b>Matrices</b> ( $W^Q, W^K, W^V, W_1, W_2$ ) | Las mismas para los 5 tokens                  | Distintas en cada una de las 6 capas               |
| <b>Que le pasa a cada token</b>               | Su propio Multi-Head Attention + Feed-Forward | Se repite el proceso completo, 6 veces             |
| <b>Resultado</b>                              | 5 vectores de 512, "enriquecidos" una vez     | Cada vuelta por una capa los enriquece un poco mas |

Así que sí: tu recuerdo es correcto — **cada token calcula su propio Multi-Head Attention** (y su propio Feed-Forward), y eso pasa **6 veces**, una vez por cada capa del diagrama que pegaste, con matrices distintas en cada vuelta.

## La aclaración honesta, con esto ya investigado

Esto **no significa** que se pueda señalar con precisión total "esta neurona representa el concepto de animales" de forma limpia y aislada – sigue siendo un patron distribuido, y la interpretación de *cada* neurona individual requiere trabajo de investigación caso por caso. Pero **si hay evidencia solida y reproducible** de que estas capas no son un misterio total: funcionan, en la practica, como un sistema de patrones-y-predicciones aprendidos, parecido a un diccionario gigante que el modelo construyo durante el entrenamiento – no es solo "números que camiban sin ningún sentido reconocible".

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Es solo dos transformaciones lineales con una función de activación **ReLU** en el medio ( $\max(0, \dots)$  – convierte todo lo negativo en cero). Esto ocurre **tanto en el Encoder como en el Decoder** – cada uno tiene su propia capa Feed-Forward, con sus propios pesos  $W_1, W_2$ , entrenados por separado.



**Ejemplo con 2 tokens, para ver que las matrices son las mismas pero el resultado cambia.** Usando el paper real, la entrada tiene  $d_{model}=512$  y la capa interna  $d_{ff}=2048$  – para poder calcular a mano, achiquemos a un ejemplo de juguete: 4 numeros de entrada, 6 internos.

**Verificado contra el paper** (Sección 3.3, cita textual): *”The dimensionality of input and output is  $d_{model}=512$ , and the inner-layer has dimensionality  $d_{ff}=2048$ ”*. \* Que es cada lietra, su tamaño (juguete vs real, y si es un peso entrenado con backpropagation:

| Símbolo                       | Que es                                                                                                                               | Por qué esta ahí (su función)                                                                                                                                                                                                        | ¿Peso entrenado?                                             | Forma matemática                                        | Nuestro ejemplo (juguete) | Tamaño real del paper   |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------|---------------------------|-------------------------|
| $x$                           | El vector de entrada del token (viene de Residual+LayerNorm , ya aplicado sobre la salida de Multi-Head Attention)                   | Es la información que hay que procesar – el punto de partida de todo el calculo                                                                                                                                                      | No – es el dato que “pasa por” la red, no algo que se ajuste | <b>Vector</b> (una sola fila de números)                | 4 numeros                 | $d_{model}=512$ numeros |
| $W_1$                         | <b>Primera matriz de pesos</b> – expande el vector                                                                                   | Le da al modelo “más espacio” para combinar la información de $x$ de formas nuevas, antes de decidir que es relevante                                                                                                                | <b>Si</b> – arranca al azar, se ajusta con backpropagation   | <b>Matriz</b> (filas x columnas)                        | 4x6                       | 512 x 2048              |
| $b_1$                         | Primer sesgo (bias) – se suma, no se multiplica                                                                                      | Le da flexibilidad al resultado: sin esto, si $x$ fuera puros ceros, la salida también seria siempre cero, sin importar que tan “buena” sea $W_1$                                                                                    | <b>Si</b> – igual que $W_1$ , se aprende                     | <b>Vector</b> (una sola fila de números, no una matriz) | 6 numeros                 | 2048 numeros            |
| $\text{Max}(0, .)$            | <b>Aca esa ReLU</b> – no es un peso ni un vector propio, es una <i>operación</i> que se aplica numero por numero al vector de arriba | Descarta (vuelve 0) las combinaciones que dieron negativo, quedándose solo con los patrones que resultaron “útiles” – sin este paso, apilar $W_1$ y $W_2$ seria matematicament e equivalente a una sola matriz, y no se ganaría nada | <b>No</b> – es una función, no un dato ni un peso            | No tiene forma propia, opera sobre el vector existente  | -                         | -                       |
| (resultado , después de ReLU) | El mismo vector de $x$ $W_1+b_1$ , pero con los negativos ya vueltos 0                                                               | Es lo que realmente entra a la segunda transformación (                                                                                                                                                                              | No – es un resultado de calculo                              | <b>Vector</b> (mismo tamaño que antes                   | 6 numeros                 | $d_{ff}=2048$ numeros   |

|          |                                                               | $W_2$ )                                                                                                                        |                                                  | de ReLU)                                           |           |             |
|----------|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|----------------------------------------------------|-----------|-------------|
| $W_2$    | Segunda matriz de pesos – achica de vuelta al tamaño original | Combina lo que sobrevivió al ReLU y lo “comprime” de nuevo al tamaño que necesita el resto del modelo para seguir funcionando  | <b>Si</b> – se aprende, independiente de $W_1$   | <b>Matriz</b>                                      | 6x4       | 2048 x 512  |
| $b_2$    | Segundo sesgo                                                 | Mismo motivo que $b_1$ : le da un margen de ajuste extra al resultado final                                                    | <b>Si</b> – se aprende                           | <b>Vector</b>                                      | 4 numeros | 512 numeros |
| $FFN(x)$ | La salida final                                               | El nuevo vector del token, ya “procesado” por esta capa – sigue camino hacia Residual + LayerNorm y de ahí al resto del modelo | <b>No</b> – es el resultado del calculo completo | <b>Vector</b> (mismo formato que x, no una matriz) | 4 numeros | 512 numeros |

Un punto importante sobre la forma: solo  $W_1$  y  $W_2$  son **matrices** (tablas de filas y columnas) – todo lo demás ( $x$ ,  $b_1$ ,  $b_2$ , los vectores de antes/despues de ReLU, y  $FFN(x)$ ) es un **vector** (una sola fila de números, sin “ancho” adicional). Nunca, aparece, en este calculo, algo del estilo “512x512” para la salida: la salida  $FFN(x)$  siempre es un vector de una sola dimensión, del mismo tamaño que  $x$  ( $d_{model}=512$ , o 4 en nuestro ejemplo) – no una tabla cuadrada como si lo era  $W^O$  en Multi-Head Attention.

Los 4 pesos entrenados ( $W_1$ ,  $b_1$ ,  $W_2$ ,  $b_2$ ) siguen el mismo mecanismo que ya vimos con los embeddings y con  $W^Q/W^k/W^v$ : arrancan con números al azar cuando se crea el modelo, y se ajustan millones de veces durante el entrenamiento (backpropagation) hasta queda fijos – ver el recuadro 🧠 del Paso 1, más arriba. Y como esas matrices son **las mismas** para cualquier token que pase por esta capa, cada token entra con su propio vector  $x$  distinto y sale con su propio  $FFN(x)$  distinta – el mismo patron que ya vimos con  $W^Q/W^k/W^v$  en Multi-Head Attention.

Y, siguiendo el diagrama de la sección anterior: el resultado de Feed-Forward (nuestro  $Sublayer_2(y)$ ) tampoco pasa directo a la siguiente capa — primero se le aplica la **segunda** Residual+LayerNorm de esta capa,  $LayerNorm(y + Sublayer_2(y))$ , con la misma mecánica que ya vimos (sumar con el vector que entró a Feed-Forward, y normalizar). Recién ese resultado es lo que sale de la capa completa.

entonces, para las matrices entrenadas, son diferentes por cada capa, si, pero son las mismas por cada token? es decir, “dog” se procesa por  $W_1$  y ese mismo  $w_1$  procesa el “the”? breve aca en el chat

Sí, exacto — confirmado:

- **Entre capas:**  $W_1/W_2$  son **distintas** en cada una de las 6 capas (cada capa entrena las suyas)
- **Dentro de una misma capa:**  $W_1/W_2$  son **las mismas** para todos los tokens – "dog", "The", "eat", "s" y "bread" pasan todos por la **misma**  $W_1$  de esa capa, solo que cada uno con su propio vector de entrada, dando resultados distintos.

Es el mismo patrón que ya vimos con  $W^Q W^K W^V$  en Multi-Head Attention: la matriz es una "receta" fija por capa, aplicada igual a cada token por separado.

**1.12.5.4 Paso 6 (capa 1/6) y así hasta la capa 6/6 (son 6 iteraciones): Residual connections + Layer #2**  
**((se aplica despues de Feed-Forward-Network))**

$x$  (entrada del token a esta capa)

↓

Multi-Head Attention  $\rightarrow$   $\text{Sublayer}_1(x)$

↓

Residual + LayerNorm #1:  $\text{LayerNorm}(x + \text{Sublayer}_1(x)) \rightarrow$  llamemos a esto  
" $y$ "  $\leftarrow$  ACÁ ESTAMOS

↓

Feed-Forward Network  $\rightarrow$   $\text{Sublayer}_2(y)$

$\leftarrow$  esto sigue, en la próxima sección

↓

Residual + LayerNorm #2:  $\text{LayerNorm}(y + \text{Sublayer}_2(y))$

↓

Esto es lo que sale de esta capa completa, y entra a la siguiente

### 1.12.5.5 El resultado del Encoder: Una representación completa de la oración en ingles

Despues de pasar por las **6 capas apiladas** del Encoder – cada una repitiendo el ciclo completo que acabamos de ver (Multi-Head Attention -> Residual + LayerNorm #1 -> Feed-Forward -> Residual + LayerNorm #2), con matrices distintas en cada una de las 6 vueltas – lo que queda no es texto ni predicciones: es un conjunto de vectores de 512 numeros, uno por cada palabra de “The Dog eats bread”. Especificamente, es la salida de **Residual+layerNorm#2 de la sexta y ultima capa** – no la salida cruda de Feed-Forward, sino ya con esa ultima normalización aplicada.

| Token   | Vector final (despues de las 6 capas)                                                              | Que “sabe” este vector                                                                                    |
|---------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| “The”   | [512 numeros]                                                                                      | Su propio significado (casi nulo, es gramatical) + psosicion 0 + 6 vueltas de contexto de toda la oración |
| “dog”   | [512 numeros – partiendo de $x(\text{dog}) = [1.5415, 0.7403, 0.6696, 1.0129, \dots]$ en la Capa 1 | Su significado (“perro” + posición 1 + 6 vueltas de contexto, cada vez mas mezclado con “eat” y “bread”   |
| “eat”   | [512 numeros] – partiendo del ejemplo que calculamos con 32%/30%/28%/14% en la Capa 1              | Su significado (“comer”) + posición 2 + 6 vueltas de contexto, reforzando su relación con “dog” y “bread” |
| “s”     | [512 numeros]                                                                                      | El sufijo verbal + posición 3 + 6 vueltas de contexto                                                     |
| “bread” | [512 numeros]                                                                                      | Su significado (“pan”) + posición 4 + 6 vueltas de contexto, reforzando su relación con “eat”             |

Cada uno de esos 5 vectores está **enriquecido** con información de todo el resto de la oración, acumulada a lo largo de las 6 vueltas — el de “dog” en la Capa 6 ya no es el mismo  $[1.5415, 0.7403, 0.6696, 1.0129, \dots]$  con el que arrancó en la Capa 1: pasó por 6 rondas de mezclarse con sus vecinos y transformarse con Feed-Forward, así que sus 512 números son, en la práctica, completamente distintos a los de partida — pero le seguimos llamando “el vector de dog” porque sigue siendo, en esa posición, la representación de esa palabra.

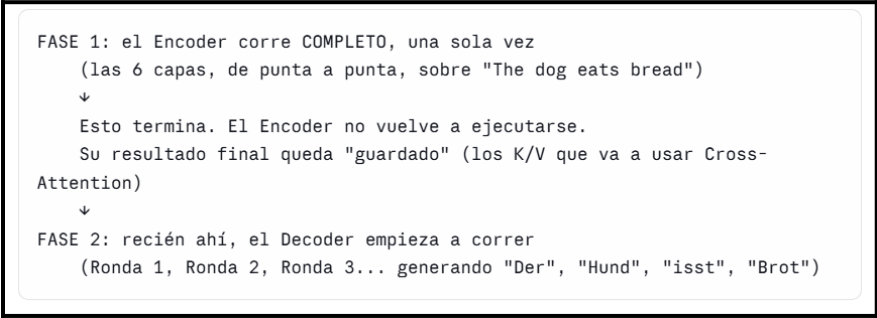
Esta representación **no se descarta** – queda “guardada” y disponible para que el Decoder la consulte, palabra por palabra, mientras genera la traducción. Ahí es donde entra el mecanismo que faltaba explica: **Cross-Attention**.

1.12.6: El Decoder: generando la traducción, palabra por palabra

**Su finalidad:** Mientras el Encoder se dedico a *leer y entender* la oración de origen (“The dog eats bread”), el **Decoder se dedica a generar la traducción**, en el idioma de destino, palabra por palabra – usando lo que el Encoder ya entendio como referencia constante en cada paso. Es la mitad del modelo que efectivamente “escribe” la salida.

Encoder y Decoder NO corren en paralelo entre sí

Antes de entrar al Decoder en detalle, vale la pena aclarar algo que se presta a confusión: el Encoder y el Decoder **no se ejecutan al mismo tiempo** — corren en dos fases separadas, una después de la otra:



Esto no contradice el paralelismo que ya vimos en los recuadros  – ese paralelismo es **entre los tokens de una misma oración** (los 5 tokens de “The dog eats bread” procesándose juntos dentro de Encoder), no entre el Encoder y el Decoder como bloques. Son dos ideas distintas:

| Tipo de paralelismo                                                                                                            | Que corre al mismo tiempo                                                                              | ¿Aplica acá?                                                    |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Entre tokens (recuadros  anteriores)</b> | Los 5 tokens de la oración, procesándose juntos dentro del Encoder (o dentro de una ronda del Decoder) | Sí, dentro de cada uno pro separado                             |
| <b>Entre Encoder y Decoder</b>                                                                                                 | -                                                                                                      | <b>No</b> – el Encoder termina antes de que el Decoder arranque |

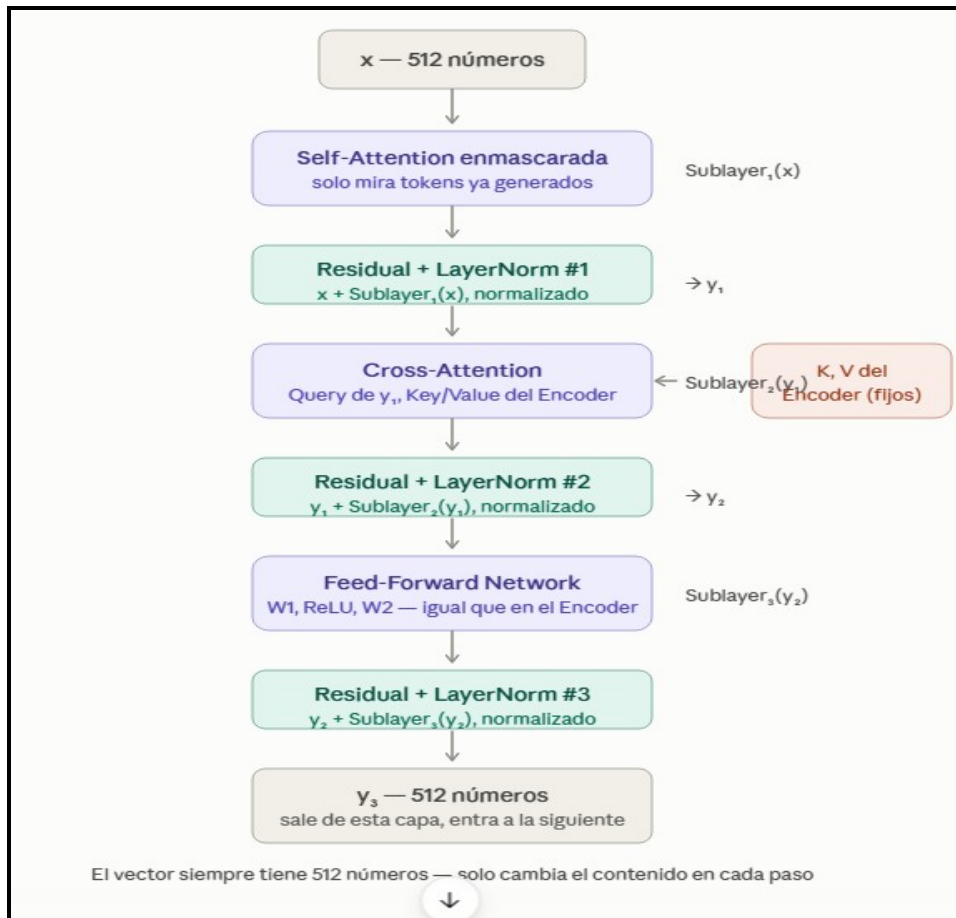
Por qué el Encoder no necesita volver a correr en cada ronda: la oración en ingles no cambia mientras el Decoder va generando la traducción, así que no tiene sentido volver a procesarla – es justo lo que vimos en “Generando la traducción completa”: *“la representación del Encoder se calcula una sola vez, al principio, y se reutiliza en las 5 rondas”*.

La estructura del Decoder: una sub-capa extra respecto al Encoder

El **Decoder** también tiene 6 capas apiladas, pero cada una tiene **una capa de atención extra**, que no existía en el Encoder – y por lo tanto, también **una aplicación extra de Residual + LayerNorm**.

|                                                    | Encoder                                                                | Decoder                                                                                                             |
|----------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Cuantas capas</b>                               | 6                                                                      | 6                                                                                                                   |
| <b>Que hace</b>                                    | Lee toda la entrada y arma una representación                          | Genera la salida, palabra por palabra                                                                               |
| <b>Sub-capas de atención</b>                       | Solo self-attention (1 sub-capa de atención)                           | Self-attention <b>enmascarada</b> + <b>cross-Attention</b> (2 sub-capas de atención)                                |
| <b>Aplicaciones de Resudial+LayerNorm por capa</b> | <b>2</b> (una despues de Self-Attention, otra despues de Feed-Forward) | <b>3</b> (una despues de Self-Attention enmascarada, otra despues de Cross-Attention, otra despues de Feed-Forward) |

Es decir: el ciclo dentro de una capa del Decoder es mas largo que el de Encoder – Self-Attention enmascarada -> Residual+LayerNorm -> Cross-Attention -> Residual+LayerNorm -> Feed-Foward -> Residual+LayerNorm – con la misma mecánica de “atajo + normalizar” que ya vimos, aplicada una vez mas por la sub-capa extra.



Ese sería el flujo completo de **una capa del Decoder**, con las 3 sub-capas.

Fijate el patrón que se repite 3 veces: **sub-cap** -> **Residual+LayerNorm**. Nunca hay dos-capas seguidas sin normalización en el medio:

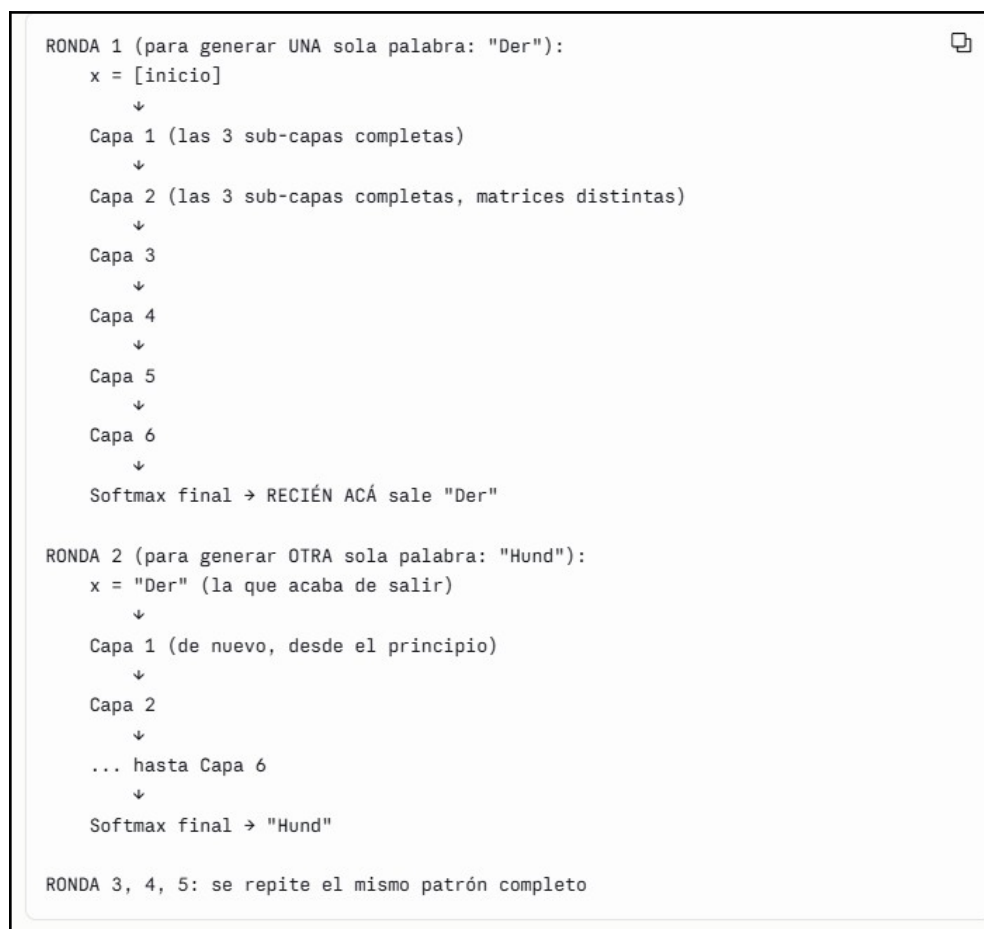
1. **Self-Attention enmascarada** (solo mira tokens ya generados) -> produce  $Sublayer1(x)$  -> se combina con  $x$  y se normaliza -> sale  $y_1$
2. **Cross-Attention** (El Query sale de  $y_1$ , pero los Key/Value vienen del Encoder, ya calculados y fijos) -> produce  $Sublayer2(y_1)$  -> se combina con  $y_1$  y se normaliza -> sale  $y_2$
3. **Feed-Forward** (la misma red que en el Encoder) -> produce  $Sublayer3(y_2)$  -> se combina con  $y_2$  y se normaliza -> sale  $y_3$ .

El vector **nunca cambia de tamaño** – siempre son 512 números – pero **si cambia el contenido**: arranca siendo solo el token aislado, y en cada paso incorpora mas contexto (primero de si mismo, después de la oración en inglés via Cross-Attention, después de transformar con Feed-Forward).

### 1.12.10.0: Sobre las capas del Decoder (6 capas en el paper)

Las capas son pasos de **procesamiento interno** – transforman un vector de 512 numeros en otro vector de 512 numeros, nada mas. La traducción (el texto real, "Der", "Hund", etc.) solo aparece **después** de la Capa 6, cuando el resultado pasa por el softmax final.

**El orden real completo, con los dos niveles que ya vimos (Ronda y Capa)**



1. **Dentro de una ronda:** las 6 capas se recorren **una sola vez, en secuencia**, y **ninguna genera nada por si sola** – solo al final de la Capa 6 aparece **una única palabra**.
2. **Entre rondas:** Cada ronda completa (Las 6 capas + softmax) genera **una sola palabra**, no la traducción entera – hacen falta **5 rondas completas** (cada una con sus propias 6 capas) para tener las 6 palabras de “Der Hund isst Brot [fin]”

### Tabla resumen para que quede grabado

|                                        | Que produce                                        |
|----------------------------------------|----------------------------------------------------|
| Una sola capa (por ejemplo, la Capa 3) | Un vector de 512 numeros – nada legible como texto |
| Las 6 capas juntas, una ronda completa | Una sola palabra (por ejemplo, “Der”)              |
| Las 5 rondas completas                 | La traducción entera (“Der Hund isst Brot”)        |



no entiendo, que es la capa 2, se le llama capa 2 a la entrada del input de "Der"

"Capa 2" es una ESTRUCTURA FIJA del modelo — no es "el momento en que entra Der"

La Capa 2 **siempre existe**, esté procesando lo que esté procesando — es un bloque de código, **con sus propias matrices**, que está ahí, fijo, sea cual sea el dato que le entre en un momento dado.

La confusión concreta: "Capa 2" vs "Ronda 2"

Creo que el problema es que estás mezclando "Capa" con "Ronda" — son ejes completamente distintos, y por eso "Capa 2 = entrada de Der" suena razonable, pero no es así.

| ¿Qué significa el número? |                                                                                                                                 |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Ronda 2                   | La <b>segunda vez</b> que arrancás desde cero con un token nuevo (en este caso, "Der" es la entrada de la Ronda 2)              |
| Capa 2                    | La <b>segunda estación de la fábrica</b> , dentro de <b>cualquier</b> ronda — no tiene nada que ver con qué token está entrando |

Viéndolo con la fábrica de nuevo, más despacio

```
RONDA 1 - materia prima: [inicio]
  Capa 1: procesa [inicio]
  Capa 2: procesa lo que salió de la Capa 1 (ya no es "[inicio] puro", es un vector transformado)
  Capa 3: procesa lo que salió de la Capa 2
  Capa 4, 5, 6: ídem
  → sale "Der"

RONDA 2 - materia prima: "Der"
  Capa 1: procesa "Der"      ← acá SÍ "Capa 1" recibe a "Der", pero es la RONDA 2
  Capa 2: procesa lo que salió de la Capa 1 de ESTA ronda
  Capa 3, 4, 5, 6: ídem
  → sale "Hund"
```

La corrección exacta de tu frase

Vos decías: "se le llama Capa 2 a la entrada del input de Der" — la corrección es: **"Der" es la entrada de la Capa 1 de la Ronda 2**" — no de la Capa 2. La Capa 2 (de la Ronda 2) recibe lo que salió de la Capa 1 (de esa misma Ronda 2) — nunca recibe "Der" directamente, salvo indirectamente, ya transformado.

Resumen, con la tabla que separa los dos ejes

|         | Qué entra en la Capa 1 | Qué entra en la Capa 2                                          |
|---------|------------------------|-----------------------------------------------------------------|
| Ronda 1 | [inicio]               | Lo que salió de la Capa 1 de la Ronda 1 (no "[inicio]" directo) |
| Ronda 2 | "Der"                  | Lo que salió de la Capa 1 de la Ronda 2 (no "Der" directo)      |
| Ronda 3 | "Hund"                 | Lo que salió de la Capa 1 de la Ronda 3                         |

vamos con el detalle completo, mostrando qué entra y qué sale de **cada** capa, con nombres distintos para cada resultado intermedio, así no se confunden entre sí.

## RONDA 1 (genera "Der")

x = [inicio] (512 números: embedding + positional encoding, posición 0)

↓

### CAPA 1

```
| Self-Attention enmascarada (con W^Q/K/V_1) |
| ↓ Sublayer1(x) |
| Residual + LayerNorm #1 → y1 |
| ↓ |
| Cross-Attention (con W^Q/K/V_cross de Capa 1) |
| ↓ Sublayer2(y1) |
| Residual + LayerNorm #2 → y2 |
| ↓ |
| Feed-Forward (con W1/W2 de Capa 1) |
| ↓ Sublayer3(y2) |
| Residual + LayerNorm #3 → y3 = SALIDA_CAPA1 |
```

↓

SALIDA\_CAPA1 (512 números) se convierte en la entrada de la Capa 2

### CAPA 2

```
| Entrada: SALIDA_CAPA1 (NO es [inicio], ya está transformado) |
| Self-Attention enmascarada (con W^Q/K/V_2, MATRICES DISTINTAS de la Capa 1) |
| ↓ ... mismo proceso de 3 sub-capas ... |
| Residual + LayerNorm #3 → SALIDA_CAPA2 |
```

↓

SALIDA\_CAPA2 se convierte en la entrada de la Capa 3

### CAPA 3

```
| Entrada: SALIDA_CAPA2 |
| ... mismo proceso, con matrices de la Capa 3 ... |
| → SALIDA_CAPA3 |
```

↓

### CAPA 4

```
| Entrada: SALIDA_CAPA3 → ... → SALIDA_CAPA4 |
```

↓

### CAPA 5

```
| Entrada: SALIDA_CAPA4 → ... → SALIDA_CAPA5 |
```

↓

### CAPA 6

```
| Entrada: SALIDA_CAPA5 → ... → SALIDA_CAPA6 |
```

↓

SALIDA\_CAPA6 (512 números) pasa por la capa lineal + softmax

↓

Se elige el Token ID más probable → 5001

↓

Se busca en el vocabulario alemán → "Der"

## RONDA 2 (genera "Hund")

`x = "Der"` (512 números: embedding + positional encoding de "Der" – DISTINTO al [inicio] de la Ronda 1, pero es la MISMA operación de "convertir texto a vector" que ya vimos)

↓

┌──────────────────┐ CAPA 1 (de la Ronda 2 – vuelve a arrancar desde acá)

└──────────────────┘

| Entrada: `x = "Der"` (NO es SALIDA\_CAPA1 de la Ronda 1 – es un dato nuevo)

|

| Self-Attention enmascarada – usa las MISMAS matrices  $W^Q/K/V_1$  que en la Ronda 1

|

| (misma matriz, pero la entrada "Der" es distinta a "[inicio]", así que el resultado difiere)|

|    ↓ ... 3 sub-capas ...

|

| Residual + LayerNorm #3 → SALIDA\_CAPA1 (de la Ronda 2 – un vector NUEVO, distinto al de antes)|

└──────────────────┘

↓

(se repite exactamente la misma secuencia: Capa 2, 3, 4, 5, 6 – con las mismas matrices de cada capa, pero procesando datos nuevos porque la entrada de esta ronda es distinta)

↓

SALIDA\_CAPA6 (de la Ronda 2) → softmax → Token ID 5002 → "Hund"

## RONDAS 3, 4, 5: exactamente el mismo patrón

Cada una arranca con `x` = la ultima palabra generada, atraviesa las 6 capas **desde cero** (Capa 1 a Capa 6, siempre en ese orden, siempre con las mismas 6 matrices por cada – las de la Capa 1 son iguales en todas las rondas, las de la Capa 6 son iguales en todas las rondas, etc), y termina en un softmax que da una palabra nueva.

## El punto que quiero que quede grabado

**Las matrices de cada capa NO cambian entre rondas** – la Capa 1 usa siempre las mismas  $W_1^Q/W_1^K/W_1^V$ , sea la Ronda 1,2,3,4 o 5. Lo que cambia entre rondas es **el dato que entra** (`x`: primero [inicio], después "Der", después "Hund"... ) – por eso el resultado final es distinto en cada ronda, aunque el "camino" (las 6 capas, con sus matrices fijas) sea siempre el mismo recorrido.

a, psea que es como lo mismo que el Encoder

Sí, exacto — es **exactamente el mismo patrón** que ya vimos con el Encoder. Buena conexión.

## El paralelo directo

|                                                  | Encoder                                                       | Decoder                                                                                                               |
|--------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Cuántas capas                                    | 6                                                             | 6                                                                                                                     |
| ¿Las matrices cambian entre capas?               | Sí — cada capa tiene las suyas                                | Sí — cada capa tiene las suyas                                                                                        |
| ¿El dato entra completo a cada capa, o se corta? | Completo — "dog" pasa entero por las 6 capas, transformándose | Completo — <span style="border: 1px solid red; padding: 2px;">[inicio]</span> /"Der"/etc. pasa entero por las 6 capas |
| ¿Cuántas veces corre esto?                       | <b>Una sola vez</b> — toda la oración de una pasada           | <b>Una vez por ronda</b> — 5 veces en total (una por palabra generada)                                                |

|                                                         | Encoder                                                       | Decoder                                                                |
|---------------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Cuántas capas</b>                                    | 6                                                             | 6                                                                      |
| <b>¿Las matrices cambian entre capas?</b>               | Si – cada capa tiene las suyas                                | Si – cada capa tiene las suyas                                         |
| <b>¿El dato entra completo a cada capa, o se corta?</b> | Completo — "dog" pasa entero por las 6 capas, transformándose | Completo — [inicio]/"Der"/etc. pasa entero por las 6 capas             |
| <b>¿Cunatas veces corre esto?</b>                       | <b>Una sola vez</b> – toda la oración de una pasada           | <b>Una vez por ronda</b> – 5 veces en total (una por palabra generada) |

### La única diferencia real entre ambos

El Encoder corre **una sola vez, para siempre** – porque la oración en ingles no cambia. El Decoder tiene que repetir ese mismo recorrido de 6 capas **una vez por cada palabra que genera** – porque cada nueva palabra depende de las anteriores, y no existe todavía cuando arranca la ronda.

### Resumen

Si – el mecanismo interno de “una capa completa, con sus matrices fijas, procesando un vector que se va transformando capa a capa” es **idéntico** en el Encoder y en el Decoder. Lo único que cambia es **cuántas veces** se recorre ese camino:

El Encoder lo recorre 1 vez; El decoder lo recorre 5 veces (una por ronda), porque tiene que generar la traducción de a una palabra por vez.

vale, y obviamente cada capa desde el 1 hasta el 6 aprenden mas relaciones complejas, etc

Sí, exacto — mismo patrón que ya confirmamos con el paper de Geva et al. para Feed-Forward, aplicado ahora al conjunto completo de las 6 capas.

### Confirmando la idea general

Las **capas tempranas** (1, 2) tienden a captar patrones más **superficiales** (relaciones sintácticas simples, posición, terminaciones de palabras), y las **capas profundas** (5, 6) tienden a captar patrones más **abstractos y semánticos** (significado, relaciones de sentido a más largo alcance).

### **Un matiz importante para no sobre-generalizar**

Esto es lo que confirmamos como **tendencia observada** en investigaciones de Interpretabilidad (como el paper de Geva et al. que ya vimos) – no es una regla matemática exacta ni algo que el paper original de 2017 defina explícitamente. Es un patrón que se **observa** al entrenar modelos y analizar que activa cada capa, no algo programado a propósito capa por capa.

### **Por qué tiene sentido, de todas formas**

Cada capa recibe como entrada lo que ya procesaron los anteriores – así que la Capa 6 está trabajando sobre la información ya “resumida y refinada” 5 veces, mientras que la Capa 1 trabaja sobre el dato más crudo posible (embedding + positional encoding, casi sin procesar). Tiene sentido que, con más “vueltas” de contexto acumulado, las capas finales puedan construir relaciones más ricas.

### **Resumen**

Sí, correcto — es la misma idea de progresión (superficial → semántico) que ya vimos con Feed-Forward en el Encoder, aplicable también al Decoder: capas tempranas más simples, capas profundas más abstractas — como tendencia observada empíricamente, no como regla fija del diseño.

### 1.12.10.1: Self-Attention enmascarada: el detalle del "masking"

**El valor de entrada:** es el valor  $x$  de token que se esta procesando en esta ronda – el mismo  $x$  que ya vimos en el flujo general, de 512 numeros.

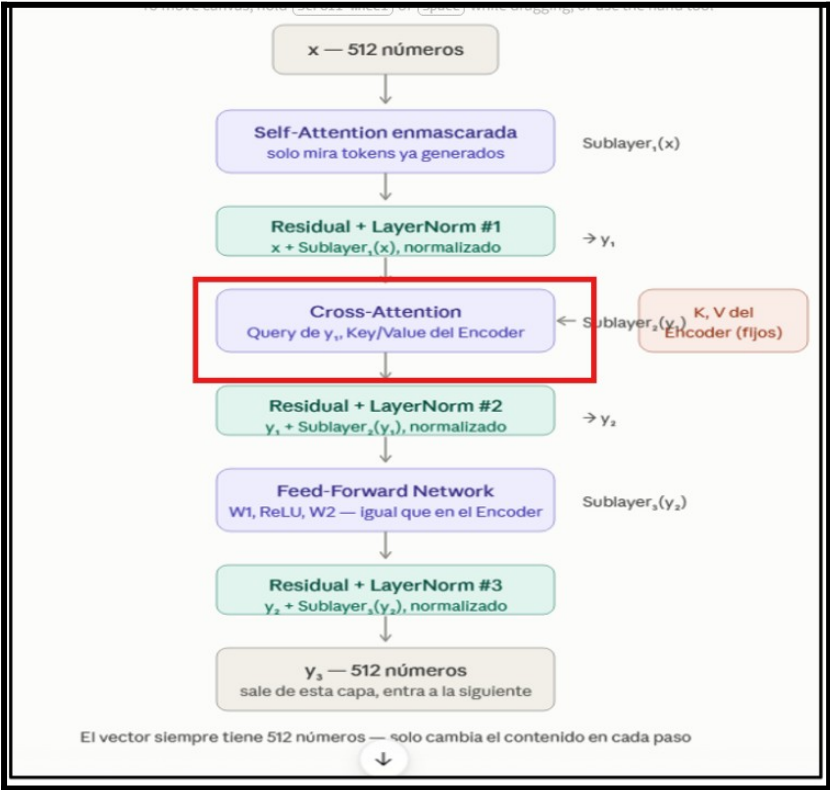
Por ejemplo, en la Ronda 3 (donde ya se generaron “Der” en la posición 0 y “Hund” en la posición 1, y toca decidir la posición 2),  $x$  es el vector de “**Hund**” – el ultimo token generado (ronda 2), que es el **que “hace la pregunta” (el Query) para ayudar a decidir que sigue.**

Pero a diferencia del Encoder, acá “entrada” no significa solo ese único token: **Self-Attention enmascarada también** necesita los Keys y Values de **todos los tokens ya generados hasta ese momento** (en este caso, tanto “Der” como “Hund”), porque el token actual tiene que comparar su Query contra ellos – igual mecánica que ya vimos en el Encoder, pero solo con la parte de la oración alemana que ya existe.

Dentro del Decoder, cuando esta generando la palabra en la posición 2 (por ejemplo, “isst”), **no puede ver** las palabras que va a generar despues (posiciones 3 y 4, es decir “Brot” y lo que venga tras ella) – porque todavía no existen. El paper resuelve esto poniendo  $-\infty$  en esas posiciones antes del softmax, así el modelo les asigna 0% de atención – eso es lo que hace que el modelo sea **autoregresivo** (genera una palabra, la usa para **predecir la siguiente, y así sucesivamente** – por ejemplo, si ya genero “Hund”, ese token se convierte en el Query de la siguiente ronda, comparándose contra los Keys de todo lo ya generado [“Der”, “Hund”] para predecir “isst”, y así en cada ronda siguiente).

🕒 ¿B c o r r e   d e   n u e v o   c a d a   v e z   q u e   e l   D e c o d e r  
?  
N - B P E ,  
e n   l a   F a s e   P r e v i a   ( s e   e n t r e n a   u n a  
,   )   .  
“   ”  
  
(   )  
,  
  
(   .   )   .  
  
/  
e s t e   e j e m p l o   y a   t i e n e n   s u   T o k e n   I D ,  
  
-  
  
“

1.12.10.2: Cross-Attention: como el Decoder consulta al Encoder



Cross-Attention es la sub-capa extra que no existe en el Encoder. Funciona **casi igual** que el Self-Attention que ya vimos, con una diferencia clave:

|                                   | Self-Attention (dentro del Encoder o del Decoder)                                                                                                     | Cross-Attention (conecta Decoder -> Encode )                                                                                                                             |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| De donde sale el <b>Query</b>     | De la misma capa donde se esta calculando – la palabra “pregunta” sobre si misma y sus vecinas, dentro de la misma secuencia                          | De la capa anterior del <b>Decoder</b> – el ultimo token alemán generado (o ya procesado por su propia Self-Attention enmascarada) es el que “pregunta”                  |
| De dónde salen <b>Key y Value</b> | De la misma secuencia que el Query (mismo idioma): en el Encoder, de las demás palabras en ingles; en el Decoder, de los tokens alemanes ya generados | De la salida ya calculada del <b>Encoder</b> (ingles) – los K/V finales de “The”, “dog”, “eats”, “bread”, tras las 6 capas del Encoder, fijos durante toda la generación |
| Que “pregunta” resuelve           | “¿Qué tan relevante es cada palabra de mi propia oración/idioma para entenderme mejor”?                                                               | “¿Qué necesito de la oración original en ingles para elegir bien mi próxima palabra en alemán?”                                                                          |
| Matrices que usa                  | Sus propias $W^Q/W^k/W^v$ (8 cabezas, 512 x 64 cada una) – distintas entre Self-Attention del Encoder y del Decoder                                   | Sus propias $W^Q/W^k/W^v$ – un tercer juego de 24 matrices, distintos de los otros dos                                                                                   |

Cuando el Decoder esta generando una palabra en alemán, si query pregunta “**¿Qué necesito de la oración original en ingles para elegir bien mi próxima palabra?**” – y busca la respuesta en los Keys y Values que armo el Encoder a partir de “The dog eats bread”, no en su propio idioma.

### 1.12.10.3: El flujo completo dentro de una capa del Decoder 1/2

Juntando las 3 sub-capas (Self-Attention enmascarada, Cross-Attention, Feed-Forward), cada una seguida de su propio Residual+LayerNorm:

X, acá, no es un token fijo – es **el ultimo token ya generado**, distinto en cada ronda: En la Ronda 1 es un marcado de “inicio” (todavía no se genero nada), en la Ronda 2 es “Der” lo que salió de la Ronda 1), en la Ronda 3 es “Hund”, y así sucesivamente. Este flujo de una capa se repite tanto **6 veces** (una por cada capa del Decoder) como **una vez por ronda de generación**.

```
x (512 números – entrada del token a esta capa)
↓
Self-Attention enmascarada (solo mira tokens ya generados) →
Sublayer1(x)
↓
Residual + LayerNorm #1:  $y_1 = \text{LayerNorm}(x + \text{Sublayer}_1(x))$ 
↓
Cross-Attention (Query de  $y_1$ , Key/Value del Encoder, ya fijos) →
Sublayer2( $y_1$ )
↓
Residual + LayerNorm #2:  $y_2 = \text{LayerNorm}(y_1 + \text{Sublayer}_2(y_1))$ 
↓
Feed-Forward Network (mismo mecanismo que en el Encoder) →
Sublayer3( $y_2$ )
↓
Residual + LayerNorm #3:  $y_3 = \text{LayerNorm}(y_2 + \text{Sublayer}_3(y_2))$ 
↓
 $y_3$  (512 números – sale de esta capa, entra a la siguiente)
```

El vector **siempre tiene 512 numeros**, en cada uno de los pasos – nunca cambia de tamaño. Lo que cambia es **el contenido**:

arranca siendo solo el token aislado, incorpora contexto de si mismo (Self-Attention),

despues contexto de la oración en ingles (Cross-Attention),

y se transforma una vez mas (Feed-Forward) – con una normalización despues de cada una de las 3 sub-capas, **para estabilizar el resultado antes de seguir**.



#### 1.12.10.4: Flujo completo [inicio: La Ronda 1, generando “Der” 2/2

Antes de arrancar, el mapa completo de la Ronda 1 — para no perder de vista dónde encaja cada pieza:

```
x = [inicio] (512 números)
↓
Capa 1 → Capa 2 → Capa 3 → Capa 4 → Capa 5 → Capa 6 (las 6, dentro
de ESTA ronda)
↓
Capa lineal + softmax → probabilidad para CADA token del vocabulario
alemán
↓
Se elige el más probable → Token ID 5001
↓
Se busca en vocab.json → "Der"
```

Ninguna capa intermedia “predice texto” — todas son pasos que transforman el vector de 512 números; la predicción ocurre **una sola vez**, después de la Capa 6, en el softmax. Y una vez que sale “Der”, para que sirva de entrada a la Ronda 2 hay que devolverlo a vector: Token ID → Embedding → + Positional Encoding — el mismo camino de ida que ya vimos para la entrada original, aplicado ahora a la salida.

El Decoder todavía no genera nada, así que  $x$  es el vector de un token especial de “inicio” (llamémoslo **[inicio]**). El objetivo de esta ronda es que el Decoder decida que la primera palabra en alemán es “Der”.

⚠ **[inicio] nunca esta “solo”**: por sí mismo, ese vector casi vacío no tendría ninguna información para predecir nada. Pero en **cada una** de las 6 capas, Cross-Attention le da acceso a los Keys y Values de **toda la oración en inglés** (“The dog eats bread”, ya procesada por el Encoder) — es literalmente **[inicio] “preguntándole” a esa oración completa que palabra corresponde generar primero.**

Sigamos la **Capa 1** de esta ronda, en detalle:

```
x (512 números — entrada del token a esta capa)
↓
Self-Attention enmascarada (solo mira tokens ya generados) →
Sublayer1(x)
↓
Residual + LayerNorm #1:  $y_1 = \text{LayerNorm}(x + \text{Sublayer}_1(x))$ 
↓
Cross-Attention (Query de  $y_1$ , Key/Value del Encoder, ya fijos) →
Sublayer2( $y_1$ )
↓
Residual + LayerNorm #2:  $y_2 = \text{LayerNorm}(y_1 + \text{Sublayer}_2(y_1))$ 
↓
Feed-Forward Network (mismo mecanismo que en el Encoder) →
Sublayer3( $y_2$ )
↓
Residual + LayerNorm #3:  $y_3 = \text{LayerNorm}(y_2 + \text{Sublayer}_3(y_2))$ 
↓
 $y_3$  (512 números — sale de esta capa, entra a la siguiente)
```

#### 1.12.10.4.1: X – entrada a la Capa 1:

$x$  = vector de [inicio] (512 números, embedding + positional encoding, posición 0)

#### 1.12.10.4.2: Self-Attention masking (para coherencia interna de la propia oración alemana que se esta construyendo)

breve, cual era el objetivo de : "Self-Attention masking" en sus 5 rondas

El objetivo es que cada palabra alemana, al generarse, **solo pueda mirar hacia atrás** (a si misma y las palabras ya generadas antes), nunca hacia adelante – así el modelo entrena y genera de forma **autoregresiva**, consistente con que en la practica no existe ninguna palabra futura todavía.

En las 5 rondas, esto se traduce en:

| Ronda | Qué puede ver Self-Attention enmascarada |
|-------|------------------------------------------|
| 1     | Solo [inicio] (1 candidato)              |
| 2     | [inicio] + "Der" (2 candidatos)          |
| 3     | [inicio] + "Der" + "Hund" (3 candidatos) |
| 4     | + "isst" (4 candidatos)                  |
| 5     | + "Brot" (5 candidatos)                  |

En cada ronda, las posiciones que todavía no existen quedan con  $-\infty$  antes del softmax (0% de atención) – así se evita que el modelo “haga trampa” mirando la respuesta que todavía no genero.

osea entender el contexto de las palabras que se generaron?

Sí, exacto — ese es el objetivo, con una precisión: **coherencia interna de la propia oración alemana que se esta construyendo**, no el significado de la oración en ingles (eso lo hace Cross-Attention).

Cada palabra nueva que el Decoder genera necesita “saber” que ya dijo antes, en su mismo idioma, para **mantener la gramática y el sentido de la frase** – concordancia de genero, numero, orden de palabras propio del alemán, etc. Self-Attention enmascarada es la que le da esa memoria de **“lo que ya llevo escrito”**, ronda a ronda

```
x (512 números – entrada del token a esta capa)
↓
Self-Attention enmascarada (solo mira tokens ya generados) →
Sublayer1(x)
↓
Residual + LayerNorm #1:  $y_1 = \text{LayerNorm}(x + \text{Sublayer}_1(x))$ 
↓
Cross-Attention (Query de  $y_1$ , Key/Value del Encoder, ya fijos) →
Sublayer2( $y_1$ )
↓
Residual + LayerNorm #2:  $y_2 = \text{LayerNorm}(y_1 + \text{Sublayer}_2(y_1))$ 
↓
Feed-Forward Network (mismo mecanismo que en el Encoder) →
Sublayer3( $y_2$ )
↓
Residual + LayerNorm #3:  $y_3 = \text{LayerNorm}(y_2 + \text{Sublayer}_3(y_2))$ 
↓
 $y_3$  (512 números – sale de esta capa, entra a la siguiente)
```

(8 cabezas (se copia 8 veces el vector de **[inicio]**), cada una con sus propias  $W_i^Q/W_i^K/W_i^V$  de 512 x 64 – matrices **propias** de esta sub-cap, **distintas de las del Encoder** y de **Cross-Attention**). Como es la Ronda 1, **[inicio]** es el único token generado hasta ahora, así que solo puede compararse consigo mismo: la salida que busca esta sub-cap, en forma matemática:

$$Sublayer_1(x) = \text{MultiHead}(Q, K, V) \in \mathbb{R}^{512}$$

Desarrollando ese MultiHead paso a paso, para cada una de las 8 cabezas – con los tamaños exactos de cada pieza, para no confundir matriz (peso) con vector (resultado):

The (query)

Key (dog, the)

value

✂ ¿[inicio] se divide en 8 pedazos? No – se **copia entero, 8 veces**. Cada una de las 8 cabezas recibe el mismo vector de 512 numeros completo, no una porción de el. Lo que si se “achica” a 64 es el **resultado** de multiplicar ese vector completo por la matriz propia de cada cabeza ( $x * x \cdot W_i^Q$ , por ejemplo) – es la matriz la que resume los 512 numeros en 64, no un recorte del vector original. Es como sacarle 8 fotocopias idénticas a un documento y repartirlas a 8 personas: cada una tiene el documento completo, pero cada una lo resume a su manera.

$$\text{Cabeza}_i : \quad Q_i = x \cdot W_i^Q \quad K_i = x \cdot W_i^K \quad V_i = x \cdot W_i^V \quad (i = 1 \dots 8)$$

| Elemento              | Tamaño            | Que es                                                              |
|-----------------------|-------------------|---------------------------------------------------------------------|
| <b>X</b>              | 512 (vector)      | La entrada                                                          |
| $W_i^Q, W_i^K, W_i^V$ | 512 x 64 (matriz) | El peso – fijo, entrenado                                           |
| $Q_i, K_i, V_i$       | 64 (vector)       | El <b>resultado</b> de x multiplicado por cada matriz – no 512 x 64 |

entonces, para los pesos de Q, K y V serian pesos diferentes para las 8 cabezas?

**R:** Si. Cada una de las 8 cabezas tiene **su propio juego completo de  $W^Q$ ,  $W^K$  y  $W^V$**  – no comparten ninguna matriz entre si.

**Contando todo, para que quede claro el total**

```
Cabeza 1: W^Q_1, W^K_1, W^V_1 (3 matrices propias)
Cabeza 2: W^Q_2, W^K_2, W^V_2 (3 matrices propias, distintas de las de la Cabeza 1)
Cabeza 3: W^Q_3, W^K_3, W^V_3
...
Cabeza 8: W^Q_8, W^K_8, W^V_8

Total: 8 cabezas × 3 matrices cada una = 24 matrices distintas
```

Ninguna cabeza reutiliza la matriz de otra – las 24 son completamente independientes entre si, cada una entrenada por separada.

### Por qué esto es a propósito

Es justamente lo que **le permite a cada cabeza “especializarse”** en algo distinto – si todas compartieran las mismas matrices, las 8 cabezas darían **exactamente el mismo resultado**, y no tendría sentido tener **8 en vez de 1**. Al tener matrices propias, cada cabeza aprende, durante el entrenamiento, a fijarse en un tipo distinto de relación (una en gramática, otra en significado, etc.).

### Ojo con un matiz que ya vimos antes: esto se multiplica también por sub-capa

Y recuerda que esto son solo las **24 matrices de una sub-capa** (por ejemplo, Self-Attention masking). Como el Decoder tiene 3 sub-capas distintas (Self-Attention enmascarada, Cross-Attention, Feed-Forward – aunque esta ultima no usa el mismo esquema de  $W^Q/W^K/W^V$ , y cada una tiene su propio juego de 24, terminas con **múltiples conjuntos de 24 matrices**, todos distintos entre si, por cada capa del modelo.

### Resumen

Sí — dentro de una misma sub-capa (por ejemplo, Self-Attention enmascarada), las 8 cabezas tienen **pesos completamente distintos** entre sí: 8 matrices  $W^Q$  distintas, 8  $W^K$  distintas, 8  $W^V$  distintas – 24 en total, ninguna compartida.

Con eso ya calculado, resolvemos la fórmula de atención para cada cabeza:

$$\text{Attention}_i(Q_i, K_i, V_i) = \text{softmax} \left( \frac{Q_i K_i^T}{\sqrt{64}} \right) V_i$$

### [Formula de Self-Attention enmascarada]

Como **[inicio]** es el único token disponible en esta Ronda 1, el “abanico” de comparación dentro del softmax tiene **un solo candidato** – el propio  $K_i$  de **[inicio]**. Softmax, aplicado a una lista de un solo numero, siempre da 100% a ese único elemento (no hay nada mas entre lo que repartir el 100%). Por eso, en este caso particular, el resultado de cada cabeza termina siendo, literalmente, su propio Value:

$$\text{Attention}_i(Q_i, K_i, V_i) = 1.0 \times V_i = V_i$$

Esto es distinto a lo que vimos con "eats" en el Encoder (donde sí había otras palabras compitiendo por atención) — acá el masking, sumado a que es la primera posición, deja a **[inicio]** sin otra opción que mirarse a sí mismo. Concatenando las 8 cabezas — cada cabeza<sub>*i*</sub> es, justamente, el resultado de  $\text{Attention}_i(Q_i, K_i, V_i)$  que acabamos de calcular arriba, uno por cada una de las 8 — y multiplicando por la  $W^O$  propia de esta sub-cap:

$$\text{Sublayer}_1(x) = \text{Concat}(\underbrace{\text{Attention}_1(Q_1, K_1, V_1)}_{\text{cabeza}_1}, \dots, \underbrace{\text{Attention}_8(Q_8, K_8, V_8)}_{\text{cabeza}_8}) W_{\text{self-dec}}^O \quad (512 \text{ números})$$

**[termina acá para el md]**

#### 1.12.10.4.2.1: P1: La fórmula completa de Self-Attention enmascarada

La fórmula completa de Self-Attention enmascarada, solo para **[inicio]** (Ronda 1, Capa 1)

Paso 1 — Calcular Q, K, V (con  $x = \text{embedding} + \text{positional encoding de [inicio]}$ , y las matrices propias de esta sub-capas):

$$Q_i = x \cdot W_i^Q \quad K_i = x \cdot W_i^K \quad V_i = x \cdot W_i^V \quad (i = 1 \dots 8)$$

Paso 2 — Resolver la atención, con un único candidato (**[inicio]** es el único token disponible, así que softmax colapsa a 100%):

$$\text{Attention}_i(Q_i, K_i, V_i) = \text{softmax} \left( \frac{Q_i K_i^T}{\sqrt{64}} \right) V_i = 1.0 \times V_i = V_i$$

Paso 3 — Concatenar las 8 cabezas y multiplicar por  $W^O$  (la propia de esta sub-capas):

$$\text{Sublayer}_1(x) = \text{Concat}(V_1, V_2, \dots, V_8) W_{\text{self-dec}}^O$$

Todo junto, en una sola línea

$$\text{Sublayer}_1([\text{inicio}]) = \text{Concat} \left( \underbrace{V_1}_{\text{cabeza}_1}, \dots, \underbrace{V_8}_{\text{cabeza}_8} \right) W_{\text{self-dec}}^O$$

Esa es la fórmula completa de Self-Attention enmascarada aplicada solo a **[inicio]** — que, como es el único token de la Ronda 1, termina simplificándose a "concatenar sus propios 8 Values" y multiplicar por  $W^O$ , sin ninguna mezcla real con otros tokens (porque no hay ninguno todavía).

#### 1.12.10.4.2.2: P2: Para [inicio], que valor podrían tener Q, K y V

para inicio, que valor podrían tener Q K y V?

Buena pregunta para poner números — vamos a inventar valores concretos para **[inicio]**, siguiendo el mismo estilo simplificado (1 dimensión) que venimos usando en todo el documento.

##### Aclaración honesta antes de dar los números

Como ya vimos: no hay forma de "calcular" estos valores sin un modelo real entrenado — así que, igual que hicimos con "eats", "dog", "Hund", estos van a ser **valores inventados**, a modo ilustrativo, no el resultado de multiplicar contra una matriz W real.

Valores inventados para **[inicio]**, en Self-Attention enmascarada (Cabeza 1, simplificado a 1 dimensión)

$$Q_i = x \cdot W_i^Q \quad K_i = x \cdot W_i^K \quad V_i = x \cdot W_i^V \quad (i = 1 \dots 8)$$

Q\_1 (de [inicio]) = 0.5

K\_1 (de [inicio]) = 0.5 ← el mismo token, así que su propio Key

V\_1 (de [inicio]) = 0.4

Por que elegi  $Q_1 = K_1$

En este caso particular, no importa mucho qué valores tengan Q y K — como es el **único** candidato, el

softmax siempre va a dar 100% sin importar el número exacto (softmax de un solo elemento siempre da 1.0, sea cual sea ese elemento). Por eso elegí números simples y parecidos, para no distraer con un cálculo innecesario.

#### Aplicando la fórmula completa con estos valores

$$\text{Attention}_i(Q_i, K_i, V_i) = \text{softmax} \left( \frac{Q_i K_i^T}{\sqrt{64}} \right) V_i$$

Paso 1 — producto punto:

$$Q_1 \cdot K_1 = 0.5 \times 0.5 = 0.25$$

Paso 2 — dividir por  $\sqrt{64} = 8$  (en nuestra versión simplificada de 1 dimensión, con  $d_k = 1$ , esto no cambia nada — igual que en los otros ejemplos):

$$0.25/1 = 0.25$$

Paso 3 — softmax de un solo número:

$$\text{softmax}([0.25]) = [1.0]$$

Paso 4 — multiplicar por  $V_1$ :

$$\text{Attention}_1([inicio]) = 1.0 \times 0.4 = 0.4$$

Resultado final para la Cabeza 1

$$\text{Attention}_1(Q_1, K_1, V_1) = V_1 = 0.4$$

Confirmando exactamente el patrón que ya establecimos: **el resultado termina siendo, literalmente, el valor de  $V_1$**  — en este caso, 0.4 — porque no hay ningún otro token con el que repartir la atención.

1.12.10.4.2.3: P3: Que hace Q, K y V y  $W_i^Q$

La función general (recordando lo que ya vimos con Q, K, V)

| Matriz  | Qué hace                                                                     |
|---------|------------------------------------------------------------------------------|
| $W_i^Q$ | Convierte el token en su versión "pregunta" — qué está buscando              |
| $W_i^K$ | Convierte el token en su versión "etiqueta" — qué ofrece para ser encontrado |
| $W_i^V$ | Convierte el token en su versión "contenido" — qué entrega si es elegido     |

**La función específica en Self-Attention enmascarada: relacionar palabras del alemán entre sí**

Acá está lo que la distingue de las otras dos sub-capas: estas matrices **aprenden a captar relaciones dentro de la oración alemana que se está generando** — no miran el inglés (eso lo hace Cross-Attention), miran solo lo que ya se generó en el propio idioma de salida.

**Con nuestro ejemplo concreto**

Cuando "Hund" (en la Ronda 3) usa su  $W_i^Q$ , esa matriz está entrenada para que "Hund" pueda "preguntarle" a las palabras alemanas anteriores algo como: *"¿qué ya se dijo antes de mí, en esta misma oración, que me da contexto gramatical?"* — y compara contra el  $K_i$  de "Der" (calculado con la  $W_i^K$  de esta misma sub-capa), que "ofrece" información como *"soy un artículo, ya aparecí antes de vos"*.

**Por qué esto importa, comparado con no tenerlo**

Sin esta sub-capa, cada palabra alemana se generaría **sin saber qué palabras alemanas la precedieron** — perdería coherencia gramatical interna (por ejemplo, concordancia de género, número, orden de palabras propio del alemán). Self-Attention enmascarada es lo que le da al Decoder **memoria de su propia frase, mientras la va construyendo**.

**Diferenciándola de las otras dos, en una tabla resumen**

| Sub-capa                   | Función de sus pesos $W^Q/W^K/W^V$                                                                                                  |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Self-Attention del Encoder | Entender relaciones <b>dentro del inglés</b> (sujeto-verbo-objeto en la oración de origen)                                          |
| Self-Attention enmascarada | Entender relaciones <b>dentro del alemán ya generado</b> (coherencia gramatical de la propia traducción, a medida que se construye) |
| Cross-Attention            | Conectar <b>alemán con inglés</b> (qué parte de la oración original es relevante para elegir la próxima palabra)                    |

**Resumen**

Los pesos de Self-Attention enmascarada tienen la función de aprender a relacionar los tokens alemanes **ya generados entre sí** — dándole al Decoder coherencia interna dentro de su propia oración de salida, sin mirar todavía la oración en inglés (eso queda para el siguiente paso, Cross-Attention).



#### 1.12.10.4.2.4-P4: K y V se reutilizan, el Q cambia por el ultimo token de salida en cada ronda

- $K_i(\text{Der})$  y  $V_i(\text{Der})$ : se calculan **una sola vez**, en la Ronda 2 (cuando "Der" recién se genera), y **se reutilizan tal cual** en la Ronda 3 y siguientes — no se recalculan.
- $Q_i$ : no tiene sentido comparar "el de Der" entre rondas, porque el Query solo se calcula para el **último** token generado en cada ronda — en la Ronda 2 es el  $Q$  de "Der", en la Ronda 3 es el  $Q$  de "Hund" (no el de "Der" otra vez).

Entonces:  $K$  y  $V$  de "Der" son **estables** una vez creados (misma matriz  $W^K/W^V$ , mismo "Der"), mientras que  $Q$  va cambiando porque en cada ronda "pregunta" un token distinto.

Entonces:  $K$  y  $V$  de "Der" son **estables** una vez creados (misma matriz  $W^K/W^V$ , mismo "Der"), mientras que  $Q$  va cambiando porque en cada ronda "pregunta" un token distinto.

#### 1.12.10.4.2.5-P5: Objetivo y función de $W_o$ de self-attention masking

$W^O$  de Self-Attention enmascarada es la matriz (512x512) que mezcla los resultados de las 8 cabezas después de contenerlas — toma los 8 pedazos de 64 números pegados uno al lado del otro y los combina en un solo vector integrado de 512, antes de pasar a Residual+LayerNorm #1. Es propia de esta sub-cap, distinta de la de  $W_o$  de Cross-Attention y de la del Encoder.

##### Funcion:

Mezclar la información de las 8 cabezas entre si — sin  $W_o$ , el resultado quedaría como 8 compartimentos separados dentro del mismo vector (nunca combinados);  **$W_o$  es la que permite que lo que aprendió una cabeza se combine con lo que aprendió otra, dando un único resultado integrado.**

### 1.12.10.4.2.6: Repaso del masking – por que se llama así – por que en la ronda de [inicio] (Ronda 1) esto “no se nota”

no me acuerdo, para self atencion enmascarada como era el flujo, como que se recortaba

Buena pregunta para retomar — te referís al **masking** (enmascaramiento), que es justo lo que hace que esta sub-capas se llame "enmascarada". Te repaso el flujo completo.

#### La idea central: "cortar" las posiciones futuras antes del softmax

El nombre técnico es **masking**, y la idea es simple: cuando un token esta calculando su attention, **no puede ver** los tokens que vienen despues de el en la secuencia – solo puede mirar hacia atrás (a si mismo y a los anteriores)

*Como se hace ,tenicamente poner  $-\infty$  antes del softmax*

Antes de aplicar softmax, tener una lista de “scores” (uno por cada token contra el que te comapras). El masking consiste en **reemplazar por  $-\infty$**  los scores de las posiciones futuras, **antes** de que softmax los procese:

Scores originales (sin mask), para "Hund" en la Ronda 3, comparándose contra los 4 tokens de una oración de 4 posiciones:

Posición 0 ("Der"): score = 0.42  
Posición 1 ("Hund"): score = 0.595  
Posición 2 (todavía no existe): score =  $-\infty$  ← "cortado"  
Posición 3 (todavía no existe): score =  $-\infty$  ← "cortado"

#### Por qué $-\infty$ específicamente, y no simplemente “borrar” esas posiciones

Porque  $e^{-\infty} = 0$  — recordá la fórmula de softmax:  $\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ . Si metés  $-\infty$  como score, al elevarlo a  $e$  te da **exactamente 0**, así que esa posición termina con **0% de atención**, sin necesidad de "eliminar" nada de la matriz — es más simple hacerlo así, matemáticamente, que reestructurar los cálculos.

#### Aplicando softmax después del "recorte"

Las dos últimas posiciones quedan en **0%**, como si no existieran — solo "Der" y "Hund" (las posiciones ya generadas) se reparten el 100% de atención.

#### Por qué esto se llama "enmascarada" (masked)

La palabra "mask" (máscara) es justamente esa — una capa que **tapa** ciertas posiciones antes de que el modelo las use, dejando visible solo lo que corresponde. Es como ponerle un cartel de "no disponible" a las posiciones futuras, para que el softmax ni siquiera las considere.

#### Resumen del flujo completo, con el masking incluido

1. Calcular  $Q_i, K_i, V_i$  para cada token (incluidos los futuros, si los hubiera)
2. Calcular los scores:  $Q_i \cdot K_i^T$ , para TODOS los tokens de la secuencia
3. MASKING: reemplazar por  $-\infty$  los scores de las posiciones futuras
4. Dividir por  $\sqrt{d_k}$
5. Softmax → las posiciones futuras dan 0%, automáticamente
6. Multiplicar por  $V_i$  y sumar → resultado final, sin ninguna influencia de tokens futuros

Por qué en [inicio] (Ronda 1) esto "no se nota"

En la Ronda 1, no hace falta ni aplicar el masking explícitamente para notar su efecto – como solo existe [inicio], no hay ninguna posición futura que “recordar” todavía. El maskin se vuelve visible recién cuando hay **varios** tokens en juego (como en la Ronda 3, con “Der” y “Hund” ya generados, y “isst”/”Brot” todavía sin existir).

1.12.10.4.2.7: Repaso 1: ¿Cuando empieza self-attention masking, como es eso que se divide?

Ah, dale — retomando esa parte específica: la respuesta corta es que **no se divide**, se **copia**. Es justo lo que aclara ese mismo recuadro que pegaste (✂). Te lo repaso con calma, porque es un punto que genera confusión fácil.

La confusión típica: "8 cabezas" suena a "partir en 8"

Es natural pensar que "8 cabezas" significa cortar el vector de 512 en 8 pedazos de 64 — pero **no es así**.

Lo que realmente pasa

```
x = [inicio] (512 números completos)

↓ se hacen 8 COPIAS IDÉNTICAS (no 8 recortes)

Copia 1: [inicio] completo, los 512 números
Copia 2: [inicio] completo, los 512 números
Copia 3: [inicio] completo, los 512 números
...
Copia 8: [inicio] completo, los 512 números
```

Cada una de las 8 cabezas recibe **el vector entero**, no una porción de él.

Entonces, ¿de dónde sale el "64"?

Cabeza<sub>i</sub> :     $Q_i = x \cdot W_i^Q$      $K_i = x \cdot W_i^K$      $V_i = x \cdot W_i^V$     ( $i = 1...8$ )

| Elemento              | Tamaño            | Que es                                                              |
|-----------------------|-------------------|---------------------------------------------------------------------|
| X                     | 512 (vector)      | La entrada                                                          |
| $W_i^Q, W_i^K, W_i^V$ | 512 x 64 (matriz) | El peso – fijo, entrenado                                           |
| $Q_i, K_i, V_i$       | 64 (vector)       | El <b>resultado</b> de x multiplicado por cada matriz – no 512 x 64 |

El 64 aparece **después**, cuando cada copia se multiplica por su propia matriz:

```
Cabeza 1: [inicio] completo (512) × W^Q_1 (matriz de 512×64) = Q_1 (64 números)
Cabeza 2: [inicio] completo (512) × W^Q_2 (matriz de 512×64) = Q_2 (64 números)
...
```

**La matriz es la que "resume" los 512 números en 64** — no es que el vector original se haya cortado. Es matemática de multiplicación: un vector de 512 multiplicado por una matriz de 512×64 da como resultado un vector de 64 — sin que se haya "perdido" ni "recortado" nada del vector de entrada, que sigue siendo completo en cada una de las 8 copias.

La analogía que ya usamos, para que quede grabada

Es como sacarle **8 fotocopias idénticas** a un mismo documento de 512 páginas, y dárselas a 8 personas distintas. Cada persona tiene el documento **completo** — pero cada una hace su propio resumen del documento (con su propio criterio, que sería la matriz), y ese resumen termina siendo más corto (64 páginas) que el original.

#### 1.12.10.4.3: Residual + LayerNorm #1 [vector de 512]: - Objetivo – Funcion

$$y_1 = \text{LayerNorm}(x + \text{Sublayer}_1(x)) \quad (512 \text{ números})$$

**Objetivo:** Evitar que el entrenamiento se “pierde” en redes tan profundas – conservar el vector original (x) sumado al resultado de Self-Attention enmascarada, en vez de reemplazarlo.

**Funcion:** suma  $x + \text{Sublayer}_1(x)$  (el “atajo” residual), y normaliza esa suma (reescala para que no se dispare de escala) – dando  $y_1$ , la versión estable que sigue camino hacia Cross-Attention.

ese vector original seria positional + embedding de inicio, para ronda 2 seria lo mismo para Der

Si, exacto – confirmado:

- **Ronda 1:**  $x = \text{embedding} + \text{positional encoding de [inicio]}$
- **Ronda 2:**  $x = \text{embedding} + \text{positional encoding de “Der” (posición 0)}$

Y en la Ronda 3, seria lo mismo pero para “Hund” (posición 1) – cada ronda arranca con el embedding + positional del **ultimo token generado**, que es el que entra como x a la Capa 1 de esa ronda.

1.12.10.4.4: Cross-Attention v1 [1 x 64 para cabeza 1] (explicación 1) (que al generar cada palabra nueva en alemán, pueda consultar oración original en ingles y decidir que parte prestar atención)

**Objetivo:**  
Que el Decode, al generar cada palabra nueva en alemán, pueda **consultar la oración original en ingles** y decidir a que parte de ella prestarle atención – así elige la traducción correcta según el contexto de la oración fuente, no solo según lo que ya escribio en alemán (eso es tarea de Self-Attention masking)

Aca entra la oración en ingles. Antes de la formula, el punto clave: **El Query sale de  $y_1$**  (el Decoder), pero **los Keys y Values salen de otro lado** – nunca de  $y_1$ .

Para calcular  $K_i/V_i$ , se multiplican **dos cosas que vienen de lugares distintos**:

|                                    | De donde sale                                                                                                                                       | Que es                                 | Función (si es un peso entrenado)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| El <b>vector</b> que se multiplica | Del <b>Encoder</b> – la representación final, tras sus 6 capas (las que vimos en “El resultado del Encoder”; ahí solo hay vectores, ninguna matriz) | El “material” a transformar            | No es un peso — es el dato de entrada                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| La <b>matriz</b> que lo transforma | Del <b>Decoder</b> - $W_i^K/W_i^V$ , propias de esta Cross-Attention (un tercer juego de 24 matrices, sin ninguna relación con el Encoder)          | La “receta” que hace la transformación | $W_i^K$ aprende a resumir cada palabra en ingles en una “etiqueta” que el Decoder pueda comparar contra su Query – que tan relevante es esa palabra para el token alemán que se esta generando.<br>$W_i^V$ aprende a resumir esa misma palabra en el “contenido” que efectivamente se entrega si el Decoder decide prestarle atención – el significado que realmente influye en la predicción, no solo su relevancia. Ambas se ajustan con backpropagaton durante el entrenamiento, igual que las demás matrices del modelo (ver el recuadro 🎮 del Paso 1) |

$$Q_i = y_1 \cdot W_i^Q \qquad K_i(\text{palabra}) = \text{vector\_final\_Encoder}(\text{palabra}) \cdot W_i^K \qquad V_i(\text{palabra}) = \text{vector\_final\_Encoder}(\text{palabra}) \cdot W_i^V$$

| Elemento                                        | Tamaño                  | Que es                                                        |
|-------------------------------------------------|-------------------------|---------------------------------------------------------------|
| $y_1$                                           | <b>1 x 512 (vector)</b> | [inicio], ya procesado por Self-Attention masking + R + Ln #1 |
| $\text{vector\_final\_Encoder}(\text{palabra})$ | <b>1 x 512 (vector)</b> | Cada palabra del ingles, tras las 6                           |

|                                                                                   |                                                             |                                                                   |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------|
|                                                                                   |                                                             | capas del Encoder – hay 4, uno por palabra                        |
| $W_i^Q$                                                                           | 512 x 64 (matriz)                                           | El peso que transforma $y_1$ en $Q_i$                             |
| $W_i^K$                                                                           | 512 x 64 (matriz)                                           | El peso que transforma cada vector del Encoder en $K_i$           |
| $W_i^V$                                                                           | 512 x 64 (matriz)                                           | El peso que transforma cada vector del Encoder $V_i$              |
| $Q_i = y_1 \cdot W_i^Q$                                                           | 1 x 64 (vector)                                             | Resultado: $[1 \times 512] \cdot [512 \times 64] = [1 \times 64]$ |
| $K_i(\text{palabra}) = \text{vector\_final\_Encoder}(\text{palabra}) \cdot W_i^K$ | 1 x 64 (vector), uno por palabra                            | Resultado, por cada una de las 4 palabras                         |
| $K_i(\text{las 4 palabras juntas})$                                               | 4 x 64 (matriz)                                             | Apilando los 4 resultados anteriores                              |
| $V_i(\text{palabra})$ y $V_i(\text{las 4 juntas})$                                | Igual patrón que $K_i$ ; 1 x 64 por palabra, 4 x 64 apilado | Misma mecánica, con $W_i^V$                                       |

**El patrón general:** todo lo que empieza como 512 (el vector de entrada ya sea  $y_1$  o el vector del Encoder) se multiplica por una matriz 512 x 64 (el peso), y da como resultado 64 – para Q hay un solo resultado (1 fila, porque hay un solo  $y_1$ ); para K y V hay 4 resultados (4 filas, uno por cada palabra del ingles), que se apilan en una matriz 4 x 64.

**[se copia 8 veces “vector\_final\_encoder” (cabeza) para The, 8 cabezas otro para “Dog”]**

Esto se repite para cada una de las 4 palabras en ingles y cada una de las 8 cabezas – en total,  $8 \times 4 \times 64 = 2.048$  numeros solo para los keys, y otros 2.048 para los Values.

$$Q_i = y_1 \cdot W_i^Q \quad K_i(\text{palabra}) = \text{vector\_final\_Encoder}(\text{palabra}) \cdot W_i^K \quad V_i(\text{palabra}) = \text{vector\_final\_Encoder}(\text{palabra}) \cdot W_i^V$$

🔄 ¿Esto se recalcula en las 5 rondas de generación? ¿Y en las 6 capas del Decoder? El **vector\_final\_Encoder** (el de "The", "dog", "eats", "bread") es siempre el mismo — se calculó una sola vez, cuando el Encoder terminó de correr, y **nunca cambia** mientras dure la generación de la traducción (ni entre rondas, ni entre capas). **Pero  $K_i/V_i$  (el resultado de multiplicar ese vector  $W_i^K/W_i^V$ ) si es distinto en cada una de las 6 capas el Decoder** (Capa 1 a Capa 6 – no tienen nombres propios, son 6 capas de **estructura idéntica** apiladas, cada una con su propio juego de matrices entradas por separado; las mismas 6 que ya vimos en el diagram de “Flujo completo dentro de una capa del Decoder”) – porque cada capa tiene su propio juego de matrices  $W_i^K/W_i^V$  de Cross-Attention. Lo que, si es constante entre las 5 rondas, para una misma capa, es que  $K_i/V_i$  de esa capa no vuelve a calcularse – solo cambia el Query, que sale de un token distinto en cada ronda.

**Resolviendo la atención**, con la fórmula general del paper (la misma "Scaled Dot-Product Attention" que ya vimos en el Encoder):

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

Acá  $d_k = 64$  (el tamaño de cada cabeza), así que en nuestro caso la fórmula queda  $\text{softmax}(QK^T/\sqrt{64})V$ . Mostrando solo el resultado de la Cabeza 1. Identificando cada elemento antes de la cuenta, con su forma matemática (vector vs. matriz, y tamaño exacto —  $Q$ : vector  $1 \times 64$ ;  $K$  de una sola palabra: vector  $1 \times 64$ , pero  $K$  de las 4 palabras juntas: matriz  $4 \times 64$ ;  $V$  sigue el mismo patrón que  $K$ ):

| Elemento                   | Quien es                            | Forma matemática      | Que representa                                                                                 |
|----------------------------|-------------------------------------|-----------------------|------------------------------------------------------------------------------------------------|
| $Q_1$                      | [inicio]                            | Vector, $1 \times 64$ | El único token alemán que existe en la Ronda 1 – el que “pregunta” que palabra generar primero |
| $K_1(\text{The})$          | “The”                               | Vector, $1 \times 64$ | Un candidato del ingles – la “etiqueta” de “The”, calculada con $W_1^K$                        |
| $K_1(\text{dog})$          | “dog”                               | Vector, $1 \times 64$ | Otro candidato del ingles                                                                      |
| $K_1(\text{eats})$         | “eats”                              | Vector, $1 \times 64$ | Otro candidato del ingles                                                                      |
| $K_1(\text{bread})$        | “bread”                             | Vector, $1 \times 64$ | Otro candidato del inglés                                                                      |
| $K_1(\text{los 4 juntos})$ | Las 4 palabras del ingles, apiladas | Matriz, $4 \times 64$ | El conjunto completo contra el que se compara $Q_1$                                            |

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$Q_1$  (de [inicio]) se compara contra **cada uno** de esos 4 Keys — nunca contra sí mismo, porque [inicio] no forma parte del inglés; acá quien “pregunta” es el token alemán, y quienes “responden” son los 4 tokens ingleses del Encoder. La multiplicación  $Q_1K_1^T$  es  $[1 \times 64] \cdot [64 \times 4] = [1 \times 4]$  — da 4 scores, uno por palabra (cada uno, en la realidad, es la suma de 64 productos, igual que en el desglose 4.1-4.4 del Encoder):

$$\text{softmax}\left(\frac{Q_1K_1^T}{\sqrt{64}}\right) \approx [0.38, 0.34, 0.15, 0.13] \quad (\text{para "The", "dog", "eats", "bread"}) \quad \text{— vector, } 1 \times 4$$

Influenciado sobre todo por “The” y “dog” — coherente con que la primera palabra a generar sea “Der” (artículo masculino de “Hund” en alemán).

**Falta el último paso — multiplicar por  $V_1$ :** los 4 porcentajes ( $1 \times 4$ ) se multiplican por  $V_1$  (matriz  $4 \times 64$ , las Values de las 4 palabras):

$$[1 \times 4] \cdot [4 \times 64] = [1 \times 64]$$

$$\text{Attention}_1 = 0.38 V_1(\text{The}) + 0.34 V_1(\text{dog}) + 0.15 V_1(\text{eats}) + 0.13 V_1(\text{bread}) \quad \text{— vector, } 1 \times 64$$



## Salida para cabeza 1: vector 1 x 64

<sup>1 2</sup><sub>3 4</sub> ¿Cómo se llega, en concreto, a esas 64 posiciones?  $V_1$  es una matriz de 4 filas (una por palabra) x 64 columnas – cada fila tiene 64 números, no 1. La multiplicación **no da un solo número que se repite 64**: cada de las 64 columnas se calcula por separado, mezclando las 4 filas con los mismos 4 porcentajes. Con un ejemplo reducido a 3 columnas (en vez de 64), para verlo completo:

```
V_1(The)    = [0.3, 0.1, 0.5]
V_1(dog)    = [0.9, 0.2, 0.4]
V_1(eats)   = [0.6, 0.7, 0.1]
V_1(bread)  = [0.2, 0.8, 0.3]
```

**Columna 1:**  $0.38(0.3) + 0.34(0.9) + 0.15(0.6) + 0.13(0.2) = 0.536$  **Columna 2:**  $0.38(0.1) + 0.34(0.2) + 0.15(0.7) + 0.13(0.8) = 0.315$  **Columna 3:**  $0.38(0.5) + 0.34(0.4) + 0.15(0.1) + 0.13(0.3) = 0.38$

Resultado (con 3 columnas de ejemplo):  $[0.536, 0.315, 0.38]$  — un vector de 3 números. En el modelo real, en vez de 3 columnas hay **64**, así que esta misma cuenta se repite 64 veces (una por columna de  $V_1$ ), dando el vector final de 64 números — no porque "aparezcan de la nada", sino porque cada Value tiene 64 números por diseño ( $d_v = d_{model}/h = 512/8 = 64$ ), y la multiplicación conserva ese tamaño.

Ese vector de 64 números es el resultado final de la Cabeza 1. Las otras 7 cabezas hacen lo **mismo en paralelo**, con sus propios K/V, posiblemente fijándose en otros aspectos de la oración.

**Las otras 7 cabezas hacen lo mismo en paralelo**, con sus propios K/V, posiblemente fijándose en otros aspectos de la oración.

### Vector\_final\_encoder para los 5 tokens se copia 8 veces.

Si: Se copia **8 veces**, nunca se divide. Es exactamente el mismo patrón que ya vimos con [inicio] y con "dog" en el Encoder — vamos a confirmarlo, aplicado ahora a cada uno de los vectores finales del Encoder.

#### Confirmando: se copia, no se divide

```
vector_final_Encoder("The") = [512 números completos]
```



↓ se copia ENTERO, 8 veces (una por cabeza)

Copia 1: [512 números completos] → se multiplica por  $W^K_1$  →  $K_1(\text{"The"}) = 64$  números

Copia 2: [512 números completos] → se multiplica por  $W^K_2$  →  $K_2(\text{"The"}) = 64$  números

...

Copia 8: [512 números completos] → se multiplica por  $W^K_8$  →  $K_8(\text{"The"}) = 64$  números

Ningun trozo del vector de 512 se “pierde” ni se recorta – cada una de las 8 cabezas recibe el vector **entero**, y es la matriz de esa cabeza la que lo resume a 64.

#### Esto pasa para CADA una de las 4 (o 5) palabras del Encoder, por separado

```
vector_final_Encoder("The") → se copia 8 veces → da  $K_1(\text{"The"}) \dots K_8(\text{"The"})$   
vector_final_Encoder("dog") → se copia 8 veces → da  $K_1(\text{"dog"}) \dots K_8(\text{"dog"})$   
vector_final_Encoder("eats") → se copia 8 veces → da  
 $K_1(\text{"eats"}) \dots K_8(\text{"eats"})$   
vector_final_Encoder("bread") → se copia 8 veces → da  
 $K_1(\text{"bread"}) \dots K_8(\text{"bread"})$ 
```

#### De ahí sale el "2.048" que calculamos antes

$$4 \text{ palabras} \times 8 \text{ copias/cabezas} \times 64 \text{ números resultantes} = 2.048$$

Cada palabra se copia 8 veces (no se divide en 8), y cada copia da un resultado de 64 numeros – nunca hay una “división” del vector original en ningún punto de este proceso.

#### Resumen, con la misma analogía que ya usamos

Es exactamente la analogía de las 8 fotocopias, aplicada 4 veces (una por palabra): sacás 8 fotocopias del vector de "The" (512 páginas cada fotocopia, completas), 8 fotocopias del vector de "dog", 8 de "eats", 8 de "bread" — 32 fotocopias en total, cada una resumida por su propia matriz a 64 páginas. Nunca se corta el documento original en pedazos — siempre se copia entero y se resume después, con la matriz.

y los pesos para las 8 cabezas para The, dog por ejemplo para  $W^K$  es el mismo peso para cabeza 1 tanto para the y dog?

Sí, exacto — confirmado. La matriz  $W^K_1$  (la de la Cabeza 1) es **la misma** para “The” y para “dog” – y para “eats” y “bread” también.

### Confirmando con la fórmula

```
K_1("The") = vector_final_Encoder("The") × W^K_1    ← misma W^K_1
K_1("dog") = vector_final_Encoder("dog") × W^K_1    ← misma W^K_1
```

**Lo que cambia** entre "The" y "dog" es el **vector de entrada** (cada palabra tiene el suyo): **Lo que NO cambia** es la matriz  $W_1^K$  — es una sola matriz, fija, que se aplica a las 5 palabras por igual dentro de la Cabeza 1.

### Es el mismo patrón que ya confirmaste en varios lugares del documento

Exactamente como pasaba con  $W_1$  de Feed-Forward (misma matriz para "dog" y "eat", distinto resultado según el vector de entrada), o con  $W_i^Q/W_i^K/W_i^V$  del Encoder para los 5 tokens de "The dog eats bread" — la matriz es la "receta" fija por cabeza, y se aplica igual a cualquier palabra que le entre.

### Resumen

Sí:  $W_1^K$  es una sola matriz, compartida entre las 4 palabras del inglés — la Cabeza 1 usa esa misma matriz sin importar si está procesando "The", "dog", "eats" o "bread". Lo que varía es solo el resultado ( $K_1$  de cada palabra), porque el vector que entra es distinto en cada caso.

#### 1.12.10.4.4.1: Sobre las 8 cabezas también para Cross-Attention

me perdi, 8 cabezas es cuando se copia el vector 8 veces (de entrada) y se calcula con sus respectivos pesos para capa 1 (8 veces el calculo), capa 2(8 veces el calculo)?

Si: en **cada capa**, el vector de entrada se copia 8 veces (una por cabeza), y se hace el calculo completo (Q, K, V, atención) **8 veces** – una vez por cabeza. Y esto se repite en **cada una** de las 6 capas.

**El total, contando todo**

```
Capa 1: 8 cálculos de atención (uno por cabeza)
Capa 2: 8 cálculos de atención (otra vez, con matrices DISTINTAS a la Capa 1)
Capa 3: 8 cálculos de atención
Capa 4: 8 cálculos de atención
Capa 5: 8 cálculos de atención
Capa 6: 8 cálculos de atención

Total: 6 capas × 8 cabezas = 48 cálculos de atención, por sub-capa, por ronda
```

**Y esto se multiplica también por las 3 sub-capas del Decoder**

Como cada capa del Decoder tiene 3 sub-capas de atención (Self-Attention masking + Cross-Attention – Feed-Forward no usa este esquema de cabezas):

```
6 capas × 2 sub-capas de atención × 8 cabezas = 96 cálculos de atención, por ronda
```

**Y esto se repite en cada una de las 5 rondas**

```
5 rondas × 96 = 480 cálculos de atención en total, para generar "Der Hund isst Brot
[fin]"
```

**Resumen de tu pregunta, confirmado**

Si – “8 veces el calculo” pasa en **cada capa** (Capa 1: 8 veces, Capa 2: 8 veces, y así hasta la capa 6), con matrices **distintas** en cada capa – y eso se repite también en cada sub-capa (Self-Attention masking y Cross attention, cada una con sus propias 8 cabezas)

```
es decir, el juego de matrices de K y V son las mismas por cada capa1 (capa 1 de the, capa
1 de dog) pero son diferentes por cabeza 1, cabeza 2, tanto para self-attention como
cross, pero son las mismas en cabeza 1 de capa 1 para the, como cabeza 1 de capa 1 para
dog
```

Sí, exacto — confirmado en las dos partes. Vamos a separarlo bien para que quede grabado.

## Parte 1: mismo juego de matrices dentro de una capa, para todas las palabras

Confirmado -  $W_1^K$ (cabeza 1, Capa 1) es la **misma** matriz tanto para “The” como para “dog” (y “eats” y “bread”):

```
K_1(The) = vector_final_Encoder(The) × W^K_1(capa1) ← misma matriz
K_1(dog) = vector_final_Encoder(dog) × W^K_1(capa1) ← misma matriz
```

## Parte 2: distintas matrices entre cabezas (dentro de la misma capa)

Confirmado también – dentro de la Capa 1, la cabeza 1 y la cabeza 2 tienen matrices **distintas** entre si:

```
Cabeza 1: W^K_1(capa1) ← una matriz
Cabeza 2: W^K_2(capa1) ← otra matriz, distinta
```

## Y esto aplica tanto a Self-Attention como a Cross-Attention – con un matiz importante:

Correcto, ambas sub-capas siguen el mismo patron (“compartida entre palabras, distinta entre cabezas”) – pero son **juegos de matrices completamente separados** entre si: el  $W_1^K$  de self-Attention (Capa 1) **no es el mismo** que el  $W_1^K$  de cross-attention (Capa 1), aunque ambos se llamen “Cabeza 1” – son matrices independientes, entrenadas por separado.

## Tabla resumen completa, con las 3 dimensiones que varían (o no)

| Eje                                                                                | ¿Cambia la matriz?                        |
|------------------------------------------------------------------------------------|-------------------------------------------|
| Entre palabras, dentro de la misma cabeza y capa (The vs. dog)                     | No — misma matriz                         |
| Entre cabezas, dentro de la misma capa (Cabeza 1 vs. Cabeza 2)                     | Sí — matrices distintas                   |
| Entre capas (Capa 1 vs. Capa 2)                                                    | Sí — matrices distintas                   |
| Entre sub-capas (Self-Attention vs. Cross-Attention), aunque sea "la misma cabeza" | Sí — matrices distintas, juegos separados |

## Resumen final, confirmando tu frase completa

Si -  $W_1^K$  de la Cabeza 1, Capa 1 es la **misma** para “The” que para “dog” (Parte 1 de tu pregunta), pero **distinta** de  $W_2^K$  (cabeza 2, misma capa) – y esto vale tanto para self-attention como para Cross-Attention, aunque cada una de esas sub-capas tiene su **propio** juego de 8 matrices, sin relación entre si.

#### 1.12.10.4.5: Cross-Attention v2 (explicación 2 a modo repaso)

Dale, vamos con el flujo completo de Cross-Attention aplicado a [inicio] en la Ronda 1, juntando todo lo que ya armamos.

##### Punto de partida

Cross-Attention arranca **despues** de que [inicio] ya paso por Self-Attention enmascarada + Residual+LayerNorm #1 – así que la entrada no es  $x$  (el [inicio] original), sino  $y_1$  (ya transformado).

```
 $y_1 = \text{LayerNorm}(x + \text{Sublayer}_1(x))$  ← esto ya lo calculamos antes, es la entrada  
Cross-Attention
```

##### Paso 1 - $y_1$ se copia 8 veces, una por cabeza (mismo mecanismo que ya vimos)

```
Copia 1:  $y_1$  completo (512 números)  
...  
Copia 8:  $y_1$  completo (512 números)
```

##### Paso 2 – cada cabeza calcula su Query a partir de $y_1$ , con las matrices propias de Cross-Attention (distintas de las de Self-Attention enmascarada)

$$Q_i = y_1 \cdot W_i^Q \quad (i = 1 \dots 8)$$

##### Paso 3 — los Keys y Values NO salen del Encoder

Aca esta la diferencia clave con Self-Attention: para cabeza, se toma el vector **final** del Encoder (tras sus 6 capas) de **cada una de las 4 palabras en ingles**, y se multiplica por las matrices  $W_i^K/W_i^V$  de esta Cross-Attention:

$$K_i(\text{palabra}) = \text{vector\_final\_Encoder}(\text{palabra}) \cdot W_i^K$$

$$V_i(\text{palabra}) = \text{vector\_final\_Encoder}(\text{palabra}) \cdot W_i^V$$

Para “the”, “dog”, “eats” y “bread” – dando 4 keys y 4 Values por cada cabeza (no solo 1, como pasaba con [inicio] en Self-Attention enmascarada).

##### Paso 4 — resolver la fórmula de atención, con 4 candidatos esta vez (no 1)

$$\text{Attention}_i(Q_i, K_i, V_i) = \text{softmax} \left( \frac{Q_i K_i^T}{\sqrt{64}} \right) V_i$$

Con valores simplificados (1 dimensión), para la Cabeza 1:

```
Q_1 = 0.6  
K_1(The)=0.1   K_1(dog)=0.3   K_1(eats)=0.9   K_1(bread)=0.2
```

Producto punto: 0.06, 0.18, 0.54, 0.12 -> dividir por  $\sqrt{64}=8$  -> softmax -> aproximadamente [0.38, 0.34, 0.15, 0.13] (influenciado sobre todo por “The” y “dog”, coherente con que la primera palabra sea “Der”).

En paso 4 no entendí, deja de simplificarlo

Tenés razón — vamos con el Paso 4 completo, sin reducir a 1 dimensión, con las 64 dimensiones reales de cada cabeza.

**Punto de partida: vectores de 64 números, no de 1**

Para la Cabeza 1, ya tenemos (del Paso 2 y 3):

$$Q_1 = [q_1, q_2, q_3, \dots, q_{64}] \quad (64 \text{ números, calculado de } y_1 \times W_1^Q)$$

Y los 4 Keys, uno por cada palabra en inglés, cada uno también de 64 números:

$$K_1(\text{The}) = [k_1, k_2, \dots, k_{64}] \quad K_1(\text{dog}) = [k'_1, k'_2, \dots, k'_{64}]$$

$$K_1(\text{eats}) = [\dots] \quad K_1(\text{bread}) = [\dots]$$

**Sub-paso 4.1 — Producto punto:**  $Q_1 \cdot K_1(\text{palabra})^T$

$$\text{Attention}_i(Q_i, K_i, V_i) = \text{softmax} \left( \frac{Q_i K_i^T}{\sqrt{64}} \right) V_i$$

Esto **no es una sola multiplicación** – es multiplicar los 64 números de  $Q_1$  contra los 64 números de cada  $K_1$ , **uno a uno, y sumar los 64 resultados**:

$$Q_1 \cdot K_1(\text{The})^T = (q_1 \times k_1) + (q_2 \times k_2) + (q_3 \times k_3) + \dots + (q_{64} \times k_{64})$$

Esto da **un solo número** – pero ese numero es la suma de 64 multiplicaciones, no una sola cuenta como veníamos simplificando. Y esto se repite, por separado, contra uno de los 4 keys:

$$\text{score}(\text{The}) = Q_1 \cdot K_1(\text{The})^T \quad \text{score}(\text{dog}) = Q_1 \cdot K_1(\text{dog})^T$$

$$\text{score}(\text{eats}) = Q_1 \cdot K_1(\text{eats})^T \quad \text{score}(\text{bread}) = Q_1 \cdot K_1(\text{bread})^T$$

Al final de este sub-paso, tenes **4 numeros** (uno por palabra) – cada uno es el resultado de 64 productos.

**Sub-paso 4.2 — Dividir cada uno de esos 4 números por  $\sqrt{64}=8$**

$$\frac{\text{score}(\text{The})}{8} \quad \frac{\text{score}(\text{dog})}{8} \quad \frac{\text{score}(\text{eats})}{8} \quad \frac{\text{score}(\text{bread})}{8}$$

Esto sí hace algo real en la versión completa (a diferencia de nuestra simplificación de 1 dimensión, donde dividir por  $\sqrt{1}=1$  no cambiaba nada) – recuerda lo que vimos antes: sumar 64 productos tienen a dar números grandes, y dividir por  $\sqrt{64}$  los “achica” de vuelta a una escala manejable.

**Sub-paso 4.3 — Softmax sobre esos 4 números**

$$\text{softmax} \left( \left[ \frac{\text{score}(\text{The})}{8}, \frac{\text{score}(\text{dog})}{8}, \frac{\text{score}(\text{eats})}{8}, \frac{\text{score}(\text{bread})}{8} \right] \right) = [p_1, p_2, p_3, p_4]$$

Donde  $p_1 + p_2 + p_3 + p_4 = 1$  (100%) – usando la formula exacta que ya vimos:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

**Sub-paso 4.4 — Multiplicar cada probabilidad por su Value completo (64 números) y sumar**

Aca esta la parte que mas se simplificaba antes: no es “multiplicar un porcentaje por un solo numero” – es

multiplicar un porcentaje por un **vector completo de 64 numeros**, y sumar los 4 vectores resultantes, **posición por posición**:

**Falta el último paso — multiplicar por  $V_1$** : los 4 porcentajes (1 X 4) se multiplican por  $V_1$  (matriz 4 x 64, las Values de las 4 palabras):

$$[1 \times 4] \cdot [4 \times 64] = [1 \times 64]$$

$$\text{Attention}_1 = p_1 \times V_1(\text{The}) + p_2 \times V_1(\text{dog}) + p_3 \times V_1(\text{eats}) + p_4 \times V_1(\text{bread})$$

Donde cada  $V_1(\text{palabra})$  es un vector de 64 numeros. Expandiendo la primera dimensión, como ejemplo:

$$\text{Attention}_1[\text{dim } 1] = p_1 \times V_1(\text{The})[1] + p_2 \times V_1(\text{dog})[1] +$$

$$- p_3 \times V_1(\text{eats})[1] + p_4 \times V_1(\text{bread})[1]$$

Y esa misma cuenta se repite **64 veces**, una por cada dimensión del vector, dando como resultado final **un vector de 64 numeros** – el resultado completo de la Cabeza 1.

#### Resumen del Paso 4 completo, sin simplificar

$$\text{Attention}_1(Q_1, K_1, V_1) = \text{softmax} \left( \frac{Q_1 K_1^T}{\sqrt{64}} \right) V_1$$

Donde:

- $Q_1$ : vector de 64 numeros
- $K_1$ : **matriz** de 4 x 64 (4 filas, una por palabras en ingles; 64 columnas)
- $Q_1 K_1^T$ : da un vector de **4 numeros** (un score por palabra, cada uno sumando 64 productos)
- Tras dividir por  $\sqrt{64}$  y aplicar softmax: **4 probabilidades**
- $V_1$ : **matriz** de 4 x 64 (igual forma que  $K_1$ )
- *El producto final* (probabilidades x  $V_1$ ): un **vector de 64 numeros** – el resultado real de esta cabeza, sin ninguna reducción a 1 dimensión.

#### Paso 5 — Multiplicar cada porcentaje por su Value y sumar

$$\text{Attention}_1 = 0.38 V_1(\text{The}) + 0.34 V_1(\text{dog}) + 0.15 V_1(\text{eats}) + 0.13 V_1(\text{bread})$$

Esto da un vector de 64 números — el resultado de la Cabeza 1. **Las otras 7 cabezas repiten el mismo proceso, con sus propias matrices.**

#### Paso 6 – concatenar las 8 cabezas y multiplicar por $W^O$ de Cross-Attention

$$\text{Sublayer}_2(y_1) = \text{Concat}(\text{cabeza}_1, \dots, \text{cabeza}_8) W_{\text{cross}}^O$$

El flujo completo, resumido



```

y1 (de [inicio], 512 números)
  ↓ se copia 8 veces
  ↓ cada cabeza: Qi = y1 · WQi
  ↓ Ki y Vi salen del Encoder (4 palabras, no de y1)
  ↓ softmax(Qi · KiT / V64) → reparte atención entre The/dog/eats/bread
  ↓ mezcla ponderada con los Vi → resultado de cada cabeza (64 números)
  ↓ concatenar las 8 → 512 números
  ↓ × WOcross
  ↓
Sublayer2(y1) – 512 números, listo para Residual+LayerNorm #2

```

#### 1.12.10.4.5.1: De donde venia V y K de cross-Attention

**Del Encoder** – específicamente, de la representación **final** del Encoder (después de sus 6 capas completas), para cada una de las 4 palabras en inglés ("The", "dog", "eats", "bread").

#### El camino completo, paso a paso

```

"The", "dog", "eats", "bread"
  ↓ pasan por las 6 capas del Encoder (Self-Attention + Feed-Forward +
Residual/LayerNorm, ×6)
  ↓
vector_final_Encoder("The"), vector_final_Encoder("dog"),
vector_final_Encoder("eats"), vector_final_Encoder("bread")
  ↓ (esto es lo que vimos en "El resultado del Encoder" – 4 vectores de 512
números cada uno)
  ↓ cada uno se multiplica por las matrices de Cross-Attention:
  ↓
Ki(palabra) = vector_final_Encoder(palabra) × WKi (de Cross-Attention)
Vi(palabra) = vector_final_Encoder(palabra) × WVi (de Cross-Attention)

```

#### Lo que NO son

No vienen de  $y_1$  (que es lo que da origen solo al **Query**), ni del Decoder, ni de ninguna palabra en alemán. Es justo la diferencia que distingue a Cross-Attention de Self-Attention: en Self-Attention, Q/K/V los tres salen de la **misma** fuente; en Cross-Attention, El Query sale del Decoder, pero K y V siempre salen del **Encoder**.

#### Resumen en una frase

K y V de Cross-Attention vienen del **Encoder**, específicamente de su salida final (tras las 6 capas), calculados con las matrices propias de Cross-Attention – nunca cambian mientras dure la generación de una traducción, porque el Encoder no vuelve a correr.

#### 1.12.10.4.6: Cerrando la sub-capa – concatenando las 8 cabezas y multiplicando por la $W^O$ propia de esta Cross-Attention:

$$\text{Sublayer}_2(y_1) = \text{Concat}(\text{cabeza}_1, \dots, \text{cabeza}_8) W_{\text{cross}}^O \quad (512 \text{ números})$$

osea, en ese resultado ahí tu tienes?

Lo que tengo, en ese momento exacto del cálculo

$$\text{Sublayer}_2(y_1) = \text{Concat}(\text{cabeza}_1, \dots, \text{cabeza}_8) W_{\text{cross}}^O$$

Esto es el resultado **completo** de Cross-Attention para [inicio], en la Capa 1 de la Ronda 1 — un vector de **512 números**.

De dónde salió ese resultado, resumiendo todo el camino

```
y1 (de [inicio], 512 números)
  ↓ 8 copias, una por cabeza
  ↓ cada cabeza calcula su Q (de y1) y su K/V (del Encoder)
  ↓ softmax(QK^T/V64) → porcentajes de atención
  ↓ mezcla ponderada con V → resultado de cada cabeza (64 números)
  ↓ concatenar las 8 cabezas → 512 números
  ↓ × WOcross
  ↓
Sublayer2(y1) – ESTO ES LO QUE TENGO: 512 números
```

Qué falta para completar la Capa 1 (y después las otras 5)

$\text{Sublayer}_2(y_1)$  todavía no es  $y_2$  — falta:

```
Residual + LayerNorm #2: y2 = LayerNorm(y1 + Sublayer2(y1))
  ↓
Feed-Forward
  ↓
Residual + LayerNorm #3: y3 = LayerNorm(y2 + Sublayer3(y2))
```

**Resumen directo**

Y recién ahí termina **la capa 1** - faltarían las Capas 2 a 6 (repitiendo todo este proceso con matrices distintas) antes de llegar al softmax final que elige "Der".

**Resumen directo**

Sí, tengo el resultado de Cross-Attention **de la Capa 1**, antes de su Residual+LayerNorm — es un vector de 512 números, calculado con las 8 cabezas ya vistas. Lo que **no tengo** calculado en detalle es el resto de la Capa 1 (Residual#2, Feed-Forward, Residual#3) con números concretos, ni ninguna de las Capas 2 a 6.

pero que representa el resultado, a eso es lo que voy

**Que representa**

Ese vector es la nueva versión de [inicio], ya “enterada” de que parte de la oración en inglés es relevante para decidir la primera palabra en alemán. Antes de Cross-Attention, [inicio] solo tenía su propio significado (casi vacío, por ser un token de arranque). Después de Cross-Attention, ese mismo vector incorpora – **mezclado, ponderado** – la información de “The” y “Dog” principalmente (según los porcentajes que calculamos: 38%, 34%, 15%, 13%).

### Con la analogía que ya usamos en el documento

Es lo mismo que vimos con “eats” en el Encoder: antes de Self-Attention, “eats” solo sabía su propio significado; después, incorporaba información de “dog” y “bread”. Aca paso algo equivalente, pero **cruzando idiomas: [inicio]** (el token alemán) incorpora información de “The” y “dog” (los tokens ingleses) – no porque se mezclen literalmente las palabras, sino porque el vector numérico que representa a [inicio] ahora “lleva consigo” una pista de que el sujeto de la oración es relevante.

### Por qué esto importa para lo que viene después

Ese vector, después de pasar por Residual+LayerNorm y Feed-Forward (y repetir todo esto 6 veces), termina siendo lo que decide – via softmax – que la primera palabra generada sea “Der” (el artículo que corresponde al sujeto “dog”/“Hund”). El resultado de Cross-Attention **no es “Der” todavía** – es un paso **intermedio que orienta al modelo hacia esa elección, acumulando evidencia capa a capa.**

### Resumen, en una frase

$Sublayer_2(y_1)$  representa: “el token alemán que se está generando, ya con la pista de a qué parte de la oración en inglés debería parecerse” – es información, no todavía una palabra – y esa información sigue transformándose (con Feed-Forward, y con las 5 capas restantes) – **antes de convertirse en la elección final de “Der”.**

#### 1.12.10.4.7: Residual + LayerNorm # 2

$$y_2 = LayerNorm(y_1 + Sublayer_2(y_1)) \quad (512 \text{ números})$$

#### 1.12.10.4.8: Feed-Forward (mismo mecanismo que en el Encoder, con $W_1/W_2$ propias de esta capa)

$$\text{Sublayer}_3(y_2) = \max(0, y_2 W_1 + b_1) W_2 + b_2 \quad (512 \text{ números})$$

Toma ese vector (ya con la pista de a que parte del ingres parecerse) y lo **transforma** — no lo mezcla con nada mas, sino que le aplica su propia “receta” de procesamiento (expandir, filtrar con ReLU, comprimir) para refinar esa información antes de pasar a la siguiente capa. Es el mismo rol que vimos en el Encoder: mientras Cross-Attention decide **de dónde** viene la información relevante, Feed-Forward la **procesa** más allá de una simple mezcla ponderada.

lo transforma en que

Honestamente, con la misma limitación que ya reconocimos antes con Feed-Forward del Encoder: **no hay una traducción limpia a “en qué concepto”** — solo sabemos que es una transformación matemática real (expandir a 2048, filtrar con ReLU, comprimir a 512), optimizada durante el entrenamiento para ayudar al softmax final a elegir mejor la palabra correcta.

Lo único verificado (el paper de Geva et al. que ya investigamos) es que estas capas funcionan como una especie de “memoria clave-valor”: ciertos patrones activan ciertas “neuronas” internas, que a su vez empujan la predicción hacia ciertos tokens del vocabulario — pero no se puede decir “esto lo transforma en el concepto de género gramatical” de forma limpia y verificable. Es una transformación real y medible, pero no interpretable en términos humanos simples.

investiga

Excelente hallazgo, con evidencia muy actual (NeurIPS 2025, octubre 2025 — investigación de vanguardia). Este paper **confirma y refuerza** lo que ya teníamos del paper de Geva et al. (2021) que ya está en el documento — no lo contradice, lo actualiza con más rigor: dice que las “memorias clave-valor” de Feed-Forward son casi tan interpretables como las técnicas más nuevas (sparse autoencoders), que es una noticia interesante para agregar a la sección que ya existe. Pero mi respuesta directa a tu “¿lo transforma en qué?” sigue siendo la correcta y honesta: **sigue sin haber una traducción limpia palabra-por-palabra**, y varias fuentes lo confirman explícitamente (índices 13, 18: “the role of feed-forward layers remains under-explored”).

Investigado. La conclusión honesta se mantiene, pero encontré evidencia más fresca (NeurIPS 2025) que la refuerza:

**Lo que no cambió:** sigue sin haber una traducción limpia “esta transformación significa X concepto humano” — varias revisiones académicas lo confirman explícitamente: “el rol de las capas Feed-Forward sigue siendo poco explorado”, con una brecha real entre patrones matemáticos y conceptos humanos.

**Lo nuevo que encontré:** un paper de NeurIPS 2025 comparó las “memorias clave-valor” de Feed-Forward (el mismo mecanismo de Geva et al. 2021 que ya está en el documento) contra las técnicas de interpretabilidad más modernas y costosas (sparse autoencoders) — y encontró que son **casi igual de interpretables**, e incluso mejores en algunos aspectos. Esto confirma, con evidencia mucho más reciente, que la interpretación “diccionario de patrones” sigue siendo de las mejores explicaciones disponibles hoy.

#### 1.12.10.4.9: Residual + LayerNorm #3:

$$y_3 = \text{LayerNorm}(y_2 + \text{Sublayer}_3(y_2)) \quad (512 \text{ números})$$

Esto fue **la Capa 1**.  $y_3$  se convierte en el x de la Capa 2, y todo el proceso se repite – con matrices distintas – hasta la Capa 6.

#### Que salida es:

La salida de Residual + LayerNorm #3 es **solo del lado alemán**. El ingles incluye en ese vector, pero no viaja dentro de el.

#### Qué sale de Residual + LayerNorm #3

Sale  $y_3$ , con **una fila de 512 numeros por cada token alemán procesado en esa ronda**. Lo que el vector lleva del ingles es información que ya absorvio en Cross-Attention (la sub-capa #2). Los vectores del Encoder no se pegan ni se suman a  $y_3$ .

| Ronda | Entrada al Decoder (alemán) | $y_3$ de esa capa | Del inglés                                        |
|-------|-----------------------------|-------------------|---------------------------------------------------|
| 1     | [inicio]                    | $1 \times 512$    | Absorbido por Cross-Attention, dentro de esa fila |
| 2     | [inicio, Der]               | $2 \times 512$    | Igual, en cada fila                               |
| 3     | [inicio, Der, Hund]         | $3 \times 512$    | Igual, en cada fila                               |

Fijate que el numero de filas es igual que el número de tokens **alemanes** de la ronda. Si el ingles viajara adentro, tendrías  $4 + n$  filas, y no es así.

#### Dónde queda el inglés

Los vectores finales del Encoder quedan aparte, fijos, y solo se usan como Keys y Values dentro de Cross-Attention.

Encoder (inglés)  $\longrightarrow$  K, V de Cross-Attention (solo se consultan)

Decoder (alemán):

$x \rightarrow$  Self-Attn enmascarada  $\rightarrow$  R+LN #1  $\rightarrow y_1$

$y_1 \rightarrow$  Cross-Attention (Query de  $y_1$ , consulta al inglés)  $\rightarrow$  R+LN #2  $\rightarrow y_2$

$y_2 \rightarrow$  Feed-Forward  $\rightarrow$  R+LN #3  $\rightarrow y_3$   $\leftarrow$  solo filas alemanas, con "pistas" del inglés

Cross-Attention hace que cada fila alemana se **mezcle** con información del ingles (como vimos con el 38%, 34%, 15% y 13% en la Ronda 1). Pwero el resultado sigue siendo un vector por token alemán.

#### Una inconsistencia que noté

En el documento, Cross-Attention habla de "4 palabras en inglés", pero después de BPE el Encoder tiene **5 tokens** (The, dog, eat, s, bread). Entonces  $K_1$  y  $V_1$  serían matrices de  $5 \times 64$  y no de  $4 \times 64$ , y el softmax tendría 5 porcentajes en vez de 4. Es un detalle que hay que unificar en el documento. ¿Lo corrijo?

#### 1.12.10.4.10: Capa lineal (después de la capa 6) – traducción entre dos “idiomas”

##### Paso 1: qué sale de las 6 capas

$y_3$  de la Capa 6 (512 números)

↓

Capa lineal (ver desglose completo, abajo)

↓

Un número (logit) por cada token posible del vocabulario

↓

Softmax – convierte esos números en probabilidades que suman 100%

↓

Token ID 5001 ("Der"): 71%

Token ID 5002 ("Hund"): 2%

Token ID 5003 ("isst"): 3%

Token ID 5004 ("Brot"): 8%

... (una probabilidad por cada token del vocabulario)

↓

Se elige el de mayor probabilidad → Token ID 5001

↓

Se busca ese ID en vocab.json → "Der"

↓

Para usarlo como entrada de la Ronda 2: "Der" → Embedding (buscar ID 5001 en la tabla) → + Positional Encoding (posición 0)

↓

$x$  de la Ronda 2 (512 números) – listo para entrar a la Capa 1 de esa nueva ronda

Entro 1 token ([inicio]), así que sale **1 fila de 512 números**:  $y_3$  de la Capa 6, de tamaño  $1 \times 512$ .

Ese vector todavía no es una palabra. Es lo que “sabe” el modelo en este punto para decidir la primera palabra en alemán.

##### Paso 2: “tomar la última fila”

Con un solo token hay una sola fila, así que ya es la última y no hay nada que descartar. Este paso solo importa desde la Ronda 2, cuando salen 2 fila y se usa solo la de “Der”.

##### Paso 3: capa lineal (de 512 números a 37.000)

Hay que pasar de un vector de 512 números a **un número por cada token del vocabulario** (el paper usa ~37.000 tokens)

$$\text{logits} = y_3 \cdot W_{emb}^T [1 \times 512] \cdot [512 \times 37.000] = [1 \times 37.000]$$

- $W_{emb}$  es la **tabla de embeddings** (37.000 filas x 512 columnas). El paper dice que comparte la misma matriz entre los embeddings y esta capa lineal
- Cada logit es un producto punto entre  $y_3$  y el embedding de un token del vocabulario. En la práctica, mide **que tan parecido es  $y_3$  a cada token**.

- Los logits son puntajes crudos: pueden ser negativos o mayores a 1

Ese paso es la **traducción entre dos “idiomas”**:  $y_3$  está en el idioma interno del modelo, **y necesitamos un puntaje por cada palabra del vocabulario.**

### Qué hay en la tabla de embeddings

$W_{emb}$  tiene 37.000 filas y 512 columnas. Cada fila es el vector de 512 de **un token** del vocabulario: la fila de “Der”, la fila de “Hund”, etc. Es la misma tabla que usamos al principio para **convertir Token ID -> vector**.

### El cálculo, con un ejemplo chico

Uso 4 tokens y vectores de 3 números en vez de 512, con valores inventados. La mecánica es idéntica:

Tabla de embeddings (4 tokens × 3 números):

Der: [0.9, 0.8, 0.1]  
Hund: [0.1, 0.2, 0.9]  
isst: [0.2, 0.1, 0.8]  
Brot: [0.3, 0.4, 0.5]

$y_3$  (lo que salió de la Capa 6): [1.0, 1.0, 0.0]

Cada logit es el **producto punto** entre  $y_3$  y la fila de ese token:

$\text{logit}(\text{Der}) = 1.0 \times 0.9 + 1.0 \times 0.8 + 0.0 \times 0.1 = 1.7$   
 $\text{logit}(\text{Hund}) = 1.0 \times 0.1 + 1.0 \times 0.2 + 0.0 \times 0.9 = 0.3$   
 $\text{logit}(\text{isst}) = 1.0 \times 0.2 + 1.0 \times 0.1 + 0.0 \times 0.8 = 0.3$   
 $\text{logit}(\text{Brot}) = 1.0 \times 0.3 + 1.0 \times 0.4 + 0.0 \times 0.5 = 0.7$

Los logits son [1.7, 0.3, 0.3, 0.7]. Después de softmax:

$$\text{softmax}([1.7, 0.3, 0.3, 0.7]) \approx [0.54, 0.13, 0.13, 0.20]$$

“Der” gana, porque  $y_3$  apunta hacia donde está el embedding de “Der”.

### Por qué aparece la traspuesta ( $W_{emb}^T$ )

Es solo para que las dimensiones encajen. En el ejemplo, la tabla es 4 x 3 (un token por fila) y  $y_3$  es 1 x 3, así que hay que voltear la tabla:

$$[1 \times 3] \cdot [3 \times 4] = [1 \times 4]$$

En el modelo real es lo mismo con otros tamaños:

$$[1 \times 512] \cdot [512 \times 37.000] = [1 \times 37.000]$$

Es una sola multiplicación, unos 19 millones de productos individuales ( $512 \times 37.000$ ).

### Por qué el paper reutiliza la misma matriz

Según el paper (Sección 3.4), la matriz de la capa lineal es la misma que la de embeddings. Como el embedding de “Der” es el punto del espacio de 512 dimensiones donde “vive” esa palabra, el modelo aprende a producir un  $y_3$  que **apunte hacia el embedding de la palabra que quiere decir**. Así el mismo espacio sirve para entrar (palabra -> vector) y para salir (vector -> palabra).

Digo que **la misma tabla se usa dos veces**: una para entrar al modelo y otra para salir.

## La tabla

Es una tabla de 37.000 filas, una por token, y cada fila son 512 números. La fila de "Der" es el vector que representa a "Der".

### Uso 1: al entrar (palabra → vector)

Tengo el Token ID de "Der" y busco su fila en la tabla:

```
Token ID 5001 → fila 5001 de la tabla → [0.9, 0.8, 0.1, ...]
```

### Uso 2: al salir (vector → palabra)

Después de las 6 capas tengo  $y_3$ , un vector de 512 números, y necesito saber a qué palabra corresponde. Lo comparo contra **cada fila de la misma tabla** con un producto punto. **Ese producto da un puntaje alto cuando los dos vectores apuntan hacia el mismo lado:**

```
 $y_3 = [1.0, 1.0, 0.0]$ 
```

```
vs. fila de Der: [0.9, 0.8, 0.1] → puntaje 1.7 (apuntan parecido)
```

```
vs. fila de Hund: [0.1, 0.2, 0.9] → puntaje 0.3 (apuntan a otro lado)
```

La palabra cuya fila queda más alineada con  $y_3$  recibe el puntaje más alto, y softmax lo convierte en la probabilidad más alta.

### Qué quiere decir "apuntar hacia el embedding"

Cada palabra tiene una posición en un espacio de 512 dimensiones, que es la fila de la tabla. Para generar "Der", el modelo aprende a dejar  $y_3$  **cerca de esa posición**. Como al salir se compara contra las mismas filas que se usaron al entrar, "Der" es a la vez el punto de entrada y el punto de llegada.

### Por qué compartir la tabla

Lo que **si dice el paper** (Sección 3.4) es que comparten la misma matriz entre las dos capas de embedding y la transformación lineal previa al softmax. Lo que **no dice** en ese párrafo es el motivo. **Una razón habitual es ahorrar parámetros**: una tabla de 37.000 x 512 son unos 19 millones de números, y así no ha que entrenar una segunda tabla igual de grande. La idea de que "entrada y salida hablan el mismo idioma" es una intuición útil sobre por qué funciona, no algo que el paper demuestre.

### En una frase

Al entrar, uso la tabla para convertir una palabra en su vector. Al salir, la uso al revés: comparo mi vector final contra todos los vectores de la tabla y gana la palabra más alineada.

### Un matiz sobre "qué tan parecido"

Dije que el logit mide qué tan parecido es  $y_3$  a cada token. Es una buena intuición, pero no es exacta: el producto punto también depende del **largo** de los vectores, no solo de hacia dónde apuntan. Lo correcto es "qué tan alineado está  $y_3$ , con cada token, ponderado por su magnitud".

### El "largo" de un vector



El largo (o magnitud) de un vector es que tan grande es en total, sin importar hacia donde apunte. Dos vectores pueden apuntar exactamente en la misma dirección y tener largos distintos:

```
[1, 1, 0] → largo corto  
[2, 2, 0] → misma dirección, el doble de largo
```

El producto punto premia las dos cosas: que apunten parecido y que sea grandes. Con el embedding de "Der" = [0.9, 0.8, 0.1]:

```
[1, 1, 0] · Der = 1.7  
[2, 2, 0] · Der = 3.4 ← misma dirección, pero el logit se duplicó
```

Por eso "que tan parecido" no es exacto: un token con un embedding mas largo tiende a sacar un logit mas alto, aunque apunte igual.

### Tu segunda pregunta: ¿ $y_3$ se parece a los embeddings del vocabulario?

Casi, con dos ajustes:

1. **No se parece a los 37.000, se alinea con uno o con pocos.**  $y_3$  es un solo vector de 512 numeros. Si el modelo "quiere decir" Der,  $y_3$  apunta hacia el embedding de "Der" y queda poco alineado con casi todos los demás. Los 37.000 logits se calculan despues, cuando lo multiplicas por la tabla.
2. **No es una copia del embedding.** Apunta hacia esa zona del espacio de 512 dimensiones, pero no es igual. Por eso el softmax reporta probabilidades (0.71 para Der, algo para Brot) en vez de dar 1.0 a una sola.

Todo esto viene de que el paper comparte la misma matriz en ambos lados: el embedding es el "lugar" donde vive cada palabra en el espacio de 512 dimensiones. El modelo, en las 6 capas, aprende a dejar  $y_3$  apuntando al lugar de la palabra que corresponde.

### Un matiz sobre el largo de $y_3$

$y_3$  sale de un LayerNorm, que reescala el vector para que su largo quede mas o menos controlado (aunque  $\gamma$  y  $\beta$ , que son aprendidos, lo ajustan un poco). Entonces las diferencias entre logits dependen sobre todo de **hacia donde apunta**  $y_3$ , y del largo de cada embedding, y no de que  $y_3$  sea mas grande o mas chico.

que es la direccion y el largo (breve)

Un vector se puede ver como una flecha:

- **Direccion:** hacia donde apunta la flecha
- **Largo:** que tan larga es, sin importar hacia donde apunte.

```
[1, 1, 0] → apunta hacia (1, 1, 0), largo corto  
[2, 2, 0] → apunta exactamente igual, el doble de largo  
[0, 0, 1] → apunta a otro lado, largo 1
```

El largo se calcula como  $\sqrt{a^2+b^2+c^2}$ . Por ejemplo, para  $[1,1,0]$  es  $\sqrt{2} \approx 1,41$ , y para  $[2,2,0]$  es  $\sqrt{8} \approx 2,83$

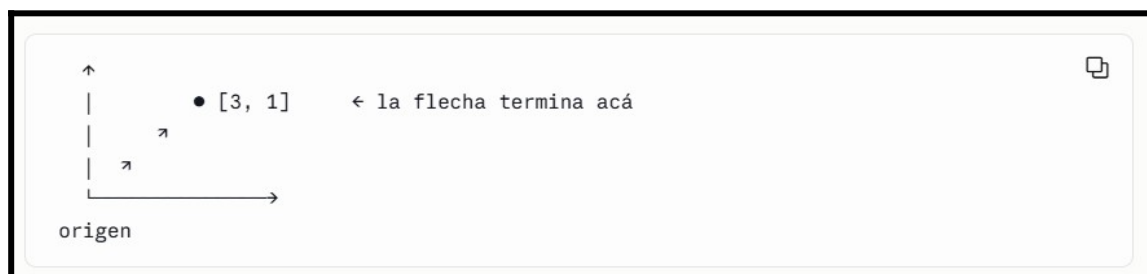
En los logits, el producto punto depende de las dos cosas: **Si dos vectores apuntan hacia el mismo lado, el puntaje sube**, y si además son largos, sube todavía mas.

no entiendo direccion

La dirección es **hacia donde apunta la flecha**, sin importar que tan larga sea. Es como decir "hacia el norte" sin decir cuantos kilómetros.

### Con un dibujo en 2 números

Un vector de 2 numeros, como  $[3,1]$ , se puede ver como una flecha desde el origen: 3 pasos hacia la derecha y 1 hacia arriba.



- **Dirección:** hacia la derecha, un poco hacia arriba.
- **Largo:** La distancia del origen a ese punto.

### Misma dirección, distinto largo

$[3, 1]$  → apunta hacia la derecha, un poco arriba  
 $[6, 2]$  → apunta EXACTAMENTE igual (es el doble), pero llega el doble de lejos

Los dos están en la misma línea, así que tienen **la misma dirección**. Cambia solo que tan lejos llegan.

### Distinta dirección

$[3, 1]$  → derecha, un poco arriba  
 $[1, 3]$  → arriba, un poco a la derecha ← apunta a otro lado  
 $[-3, -1]$  → apunta hacia el lado contrario

Aunque tengan el mismo largo, apuntan a lugares distintos.

### Cómo se relaciona con nuestro ejemplo

Con 3 números,  $y_3 = [1, 1, 0]$  y el embedding de "Der" =  $[0.9, 0.8, 0.1]$  tienen **direcciones parecidas**: los dos son altos en las dos primeras posiciones y casi cero en la tercera. "Hund" =  $[0.1, 0.2, 0.9]$  es al revés (alto en la tercera), así que apunta a otro lado. Por eso el producto punto da alto con "Der" y bajo con "Hund".

Con 512 numeros no se puede dibujar, pero la idea es la misma: dos vectores "apuntan parecido" cuando sus numeros altos y bajos coinciden en las mismas posiciones.

### Sobre logit:

Lo segundo: "logit" es solo el **nobmre del resultado** de esa multiplicación. No hay

ningún paso extra que le saque dirección y largo.

El producto punto ya mezcla las dos cosas por si solo: sube si y3 y la fila apuntan parecido, y sube mas si además son largas. Es una propiedad de la cuenta, no una operación aparte.

#### 1.12.10.4.11+Agregado Aclaracion: escalado $\sqrt{d\_model}$ en embeddings de entrada (capa lineal usa la tabla cruda) – Softmax

##### Una sola tabla, tres usos

- La tabla de embeddings  $E$  tiene ~37.000 filas x 512 columnas. La fila  $E_j$  es el vector del token  $j$ .
- El paper (Seccion 3.4) dice que comparte la misma matriz entre las dos capas de embedding y la transformación lineal previa al softmax. Son tres usos de la misma tabla:
  1. Embedding de entrada del Encoder (ingles).
  2. Embedding de entrada de Decoder (alemán)
  3. Capa lineal de salida (antes del softmax)
- Es posible porque el vocabulario BPE es único y compartido (Seccion 5.1).
- Los 512 numeros de cada fila salen del entrenamiento, así que usa notación en vez de valores inventados.

##### Al entrar (Encoder y Decoder)

Formula, dimensión por dimensión ( $k = 0, \dots, 511$ ):

$$x[k] = \sqrt{512} \cdot E[\text{ID}][k] + PE_{(pos, k)}$$

- $\sqrt{512} \approx 22,627$ . El paper: "In the embedding layers, we multiply those weights by  $\sqrt{d_{model}}$ ".
- Positional Encoding (con  $d_{model} = 512$ ):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/512}}\right) \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/512}}\right)$$

- Primeras 4 dimensiones, calculadas:

$$PE_{(0, 0:4)} = [0, 1, 0, 1]$$

$$PE_{(1, 0:4)} = [0.8415, 0.5403, 0.8219, 0.5697]$$

- En la Ronda 2, "Der" va en la posición 1, porque [inicio] ocupa la posición 0 (entrada "desplazada a la derecha", Figura 1):

$$x(\text{Der}) = \sqrt{512} \cdot E[\text{ID de "Der"}] + PE_{(1)} \quad (512 \text{ números})$$

- El PE se suma **después** de escalar.

##### Al salir (capa lineal)

$$\text{logits} = y_3 \cdot E^T$$

- Tamaños:

$$[n \times 512] \cdot [512 \times 37.000] = [n \times 37.000]$$

donde  $n$  es la cantidad de tokens que entraron al Decoder. Para predecir la palabra nueva solo se usa la última fila:

$$[1 \times 512] \cdot [512 \times 37.000] = [1 \times 37.000]$$

- Se usa  $E$  **cruda**: sin multiplicar por  $\sqrt{512}$  y sin posición.
- $E^T$  es  $E$  traspuesta, solo para que las dimensiones encajen.
- Cada logit es un producto punto:

$$\text{logit}_j = \sum_{k=0}^{511} y_3[k] \cdot E[j][k]$$

Son 512 productos por token y  $37.000 \times 512 = 18.944.000$  productos en total por fila.

- Ese producto punto equivale a:

$$\text{logit}_j = \|y_3\| \cdot \|E_j\| \cdot \cos \theta$$

Depende de las tres cosas a la vez: el largo de  $y_3$ , el largo de la fila y el ángulo entre ambos. No hay ningún paso extra que "saque" dirección y largo. "Logit" es solo el nombre del resultado.

- Los logits son puntajes crudos: pueden ser negativos o mayores que 1.

## Softmax

$$p_j = \frac{e^{\text{logit}_j}}{\sum_{l=1}^{37.000} e^{\text{logit}_l}}$$

- Da 37.000 probabilidades que suman exactamente 1.
- No hay filtro de idioma: el softmax se calcula sobre todo el vocabulario, y el entrenamiento hace que los tokens que no corresponden queden con probabilidad casi cero.

## Escalado $\sqrt{d_{model}}$ : que es y que no es

- No es LayerNorm. LayerNorm vive dentro de Residual + LayerNorm y reajusta promedio y dispersión del vector.
- El paper solo menciona el escalado para las capas de embedding. Que la capa lineal use la matriz cruda es lo que se deduce del texto, y las implementaciones habituales lo hacen así.
- Si todos los logits se multiplicaran por una constante  $c > 1$ , no cambiaría que token gana, pero el softmax quedaría más "puntiagudo".
- Por que se escala al entrar (explicación habitual, no del paper): los valores de E suelen ser chicos, y el PE (entre -1 y 1) podría taparlos.

## Resumen

$$\text{Al entrar: } x = \sqrt{512} \cdot E[\text{ID}] + PE_{(pos)}$$

$$\text{Al salir: } \text{logits} = y_3 \cdot E^T \quad (\text{sin escalar})$$

## 2da explicación:

### Tabla de embeddings: se usa dos veces

- Es una tabla de ~37.000 filas x 512 columnas. Cada fila es el vector de un token
- **Al entrar** (palabra -> vector): se busca la fila del Token ID.
- **Al salir** (vector -> palabra): el vector final  $y_3$ , se compara contra todas las filas de la misma tabla.
- El paper (Sección 3.4) dice que comparte la misma matriz entre las capas de embedding y la transformación lineal previa al softmax.

### Al entrar: embedding de "Der" (Ronda 2, posición 1)

Ejemplo de juguete con 3 números en vez de 512 (valores inventados):

```
Fila de la tabla:      [0.9,  0.8,  0.1 ]
× √3 ≈ 1,73:          [1.56,  1.39,  0.17 ]
+ Positional (pos. 1): [0.84,  0.54,  0.002]
= x que entra a la Capa 1: [2.40,  1.93,  0.18 ]
```

En el modelo real el escalado es  $\sqrt{512} = 22.6$ . "Der" esta en la posición 1 porque **[inicio]** ocupa la posición 0.

### Al salir: capa lineal (logits)

Formula:  $\text{logits} = y_3 \cdot W_{\text{emb}}^T$

- Tamaños reales:  $[1 \times 512] \cdot [512 \times 37.000] = [1 \times 37.000]$
- Se usa la tabla **cruda**, sin escalar y sin posición.
- $W_{\text{emb}}^T$  es la tabla traspuesta, solo para que las dimensiones encajen.
- Un logit es un puntaje crudo por token. Puede ser negativo o mayor que 1, y despues pasa por softmax.
- Cada logit es el producto punto entre  $y_3$ , y la fila del token. Sube si los dos apuntan hacia el mismo lado, y también si son largos. No hay ningún paso extra que "saque" dirección y largo

Ejemplo de juguete (4 tokens, 3 números, valores inventados):

```
Der:  [0.9, 0.8, 0.1]      y3 = [1.0, 1.0, 0.0]
Hund: [0.1, 0.2, 0.9]
isst: [0.2, 0.1, 0.8]
Brot: [0.3, 0.4, 0.5]

logits = [1.7, 0.3, 0.3, 0.7]
softmax ≈ [0.54, 0.13, 0.13, 0.20] → gana "Der"
```

$y_3$  no es igual a la fila de "Der", pero apunta hacia el mismo lado (numeros altos en las dos primeras posiciones y casi cero en la tercera).

### Escalado por $\sqrt{d_{\text{model}}}$ (no es LayerNorm)

El paper dice: "In the embedding layer, we multiply those weights by  $\sqrt{d_{\text{model}}}$ ".

- Embedding de entrada del **Encoder** (ingles): si escala
- Embedding de entrada del **Decoder** (alemán): si escala
- **Capa línea final**: No escala, usa la matriz cruda. El paper solo menciona el escalado para las capas de embedding, así que esto es lo que se deduce del texto, y las implementaciones habituales lo

hacen así.

- Después de escalar, a ambas entradas se les suma el Positional Encoding
- LayerNorm es otra cosa: vive dentro de Residual + LayerNorm y reajusta promedio y dispersión del vector.

### Que pasaría si la capa lineal escalara

Multiplicar la tabla por 1.73 multiplica todos los logits por 1,73:

```
Tabla cruda: logits [1.7, 0.3, 0.3, 0.7] → softmax [0.54, 0.13, 0.13, 0.20]
Tabla escalada: logits [2.9, 0.5, 0.5, 1.2] → softmax [0.74, 0.07, 0.07, 0.13]
```



No cambia quien gana, pero el softmax queda mas “puntiagudo”.

### Por qué se escala al entrar (explicación habitual, no del paper)

Los valores del embedding suelen ser chicos, y el Positional Encoding (seno y coseno entre -1 y 1) podría taparlos. Multiplicar por 22.6 los agranda para que la posición se sume sin dominar el significado.

### Resumen

- **Al entrar (Encoder y Decoder):** Fila de la tabla  $\times \sqrt{512}$  + Positional Encoding
- **Al salir (capa lineal):**  $y_3 \times$  tabla cruda, sin escalar

#### 1.12.10.4.12: Softmax (salida de 37.000 probabilidades que suman 1.0)

### Paso 4: softmax

Convierte los 37.000 logits en 37.000 probabilidades que suman exactamente 1.0, con la formula que vimos  $e^{x_i} / \sum_j e^{x_j}$ . Un ejemplo:

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

```
"Der": 0.71  
"Hund": 0.02  
"isst": 0.03  
"Brot": 0.08  
resto (~36.996 tokens): 0.16 repartido entre todos
```

1 x 37.000 (bpe)

Embedding

37.000 (palabras) x 512 (pesos que significa la palabra) brackprohgapagioon

Con un vocabulario real, estos cuatro no tienen por qué sumar 100%, porque el resto se reparte entre los demás tokens.

Lo desarrollo con un vocabulario de solo 3 tokens ("Der", "Hund", "Brot") y logits inventados, para que se vea cada paso. Con 37.000 tokens la mecánica es idéntica, solo que la suma de abajo tiene 37.000 términos.

#### La fórmula

$$p_j = \frac{e^{z_j}}{e^{z_1} + e^{z_2} + e^{z_3}} \quad j = 1, 2, 3$$

donde  $z_j$  es el logit del token  $j$ .

**Los logits de partida (inventados)**

$$z = [z_1, z_2, z_3] = [2.0, 1.0, 0.1] \quad (\text{para "Der", "Hund", "Brot"})$$

**Paso 1: elevar  $e$  a cada logit**

$$e^{z_1} = e^{2.0} \approx 7.3891$$

$$e^{z_2} = e^{1.0} \approx 2.7183$$

$$e^{z_3} = e^{0.1} \approx 1.1052$$



**Paso 2: sumar (el denominador, el mismo para los tres)**

$$e^{z_1} + e^{z_2} + e^{z_3} \approx 7.3891 + 2.7183 + 1.1052 = 11.2125$$

**Paso 3: dividir cada uno por esa suma**

$$p_1 = \frac{7.3891}{11.2125} \approx 0.6590 \quad (\text{"Der"})$$

$$p_2 = \frac{2.7183}{11.2125} \approx 0.2424 \quad (\text{"Hund"})$$

$$p_3 = \frac{1.1052}{11.2125} \approx 0.0986 \quad (\text{"Brot"})$$

**Comprobación: suman 1**

$$p_1 + p_2 + p_3 \approx 0.6590 + 0.2424 + 0.0986 = 1.0000$$

Suman exactamente 1 porque el denominador es la suma de los tres numeradores. Cada  $p_j$  es la parte que le toca a  $e^{z_j}$  del total.

**Dos cosas que muestra el ejemplo**

- **Siempre da positivo, aunque el logit sea negativo.** Con  $z = [2.0, 1.0, -1.0]$ ,  $e^{-1} \approx 0.3679$  sigue siendo positivo, y el resultado es  $[0.7054, 0.2595, 0.0351]$ .
- **Los logits chicos igual se convierten en probabilidades grandes de diferencia.** Una diferencia de 1 punto entre logits (2.0 vs. 1.0) hace que "Der" tenga casi 3 veces la probabilidad de "Hund" ( $0.6590/0.2424 \approx 2.7$ ). Es la razón por la que  $e^z$  "exagera" las diferencias.

### 1.12.10.4.13: Paso 5: Elegir la palabra (greedy decoding o beam search GNMT) – recordando tamaño de embedding

Después del softmax hay 37.000 probabilidades y hay que quedarse con una decisión.

- **Opcion simple (greedy):** tomar la más alta. Acá sería "Der" (0.71). El problema es que la mejor **palabra de ahora puede llevar a una oración peor más adelante.**
- **Lo que usa el paper (Sección 6.1):** beam search con tamaño de haz 4 y penalización de longitud  $\alpha=0.6$ . El paper cita a Wu et al. (GNMT) para esto y no explica el algoritmo, así que lo que sigue es el algoritmo estándar.

Inicio, Der

Inicio, hund

Inicio, isst

Inicio, Trinken

**Cómo funciona (para beam search de forma simplificada):**

1. Se guardan **4 secuencias parciales** a la vez.
2. En cada ronda, el Decoder corre para cada una y da 37.000 probabilidades por secuencia, o sea 4 x 37.000 extensiones posibles.
3. Se conservan las **4 mejores** por puntaje, y el puntaje es la suma de los logaritmos de las probabilidades. Ejemplo para la primera palabra:  $\ln(0.71) \approx -0.342$ .

Por qué hace falta la penalización de longitud: los logaritmos de probabilidades son negativos, así que cada palabra extra baja el puntaje y las oraciones largas salen perjudicadas. Wu et al. lo compensan con  $\alpha$ : el puntaje es  $\log P(y|x)$  multiplicado por  $(1 + 5)^\alpha / (1 + |y|)^\alpha$ .

**Una simplificación que hice yo:** dije "seguimos con la mejor, 'Der'". En beam search real, en la Ronda 2 siguen las 4 secuencias, y al final gana la secuencia completa con mejor puntaje, que no siempre empezó con la palabra más probable. El ejemplo sigue un solo camino ("Der Hund isst Brot") solo para que se lea fácil.

### Otra explicación

**Acá va la versión solo con greedy.**

- **Lo que dice el paper:** El paper no usa greedy son beam search, que veremos en el próximo mensaje. Esta es la versión simplificada.
- **Lo que si sale del paper:** La entrada "desplazada a la derecha", el escalado por  $\sqrt{d_{model}}$ , el positional encoding y el largo máximo de salida.
- **Lo que cambié:** Inverti los Pasos 6 y 7, preparar la ronda siguiente ocurre **dentro del bucle**, y pasar a texto ocurre **una vez, al final**

### Paso 5: elegir la palabra (greedy)

El softmax dejó 37.000 probabilidades (Der 0.71, Brot 0.08, isst 0.03, Hund 0.02 y el resto repartido). Greedy toma el token con la probabilidad más alta:

$$\hat{y} = \arg \max_j p_j$$

Acá gana "Der" (0.71), con Token ID 5001 (inventado).

- **Sale un ID, no texto**
- **Es definitiva:** Una vez elegido, el token no se corrige mas adelante.
- **Es determinista:** Con la misma entrada siempre sale lo mismo
- **La limitación:** La mejor palabra de ahora puede llevar a una oración peor mas adelante. Por eso el paper usa beam search.

### La fórmula en una frase

$$\hat{y} = \arg \max_j p_j$$

Se lee: acute {y es la posición j donde p<sub>j</sub> es más grande”.

Tenes una lista de 37.000 probabilidades, una por token. Busas cual es la mayor y te quedas con **su posición** en la lista. Esa posición es el ID del token elegido.

| Símbolo           | Que es                                                                                                              | Ejemplo               | Forma                            |
|-------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------|----------------------------------|
| P                 | La lista completa de probabilidades (salida del softmax). Hay una por token y suman 1                               | [0.0001,...,0.71,...] | Vector de 1 x 37.000             |
| J                 | Una <b>posición</b> de esa lista, que es también el ID de un token                                                  | 0, 1, 2, ..., 36.999  | Un entero                        |
| $p_j$             | La probabilidad que esta en la posición j                                                                           | $p_{5001}=0.71$       | Un numero                        |
| $\arg \max_j p_j$ | “el j que hace mas grande a p <sub>j</sub> ”: recorre la lista y devuelve la <b>posición</b> del mayor, no su valor | 5001                  | Operación que devuelve un entero |
| $\hat{y}$         | El resultado: el ID elegido                                                                                         | 5001                  | Un entero (1x1)                  |
| =                 | “Se asigna” guarda el resultado en $\hat{y}$                                                                        |                       |                                  |

Ninguno de estos símbolos es un peso entrenado.  $p$  depende de los pesos porque se calcula con ellos, pero no es un peso.

### Ejemplo con 5 posiciones (valores inventados)

```
posición j:  0      1      2      3      4
p           = [0.02, 0.71, 0.08, 0.03, 0.16]

p_1 = 0.71           ← el valor que hay en la posición 1
max p_j = 0.71       ← el valor más grande
argmax p_j = 1       ← la posición del más grande
ŷ = 1                ← el ID elegido
```

Con el vocabulario real es lo mismo: si el 0.71 está en la posición 5001, entonces  $\arg \max = 5001$  y  $\hat{y}=5001 = 5001$ .

### Por qué la posición es el ID

Porque el número de la posición  $j$  se calculó con la fila  $j$  de la tabla de embeddings  $E$ , que es la del token con ID  $j$ . El ID no contiene la probabilidad: solo dice **dónde** está.

## Qué es de fuente y qué es mío

- **La fórmula es estándar.** Aparece como  $w_t = \arg \max_w P(w \mid w_{1:t-1})$  en el blog de Hugging Face sobre métodos de decodificación y en un survey reciente de decodificación.
- **La notación  $(\hat{y}, p_j)$  es mía.** El gorrito para "predicho" es una convención habitual, pero no la verifiqué en esas fuentes.
- **El paper no usa greedy.** Usa beam search (Sección 6.1), que veremos en el próximo mensaje.
- **Tres cosas de mi nota original no las verifiqué en ninguna fuente:** que en entrenamiento no se usa el argmax, la regla de empates y que el ID coincide con el número de fila de  $E$  (es la convención de las implementaciones habituales). Las sacaré o las marcaré como no verificadas.

## Mas aclaraciones

posición (ID): 0 1 2 3 4 ... 5001 ...  
probabilidad: [0.0001, 0.0002, 0.0001, 0.0003, 0.0001, ... 0.71 ... ]  
como probabilidad tiene tamaño  $1 \times 37.000$ , me imagino que el token id tambien tiene lo mismo

No: el token ID es **un solo numero entero** (tamaño  $1 \times 1$ ). Lo que mide  $1 \times 37.000$  es la lista de probabildiades, porque tiene una probabilidad por cada ID posible.

## La confusión típica

El 37.000 aparece por dos motivos distintos:

| Qué es                  | Tamaño                   | Qué significa el 37.000                     |
|-------------------------|--------------------------|---------------------------------------------|
| Lista de probabilidades | $1 \times 37.000$        | Hay una probabilidad por cada token posible |
| Token ID elegido        | $1 \times 1$ (un número) | Es un número entre 0 y ~36.999              |

El ID es un valor que puede ir de 0 a unos 36.999. No es una lista de 37.000 elementos.

## Qué hace el argmax con los tamaños

$$\underbrace{p}_{1 \times 37.000} \xrightarrow{\arg \max} \underbrace{\hat{y}}_{1 \times 1} = 5001$$

Entran 37.000 numeros y sale uno solo: la posición del mas grande. En la lista, esa posición es el ID.

## Los tamaños que puede tener "un ID"

| Forma de representar el ID de "Der" | Tamaño            | Contenido                                    |
|-------------------------------------|-------------------|----------------------------------------------|
| El ID como número                   | $1 \times 1$      | 5001                                         |
| Vector <i>one-hot</i> del ID        | $1 \times 37.000$ | Un 1 en la posición 5001 y ceros en el resto |
| Embedding del ID                    | $1 \times 512$    | La fila $E[5001]$                            |

Tu intuición de  $1 \times 37.000$  coincide con la representación **one-hot**, que es equivalente al ID pero mucho más grande. En el entrenamiento se suele comparar con el token correcto en esa forma. No lo verifiqué en el paper.

#### Resumen:

- **Probabilidades:**  $1 \times 37.000$ , una por token, y la posición en la lista es el ID
- **Token ID:**  $1 \times 1$ , un numero entero
- **One-hot del ID:**  $1 \times 37.000$ , una forma mas grande de escribir lo mismo.

#### Si es verificada o no:

pero esperate, osea que esa explicacion es de mentira? o es un aproximado a lo que realmente sería

No es de mentira. La fórmula es exacta para lo que describe, que es **greedy decoding**. Lo que simplifiqué es otra cosa.

#### Que es exacto

- **La formula:**  $\arg \max$  sobre las probabildiades es la definición estándar de greedy: elegir en cada paso la palabra con mayor probabilidad. Un survey reciente de decodificación da la misma dirección.
- **Que el resultado sea un ID (un entero):** es lo que hace  $\arg \max$  en el código real. Devuelve la posición del mayor, no su valor.
- **Que el vector de probabilidades mida  $1 \times 37.000$ :** el tamaño del vocabulario ( $\sim 37.000$ ) es el paper (Seccion 5.1).

#### Qué es una simplificación o una convención

| Punto                                                       | Estado                                                                                                                                 |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Greedy no es lo que usa el paper</b>                     | El paper usa beam search (Sección 6.1). Greedy es la versión más simple, útil para entender. Como descripción del paper, es incompleta |
| <b>Los números</b> (0.71, 5001, 0.08...)                    | Inventados, para ilustrar. No salen de ningún modelo real                                                                              |
| <b>La notación</b> $(\hat{y}, p_j)$                         | Es mía. Las fuentes escriben $w_t$ y $P(w \mid w_{1:t-1})$ , con el mismo significado                                                  |
| <b>"La posición <math>j</math> es el ID <math>j</math>"</b> | Es una convención estándar de las implementaciones. Se deduce de que la matriz $E$ se comparte, pero el paper no lo dice               |
| <b>argmax sobre probabilidades vs. logits</b>               | En el código real suele aplicarse sobre los logits. Da el mismo resultado, porque el softmax conserva el orden                         |
| <b>Chat de un LLM</b>                                       | Suele muestrear en vez de usar argmax, como te dije antes                                                                              |

En resumen: **la fórmula y la mecánica son reales**, y lo que cambia respecto de un modelo de producción es que los números son inventados y que el paper usa beam search en vez de greedy.

## GNMT beam search

El sistema [GNMT \(Google's Neural Machine Translation\)](#) utiliza **beam search** (búsqueda de haces) como **algoritmo de decodificación para generar traducciones palabra por palabra maximizando la probabilidad condicional del texto de salida.**

### ¿Cómo funciona?

- **Mantenimiento de candidatos:** En cada paso de tiempo, el decodificador guarda un numero fijo de secuencias activas parciales conocido como **ancho de haz** (*beam size, comunmente  $k=5$* )
- **Expansion y puntuación:** Cada secuencia actual se expande con todas las palabras posibles del vocabulario. Se evalua la probabilidad conjunta acumulada de toda la frase resultante.
- **Poda (Pruning):** Se cescartar las opciones con menor puntuación y solo se conservan las k mejores rutas para el siguiente paso, evitando explorar un arbol de decisiones infinitamente grande.

### Ajustes clave en GNMT

- **Normalización de longitud (Length Normalization):** Como las oraciones mas largas tienen a acumular probabildiades mas bajas por la multiplicación continua, GNMT aplica un factor de normalización para evitar sesgar el resultado hacia traducciones excesivamente cortas.
- **Penalización por cobertura (Coverage Penalty):** Penaliza las palabras del texto de origen que no han sido traducidas adecuamdanente o que se repiten en exceso, fomentando que toda la fuente sea cubierta de forma equilibrada.

que forma matematica tiene el embedding a todo esto?, que tamaño tiene?

El embedding es una **matriz E** con una fila por token del vocabulario. Mide  $V \times 512$ , donde  $V \approx 37.000$  es el tamaño del vocabulario compartido del paper (Seccion 5.1) y 512 es  $d_{model}$ .

$$E \in \mathbb{R}^{V \times 512} \quad V \approx 37.000$$

### Cada pieza del recorrido, con su forma

| Símbolo                 | Qué es                                                       | Forma                 | ¿Entrenado?                     |
|-------------------------|--------------------------------------------------------------|-----------------------|---------------------------------|
| <b>E</b>                | La tabla de embeddings completa: una fila por token          | 37.000 x 512 (matriz) | <b>Si</b>                       |
| <b>E[j]</b>             | La fila j: el vector del token con ID j                      | 1 x 512 (vector)      | Si, es parte de E               |
| $\sqrt{512} \cdot E[j]$ | La fila escalada al entrar (Seccion 3.4 del paper)           | 1 x 512               | No es un peso nuevo             |
| <b>P E (pos)</b>        | El Positional Encoding de esa posición, con seno y coseno    | 1 x 512               | <b>No</b> (es una formula fija) |
| <b>X</b>                | Lo que entra a la Capa 1: $\sqrt{512} \cdot E[j] + PE(pos)$  | 1 x 512               | <b>No</b> , es un resultado     |
| $E^T$                   | La misma tabla traspuesta, usada en la capa lineal de salida | 512 x 37.000          | Es la misma E                   |

Para una secuencia de n tokens, las filas x se apilan y entra una matriz de  $n \times 512$ . Sobre el PE, el paper probó también posiciones aprendidas y obtuvo resultados casi idénticos.

### Tamaño total

$$37.000 \times 512 = 18.944.000 \text{ números}$$

Son unos 19 millones de números que se entrenan. Como la misma matriz se usa al entrar (Encoder y Decoder) y al salir (capa lineal), se guardan una sola vez.

### Cómo se ve la operación

**Buscar una fila.** En la practica es solo una consulta: se toma la fila  $j$  de  $E$ .

**Equivalencia matemática.** Es lo mismo que multiplicar un vector “one-hot” (con un 1 en la posición  $j$  y ceros en el resto) por la tabla:

$$\underbrace{[0, \dots, 1, \dots, 0]}_{1 \times V} \cdot \underbrace{E}_{V \times 512} = \underbrace{E[j]}_{1 \times 512}$$

Las implementaciones usan la consulta directa porque multiplicar por un vector casi todo de ceros seria un gasto inútil. La equivalencia es matemática, y no la verifiqué en el paper.

**Al salir,** se usa la misma tabla al revés:

$$\text{logits} = y_3 \cdot E^T \quad [1 \times 512] \cdot [512 \times 37.000] = [1 \times 37.000]$$

### Resumen

- **Entrar:**  $ID \rightarrow \text{fila } E[ID] (1 \times 512) \rightarrow \text{escalar} \rightarrow \text{sumar PE} \rightarrow x$ .
- **Salir:**  $y_3 (1 \times 512) \cdot E^T \rightarrow 37.000 \text{ logits} \rightarrow \text{softmax} \rightarrow 37.000 \text{ probabilidades}$ .
- **Mismo  $E$**  en los dos sentidos y en Encoder y Decoder (paper, Sección 3.4).



#### 1.12.10.4.14: Paso 6: Preparar la Ronda 2 (dentro del bucle)

El ID elegido en el Paso 5 (5001, inventado) se agrega a la secuencia y se vuelve a convertir en vector, con el mismo camino de entrada:

$$x(\text{Der}) = \sqrt{512} \cdot E[5001] + PE_{(1)} \quad [1 \times 512]$$

- **[inicio]** ocupa la posición 0 y “Der” la posición 1 (la entrada esta “desplazada a la derecha”, Figura 1 del paper)
- **Version conceptual:** entra la secuencia **[inicio, Der]** como una matriz  $[2 \times 512]$ , y de salida se usa solo la ultima fila, la de “Der”.
- **En la practica (no cache):** Entra solo el vector de “Der”  $(1 \times 512)$ , y los K/V de **[inicio]** salen de un cache. Da el mismo resultado, porque el masking impide que **[inicio]** cambie.
- El Encoder no vuelve a correr. La ronda repite: 6 -> capa lineal -> softmax -> Paso 5.
- **Cuando se detiene:** el token elegido es **</s>**, o se llega al largo máximo (la entrada + 50).

#### 1.12.10.4.15: Paso 7: de numero a texto (una sola vez, al final)

- La secuencia final de IDs (inventados): **[inicio] 5001 5002 5003 5004 </s>**.
- Le sacan **[inicio]** y **</s>**, y cada ID se busca en el **vocab.json**: 5001 -> “Der”, 5002 -> “Hund”, 5003 -> “isst”, 5004 -> “Brot”.
- Con BPE, un token puede ser solo un pedazo de palabra (como **eat + s**). El texto final se arma pegando los pedazos: **“Der Hund isst Brot”**.
- No confundir dos tablas: **vocab.json** convierte ID <-> texto, y la tabla de embeddings E convierte ID -> vector de 512 numeros.

#### En una línea

$y_3(512 \text{ numeros}) \rightarrow 37.000 \text{ puntajes} \rightarrow 37.000 \text{ probabilidades} \rightarrow \arg \max \rightarrow \text{ID} \rightarrow \text{vector } \sqrt{512} \cdot E[\text{ID}] + PE \rightarrow$  siguiente ronda. Cuando sale **</s>**, los IDs se pasan a texto.

ahi cuando pasa al ID, el ID indexa a su respectivo embedding

Si. El ID es la dirección de una fila de la tabla de Embeddings E, la misma idea de indexar que vimos con las probabilidades.

#### Qué pasa en ese paso

```
ID = 5001                (1x1, un entero)
  ↓ indexar la tabla E
E[5001]                  (1x512, la fila 5001 de la matriz de 37.000 x 512)
  ↓ × √512
√512 · E[5001]           (1x512)
  ↓ + Positional Encoding de la posición 1
x(Der)                   (1x512, lo que entra a la Capa 1)
```

- **Indexar es solo una consulta**, no un cálculo: se toma la fila 5001.
- **Los pasos que siguen si calculan:** Multiplicar por  $\sqrt{512}$  y sumar el Postional Encoding. Por eso lo que entra a la Capa 1 no es la fila cruda sino x.

### Cómo se relaciona con lo anterior

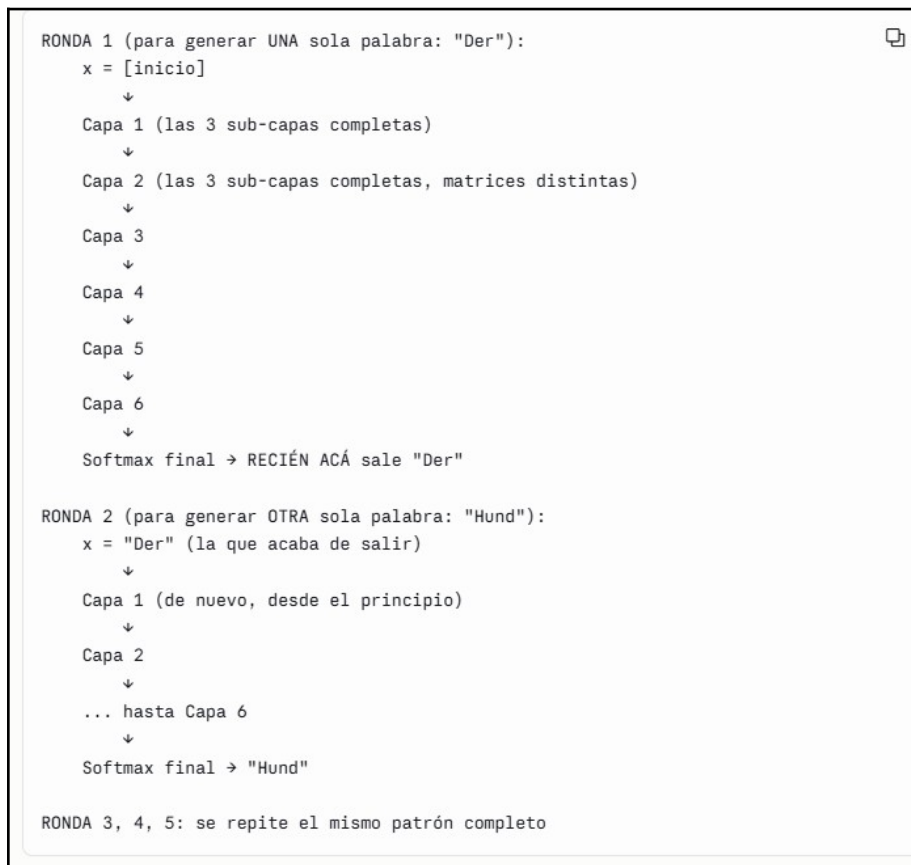
| Momento                              | Que se hace con el ID                                   | Tamaño del resultado |
|--------------------------------------|---------------------------------------------------------|----------------------|
| Al salir ( <b>Paso 5</b> )           | El arg max <b>devuelve</b> el ID: la posición del mayor | 1x1                  |
| Al volver a entrar ( <b>Paso 6</b> ) | El ID <b>indexa</b> la fila j de E                      | 1 x 512              |

La misma tabla  $E$  se usa en las dos puntas. Al salir, la fila  $j$  se usa para calcular el logit número  $k$  (con  $E^T$ ), y al volver a entrar, la fila  $j$  se usa para armar el vector del token elegido.

### Lo que está en el paper y lo que no

El paper (Sección 3.4) dice que usa embeddings aprendidos para convertir los tokens de entrada y se salda en vectores de  $d_{model}$ , y que multiplica esos por  $d_{model}$ . Que el ID sea el número de fila de  $E$  es la convención de las implementaciones habituales, y el paper no lo dice.

### 1.12.10.5: Ejemplo completo 2: La Ronda 2, generando "Der" 2/2



Sigo con la Ronda 2 (entra "Der", sale "Hund") en greedy con cache. Uso de los tamaños reales del paper:  $d_{model}=512$ , 8 cabezas de 64,  $d_{ff}=2048$ , 6 capas y 37.000 tokens de vocabulario. La oración en ingles tiene 5 tokens tras BPE (**the, dog, eat, s, bread**).

#### 1.12.10.5.1: 0. Estado antes de arrancar

| Que                        | Contenido                                          | Tamaño             |
|----------------------------|----------------------------------------------------|--------------------|
| Salida del Encoder         | Un vector por token ingles, ya calculada y foja    | [5 x 512]          |
| Cache de cada capa (K y V) | Lo que dejo la Ronda 1: la fila de <b>[inicio]</b> | [1 x 512] cada uno |
| ID elegido en la Ronda 1   | 5001 (inventado, "Der")                            | 1x1                |
| Posicion que toca          | 1                                                  |                    |

#### 1.12.10.5.2: A. Del ID al Vector

$$x = \sqrt{512} \cdot E[5001] + PE_{(1)} \quad [1 \times 512]$$

1. Se toma la **fila 5001** de E (consulta): [1 x 512]
2. Se multiplica por  $\sqrt{512} \approx 22,627$
3. Se suma el positional encoding de la posición 1. Sus primeras 4 dimensiones, calculadas antes, son **[0.8415, 0.5403, 0.8219, 0.5697]**.

1.12.10.5.2: B. Capa 1, paso a paso

1.12.10.5.2.1: B1. Self-Attention enmascarada (8 cabezas) (armar una versión enriquecida con la información que ya se tiene)

Acaba de entrar “Der” y [inicio] ya estaba escrito.

La idea

“Der” mira lo que ya está escrito en alemán ([inicio] y él mismo) y arma una versión enriquecida de sí mismo con esa información. Para hacerlo, se calculan tres cosas por cabeza: una pregunta, unas etiquetas y unos contenidos. La pregunta se compara con las etiquetas para decidir a quién mirar, y con eso se mezclan los contenidos.

Paso 1: crear q, k y v

A “Der” le hacen falta tres versiones de sí mismo:

- Q (la pregunta): Lo que “Der” busca en los demás.
- K (la etiqueta): como se presenta “Der” para que otros lo encuentren.
- V (el contenido): Lo que “Der” entrega si alguien le presta atención.

Punto de partida

| Elemento                   | Tamaño                   | Qué es                                                                |
|----------------------------|--------------------------|-----------------------------------------------------------------------|
| $x$                        | $1 \times 512$           | “Der”, ya convertido en vector: $\sqrt{512} \cdot E[5001] + PE_{(1)}$ |
| $k_i^{cache}, v_i^{cache}$ | $1 \times 64$ por cabeza | Clave y valor de [inicio], guardados en la Ronda 1                    |
| $W_i^Q, W_i^K, W_i^V$      | $512 \times 64$ cada una | 3 tablas de pesos por cabeza, entrenadas                              |
| $W^O$                      | $512 \times 512$         | Tabla que mezcla las 8 cabezas, entrenada                             |

Como leer la notación:

| Símbolo   | Qué señala                                              | Va de   |
|-----------|---------------------------------------------------------|---------|
| $i$       | Qué cabeza                                              | 1 a 8   |
| $m$       | Una posición dentro de los 512 números de $x$           | 0 a 511 |
| $c$       | Una posición dentro de los 64 números de una cabeza     | 0 a 63  |
| $j$       | Una fila del cache (un token ya escrito)                | 0 y 1   |
| $A[m][c]$ | El número de la matriz $A$ en la fila $m$ , columna $c$ |         |

| Simbolo   | Que señala                                          | Va de   |
|-----------|-----------------------------------------------------|---------|
| $i$       | Que cabeza                                          | 1 a 8   |
| M         | Una posición dentro de los 512 números de           | 0 a 511 |
| $c$       | Una posición dentro de los 64 números de una cabeza | 0 a 63  |
| J         | Una fila del cache (un token ya escrito)            | 0 y 1   |
| $A[m][c]$ | El numero de la matriz A en la fila                 |         |

|  |              |  |
|--|--------------|--|
|  | m, columna c |  |
|--|--------------|--|

| Elemento                   | Tamaño            | Que es                                                                    |
|----------------------------|-------------------|---------------------------------------------------------------------------|
| X                          | 1 x 512           | “Der”, ya convertido en vector:<br>$\sqrt{512} \cdot E[5001] + P E_{(1)}$ |
| $k_i^{cache}, v_i^{cache}$ | 1 x 64 por cabeza | Clave y valor de <b>[inicio]</b>                                          |
| $W_i^Q, W_i^K, W_i^V$      | 512 x 64 cada una | 3 tablas de pesos por cabeza, entrenadas                                  |
| $W^O$                      | 512 x 512         | Tabla que mezcla las 8 cabezas, entrenada                                 |

**Paso 1: convertir x en q, k y v de “Der”**

**Que se hace:** De los 512 números de “Der” se sacan tres resúmenes de 64 números, cada uno con su propia tabla de pesos.

$$q_i = xW_i^Q, \quad k_i = xW_i^K, \quad v_i = xW_i^V$$

Un numero de  $q_i$ , con la formula completa:

$$q_i[c] = \sum_{m=0}^{511} x[m] W_i^Q[m][c]$$

Se toma la columna c de  $W_i^Q$  (512 pesos), se multiplica da peso por el numero de x que esta en la misma posición, y se suma todo. Eso da un solo número. Se repite 64 veces (una por c) para tener  $q_i$  completo, y otras 128 veces mas para  $k_i$  y  $v_i$ .

**Control de tamaños:**  $[1 \times 512] * [512 \times 64] = [1 \times 64]$

**Cuanto cuesta:** Cada numero suma 512 productos, y cada resumen tiene 64 numeros:  $512 \times 64 = 32.764$  productos. Son 3 resúmenes por cabeza y 8 cabezas:  $3 \times 8 \times 32.768 = 786.432$  productos en total, solo para este paso.

Las 8 cabezas parten del mismo x completo, cada una con sus propias tablas.

**Paso 2: leer el cache y agregar la fila nueva**

**Que se hace:** **[inicio]** ya calculo su  $k$  y su  $v$  en la Ronda 1, y esos valores no cambian. Se leen del cache y se les agrega la fila de “Der”.

$$K_i = \begin{bmatrix} K_i^{cache} \\ k_i \end{bmatrix}, \quad V_i = \begin{bmatrix} V_i^{cache} \\ v_i \end{bmatrix} \quad [2 \times 64]$$

**Como queda:** una matriz de 2 filas y 64 columnas.

[2 x 64]

1. Fila 0: **[inicio]**.
2. Fila 1: "Der".

- Fila 0: **[inicio]**, calculada en la Ronda 1 y **no se recalcula**.
- Fila 1: "Der", recién calculada.
- Después de la ronda se guarda:  $K_i^{cache} \leftarrow K_i$  y  $V_i^{cache} \leftarrow V_i$ , para que la Ronda 3 encuentre 2 filas.
- Se guardan  $k$  y  $v$ , pero **no**  $q$ , porque la pregunta solo se usa en su propia ronda.

Después de esta ronda se guarda para la Ronda 3, y va a tener 3 filas.

### Paso 3: calcular los puntajes

**Que se hace:** Comparar la pregunta de "Der" con la clave de cada fila del cache, para ver cual le sirve mas.

$$s_i = \frac{q_i K_i^T}{\sqrt{64}} \quad [1 \times 64] \cdot [64 \times 2] = [1 \times 2]$$

**Como se calcula cada puntaje:** es un producto punto de 64 términos, uno por fila del cache:

$$s_i[j] = \frac{1}{\sqrt{64}} \sum_{c=0}^{63} q_i[c] K_i[j][c], \quad j \in \{0, 1\}$$

- $s_i[0]$ : producto punto de  $q_i$  con la fila 0 ( $k_i^{cache}$ , de **[inicio]**).
- $s_i[1]$ : producto punto de  $q_i$  con la fila 1 ( $k_i^{cache}$ , de "Der").

- $s_i[0]$ : qué tan bien encaja la pregunta de "Der" con la clave de **[inicio]**.
- $s_i[1]$ : qué tan bien encaja con su propia clave.
- Cada puntaje suma 64 productos.
- Se divide por  $\sqrt{64} = 8$  porque el paper (Sección 3.2.1, nota al pie 4) explica que el producto punto de  $q$  y  $k$  crece con  $d_k$ , y eso empuja al softmax a zonas de gradientes muy chicos.

**Control de tamaños:**  $[1 \times 64] * [64 \times 2] = [1 \times 2]$

**Por que se divide por 8:** El paper explica que sumar 64 productos tienen a dar numeros grandes, y eso hace que el softmax siguiente se vuelva casi "todo o nada". Dividir por  $\sqrt{64}=8$  mantiene los numeros en una escala razonable.

### Paso 4: el masking

Con cache **no hace falta hacer nada**: en el cache solo hay posiciones ya escritas (**[inicio]** y "Der" mismo), nunca una posición futura. El paper pone  $-\infty$  en las posiciones futuras cuando se procesan todas las posiciones juntas, pero acá esas posiciones ni siquiera están el cache.

**Otra exp:**

Con cache **no hace falta hacer nada**: el cache solo contiene posiciones anteriores y la actual, así que no existe ninguna posición futura para tapar. El paper (Sección 3.2.3) pone  $-\infty$  en las conexiones ilegales cuando se procesan todas las posiciones a la vez. Con cache, esas posiciones simplemente no están.

### Paso 5: softmax

**Que se hace**: convertir los 2 puntajes en 2 porcentajes que sumen 100%.

$$p_i[j] = \frac{e^{s_i[j]}}{e^{s_i[0]} + e^{s_i[1]}}, \quad p_i[0] + p_i[1] = 1$$

Sale el porcentaje de atención que va a [inicio] y el que va a "Der". Cada cabeza reparte distinto, porque tiene sus propias matrices.

$p_i[0]$  es cuanta atención le presta "Der" a [inicio], y  $p_i[1]$  cuanta a si mismo.

**Salida**:  $[1 \times 2]$

### Paso 6: mezclar los valores

**Que se hace**: Con esos porcentajes, se hace un promedio ponderado de los valores de las 2 filas.

$$o_i = p_i V_i \quad [1 \times 2] \cdot [2 \times 64] = [1 \times 64]$$

Un número,  $c = 0, \dots, 63$ :

$$o_i[c] = p_i[0] V_i[0][c] + p_i[1] V_i[1][c]$$

Cada número es una suma ponderada de 2 valores: el de [inicio] y el de "Der", con los mismos 2 pesos.

$$o_i = p_i V_i \quad o_i[c] = p_i[0] V_i[0][c] + p_i[1] V_i[1][c]$$

**Control de tamaños**:  $[1 \times 2] * [2 \times 64] = [1 \times 64]$ . Cada uno de los 64 numeros es una suma de 2 terminos.

Cada numero es una suma ponderada de 2 valores: el de [inicio] y el de "Der", con los mismos 2 pesos.

### Paso 7: pegar las 8 cabezas

$$O = [o_1 \mid o_2 \mid \dots \mid o_8] \in \mathbb{R}^{1 \times 512}$$

Cada bloque de 64 viene de una sola cabeza. Hasta acá las cabezas no se mezclaron entre si.

Es lo mismo que:

$$Sublayer_1(x) = \text{Concat}(\underbrace{\text{Attention}_1(Q_1, K_1, V_1)}_{\text{cabeza}_1}, \dots, \underbrace{\text{Attention}_8(Q_8, K_8, V_8)}_{\text{cabeza}_8}) W_{\text{self-dec}}^O \quad (512 \text{ números})$$

Lo único que agrega tu texto es multiplicar por  $W^O$  al final, que en mi versión era el Paso 8, separado.

## Pasos 8: $W^O$ (mezclar las cabezas)

$$\text{Sublayer}_1 = O W^O [1 \times 512] \cdot [512 \times 512] = [1 \times 512]$$

$$\text{Sublayer}_1[c] = \sum_{m=0}^{511} O[m] W^O[m][c]$$

Cada numero de salida usa los 512 de entrada, o sea las 8 cabezas a la vez. Son  $512 \times 512 = 262.144$  productos.

## Qué sale y qué sigue

$\text{Sublayer}_1(1 \times 512)$  es "Der" enriquecido con lo que ya estaba escrito en alemán. Sigue Residual + LayerNorm #1 y despues Cross-Attention.

## Qué cambia en cada ronda con cache

| Ronda | Filas del cache tras agregar | Puntajes por cabeza | Mezcla                             |
|-------|------------------------------|---------------------|------------------------------------|
| 1     | 1                            | $1 \times 1$        | $[1 \times 1] \cdot [1 \times 64]$ |
| 2     | 2                            | $1 \times 2$        | $[1 \times 2] \cdot [2 \times 64]$ |
| 5     | 5                            | $1 \times 5$        | $[1 \times 5] \cdot [5 \times 64]$ |
| R     | R                            | $1 \times r$        | $[1 \times r] \cdot [r \times 64]$ |

Las proyecciones (786.432 multiplicaciones) y  $W^O(262.144)$  cuestan lo mismo en todas las rondas. Solo los pasos 3 y 6 crecen con r:  $8 \times r \times 64$  multiplicaciones cada uno. Para  $r = 2$  el total de esta sub-capa es 1.050.624, y para  $r = 5$  es 1.054.720.

## Qué es verificado y qué no

- **Del paper:** La formula de atención las 8 cabezas de 64, la división por  $\sqrt{d_k}$  y su motivo,  $W^O$  y el masking con  $-\infty$
- **De la implementación oficial (verificado):** entra solo el último ID y hay un cache por capa.
- **No verifiqué:** cómo se guarda el cache por dentro, ni que las 8 matrices se junten en una de  $512 \times 512$ , como suelen hacer las implementaciones.

---

## Preguntas

### Qué dicen los porcentajes

Cuánto de "Der" se arma con el contenido de [inicio] y cuánto con su propio contenido. Con  $p_i = [0.5288, 0.4712]$ : un 53% viene de [inicio] y un 47% de sí mismo. En este caso los pesos  $q, k, v$  son inventados, así que no reflejan nada real sobre [inicio] o "Der" en un modelo entrenado.



y por ejemplo para hund como se distribuiría el procenatje?

En la **Ronda 3**, "Hund" reparte su atención entre **[inicio]**, "Der" y él mismo: 3 porcentajes que suman 100%, uno por cada fila del cache.

**Por qué son 3 y no 2**

| Ronda | Quién entra | Filas en el cache antes de esta ronda | Candidatos para el softmax |
|-------|-------------|---------------------------------------|----------------------------|
| 2     | Der         | 1 ([inicio])                          | 2                          |
| 3     | Hund        | 2 ([inicio], Der)                     | 3                          |

**Cómo se calcula (mismos pasos, con 3 filas)**

$$s_i = \frac{q_i K_i^T}{\sqrt{64}} \quad [1 \times 64] \cdot [64 \times 3] = [1 \times 3]$$
$$p_i[j] = \frac{e^{s_i[j]}}{e^{s_i[0]} + e^{s_i[1]} + e^{s_i[2]}}, \quad j \in \{0, 1, 2\}$$

**Ejemplo inventado (con los mismos números de juguete, 3 dimensiones)**

```
q (Hund)      = [0.6, 0.3, 0.2]
k([inicio])   = [0.8, 0.2, 0.4]    k(Der) = [0.4, 0.6, 0.1]    k(Hund) = [0.5, 0.5, 0.3]

s[0] = 0.6×0.8+0.3×0.2+0.2×0.4 = 0.48+0.06+0.08 = 0.62
s[1] = 0.6×0.4+0.3×0.6+0.2×0.1 = 0.24+0.18+0.02 = 0.44
s[2] = 0.6×0.5+0.3×0.5+0.2×0.3 = 0.30+0.15+0.06 = 0.51

s / √3 ≈ [0.358, 0.254, 0.294]
softmax ≈ [0.372, 0.335, 0.293]
```

"Hund" reparte 37% a [inicio], 34% a "Der" y 29% a sí mismo. Son valores inventados, no algo que salga de un modelo entrenado.

### En una línea

Cada ronda agrega un candidato más al softmax, porque hay una fila más en el cache. Los pesos que le tocan a cada uno dependen de las matrices  $W^Q$ ,  $W^K$  del modelo, que acá seguimos inventando para poder mostrar la cuenta.

pero no entiendo, que significa que inicio tenga 37%, der tenga 34%, que significa?.  
que representa este softmax

Significa **qué tan relevante es cada palabra ya escrita para ayudar a decidir qué sigue después de "Hund"**. Son pesos de mezcla, no una probabilidad de que salga esa palabra.

### Qué representa cada número

|          | 37% para [inicio]           | 34% para "Der"               | 29% para "Hund"        |
|----------|-----------------------------|------------------------------|------------------------|
| Qué dice | Aportá algo de tu contenido | Aportá algo del tuyo también | Aportá algo del propio |

No es "hay 37% de probabilidad de que la respuesta sea [inicio]". Es "al armar la nueva versión de 'Hund', tomá un 37% del contenido de [inicio], un 34% del de 'Der' y un 29% del de 'Hund' mismo".

### Para qué sirven esos porcentajes en el paso siguiente

Se usan para mezclar los **valores** ( $v$ ) de cada palabra, con esos pesos exactos:

$$o = 0.37 \cdot v(\text{inicio}) + 0.34 \cdot v(\text{Der}) + 0.29 \cdot v(\text{Hund})$$

El resultado es un vector nuevo de "Hund", que ya no es solo su propio significado: es una mezcla de las tres palabras, ponderada por esos porcentajes.

### Comparación con la probabilidad del final (para no confundirlas)

|                              | Este softmax (dentro de Self-Attention)                          | El softmax final del modelo                            |
|------------------------------|------------------------------------------------------------------|--------------------------------------------------------|
| Sobre qué reparte            | Sobre las palabras <b>ya escritas</b> (3, en la Ronda 3)         | Sobre los ~37.000 tokens del vocabulario               |
| Qué significa un número alto | "Este token aporta mucho contenido a la nueva versión de 'Hund'" | "Es probable que la palabra siguiente sea este token"  |
| Qué se hace con el resultado | Se mezclan los $v$ de esas palabras                              | Se elige el token con más probabilidad (o beam search) |

Son dos softmax con propósitos distintos, y en este ejemplo aparecen dentro de la misma sub-capa por casualidad de tamaños (3 palabras).

### Con una analogía

Es como pedirle opinión a 3 personas sobre que decir después, y darle mas peso a la opinión de una que a las otras. El 37% de [inicio] no dice "la respuesta va a ser [inicio]": Dice "voy a escuchar mas a [inicio]" al armar mi propia opinión nueva.

### Por qué esto ayuda a predecir la palabra siguiente

La nueva versión de "Hund" (la mezcla) sigue viaje por el resto de la Capa 1, y después por las Capas 2 a 6, hasta llegar al softmax final, que es el que sí da una probabilidad por cada palabra del vocabulario. **Este softmax de Self-Attention es un paso intermedio que ayuda a construir esa predicción, no la predicción en sí.**

### 1.12.10.5.2.2: B2. Residual + LayerNorm #1

$$y_1 = \text{LayerNorm}(x + \text{Sublayer}_1) \quad [1 \times 512]$$

### 1.12.10.5.2.3: B3. Cross-Attention (8 cabezas)

Es la **Cross-Attention**, la sub-capa siguiente a la Self-Attention enmascarada, dentro de la misma Capa 1. La diferencia clave: El Query sale del Decoder, pero la Clave y el Valor salen del **Encoder**, no de [inicio] ni de "Der".

#### Punto de partida

| Elemento              | Tamaño                      | Qué es                                                                                                                | ¿Se entrena? |
|-----------------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------|
| $y_1$                 | $1 \times 512$              | La salida de "Der" tras Self-Attention enmascarada + Residual+LayerNorm #1                                            | No           |
| $E_{out}$             | $5 \times 512$              | La salida final del Encoder: 5 filas, una por token inglés (The, dog, eat, s, bread), fija durante toda la generación | No           |
| $W_i^Q, W_i^K, W_i^V$ | $512 \times 64$<br>cada una | Pesos de esta Cross-Attention, distintos de los de Self-Attention                                                     | Sí           |
| $W^O$                 | $512 \times 512$            | Mezcla las 8 cabezas, propia de esta sub-capa                                                                         | Sí           |

| Elemento              | Tamaño                   | Qué es                                                                                                                | ¿Se entrena? |
|-----------------------|--------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------|
| $y_1$                 | $1 \times 512$           | La salida de "Der" tras Self-Attention enmascarada + Residual + LayerNorm #1                                          | No           |
| $E_{out}$             | $5 \times 512$           | La salida final del Encoder: 5 filas, una por token inglés (The, dog, eat, s, bread), fija durante toda la generación | No           |
| $W_i^Q, W_i^K, W_i^V$ | $512 \times 64$ cada una | Pesos de esta Cross-Attention, distintos de los de Self-Attention                                                     | Si           |
| $W^O$                 | $512 \times 512$         | Mezcla las 8 cabezas, propia de esta sub-capa                                                                         | Si           |

### Paso 1: crear el Query, del Decoder

$$Q_i = y_1 W_i^Q \quad [1 \times 512] \cdot [512 \times 64] = [1 \times 64]$$

$$Q_i[c] = \sum_{m=0}^{511} y_1[m] W_i^Q[m][c]$$

Misma mecánica que ya vimos: se toman los 512 números de  $y_1$  y se resumen en 64, con la tabla de pesos  $W_i^Q$ . Esta es la pregunta que "Der" hace, pero ahora dirigida al inglés.

## Paso 2: crear la Clave y el Valor, del Encoder (no de "Der")

$$K_i = E_{out} W_i^K, \quad V_i = E_{out} W_i^V \quad [5 \times 512] \cdot [512 \times 64] = [5 \times 64]$$

Un número de  $K_i$  (fila  $r$ , columna  $c$ , con  $r = 0, \dots, 4$ ):

$$K_i[r][c] = \sum_{m=0}^{511} E_{out}[r][m] W_i^K[m][c]$$

- Cada una de las 5 filas de  $E_{out}$  (una por palabra inglesa) pasa por la misma tabla  $W_i^K$ , y da una fila de  $K_i$ .
- Resultado: una tabla de 5 filas y 64 columnas, una fila por palabra inglesa.
- Lo mismo con  $V_i$ , usando  $W_i^V$ .

**Por que no se guarda un cache acá:**  $E_{out}$  es fila desde que el Encoder corrió, así que  $K_i$  y  $V_i$  de Cross-Attention **son las mismas en las 5 rondas**. En las implementaciones se calculan una vez y se reutilizan; no lo verifiqué línea por línea en el código.

## Paso 3: puntajes

$$s_i = \frac{Q_i K_i^T}{\sqrt{64}} \quad [1 \times 64] \cdot [64 \times 5] = [1 \times 5]$$

$$s_i[j] = \frac{1}{\sqrt{64}} \sum_{c=0}^{63} Q_i[c] K_i[j][c], \quad j = 0, \dots, 4$$

Cinco puntajes: que tan bien encaja la pregunta de "Der" con la clave de "The" ( $j = 0$ ), "dog" ( $j = 1$ ), "eat" ( $j = 2$ ), "s" ( $j = 3$ ) y "bread" ( $j = 4$ ).

## Paso 4: El masking

**No hace falta.** El masking existe para que una palabra alemana no mire a una palabra alemana futura. Aca se esta mirando el ingles, que ya esta completo desde que corrió el Encoder: no hay ninguna posición "futura" que tapar

## Paso 5: softmax

$$p_i[j] = \frac{e^{s_i[j]}}{\sum_{l=0}^4 e^{s_i[l]}}$$

Ejemplo inventado: [0.10, 0.55, 0.20, 0.05, 0.10]. **Dice cuanta atención le presta "Der" a cada palabra inglesa**, no la probabilidad de que la traducción sea esa palabra: acá "dog" (55%) es la que mas aporta contenido a la mezcla, coherente con que "Der" sea el articulo de "Hund" en alemán.

## Paso 6: mezclar los valores

$$o_i = p_i V_i \quad [1 \times 5] \cdot [5 \times 64] = [1 \times 64]$$

$$o_i[c] = \sum_{j=0}^4 p_i[j] V_i[j][c]$$

Cada uno de los 64 números es una suma ponderada de 5 terminos (uno por palabra inglesa), con los mismos 5 pesos.

### Paso 7: pegar las 8 cabezas

$$O = [o_1 \mid o_2 \mid \dots \mid o_8] \quad 8 \times 64 = 1 \times 512$$

Cada cabeza tiene sus propios  $W_i^Q, W_i^K, W_i^V$ , así que puede fijarse en palabras inglesas distintas.

### Paso 8: mezclar las cabezas con $W^O$

$$Sublayer_2 = O W^O \quad [1 \times 512] \cdot [512 \times 512] = [1 \times 512]$$

$$Sublayer_2[c] = \sum_{m=0}^{511} O[m] W^O[m][c]$$

### Comparación con Self-Attention enmascarada

|                         | Self-Attention enmascarada                         | Cross-Attention                         |
|-------------------------|----------------------------------------------------|-----------------------------------------|
| Query viene de          | El propio token del decoder                        | El propio token del Decoder             |
| Clave y Valor vienen de | El cache de tokens alemanes ya escritos            | La salida fija del Encoder (inglés)     |
| Cantidad de candidatos  | Crece ronda a ronda (1, 2, 3...)                   | Fija: 5 (siempre las mismas 5 palabras) |
| Masking                 | Sí (en la versión sin cache)                       | No hace falta                           |
| Qué dice el softmax     | <b>Cuánto pesa cada palabra alemana ya escrita</b> | <b>Cuánto pesa cada palabra inglesa</b> |

### Qué sale

$Sublayer_2$ , de  $1 \times 512$ : "Der" enriquecido ahora con información del ingles. Sigue Residual + LayerNorm # 2 y despues Feeed-Forward.

### Qué es verificado y qué no

- **Del paper (Sección 3.2.3):** que en Cross-Attention las queries vienen de la capa anterior del decoder y las keys/values de la salida del encoder.
- **Mio:** que  $K_i, V_i$  de Cross-Attention no cambian entre rondas y por eso no llevan cache propio; es una consecuencia lógica de que  $E_{out}$  sea fija, pero no lo vi confirmado línea por línea en el código.
- Los porcentajes del ejemplo son inventados.

#### 1.12.10.5.2.4: B4. Residual + LayerNorm #2

$$y_2 = \text{LayerNorm}(y_1 + \text{Sublayer}_2) \quad [1 \times 512]$$

$$\text{Sublayer}_3 = \max(0, y_2 W_1 + b_1) W_2 + b_2$$

$$[1 \times 512] \xrightarrow{W_1: 512 \times 2048} [1 \times 2048] \xrightarrow{\text{ReLU}} [1 \times 2048] \xrightarrow{W_2: 2048 \times 512} [1 \times 512]$$

#### 1.12.10.5.2.5: B5. Feed-Forward

Es el **Feed-Forward** de la Capa 1: la última sub-capa, después de Residual + LayerNorm #2. A diferencia de Self-Attention y Cross-Attention, no compara "Der" con nadie mas: solo lo transforma con dos tablas de pesos y una regla intermedia.

#### Punto de partida

| Elemento | Tamaño            | Qué es                                              | ¿Se entrena? |
|----------|-------------------|-----------------------------------------------------|--------------|
| $y_2$    | $1 \times 512$    | "Der", tras Cross-Attention + Residual+LayerNorm #2 | No           |
| $W_1$    | $512 \times 2048$ | Primera tabla de pesos: agranda                     | Sí           |
| $b_1$    | $1 \times 2048$   | Sesgo, se suma después de $W_1$                     | Sí           |
| $W_2$    | $2048 \times 512$ | Segunda tabla de pesos: achica                      | Sí           |
| $b_2$    | $1 \times 512$    | Sesgo, se suma después de $W_2$                     | Sí           |

#### La fórmula completa

$$\text{Sublayer}_3 = \max(0, y_2 W_1 + b_1) W_2 + b_2$$

#### Paso 1: agrandar de 512 a 2048

$$h = y_2 W_1 + b_1 \quad [1 \times 512] \cdot [512 \times 2048] = [1 \times 2048]$$

Un número, misma mecánica de siempre: tomás una columna de  $W_1$  (512 pesos), la multiplicás con los 512 números de  $y_2$  y sumás, y al final le sumás el sesgo:

$$h[c] = \sum_{m=0}^{511} y_2[m] W_1[m][c] + b_1[c], \quad c = 0, \dots, 2047$$

[h y ReLU(h) miden 1 x 2048]

Se repite 2048 veces. Cuesta  $512 \times 2048 = 1.048.576$  productos.

## Paso 2: ReLU

$$\text{ReLU}(h)[c] = \max(0, h[c])$$

[h y ReLU(h) miden 1 x 2048]

Se aplica a cada uno de los 2048 numeros por separado: si es negativo, queda en 0; si es positivo, queda igual. No mezcla numeros entre si, y no tiene pesos entrenados.

## Paso 3: achicar de 2048 a 512

$$\text{Sublayer}_3 = \text{ReLU}(h) W_2 + b_2 \quad [1 \times 2048] \cdot [2048 \times 512] = [1 \times 512]$$

$$\text{Sublayer}_3[c] = \sum_{m=0}^{2047} \text{ReLU}(h)[m] W_2[m][c] + b_2[c], \quad c = 0, \dots, 511$$

Se repite 512 veces. Cuesta  $2048 \times 512 = 1.048.576$  productos, el doble que la Self-Attention y el Cross-Attention juntos.

## Por qué "agrandar y después achica"

Con la tabla que ya vimos:

| Paso             | Qué es                                                     | Tamaño          |
|------------------|------------------------------------------------------------|-----------------|
| $y_2 W_1 + b_1$  | El resultado, antes de ReLU. Puede tener números negativos | $1 \times 2048$ |
| $\max(0, \cdot)$ | La regla ReLU en sí: no es un dato, es una operación       | —               |
| Después de ReLU  | El mismo vector, con los negativos en 0                    | $1 \times 2048$ |

Con  $W_1$  el modelo puede combinar los 512 numeros de “Der” de 2048 formas distintas, y ReLU descarta las combinaciones que dieron negativo.  $W_2$  toma lo que quedo y lo comprime de nuevo a 512, para que encaje con el resto del modelo. Si no hubiera ReLU en el medio, hacer  $W_1$  y depues  $W_2$  seria mateamticamente igual a una sola tabla, y nose ganaría nada por tener dos.

**Ejemplo chico (4 números en vez de 512, 6 números intermedios en vez de 2048)**

```
y2 = [0.5, 0.3, -0.2, 0.7]
```



```
W1 (4x6, inventada):
```

```
[0.2, 0.1, -0.3, 0.4, 0.0, 0.2]
```

```
[0.1, -0.2, 0.3, 0.1, 0.3, -0.1]
```

```
[-0.1, 0.3, 0.2, -0.2, 0.1, 0.4]
```

```
[0.3, 0.0, -0.1, 0.2, -0.2, 0.1]
```

```
b1 = [0.1, 0.0, -0.1, 0.2, 0.0, 0.1]
```

```
h = y2 · W1 + b1 = [0.46, -0.07, -0.27, 0.61, -0.07, 0.16]
```

```
ReLU(h) = [0.46, 0, 0, 0.61, 0, 0.16]
```

Con una  $W_2$  (6x4) y  $b_2$  que no repito acá, se vuelve a comprimir a un vector de 4 números.

### Cuánto pesa este paso, comparado con lo anterior

| Sub-capa                   | Productos        |
|----------------------------|------------------|
| Self-Attention enmascarada | ~1.050.624       |
| Cross-Attention            | ~1.058.816       |
| <b>Feed-Forward</b>        | <b>2.097.152</b> |

Estos números los calculé yo a partir de los tamaños. Feed-Forward es la parte más pesada de una capa: cerca del doble de las otras dos juntas.

### Qué sale y qué sigue

*Sublayer*<sub>3</sub>, de 1 x 512: pasa por Residual + LayerNorm # 3, y ahí termina la Capa 1. Lo que salga entra como  $x$  de la Capa 2.

### Qué es del paper y qué mío

- **Del paper (Sección 3.3):** La formula,  $d_{ff}=2048$ , y que las mismas matrices se aplican a cada posición por separado.
- **Verificado (paper, Sección 3.3):** "the linear transformations are the same across different positions, they use different parameters from layer to layer". Es la misma tabla para "Der" que para cualquier otro token, y distinta en cada una de las 6 capas.
- **Mío:** el ejemplo de los 4 números, el conteo de productos y la explicación de "agrandar y después achicar".



super breve, el cual sub-capa te asociada el "der" con el "hund" o te indicaba la direccion?

Ninguna las asocia como "la traducción correcta va a ser Hund".

La que mas se acerca a "dirección" es **Self-Attention enmascarada**, porque relaciona a "Der" con los tokens alemanes ya escritos (y con "Hund" recién en la Ronda 3, cuando "Hund" ya existe y mira hacia atrás a "Der"). Cross-Attention conecta "Der" con el ingles, no con "Hund". No con "Hund". La decisión de que siga "Hund" la da el **softmax final**, después de las 6 capas.

#### 1.12.10.5.2.6: B6. Residual + LayerNorm #3

$$y_3 = \text{LayerNorm}(y_2 + \text{Sublayer}_3) \quad [1 \times 512]$$

Este  $y_3$  es la entrada de la Capa 2

#### 1.12.10.5.3: C. Capas 2 a 6

Repiten B1 a B6, cada una con sus propias matrices y su propio cache. En cada capa, el cache de K y V pasa de 1 fila a 2. Los puntajes son de  $[1 \times 2]$  en self-attention y  $[1 \times 5]$  en Cross-Attention.

1.12.10.5.4: D. De la Capa 6 al ID

| Paso | Operación                    | Tamaño                                                         |
|------|------------------------------|----------------------------------------------------------------|
| 1    | $y_3$ de la Capa 6           | $[1 \times 512]$                                               |
| 2    | Capa lineal: $y_3 \cdot E^T$ | $[1 \times 512] \cdot [512 \times 37.000] = [1 \times 37.000]$ |
| 3    | Softmax                      | $[1 \times 37.000]$ , suma 1                                   |
| 4    | arg max                      | $[1 \times 1]$ : ID 5002 ("Hund", inventado)                   |

de los pasos vistos, cual es el que le da una idea de que la siguiente palabra es hund?

Ninguno por si solo: es **la acumulación de todos los pasos anteriores**, concentrada en  $y_3$ . La respuesta corta a "¿Cuál da la idea?" es que ningún paso "decide" nada solo - recién el conjunto completo termina apuntando a "Hund". Pero si hay que señalar **dónde se acumula esa evidencia**, son dos momentos.

Los dos momentos que más aportan

| Paso                                    | Qué hace                                                                                      | Por qué contribuye a "Hund"                                                                                                                 |
|-----------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cross-Attention</b> (de las 6 capas) | "Der" mira las 5 palabras inglesas y toma más contenido de "dog" (según nuestro ejemplo, 55%) | "dog" en inglés se traduce como "Hund" en alemán. Es la pista más directa hacia esa palabra                                                 |
| <b>Feed-Forward</b> (de las 6 capas)    | Transforma ese contenido ya mezclado, capa por capa, afinándolo                               | Es donde el modelo, según lo que investigamos con el paper de Geva et al., termina "empujando" hacia predicciones concretas del vocabulario |

Por qué no es un solo paso

- **Self-Attention enmascarada** solo mira lo ya escrito en alemán ([inicio], "Der"). No tiene ninguna pista de "dog" ahí.
- **Cross-Attention** es la única sub-capa que toca el inglés, así que es donde entra la información necesaria para saber que hay que traducir "dog".
- **Feed-Forward** no mira ninguna otra palabra: solo transforma lo que ya llegó. No agrega información nueva sobre el inglés, pero sí "decide" qué hacer con la que ya tiene.
- Esto se repite 6 veces (una por capa), y recién  $y_3$  de la Capa 6 es la versión final: la capa lineal y el softmax solo **leen** esa versión, no agregan ninguna pista nueva.

### En una línea

La pista de "Hund" entra en la Cross-Attention (porque ahí se consulta "dog"), y se va refinando en cada una de las 6 vueltas de Feed-Forward, hasta que  $y_3$  **queda lo bastante cerca del embedding de "Hund" como para que gane el softmax**. La capa lineal y el softmax no "deciden": solo miden qué tan cerca quedó.

Es el tramo final de la Ronda 2: toma  $y_3$  (lo que salió de la Capa 6, ya con las 3 sub-capas repetidas 6 veces) y lo convierte en el ID de la palabra siguiente. Cuatro pasos, ninguno vuelve a comparar tokens entre sí: acá se decide, no se mezcla información.

### Paso 1: $y_3$ de la Capa 6

Es la salida de Residual + LayerNorm #3 de la **sexta** capa, no de la primera. "Der" pasó por 6 vueltas completas de Self-Attention enmascarada, Cross-Attention y Feed-Forward, cada una con sus propias matrices.  $y_3$  es esa versión final, de  $1 \times 512$ . Ya no se va a transformar más: lo único que falta es leerla.

### Paso 2: capa lineal

$$\text{logits} = y_3 \cdot E^T \quad [1 \times 512] \cdot [512 \times 37.000] = [1 \times 37.000]$$

Un numero de la salida:

$$\text{logits}[j] = \sum_{m=0}^{511} y_3[m] E[j][m], \quad j = 0, \dots, 36.999$$

Es un producto punto entre  $y_3$  y la fila  $j$  de  $E$ , la tabla de embeddings (Sección 3.4 del paper: es la misma tabla que se usó al entrar). Mide que tan alineado está  $y_3$  con el token  $j$ : si  $y_3$  apunta parecido a la fila de "Hund", el logit de "Hund" sale alto.

- 37.000 productos punto, cada uno de 512 términos:  $37.000 \times 512 = 18.944.000$  productos.
- No hay sesgo (no verificado en el paper que lo tenga).
- Los logits son puntajes crudos: pueden ser negativos o mayores a 1.

### Paso 3: softmax

$$p_j = \frac{e^{\text{logits}[j]}}{\sum_{l=0}^{36.999} e^{\text{logits}[l]}}$$

Convierte los 37.000 logits en 37.000 probabilidades que suman exactamente 1. Cada una dice qué tan probable es, según el modelo, que la palabra siguiente sea ese token.

### Paso 4: arg max

$$\hat{y} = \arg \max_j p_j$$

Recorre la lista de 37.000 probabilidades y devuelve **la posición** de la más alta, no su valor. Esa posición es el ID: en el ejemplo, 5002 ("Hund", inventado).

$$\underbrace{p}_{1 \times 37.000} \xrightarrow{\arg \max} \underbrace{\hat{y}}_{1 \times 1}$$

### Los cuatro pasos, en una tabla

| Paso | Entra               | Sale                         | Qué hace                            |
|------|---------------------|------------------------------|-------------------------------------|
| 1    | —                   | $[1 \times 512]$             | Nada, ya está calculado             |
| 2    | $[1 \times 512]$    | $[1 \times 37.000]$          | Compara $y_3$ con cada fila de $E$  |
| 3    | $[1 \times 37.000]$ | $[1 \times 37.000]$ , suma 1 | Convierte a probabilidades          |
| 4    | $[1 \times 37.000]$ | $[1 \times 1]$               | Se queda con la posición del máximo |

Se ve el mismo patrón que en toda la Capa 1: entra un vector grande, pasa por una tabla, y sale algo más chico. Acá el "achicamiento" final es extremo: de 37.000 números a uno solo.

### Qué representa el resultado

El ID 5002 no lleva la probabilidad adentro: es solo una dirección, como vimos antes. Con ese ID se busca la palabra en vocab.json (para el texto) y su fila en \$E\$ (para convertirlo en vector y arrancar la Ronda 3).

### Qué es del paper y qué mío

- **Del paper (Sección 3.4):** que la capa lineal y el softmax convierten la salida del decoder en probabilidades del siguiente token, y que comparte la matriz de embeddings.
- **arglmax es greedy, y el paper usa beam search** (Sección 6.1), como ya vimos.
- **Mío:** que no hay sesgo en la capa lineal (lo saqué del documento por no estar en el paper), el conteo de productos y la lectura de "qué tan alineado está  $y_3$ "
- El ID 5002 es inventado.

#### 1.12.10.5.5: E. Estado al termina la Ronda 2

- La secuencia queda [inicio, 5001, 5002].
- Cada capa tiene 2 filas en su cache. En total hay guardados 6 capas x 2 (K y V) x 2 filas x 512 = 12.288 numeros (6.144 al terminar la Ronda 1).
- El ID 5002 arranca la Ronda 3 y repite el Paso A con la posición 2.

#### 1.12.10.5.6: Que es de la implementación oficial y que es mio

- **Verificado en el código oficial de TensorFlow:** entra solo el último ID, hay un cache por capa y el Encoder no vuelve a correr. La capa lineal usa la misma matriz que los embeddings.
- **Ese código normaliza antes de cada subcapa**, no despues. no después. Mantuve la fórmula del paper,  $\text{\$LayerNorm}(x + \text{Sublayer})\text{\$}$ , por consistencia con el documento.
- **No verifiqué** si los K/V de Cross-Attention se recalculan en cada ronda o se guardan. El resultado matemático es el mismo.

#### Verificado

Sí, se guardan. Confirmado en dos fuentes independientes.

- HuggingFace (**modeling\_tf\_encoder\_decoder.py**): la Cross-Attention del decoder usa **encoder\_outputs.last\_hidden\_state**, ya calculado una sola vez, y la caché documentada (**past\_key\_values**) sirve para "acelerar la decodificación", lo mismo que ya vimos para Self-Attention.
- **EncoderDecoderCache** de la librería **transformers**: gestiona por separado la caché de self-attention y la de cross-attention, confirmando que la de Cross-Attention también se guarda y no se recalcula.

Esto corrige lo que había dejado como "no verificado":  $K_i$  y  $V_i$  de Cross-Attention se calculan una vez (cuando termina el Encoder) y se reutilizan en las 5 rondas, con un cache propio, igual patrón que la Self-Attention.

Los números (5001, 5002, los pesos de atención) son inventados.

### 1.12.7: Por que esto es mejor que lo anterior (RNN) – los 3 criterios del paper

El paper no compara Self-Attention contra RNN de forma vaga – plantea explícitamente **3 criterios** (Sección 4, "Why Self-Attention") para justificar la elección.

1. **Complejidad computación total por capa**
2. **Cuanto computo se puede paralelizar**, medido como la cantidad mínima de operaciones secuenciales necesarias
3. **La longitud del camino** entre dependencias de largo alcance en la red – cuanto mas cortos esos caminos, más fácil es aprender relaciones entre palabras lejanas.

| Tipo de capa                        | Complejidad por capa     | Operaciones secuenciales | Longitud máxima de camino |
|-------------------------------------|--------------------------|--------------------------|---------------------------|
| Self-Attention                      | $O(n^2 \cdot d)$         | <b>O(1)</b>              | <b>O(1)</b>               |
| Recurrente (RNN)                    | $O(n \cdot d^2)$         | <b>O(n)</b>              | <b>O(n)</b>               |
| <b>Convolutioal</b>                 | $O(k \cdot n \cdot d^2)$ | <b>O(1)</b>              | $O(\log_k(n))$            |
| <b>Self-attention (restringida)</b> | $O(r \cdot n \cdot d)$   | <b>O(1)</b>              | $O(n/r)$                  |

La ventaja clave: en una RNN, para que la palabra 1 “se entere” de la palabra 100, la información tiene que pasar por 99 intermedios. En el Transformer, **cualquier palabra puede conectarse directamente con cualquier otra en un solo paso** – por eso entiende mejor relaciones entre palabras lejanas, y por eso el Cross-Attention puede “mirar” cualquier palabra de la oración en ingles sin importar que tan lejos este de la palabra alemana que se está generando.

**Dato adicional del paper:** cuando la longitud de la secuencia  $n$  es menor que la dimensión del modelo  $d$  (que es el caso mas común con las representaciones de oraciones que usan los modelos de traducción de vanguardia), Self-Attention **es directamente mas rápida** que un capa recurrente, no solo mas paralelizable.

**Beneficio adicional que menciona el paper:** Self-Attention puede producir modelos mas “interpretables” – los autores inspeccionaron las distribuciones de atención de sus modelos entrenados y encontraron que las distintas cabezas aprenden a realizar tareas diferentes, muchas de ellas relacionadas con la estructura sintáctica y semántica de las oraciones (esto esta desarrollado con ejemplos visuales en el apéndice del paper).

---

**Qué significa cada columna**

### 1.12.7.1 Complejidad por capa –

cuanto trabajo (cuantas operaciones) hace **una sola capa** para procesar toda la secuencia, en función de  $n$  (cantidad de tokens), y  $d$  (tamaño del vector, 512 en nuestro caso). Es el costo total de cómputo, sumando todo lo que se calcula, aunque se haga en paralelo.

#### Complejidad pro capa: $O(n^2 \cdot d)$ para self-attention.

Recorda el calculo que ya hicimos: cada uno de los 5 tokens compara su query contra los keys de **todos** los demás tokens (incluido el mismo). Eso son  $5 \times 5 = 25$  comparaciones, cada uno con vectores de  $d = 512$  numeros:

$$n^2 \cdot d = 5^2 \times 512 = 25 \times 512 = 12.800 \text{ operaciones (aprox.)}$$

Si la oración tuviera 10 tokens en vez de 5, serian  $10^2 \times 512 = 51.200$  – **4 veces más**, no el doble, porque crece con  $n^2$ .

no entendi que significa que  $n \cdot d$  al cuadrado para RNN, y la explicaion de ahi

Vamos con  $O(n \cdot d^2)$  de la RNN, viendo primero de donde sale cada factor, y después por que en la practica compensa el resultado que viste.

#### De dónde sale cada parte de la fórmula

Una RNN procesa la oración **token por token**, en  $n$  pasos (uno por palabra). En **cada uno** de esos pasos, hace un cálculo interno que combina el vector del token actual con el “estado oculto” (la memoria acumulada hasta ese momento) – y ese cálculo es una multiplicación de matrices con vectores de tamaño  $d$ :

$$h_t = f(W \cdot h_{t-1} + U \cdot x_t)$$

Esa multiplicación de matriz ( $d \times d$ ) por vector ( $d$ ) cuesta  $d^2$  operaciones – es la misma lógica que ya vimos con nuestras matrices  $W_i^O$  de  $512 \times 64$ : multiplicar una matriz por un vector cuesta (filas x columnas) operaciones.

$$\underbrace{n}_{\text{pasos, uno por token}} \times \underbrace{d^2}_{\text{costo de CADA paso}} = O(n \cdot d^2)$$

#### Con nuestra oración, número por número

“the dog eats bread” tiene  $n = 5$  tokens, y  $d = 512$  (el tamaño del vector).

En CADA uno de los 5 pasos, la RNN hace una multiplicación matriz-vector de tamaño  $d \times d$ :

Paso 1 ("The"): cuesta  $d^2 = 512^2 = 262.144$  operaciones

Paso 2 ("dog"): cuesta  $d^2 = 262.144$  operaciones (usa el resultado del paso 1)

Paso 3 ("eats"): cuesta  $d^2 = 262.144$  operaciones (usa el resultado del paso 2)

Paso 4 ("s"): cuesta  $d^2 = 262.144$  operaciones

Paso 5 ("bread"): cuesta  $d^2 = 262.144$  operaciones

Total:  $5 \times 262.144 = 1.310.720$  operaciones

Eso coincide exactamente con el número que ya calculamos antes:  $5 \times 512^2 = 1.310.720$

### Por qué esto "compensa" comparado con Self-Attention

Recordá los dos números que ya vimos, lado a lado:

|                | Fórmula       | Con $n = 5, d = 512$           |
|----------------|---------------|--------------------------------|
| Self-Attention | $n^2 \cdot d$ | $25 \times 512 = 12.800$       |
| RNN            | $n \cdot d^2$ | $5 \times 262.144 = 1.310.720$ |

Con estos números concretos, **Self-Attention es mucho más barata** en operaciones totales – 100 veces menos. Esto pasa porque  $d = 512$  es un número mucho más grande que  $n = 5$ , y en la fórmula de la RNN ese  $d$  está **elevado al cuadrado** ( $d^2$ ), mientras que en Self-Attention el que está al cuadrado es  $n$  (el más chico de los dos).

### Cuándo se invertiría la comparación

Si la operación fuera muchísimo más larga que  $d$  – por ejemplo,  $n = 10.000$  tokens con el mismo  $d = 512$  – ahí si Self-Attention ( $n^2 \cdot d$ ) empezaría a ser más cara que la RNN ( $n \cdot d^2$ ), porque el  $n^2$  ya crecería más rápido que el  $d^2$  fijo:

Self-Attention:  $10.000^2 \times 512 = 51.200.000.000$

RNN:  $10.000 \times 512^2 = 2.621.440.000$

Aca la RNN sería -20 veces más barata. Es exactamente lo que menciona el fragmento que citaste: Self-Attention es peor cuando  $n$  es enorme, pero mejor cuando  $d$  es el que domina – que es el caso típico en traducción, donde las oraciones (los  $n$ ) suelen ser mucho más cortas que el tamaño del vector interno del modelo ( $d = 512$  o más).

### Se infiere que la $d$ tiene que ser mayor que la entrada para mayor contexto en significado de cada palabra individual

#### Resumen

$O(n \cdot d^2)$  para la RNN significa:  $n$  **pasos obligatoriamente en fila** (uno por token), y **cada paso** cuesta  $d^2$  operaciones porque involucra multiplicar una matriz de  $d \times d$  por un vector – el costo total es el producto de esas dos cosas.





### 1.12.7.2 Operaciones secuenciales –

cuantos pasos **tienen que hacerse uno detrás de otro**, sin poder paralelizarse, porque cada uno depende del resultado del anterior. Es la matrice de **latencia mínima**: aunque tuvieras infinitas GPUs, no podrías bajar de ese numero de pasos.

#### Operaciones secuenciales $O(1)$

**Con self-attention (nuestro caso)**: las 25 comparaciones de arriba se calculan **todas a la vez**, en paralelo – “eats” comparándose con “dog” no tiene que esperar a que “dog” termine de compararse con “The”. Por eso es  $O(1)$ : **1 solo paso**, sin importar si la oración tiene 5 tokens o 500.

**Con una RNN (para comparar)**: para procesar “The dog eats bread”, una RNN tiene que leer **The**, esperar a terminar, recién ahí leer **dog**, esperar, despues **eats**, despues **bread**. Son **4 pasos obligatoriamente en fila** – no se puede empezar el paso 3 sin haber terminado el paso 2. Con  $n = 5$  tokens, son  $n = 5$  pasos secuenciales; con 500 tokens, serian 500 pasos, uno atrás del otro.

### 1.12.7.3: Longitud máxima de camino –

cuantos “saltos” como máximo tiene que dar la información para ir de un token a otro cualquiera de la secuencia. Es la matrice que ya vimos: en Self-Attention es  $O(n)$  porque la información tiene que pasar por cada palabra intermedia.

Cuantos “saltos” como máximo tiene que dar la información para ir de un token a otro cualquiera de la secuencia. Es la métrica que ya vimos en Self-Attention es  $O(1)$  porque cualquier palabra se conecta directo con cualquier otra; en una RNN es  $O(n)$  porque la información tiene que pasar por cada palabra intermedia.

#### Longitud máxima de camino: $O(1)$ vs $O(n)$

Con “eats”: para que “eats” (posición 2) se entere de “bread” (posición 4), en Self-Attention es **un solo salto directo** – query de “eats” contra key de “bread”, sin pasar por nada intermedio. Eso es  $O(1)$ .

En una RNN, para que la información de “The” (posición 0) llegue hasta “bread” (posición 4), tiene que **pasar por** “dog” y “eats” en el camino – 3 saltos intermedios para llegar. Con una oración de  $n$  palabras, en el peor caso (la primera palabra influyendo la ultima) son  $n - 1$  saltos, que se escribe como  $O(n)$ .

#### Resumen, con nuestra oración concreta ( $n = 5$ )

| Métrica                          | Self-Attention            | RNN                          |
|----------------------------------|---------------------------|------------------------------|
| Complejidad por capa             | $5^2 \times 512 = 12.800$ | $5 \times 512^2 = 1.310.720$ |
| Operaciones secuenciales         | 1 (todo en paralelo)      | 5 (una palabra por vez)      |
| Distancia máxima "The" → "bread" | 1 salto directo           | 3 saltos intermedios         |

Fijate el dato interesante en la primera fila: con  $d = 512$  (grande) y  $n=5$  (chico) , self-attention termina siendo **mas barata** en operaciones totales que la RNN, a pesar de tener  $n^2$  en su formula – es justo el caso que mencionamos en el documento, donde  $n \ll d$  favorece a Self-Attention.

### 1.12.7.4 La jerarquía de Big-O, de más rápida a mas lenta.

Esta es la escala estándar que se usa en diseño de algoritmos, de menor a mayor crecimiento:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$$

| Notación      | Nombre       | Qué tan rápido crece                                            |
|---------------|--------------|-----------------------------------------------------------------|
| $O(1)$        | Constante    | No crece — siempre el mismo costo, sin importar $n$             |
| $O(\log n)$   | Logarítmica  | Crece muy despacio (ej: búsqueda binaria)                       |
| $O(n)$        | Lineal       | Crece proporcional a $n$                                        |
| $O(n \log n)$ | Cuasi-lineal | Un poco peor que lineal (ej: buenos algoritmos de ordenamiento) |
| $O(n^2)$      | Cuadrática   | Crece con el cuadrado de $n$                                    |
| $O(n^3)$      | Cúbica       | Peor todavía                                                    |
| $O(2^n)$      | Exponencial  | Crece explosivamente — la peor categoría común                  |

### Ubicando la tabla del documento en esa jerarquía

| Columna de la tabla                                | Categoría de Big-O | Dónde queda en la jerarquía                                   |
|----------------------------------------------------|--------------------|---------------------------------------------------------------|
| Self-Attention: $O(n^2 \cdot d)$                   | Cuadrática en $n$  | Peor que $O(n)$ , mejor que $O(n^3)$                          |
| RNN: $O(n \cdot d^2)$                              | Lineal en $n$      | Mejor que $O(n^2)$ en $n$ , pero peor en $d$ si $d$ es grande |
| Convolutacional: $O(k \cdot n \cdot d^2)$          | Lineal en $n$      | Similar a RNN en $n$ , con una constante $k$ extra            |
| Self-Attention restringida: $O(r \cdot n \cdot d)$ | Lineal en $n$      | Mejor que Self-Attention completa cuando $n$ es muy grande    |

| Columna de la tabla                                       | Categoría de Big-O | Donde queda en la jerarquía                                   |
|-----------------------------------------------------------|--------------------|---------------------------------------------------------------|
| <b>Self-Attention:</b> $O(n^2 \cdot d)$                   | Cuadrática en $n$  | Peor que $O(n)$ , mejor que $O(n^3)$                          |
| <b>RNN:</b> $O(n \cdot d^2)$                              | Lineal en $n$      | Mejor que $O(n^2)$ en $n$ , pero peor en $d$ si $d$ es grande |
| <b>Convolutacional:</b> $O(k \cdot n \cdot d^2)$          | Lineal en $n$      | Similar a RNN en $n$ , con una constante $k$ extra            |
| <b>Self-Attention restringida:</b> $O(r \cdot n \cdot d)$ | Lineal en $n$      | Mejor que Self-Attention completa cuando $n$ es muy grande    |

**Un matiz importante:** Estas formulas tienen dos variables ( $n$  y  $d$ ), así que no se puede decir “una es siempre mas rápida que otra” sin saber los valores concretos. Self-Attention es  $O(n^2 \cdot d)$ : peor que las otras cuando  $n$  (el largo de la secuencia) es enorme, pero mejor cuando  $d$  (el tamaño del vector) es el que domina – que es justo el caso típico en traducción, como ya vimos en el documento (donde  $n$  suele ser menor que  $d$ ).

supert breve, que es restringido?

Es una variante de Self-Attention donde cada token **no** mira a todos los demás, sino solo a los  $r$  vecinos mas cercanos (una “ventana” alrededor suyo), en vez de comparar contra la secuencia entera.

Por eso su formula es  $O(r \cdot n \cdot d)$  en vez de  $O(n^2 \cdot d)$ : en lugar de n comparaciones por token, hace solo r (mucho menor que n si la secuencia es larga) – y por eso la longitud máxima de camino sube a  $O(n/r)$ , porque ya no hay conexión directa entre dos tokens muy lejanos. El paper la menciona como algo a futuro, no como parte del modelo que implementaron.

#### 1.12.7.5: La jerarquía de “Operaciones secuenciales” en esta tabla, en particular

Acá la comparación es más directa, porque todas dependen solo de n:

$$\underbrace{O(1)}_{\text{Self-Attention y Convolucional}} < \underbrace{O(n)}_{\text{RNN}}$$

| Tipo de capa                 | Complejidad por capa     | Operaciones secuenciales | Longitud máxima de camino |
|------------------------------|--------------------------|--------------------------|---------------------------|
| Self-Attention               | $O(n^2 \cdot d)$         | <u><math>O(1)</math></u> | <u><math>O(1)</math></u>  |
| Recurrente (RNN)             | $O(n \cdot d^2)$         | $O(n)$                   | $O(n)$                    |
| Convolutioal                 | $O(k \cdot n \cdot d^2)$ | <u><math>O(1)</math></u> | $O(\log_k(n))$            |
| Self-attention (restringida) | $O(r \cdot n \cdot d)$   | <u><math>O(1)</math></u> | $O(n/r)$                  |

Esta es la razón principal por que la que Self-Attention le gana a las RNN en velocidad de entranamiento: No importa que tan larga sea la oración, siempre necesita la misma cantidad mínima de pasos secuenciales ( $O(1)$ ), mientras que una RNN necesita mas pasos cuanto mas larga es la oración  $O(n)$ .

## 1.12.8: Como se entreno realmente este modelo (Seccion 5 del paper) – 65.000.000 de parámetros

### Datos de entrenamiento

El paper entreno con el dataset estándar **WMT 2014 English-German**, de aproximadamente **4.5 millones de pares de oraciones**. El vocabulario compartido entre ingles y alemán, generado con byte-pair encoding, tiene unos **37.000 tokens** (el mismo numero que ya mencionamos en la Fase Previa de este documento). Para ingles-frances usaron un dataset bastante mas grande: 36 millones de oraciones, con un vocabulario de 32.000 "word-pieces".

### Hardware y tiempos reales

Entrenaron en **una sola maquina con 8 GPUs NVIDIA P100**. El modelo base (el de 65 millones de parámetros que venimos usando de referencia) tardo **12 horas** (100.000 pasos, ~0.4 segundos cada uno). El modelo "grande" (213 millones de parámetros) tardo **3.5 dias** (300.000 pasos, ~1 segundo cada uno).

### El optimizador: Adam, con una formula especifica de larning rate

Usaron el optimizador **Adam**, con  $\beta_1=0.9$ ,  $\beta_2=0.98$ ,  $\epsilon=10^{-9}$ . Pero lo interesante es que el *learning rate* (que tan grandes son los ajustes en cada paso de backpropagation) no fue constante – sigui esta formula:

$$lrate = d_{model}^{-0.5} \cdot \min(step\_num^{-0.5}, step\_num \cdot warmup\_steps^{-1.5})$$

### Qué es el optimizador Adam

Recordá la fórmula básica del descenso del gradiente que vimos antes:  $w \leftarrow w - \eta \nabla_w L$  – restarle a cada peso una fracción ( $\eta$ , el learning rate) de cuánto empeora el error si ese peso cambia.

**El problema que Adam resuelve:** usar el **mismo** learning rate para **todos** los pesos del modelo no es ideal – algunos pesos necesitan ajustes grandes, otros necesitas ajustes muy finos, y esto puede cambiar durante el entrenamiento. Adam ajusta el "tamaño de paso" **de forma individual para cada peso**, basándose en su historial de gradientes.

Adam lleva un promedio de los gradientes recientes de cada peso (para suavizar el "ruido" de un solo paso) y también un promedio de que tan grandes han sido esos gradientes al cuadrado (para saber si ese peso necesita pasos grandes o chicos). Combina ambas cosas para decidir el ajuste de cada peso, paso a paso.

### Que son $\beta_1, \beta_2, \epsilon$

| Parámetro  | Qué es                                                                                          | Valor del paper |
|------------|-------------------------------------------------------------------------------------------------|-----------------|
| $\beta_1$  | Qué tan "memoria" tiene el promedio del gradiente reciente (más cerca de 1 = memoria más larga) | 0.9             |
| $\beta_2$  | Lo mismo, pero para el promedio de los gradientes al cuadrado                                   | 0.98            |
| $\epsilon$ | Un número muy chico que se suma para evitar dividir por cero en los cálculos internos de Adam   | $10^{-9}$       |

| Parametro  | Que es                                                                                         | Valor del paper |
|------------|------------------------------------------------------------------------------------------------|-----------------|
| $\beta_1$  | Que tan “memoria” tiene el promedio de gradiente reciente (mas cerca de 1 = memoria mas larga) | 0.9             |
| $\beta_2$  | Lo mismo, pero para el promedio de los gradientes al cuadrado                                  | 0.98            |
| $\epsilon$ | Un numero muy chico que se suma para evitar dividir por cero en los cálculos internos de Adam  | $10^{-9}$       |

Estos tres son **hiperparametros estándar** de Adam – el paper usa valores ligeramente distintos a los típicos que (que suelen ser  $\beta_2=0.999$ ) para adaptarlo al entrenamiento de este modelo específico.

### El learning rate: por qué no es un número fijo

El *learning rate* es “que tan grande es cada ajuste” que se le hace a los pesos del modelo en cada paso de entrenamiento (recorda:  $w \leftarrow w - \eta \nabla_w L$ , donde  $\eta$  es justo este numero). El paper no usa un  $\eta$  constante – lo hace variar siguiendo esta formula:

$$lrate = d_{model}^{-0.5} \cdot \min(step\_num^{-0.5}, step\_num \cdot warmup\_steps^{-1.5})$$

| Símbolo                                | Que es                                                                                    | Valor en el paper                  |
|----------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------|
| $lrate$                                | El learning rate (tasa de aprendizaje) resultante, para ese paso específico               | Varia en cada paso                 |
| $d_{model}^{-0.5}$                     | Una constante que depende del tamaño del modelo – “escala” todo el resultado hacia abajo. | $512^{-0.5} \approx 0.0442$        |
| $Step\_num$                            | El numero de paso de entrenamiento en el que estas (1,2,3..., 100.000)                    | Variable, el eje del entrenamiento |
| $step\_num^{-0.5}$                     | El primer termino dentro del min – decrece a medida que avanza el entrenamiento           | $\frac{1}{\sqrt{step\_num}}$       |
| Warmup_steps                           | Cuantos pasos dura la fase de “calentamiento”                                             | 4.000 (fijo)                       |
| $step\_num \cdot warmup\_steps^{-1.5}$ | El segundo termino dentro del min – crece linealmente con el paso                         | Crece mientras step_num sube       |
| Min(,,)                                | Se queda con el <b>menor</b> de los dos términos                                          | Decide que tramo de la curva estas |

### Ejemplo numérico: calculando $lrate$ en distintos pasos

Con  $d_{model}=512$  (así que  $d_{model}^{-0.5}=512^{-0.5} \approx 0.04419$ ) y  $warmup\_steps=4.000$ :

#### Paso 1 (recién arrancando):

```

Término 1: step_num^-0.5 = 1^-0.5 = 1.0
Término 2: step_num * warmup_steps^-1.5 = 1 * 4000^-1.5 ≈ 1 * 0.00000395 = 0.00000395

min(1.0, 0.00000395) = 0.00000395 ← gana el término 2 (el más chico)

lrate = 0.04419 * 0.00000395 ≈ 0.000000175

```

## Paso 2.000 (en medio del warmup):

```
Término 1:  $2000^{-0.5} \approx 0.02236$   
Término 2:  $2000 \times 0.00000395 = 0.0079$   
  
 $\min(0.02236, 0.0079) = 0.0079$  ← sigue ganando el término 2  
  
 $lrate = 0.04419 \times 0.0079 \approx 0.000349$ 
```

## Paso 4.000 (justo en el punto de cambio — el pico):

```
Término 1:  $4000^{-0.5} \approx 0.01581$   
Término 2:  $4000 \times 0.00000395 = 0.0158$   
  
 $\min(0.01581, 0.0158) \approx 0.01581$  ← acá los dos términos son casi iguales (el pico)  
  
 $lrate = 0.04419 \times 0.01581 \approx 0.000699$  ← ESTE es el learning rate máximo
```

## Paso 10.000 (bajando, después del warmup):

```
Término 1:  $10000^{-0.5} \approx 0.01$   
Término 2:  $10000 \times 0.00000395 = 0.0395$   
  
 $\min(0.01, 0.0395) = 0.01$  ← ahora gana el término 1 (ya cambió de tramo)  
  
 $lrate = 0.04419 \times 0.01 \approx 0.000442$ 
```

## La tabla completa, viendo la curva

| Paso         | <i>lrate</i> aproximado | Tramo                |
|--------------|-------------------------|----------------------|
| 1            | 0.000000175             | Subiendo (warmup)    |
| 2.000        | 0.000349                | Subiendo (warmup)    |
| <b>4.000</b> | <b>0.000699</b>         | <b>Pico (máximo)</b> |
| 10.000       | 0.000442                | Bajando              |

Se confirma exactamente el patrón que ya describimos: el learning rate sube durante los primeros 4.000 pasos, llega a su máximo justo ahí, y después empieza a bajar gradualmente.

El *lrate* que calcula esa formula es exactamente el  $\eta$  de la formula de descenso del gradiente ( $w \leftarrow w - \eta \nabla_w L$ ). Backpropagation calcula el gradiente ( $\nabla_w L$ ); esta formula decide **que tan grande** es el paso que se da en esa direccion, en cada paso de entrenamiento.

## Desarmando la formula en sus dos tramos

La función  $\min(.,.)$  se queda con el **menor** de dos valores, y eso genera dos comportamientos distintos según en que paso del entrenamiento estes.

**Tramo 1- los primeros 4.000 pasos (warmup):** acá gana el segundo termino,  $step_{num} \cdot warmup_{step} s^{-1.5}$ , que **crece linealmente** con el numero de paso – cuantos mas pasos van pasando, mas grande es el learning rate.

**Tramo 2 – después del paso 4.000:** acá gana el primer termino,  $step_{nu} m^{-0.5}$ , que **decrece** – cuanto mas avanza el entrenamiento, mas chico se vuelve el learning rate (proporcional a  $1/\sqrt{step_{num}}$ ).

### Con números concretos, para ver el patrón

```
Paso 1:      lrate pequeño (recién arrancando)
Paso 2.000:  lrate más grande (subiendo)
Paso 4.000:  lrate en su punto MÁXIMO (acá cambia de tramo)
Paso 8.000:  lrate bajando otra vez
Paso 50.000: lrate todavía más chico
```

Es como una montaña: sube durante el warmup, llega a un pico en el paso 4.000, y después baja gradualmente el resto del entrenamiento.

### Por qué tiene sentido este diseño

- **Al principio (warmup)**, las matrices todavía están casi al azar – si el learning rate fuera grande desde el primer paso, los ajustes podrías ser tan bruscos que el entrenamiento se “rompa” o se vuelve inestable.
- **Al final**, el modelo ya aprendió patrones útiles – si el learning rate siguiera siendo grande. Cada ajuste podría “pisotear” lo que ya se aprendió bien, en vez de afinarlo.

### Regularización: Evitando que el modelo memorice de mas

El paper usa 3 tecnicas para evitar el sobreajuste (que el modelo memorice los datos de entrenamiento en vez de aprender patrones generales):

Un modelo puede **memorizar** los ejemplos de entrenamiento en vez de aprender patrones que se generalicen a oraciones nuevas – eso se llama sobreajuste (overfitting). El paper usa 2 tecnicas para evitarlo.

#### 1. Técnica 1: Residual Dropout

Durante el entrenamiento, se “apaga” aleatoriamente una fracción de las neuronas en cada sub-capas (tasa  $P_{drop}=0.1$  para el modelo base) – tanto en las salidas de attention/feed-forward como en la suma de embeddings + positional encoding)

Durante el entrenamiento (nunca en inferencia), se **apaga al azar** una fracción de las neuronas – con  $P_{drop}=0.1$ , eso es un 10% de las conexiones, elegidas al azar en cada paso.

**Por que ayuda:** Si el modelo supiera que siempre puede confiar en una neurona especifica, podría “especializarse” demasiado en esa única ruta – memorizando en vez de generalizando. Al apagar neuronas al azar, el modelo se ve obligado a aprender patrones **redundantes**, repartidos entre varias rutas, lo que lo hace mas robusto frente a datos nuevos.

**Donde se aplica**, según el paper: en la salida de cada sub-capas (antes de sumarla con Residual) y también en la suma de embeddings + positional encoding.

#### 2. Técnica 2: Label Smoothing

En vez de decirle al modelo “la respuesta correcta es 100% esta palabra, 0% todas las demás”, se usa un valor de  $\epsilon_{ls}=0.1$  que le resta un poco de certeza absoluta al objetivo de entrenamiento – esto empeora la “perplejidad” (que tan segura esta la predccion) pero **mejora** la precision y el puntaje BLEU final.



Normalmente, entrenar un modelo significa decirle “la respuesta correcta es 100% esta palabra, 0% todas las demás” – un objetivo muy “duro” y absoluto. Label Smoothing,  $\epsilon_{ls}=0.1$ , **suaviza** eso: en vez de 100%/0%, le dice algo como “95%” esta palabra, y reparte el 5% restante entre las demás”.

**Por que esto ayuda, aunque suene contraintuitivo:** Si entrenas el modelo para estar 100% seguro siempre, se vuelve **demasiado confiado** – y eso lo hace frágil ante situaciones ambiguas (donde en realidad hay más de una traducción razonable). Suavizar el objetivo hace que el modelo aprenda a repartir algo de probabilidad entre alternativas plausibles, lo cual mejora que tan bien generaliza a oración nuevas - aunque, como contrapartida, hace que la "perplejidad" (una métrica de qué tan segura está cada predicción) empeore un poco, porque el modelo ya no apunta a estar 100% seguro

### Resumen de las tres técnicas juntas

| Tecnica                 | Que hace                                        | Para que sirve                                     |
|-------------------------|-------------------------------------------------|----------------------------------------------------|
| Learning rate variable  | <b>Sube despacio al princpio, baja despues</b>  | Evitar inestabilidad al arrancar, afinar al final  |
| <b>Residual Dropout</b> | <b>Apaga neuronas al azar (10%)</b>             | Evitar que el modelo dependa de rutas específicas  |
| <b>Label Smoothing</b>  | <b>Suaviza el objetivo de 100%/0% a 90%/10%</b> | Evitar exceso de confianza, mejorar generalización |

### 1.12.9: Beam Search: como se elige la traducción final + Resultados: comparado contra que otros modelos + que pasa si cambias las piezas del modelo

Un detalle que no mencionamos antes: en vez de simplemente elegir siempre "la palabra más probable" en cada ronda de generación (lo que se llama *greedy decoding*), el paper usa **beam search** con un tamaño de haz de 4 (*beam size* = 4) y una penalización de longitud  $\alpha=0.6$ . Esto significa que, en cada ronda, el modelo no se queda con una sola opción – mantiene las **4 mejores secuencias candidatas** en paralelo, y al final elige la que globalmente resulto mejor, en vez de tomar decisiones “verdes” (greedy) palabra por palabra que podrían no se la mejor traducción completa.

#### Resultados: comparado contra qué otros modelos

La tabla 2 del paper compara el Transformer contra los mejores modelos de la época:

| Modelo                                          | BLEU (ingles-aleman) | BLEU (ingles-aleman) |
|-------------------------------------------------|----------------------|----------------------|
| ByteNet                                         | 23.75                | -                    |
| GNMT + RL (Google, el estado del arte anterior) | 24.6                 | 39.92                |
| ConvS2S                                         | 25.16                | 40.46                |
| MoE (Mixture of Experts)                        | 26.03                | 40.56                |
| <b>Transformer (base)</b>                       | <b>27.3</b>          | <b>38.1</b>          |
| <b>Transformer (grande)</b>                     | <b>28.4</b>          | <b>41.0</b>          |

El Transformer grande superó a todos los modelos anteriores (incluidos *ensembles*, que combinan varios modelos) por más de 2 puntos BLEU en inglés-alemán, y lo hizo entrenando en una fracción del costo computacional de los mejores modelos anteriores — algunos de esos modelos anteriores costaron hasta  **$8.0 \times 10^{20}$  FLOPs** en su versión ensemble, contra los  **$2.3 \times 10^{19}$  FLOPs** del Transformer grande — un ahorro de más de 30 veces.

#### BLEU

Una métrica que mide que tan parecida es la traducción que genero el modelo a una traducción de referencia, comparando que fracción de palabras y secuencias cortas de palabras (n-gramas) coinciden entre las dos. Va de 0 a 100 (o 0 a 1): mas alto es mejor. **No mide significado, solo coincidencia de texto.**

#### Qué pasa si cambiás las piezas del modelo (Tabla 3 del paper)

El paper también prueba variantes del modelo base, cambiando una pieza a la vez, para ver que tan importante es cada una:

- **Menos cabezas de atención (de 8 a 1):** El BLEU baja de 25.8 a 24.9 – confirma que Multi-Head attention realmente ayuda.
- **Demasiadas cabezas (32):** El BLEU también empeora un poco (25.4) – hay un punto optimo, no es “cuantas más mejor”.
- **Reducir  $d_k$  (la dimensión de cada cabeza):** empeora la calidad – sugiere que la función de compatibilidad entre Query y Key no es trivial de calcular con pocas dimensiones.
- **Modelos mas grandes (mas capas, mas  $d_{model}$ ):** mejoran los resultados, como es esperable.
- **Dropout:** quitarlo del todo ( $P_{drop}=0$ ) empeora el resultado por sobreajuste; con demasiado dropout (0.2) también empeora, por quedarse “corto” de capacidad.

- **Positional encoding aprendido, en vez de las formulas de seno/coseno:** dio resultados casi idénticos (25.7 vs 25.8) – confirma que la elección específica de la fórmula no es crítica, ambas funcionan parecido.

### 1.12.10: Los números concretos de este paper específico + Lo que quedo suelto: que usa realmente GPT (investigado) + Por que se llama “decoder-only”

| Parametro                                 | Valor                                                                     |
|-------------------------------------------|---------------------------------------------------------------------------|
| Capas (encoder y decoder)                 | 6 cada uno                                                                |
| Dimensión del modelo ( <b>d_model</b> )   | 512                                                                       |
| Cabezas de atención ( <b>h</b> )          | 8                                                                         |
| Dimension de cada cabeza ( <b>dk,dv</b> ) | 64                                                                        |
| Dimension interna del feed-forward        | 2048                                                                      |
| Parámetros totales (modelo base)          | 65 millones                                                               |
| Tiempo de entrenamiento                   | 12 horas, en 8 GPUS                                                       |
| Resultado real (BLEU, ingles -> alemán)   | 27.3 (modelo base) / 28.4 (modelo grande) – nuevo estado del arte en 2017 |

Para tener perspectiva: este modelo "chico" de 65 millones de parámetros es **minúsculo** comparado con los modelos actuales (GPT-5, Claude) que tienen cientos de miles de millones o billones de parámetros — pero el mecanismo de atención en sí sigue siendo esencialmente el mismo.

### Lo que quedó suelto: qué usa realmente GPT (investigado)

El paper de 2017 describe la arquitectura **completa** que acabamos de recorrer — Encoder + Decoder + Cross-Attention, pensada específicamente para **traducción** (Encoder lee una oración en un idioma, Decoder genera la traducción en otro). Pero **GPT no usa esta arquitectura completa** – usa solo una parte, con modificaciones. Esto es clave para no quedarse con una idea incompleta.

### GPT es "decoder-only" (solo decodificador)

Confirmado con múltiples fuentes técnicas: a partir de GPT-1 (2018), OpenAI tomo **solo el decoder** del transformer original, y le saca la parte de **Cross-Attention** – porque GPT no necesita “traducir” de una secuencia a otra, solo necesita **continuar** una única secuencia de texto en el mismo idioma.

|                       | Transformer original (2017)                          | GPT (decoder-only)                      |
|-----------------------|------------------------------------------------------|-----------------------------------------|
| Estructura            | Encoder + Decoder, dos pilas separadas               | Solo una pila de bloques decoder        |
| Para qué está pensado | Traducción (secuencia → secuencia distinta)          | Generar texto (continuar una secuencia) |
| Cross-Attention       | Sí, conecta Decoder con Encoder                      | No existe, no hay Encoder               |
| Self-Attention        | Sin máscara en el Encoder, con máscara en el Decoder | Siempre con máscara (causal)            |

### Por qué se llama "decoder-only" aunque el nombre confunda un poco

Una fuente lo aclara bien: la etiqueta es un poco engañosa — GPT no "decodifica" la salida de ningún encoder separado, simplemente lee y genera texto dentro de un mismo flujo continuo de tokens. Un nombre

más preciso sería "transformer autorregresivo", pero el nombre "decoder-only" quedó fijado por convención histórica.

**Por qué esta simplificación ganó (y se volvió el estándar)**

Confirmado en varias fuentes: la arquitectura decoder-only terminó **desplazando** a la arquitectura completa encoder-decoder para la mayoría de los usos, por razones prácticas:

- **Unificación:** cualquier tarea se puede plantear como "continuar este texto" — no hace falta diseñar una entrada y salida separadas
- **Escala más eficiente:** una sola pila, no dos — más simple de entrenar a tamaños enormes
- **El entrenamiento en paralelo sigue funcionando:** gracias al masking causal, cada predicción de "siguiente token" solo mira hacia atrás, así que todas las posiciones de una secuencia se pueden entrenar a la vez

**Dónde quedó la arquitectura completa (encoder-decoder)**

No desapareció – sigue siendo la mejor opción para tareas específicas de “traducir una cosa en otra”, exactamente el caso de uso que recorrimos en este documento (o resumir un texto largo a uno corto). El ejemplo mas conocido es **T5** de Google, que si usa la arquitectura completa con Cross-Attention.

**Y el otro lado: los modelos "encoder-only"**

También existe una tercera variante, que **no genera texto** – se usa para **entender/clasificar** texto (por ejemplo, análisis de sentimiento, búsqueda). El ejemplo mas conocido es **BERT** (Google, 2019).

**Resumen de las 3 variantes que nacieron del mismo paper**

| Variante                                                     | Usa                                          | Para que sirve                                       | Ejemplo                                                                  |
|--------------------------------------------------------------|----------------------------------------------|------------------------------------------------------|--------------------------------------------------------------------------|
| Encoder-only                                                 | Solo la mitad “encoder”                      | Entender/clasificar texto                            | BERT                                                                     |
| Decoder-only                                                 | Solo la mitad “decoder”, sin Cross-Attention | Generar texto                                        | GPT, Claude, Llama, Gemini, Qwen — prácticamente todos los LLMs modernos |
| Encoder-Decoder<br>(completo, el que explica este documento) | Las dos mitades, con Cross-Attention         | Traducción, tareas de "transformar" un texto en otro | T5, BART, y el propio paper original                                     |

### 1.12.11: Conclusión: qué parte de "Attention Is All You Need" seguís usando hoy + La conclusión del propio paper (Sección 7)

Cuando usás GPT-5, Claude o Gemini, **seguís usando exactamente el mismo mecanismo central** que describe el paper de 2017 y que recorrimos en este documento — el Self-Attention, el Multi-Head Attention, el Feed-Forward, el Residual + Layer Norm. Lo único que cambió es que se **descartó la mitad "Encoder" y el Cross-Attention** de la arquitectura, quedándose solo con el Decoder (con su Self-Attention enmascarada), porque resultó ser la forma más simple y escalable de resolver la tarea que estos modelos realmente hacen: predecir la siguiente palabra, una y otra vez, en un único flujo continuo de texto — en vez de traducir entre dos secuencias distintas, como sí hace el modelo completo que explicamos acá, fiel al paper original.

---

### La conclusión del propio paper (Sección 7)

Vale la pena cerrar con lo que los propios autores escribieron al final, porque muestra que **ya intuían hacia donde iba a evolucionar esto** – aunque no midieran del todo el alcance real, como mencionamos al principio de este documento:

- Se presento el Transformer como **"el primer modelo de transducción de secuencias basado enteramente en atención"**, reemplazando las capas recurrentes mas comunes en arquitecturas encoder-decoder con self-attention multihead
- Para tareas de traducción, el Transformer se entrena **significativamente más rápido** que las arquitecturas basadas en capas recurrentes o convolucionales
- En ambas tareas de traducción (inglés-alemán e inglés-francés) lograron un nuevo estado del arte — en la primera, su mejor modelo **superó incluso a todos los ensembles reportados anteriormente**
- Los autores se declararon **entusiasmados por el futuro de los modelos basados en atención**, y plantearon extenderlo a **"modalidades de entrada y salidas distintas del texto"** – específicamente mencionan **imágenes, audio y video** – e investigar mecanismos de atención locales y restringidos para manejar eficientemente entradas y salidas grandes.
- También plantearon como objetivo futuro **"hacer la generación menos secuencial"** — es decir, reducir la necesidad de generar token por token, un problema que (como vimos en otro documento de este curso) sigue siendo un área activa de investigación hoy, con técnicas como el *test-time compute* y arquitecturas alternativas como Mamba

**Test-time compute:** dejar que el modelo "piense más" en el momento de responder (generando más pasos de razonamiento, o explorando varias respuestas candidatas) en vez de solo hacerlo más grande en el entrenamiento — es la idea detrás del budget forcing que ya vimos (el paper s1), y de los modelos "reasoning".

**Mamba:** una arquitectura alternativa al Transformer (basada en modelos de espacio de estados, SSM) que ya mencionamos en el Día 4 — procesa secuencias en  $O(n)$  en vez de  $O(n^2)$ , así que escala mejor con oraciones muy largas, a costa de perder algo de capacidad de in-context learning comparado con el Transformer.

Ambas apuntan al mismo problema que el paper dejó como objetivo futuro: "hacer la generación menos secuencial" — reducir la dependencia de generar token por token, cada uno esperando al anterior.

Ocho años después, buena parte de esa visión se cumplió: la atención se extendió a imágenes (Vision Transformers), a audio (Whisper, que ya usaste en tu curso), y es la base de prácticamente todos los sistemas de IA generativa modernos — confirmando que lo que los propios autores describieron modestamente como "una optimización" terminó siendo, en la práctica, el cambio de arquitectura más influyente de la última década en inteligencia artificial.

1.12.12 Alternativas al Transformer + El patrón que domina – presentación de algunos benchmark

Las alternativas principales

| Arquitectura                    | Idea central                                                                                                     | Complejidad                                 |
|---------------------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| Mamba / Mamba-2 (SSM)           | Modelos de espacio de estado: comprimen la secuencia en un "estado" en vez de comparar todos los tokens entre si | $O(n)$ lineal vs $O(n^2)$ de self-attention |
| RWKV                            | Una RNN diseñada para poder entrenarse en paralelo como un Transformer                                           | $O(n)$                                      |
| x-LSTM                          | Una versión modernizada de las LSTM clásicas                                                                     | $O(n)$                                      |
| Modelos de difusión de lenguaje | En vez de generar token por token, "refinan" varios tokens a la vez en pasadas sucesivas                         | Genera varios tokens por pasada             |

El patrón que domina: híbridos, no reemplazos puros

La idea de "Mamba reemplaza al Transformer" perdió fuerza. Lo que se consolido es intercalar capas de Attention con capas de Mamba en el mismo modelo – ejemplo real:

**Jamba** (AI21 Labs), con 43% Mamba-2 + 7% Attention + 50% MLP, superando a Llama-3.1-70B/405B en algunos benchmarks. La lógica: Attention sigue siendo mejor para razonamiento complejo y comaprar información a largo alcance; **Mamba es mucho mas barato para la mayoría del procesamiento**. Un dato concreto de 2026.

Qwen3.5 adoptó esta arquitectura híbrida en febrero, con buenos resultados en tareas de agentes y código usando solo 3B parámetros activos.

El trade-off que sigue sin resolverse

Mamba-3B iguala la perplejidad de un Transformer de 6B con 5x más throughput — pero **se queda atrás en in-context learning** (MMLU: 46.3% vs 51.2% en modelos de 8B). Es la misma limitación que ya vimos en tu documento: Mamba gana en velocidad para secuencias largas, pierde algo de la capacidad de "aprender del contexto sobre la marcha" que tiene Self-Attention.

Una estimación citada en una de las fuentes: ~45% de probabilidad de que los híbridos se vuelvan dominantes, contra ~35% de que el Transformer puro se mantenga — aunque es una proyección de un sitio de pronósticos, no un dato duro.

En una frase

A día de hoy no hay "el sucesor" del Transformer — hay un puñado de arquitecturas lineales (Mamba, RWKV) que se usan sobre todo **combinadas** con Attention, y los modelos de difusión como paradigma genuinamente distinto, todavía compitiendo parejo en algunos benchmarks y atrás en otros.

- 1. **Perplejidad:** Que tan “sorprendido” esta el modelo al ver el texto real – mas baja es mejor; mide que tan bien predice la siguiente palabra.
- 2. **Throughput:** cuantos tokens procesa el modelo por segundo – mide velocidad, no calidad
- 3. **MLU:** un benchmark de preguntas de opción múltiple sobre muchas materias (matemática, historia, derecho, etc.) que mide conocimiento y razonamiento general del modelo — se reporta como % de aciertos.

que hace mamba? a ver un ejemplo

Mamba reemplaza el mecanismo de Self-Attention (donde cada token compara con *todos* los demás) por una “**memoria**” que **se va actualizando token por token**, como una RNN pero con **un diseño que si permite entrenarla en paralelo**. Vamos con un ejemplo para que se vea la diferencia concreta.

La diferencia de fondo con Self-Attention

|                         | Self-Attention (Transformer)                                        | Mamba (SSM)                                                                                |
|-------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Cómo mira el contexto   | Cada token compara con <b>todos</b> los anteriores, explícitamente  | Mantiene un <b>estado comprimido</b> (un vector fijo) que resume todo lo visto hasta ahora |
| Costo                   | $O(n^2)$ — crece al cuadrado con el largo                           | $O(n)$ — crece lineal                                                                      |
| Memoria por token nuevo | Necesita guardar el K/V de cada token anterior (el cache que vimos) | Solo actualiza <b>un estado de tamaño fijo</b> , sin importar cuántos tokens ya pasaron    |

Ejemplo con nuestra misma oración, "The dog eats bread"

**Con Self-Attention** (lo que ya vimos): cuando "eats" calcula su atención, compara explícitamente contra "The", "dog" y "bread" — 3 comparaciones directas, con sus propios Keys guardados en el cache.

**Con Mamba**, en cambio, no hay comparación directa entre tokens. Hay un **estado** (un vector, pensalo como una "memoria resumida") que se va actualizando:

```
Estado inicial (vacío):      h0 = [0, 0, 0, ...]
↓ entra "The"
h1 = f(h0, "The")          ← el estado "absorbe" algo de "The"
↓ entra "dog"
h2 = f(h1, "dog")          ← el estado ahora es una mezcla de "The" + "dog"
↓ entra "eats"
h3 = f(h2, "eats")          ← mezcla de "The"+"dog"+"eats", comprimida
↓ entra "bread"
h4 = f(h3, "bread")        ← el estado final, "resumen" de toda la oración
```

Para predecir que sigue despues de “bread”, mamba **no** vuelve a mirar “The” ni “dog” directamente – usa solo  $h_4$ , el estado acumulado, que ya “contiene” (de forma comprimida) la información relevante de todo lo anterior.

La pieza clave que hace a Mamba distinto de una RNN vieja

La palabra "Selective" en "Selective State Space Model" es la novedad: a diferencia de una RNN clásica (que trata a todos los tokens igual), Mamba **decide dinámicamente** qué tan importante es cada token nuevo para el estado — puede "darle mucho peso" a una palabra clave y casi ignorar una palabra de relleno, ajustando ese peso de forma distinta según el contenido. Eso es lo que recupera parte de la capacidad que las RNN clásicas no tenían.

**Por qué esto da  $O(n)$  en vez de  $O(n^2)$**

Cada token nuevo solo necesita **una actualización del estado** (un calculo de tamaño fijo) – no comparaciones contra todos los tokens anteriores. Por eso, si la oración tiene 10.000 palabras, el costo crece proporcional 10.000, no a  $10.000^2$ .

**El costo de esto: por qué pierde en in-context learning**

Como toda la información pasada queda **comprimida en un solo vector de tamaño fijo (h)**, hay un limite a cuanto detalle puede "recordar" – es como resumir un libro entero en una sola pagina: perdes matices. Self-Attention, en cambio, mantiene acceso directo a cada token anterior (via el cache de K/V), sin comprimir nada – **por eso le va mejor en tareas donde hace falta recuperar un detalle especifico de muy atrás en el contexto.**



## 1.13 DIA 4 [01-10] - Evolución de los Modelos de Lenguaje: De la Ingeniería de Prompts a la

### IA Agéntica

Transformers: Attention, Emergent Intelligence & Agentic AI

Along the way



Prompt Engineers  
The rise and fall



Copilots  
like MS Copilot and Github Copilot



Context Engineering  
RAG



Agentic AI  
Claude Code, Cursor Agents

### Contexto Previo (El texto)

Antes de llegar a estas etapas, el texto menciona el origen del *Transformer* (la arquitectura actual de la IA). Antes de él, existía la **LSTM** (memoria a largo corto plazo), que era potente pero muy difícil de paralelizar (entrenar paso a paso era lento). El **Transfoormer** simplifico esto con el concepto de “**Attention**”, permitiendo procesar todo en paralelo, lo que llevo a la **Inteligencia Emergente**: La capacidad de que una maquina no solo prediga palabras probables, sino que de respuestas exactas y precisas (como resolver un problema matemático), algo que aún sorprende a los expertos (y que fue muy criticado en el famoso paper de los "loros estocásticos" en 2023).

### Las 4 Etapas de la Imagen (Explicadas con el texto)

#### 1. Prompt Engineers (Ingenieros de Prompts) - "El auge y la caída"

- **Qué era:** Hubo un tiempo en que saber escribir instrucciones (prompts) perfectas para la IA era un trabajo muy bien pagado (de seis cifras).
- **Qué pasó:** La gente aprendió los trucos (dar contexto, definir el estilo). Ahora "todos somos ingenieros de prompts". Se volvió una habilidad básica y dejó de ser un título laboral único. La imagen muestra a un humano dictando instrucciones a un robot.

#### 2. Copilots (Copilotos)

- **Qué es:** La llegada de herramientas como Microsoft Copilot o GitHub Copilot.
- **El cambio:** Se pasó de solo dar órdenes a una **colaboración humano-IA**. La IA actúa como un asistente que automatiza tareas manuales y enriquece el trabajo, trabajando "mano a mano" con el humano (como se ve en la imagen, el humano trabajando junto a la pantalla del robot).

### 3. Context Engineering (Ingeniería de Contexto) - RAG

- **Qué es:** La evolución del prompt engineering. Ya no se trata solo de escribir una buena pregunta, sino de **reflexionar ampliamente sobre toda la información** que se le da a la IA.
- **Cómo funciona:** Incluye darle información específica del negocio, unir datos de manera convincente y equipar al modelo con **herramientas**. Todo se reduce a darle mejores secuencias de entrada para que la salida esté alineada con lo que necesitas (ejemplo del texto: si quieres informar sobre precios de billetes, debes asegurarte de incluir esos precios en el contexto de entrada). RAG (Retrieval-Augmented Generation) es una técnica clave aquí.

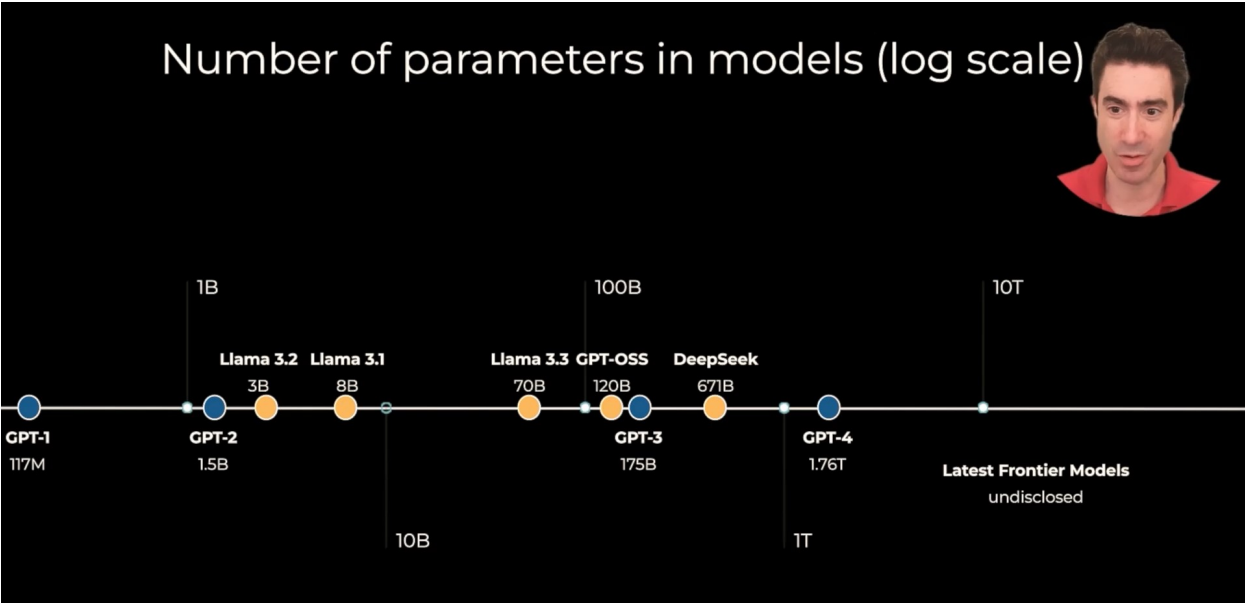
### 4. Agentic AI (IA Agéntica) - Claude Code, Cursor Agents

- **Qué es:** El tema más candente actualmente. Se define de dos formas principales:
  1. Un sistema donde un LLM controla el flujo de trabajo (decide qué hacer a continuación, llama a otras IAs o usa herramientas).
  2. Un LLM en un **bucle** (loop). Se llama a sí mismo repetidamente y tiene acceso a herramientas para tomar acciones.
- **La clave - Autonomía:** La IA es dueña de su propio destino. Planifica, ejecuta y marca tareas pendientes (como se ve en Claude Code). Aunque suene a "vudú", todo se reduce a predecir tokens: la IA genera una secuencia de salida que incluye instrucciones sobre qué actividad realizar a continuación. La imagen muestra a un robot integrado completamente en un entorno de oficina, tomando el control de tareas.

### En resumen

El texto y la imagen narran el viaje desde que **escribíamos instrucciones manuales** (Prompt Engineering), pasando por **trabajar junto a la IA** (Copilots), luego **darle un contexto rico y herramientas** (Context Engineering/RAG), hasta llegar al presente, donde **la IA se ejecuta a sí misma en bucles autónomos** para resolver problemas complejos (Agentic AI). El autor menciona al final que construirán algunos de estos agentes en la última semana del curso, como el que reservó un pastel de banoffee.

1.14 DIA 4 [02-10] – Numero de parámetros en modelos (log scale)



1.14.1 Que es un parámetro – abreviaturas

Son los números que la red ajusta mientras entrena (pesos y sesgos). Con ellos transforma una entrada en una salida. Mas parámetros significa mas “perillas” para guardar patrones.

| Época                    | Parámetros                   | Ejemplo                                                 |
|--------------------------|------------------------------|---------------------------------------------------------|
| ML tradicional           | -20 a 200                    | Solvencia crediticia con 20–30 factores                 |
| Primeras redes profundas | Millones                     | GPT-1: 117 M                                            |
| LLMs actuales            | miles de millones a billones | Llama 3.1: 8 mil millones; GPT-4: 1,76 billones (rumor) |

Ejemplo práctico con cuentas reales.  
Scoring con 3 factores:  $y = w_1 \cdot \text{ingreso} + w_2 \cdot \text{deuda} + w_3 \cdot \text{edad} + b$ , son 4 parámetros.  
Una capa lineal de 512 a 512:  $512 \times 512 + 512 = 262.656$  parámetros.  
Un modelo de 8 mil millones equivale a unas  $8.000.000.000 / 262.656 \approx 30.458$  capas de ese tamaño.

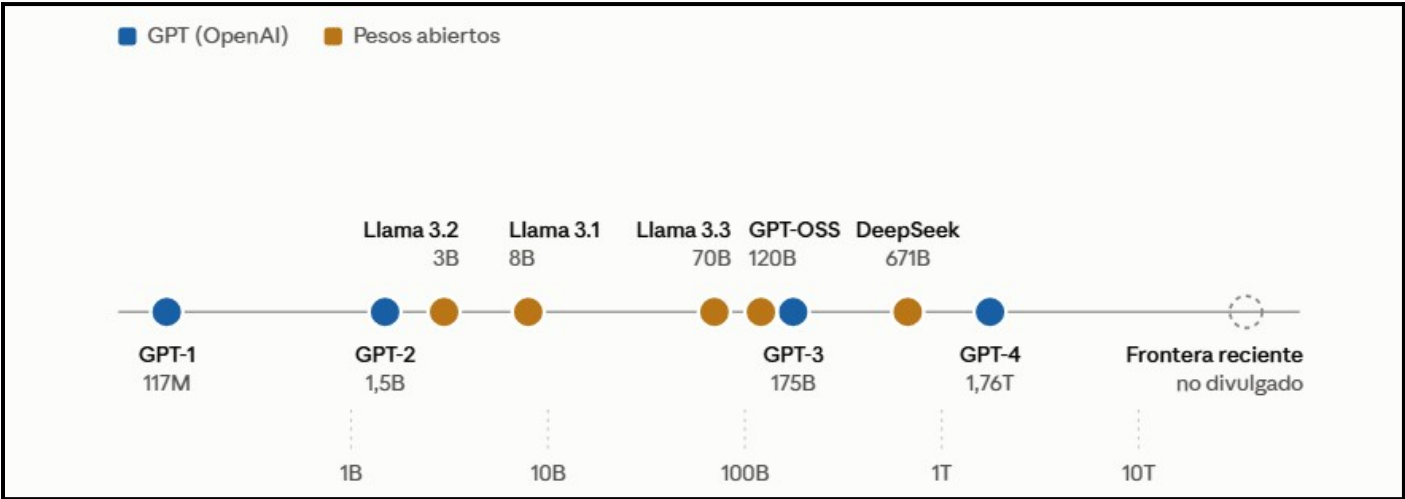
Ejemplo práctico con cuentas reales.  
Scoring con 3 factores:  $y = w_1 \cdot \text{ingreso} + w_2 \cdot \text{deuda} + w_3 \cdot \text{edad} + b$ , son 4 parámetros  
Una capa lineal de 512 a 512 + 512 = 262.656 parámetros.  
Un modelo de 8 mil millones equivale a unas  $8.000.000.000 / 262.656 \approx 30.458$  capas de ese tamaño.

Abreviaturas

| Sigla | Ingles   | Español      | Potencia  |
|-------|----------|--------------|-----------|
| M     | Million  | Millón       | $10^6$    |
| B     | Billion  | mil millones | $10^9$    |
| T     | Trillion | Billón       | $10^{12}$ |

Por eso "1,76 billones" de la transcripción es lo mismo que 1.76T de la diapositiva.

1.14.2 La diapositiva: parámetros en escala logarítmica – Ordenes de magnitud



Cada marca vale 10 veces la anterior. Los colores son los de la diapositiva; la leyenda es interpretación mía.

| Cada marca vale 10 veces la anterior. Los colores son los de la diapositiva; la leyenda es interpretación mía. |            |         |                             |
|----------------------------------------------------------------------------------------------------------------|------------|---------|-----------------------------|
| Modelo                                                                                                         | Parámetros | Tipo    | Estado de la cifra          |
| GPT-1                                                                                                          | 117 M      | GPT     | Publicada                   |
| GPT-2                                                                                                          | 1,5 B      | GPT     | Versión más grande          |
| Llama 3.2                                                                                                      | 3 B        | Abierto | Según la diapositiva        |
| Llama 3.1                                                                                                      | 8 B        | Abierto | Según la diapositiva        |
| Llama 3.3                                                                                                      | 70 B       | Abierto | Según la diapositiva        |
| GPT-OSS                                                                                                        | 120 B      | Abierto | Según la diapositiva        |
| GPT-3                                                                                                          | 175 B      | GPT     | Publicada por OpenAI        |
| DeepSeek                                                                                                       | 671 B      | Abierto | Total (mezcla de expertos)  |
| GPT-4                                                                                                          | 1,76 T     | GPT     | Rumor OpenAI no la confirmó |
| Frontera reciente                                                                                              | —          | —       | No divulgado                |

| Modelo            | Parametros | Tipo    | Estado de la cifra          |
|-------------------|------------|---------|-----------------------------|
| GPT-1             | 117M       | GPT     | Publicada                   |
| GPT-2             | 1,5 B      | GPT     | Version mas grande          |
| Llama 3.2         | 3B         | Abierto | Según la diapositiva        |
| Llama 3.1         | 8 B        | Abierto | Según la diapositiva        |
| Llama 3.3         | 70B        | Abierto | Según la diapositiva        |
| GPT-OSS           | 120 B      | Abierto | Según la diapositiva        |
| GPT-3             | 175 B      | GPT     | Publicada por OpenAI        |
| DeepSeek          | 671 B      | Abierto | Total (mezcla de expertos)  |
| GPT-4             | 1,76 T     | GPT     | Rumor OpenAI no la confirmó |
| Frontera reciente | —          | —       | No divulgado                |

Cuentas rápidas sobre la escala.

GPT-2 a GPT-3: 175 B / 1,5 B  $\approx$  117 veces (casi 2 marcas).

GPT-3 a GPT-4: 1,76 T / 175 B  $\approx$  10 veces (1 marca).

GPT-1 a GPT-4: 1,76 T / 117 M  $\approx$  15.043 veces.

### Cuentas rápidas sobre la escala.

1. GPT-2 a GPT-3: 175B / 1,5B  $\approx$  117 (casi 2 marcas)
2. GPT-3 a GPT4: 1,76T / 175B  $\approx$  10 veces (1 marca)
3. GPT1-1 a GPT-4: 1,76 T / 117M  $\approx$  15.043 veces

Muestran **cuántas veces más grande y complejo** se volvió cada modelo de IA en comparación con versión anterior, basándose en su cantidad de **parámetros**.

### 1. Las letras (M, B, T)

Representan la cantidad de parámetros del modelo. **Los parámetros son los valores internos (como "conexiones neuronales")** que la IA ajusta durante su entrenamiento para aprender a procesar el lenguaje.

### 2. El cálculo de las "veces"

Las divisiones muestran el factor de crecimiento entre una generación y otra:

- **175 B / 1,5 B  $\approx$  117 veces:** Para pasar de GPT-2 a GPT-3, OpenAI hizo el modelo 117 veces más masivo.
- **1,76 T / 175 B  $\approx$  10 veces:** GPT-4 es aproximadamente 10 veces más grande que GPT-3.
- **1,76 T / 117 M  $\approx$  15.043 veces:** Si comparas el primer modelo (GPT-1) con GPT-4, el tamaño de la red neuronal creció más de 15.000 veces.

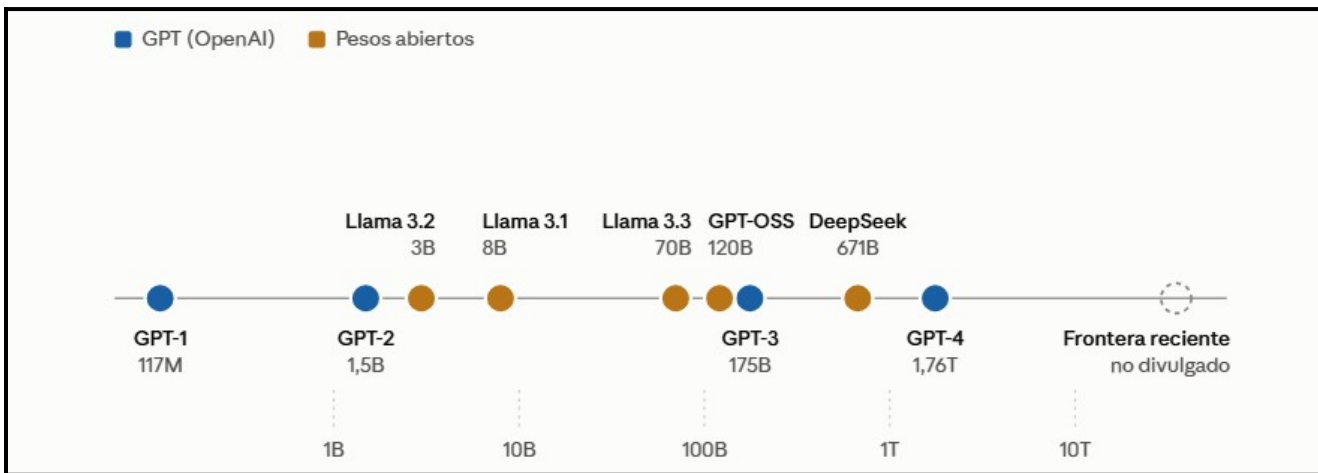
### 3. ¿Qué significan las "marcas"?

El texto usa "marcas" para referirse a los **ordenes de magnitud** (saltos multiplicados por 10). En tecnología, medir en ordenes de magnitud ayuda a entender crecimientos exponenciales:

- Multiplicar por 10 = 1 orden de magnitud (1 marca)
- Multiplicar por 100 = 2 ordenes de magnitud (2 marcas). Como 117 está cerca de 100, el texto dice "casi 2 marcas"

En resumen, esas cuentas ilustran el crecimiento explosivo de los recursos y la escala que OpenAI necesitó inyectar en sus modelos para lograr que pasaran de generar textos básicos (GPT-1) a tener razonamiento complejo (GPT-4).

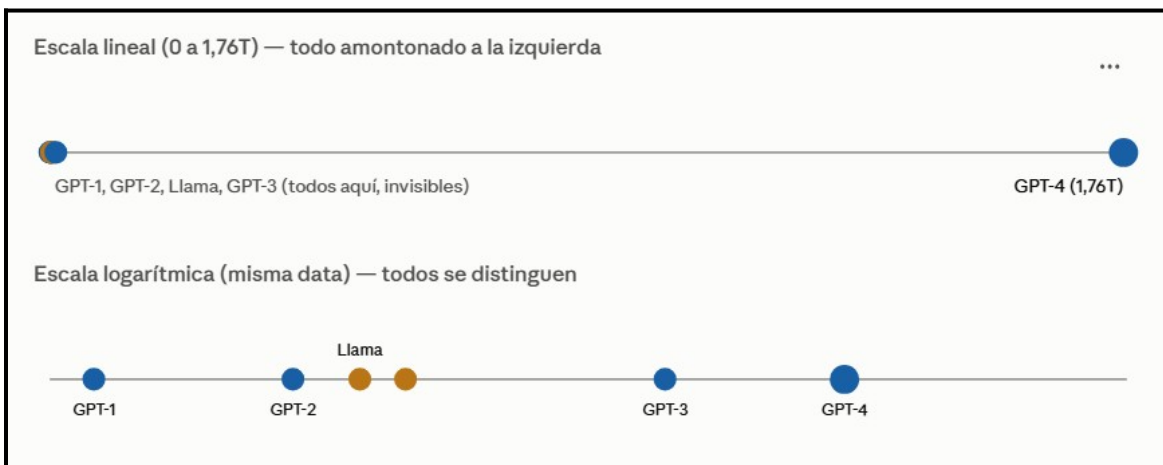
### 1.14.3 ¿Qué es escala logarítmica?



**El problema con una escala normal (lineal):** si pusieras GPT-1 (117M) y GPT-4 (1,76T) en una recta donde cada centímetro vale lo mismo, GPT-1 prácticamente desaparecería pegado al cero, porque 117 millones es casi nada comparado con 1,76 billones. Todos los modelos chicos (Llama 3.2 con 3B, GPT-2 con 1,5B) quedarían amontonados e ilegibles en un mismo punto.

**Qué hace la escala logarítmica:** En vez de que cada marca sume una cantidad fija, cada marca *multiplica* por 10 (1B → 10B → 100B → 1T → 10T). Así:

- La distancia entre 1B y 10B es la misma entre 100B y 1T, aunque en números absolutos enna diferencia es "solo" 9 mil millones y la otra son 900 mil millones.
- No importa cuanto creció en términos absolutos, sino cuantas **veces se multiplica (el orden de magnitud)**.



Regla practica: en escala log, moverse una marca a la derecha = multiplicar por 10. Moverse dos marcas = multiplicar por 100. No importa si partes de 1B o de 100B, la misma distancia visual siempre representa el mismo factor multiplicativo.

**En corto:** Se usa escala logarítmica porque el rango de tamaños (de 117M a 1,76T) abarca más de 4 órdenes de magnitud. Con escala lineal, los modelos chicos serían invisibles. Con escala log, cada modelo es visualmente comparable, aunque uno sea mil veces más grande que otro.

## Con más analogías

Dale, vamos más despacio, sin gráficos esta vez.

**La idea central:** una escala normal (lineal) mide "cuánto sumaste". Una escala logarítmica mide "cuántas veces multiplicaste".

Pensemos en una regla común, de esas que usas en el colegio:

- Del 0 al 1 hay 1 cm.
- Del 1 al 2 hay 1 cm.
- Del 99 al 100 hay 1 cm.

Siempre la misma distancia = siempre la misma suma (+1). Esa es la escala lineal.

Ahora la escala logarítmica funciona distinto:

- Del 1 al 10 hay 1 cm.
- Del 10 al 100 hay 1 cm (otra vez la misma distancia).
- Del 100 al 1.000 hay 1 cm.

Es decir: la misma distancia = siempre multiplicar por 10. No importa si vas de 1 a 10, o de 1.000.000 a 10.000.000 — la misma marca en el eje representa "le agregué un cero".

## ¿Por qué sirve para los parámetros de los modelos?

Mira estos números:

- GPT-1: 117.000.000
- GPT-4: 1.760.000.000.000

Si usaras una regla normal donde 1 cm = 1 millón, para llegar a GPT-4 necesitarías una regla de casi 18 kilómetros de largo. Y en esos 18 km, GPT-1 estaría a solo 1,17 metros del inicio: un punto invisible pegado al cero.

En cambio, en escala logarítmica, cada "salto de 10 veces" ocupa el mismo espacio chico. Así:

- 117M está en una posición.
- 1,5B (GPT-2) está un poco más allá.
- 1,76T (GPT-4) está más allá todavía.

Y los cuatro modelos caben cómodos en una sola imagen, legibles, sin que ninguno desaparezca.

---

ejemplo de como se mide en escala logarítmica

**La fórmula clave:** en escala logarítmica, la posición de un número es su *exponente* en base 10, no el número mismo.

Para un número N, su posición es:  $\log_{10}(N)$

**Ejemplo breve:**

| Número | Se escribe como | Exponente ( $\log_{10}$ ) | Posición en la escala |
|--------|-----------------|---------------------------|-----------------------|
| 1      | $10^0$          | 0                         | 0                     |
| 10     | $10^1$          | 1                         | 1                     |
| 100    | $10^2$          | 2                         | 2                     |
| 1.000  | $10^3$          | 3                         | 3                     |
| 10.000 | $10^4$          | 4                         | 4                     |

Cada vez que el número se multiplica por 10, la posición solo sube de a **1 unidad**. Por eso la distancia entre cada marca es siempre igual, aunque el número crezca muchísimo más rápido.

### Con tus modelos:

| Modelo | Parámetros                                      | $\log_{10}(\text{parámetros})$ | Posición  |
|--------|-------------------------------------------------|--------------------------------|-----------|
| GPT-1  | $117.000.000 \approx 1,17 \times 10^8$          | $\approx 8,07$                 | $\sim 8$  |
| GPT-2  | $1.500.000.000 \approx 1,5 \times 10^9$         | $\approx 9,18$                 | $\sim 9$  |
| GPT-3  | $175.000.000.000 \approx 1,75 \times 10^{11}$   | $\approx 11,24$                | $\sim 11$ |
| GPT-4  | $1.760.000.000.000 \approx 1,76 \times 10^{12}$ | $\approx 12,25$                | $\sim 12$ |

Entonces en el grafico, GPT-1 se dibuja en la posición 8, GPT-2 en la 9, GPT-3 en la 11, GPT-4 en la 12. Son posiciones chicas y cercanas entre sí, aunque los números reales sean brutalmente distintos (de 117 millones a 1,76 billones).

### La cuenta a mano, paso a paso con GPT-3:

1. GPT-3 tiene 175.000.000.000 parámetros.
2. Lo escribes en notación científica:  $1,75 \times 10^{11}$ .
3.  $\log_{10}(1,75 \times 10^{11}) = \log_{10}(1,75) + 11 \approx 0,24 + 11 = 11,24$ .
4. Esa es su posición en el eje: 11,24.



#### 1.14.4 ¿Qué son los parámetros? (números que un modelo ajusta durante el entrenamiento) +

##### $z = Wx + b$ + función de activación

Los parámetros son los números que un modelo ajusta durante el entrenamiento: pesos y sesgos que determinan como transforma entradas en salidas. Todo lo que el modelo “sabe” esta guardado en esos valores.

- **Pesos:** Miden la importancia de cada entrada.
- **Sesgos:** desplazan el resultado
- Cada neurona calcula  $z = Wx + b$ , y los parámetros son el conjunto de todas las **W y b**.
- Empizan con valores aleatorios pequeños, así que al inicio el modelo se equivoca casi siempre; el entrenamiento los va afinando.

#### Ejemplo práctico (números inventados, para visualizar):

Una neurona que predice si aprueban un crédito con 2 entradas: ingreso = 3 y deuda = 1.

- Pesos:  $w_1 = 0,8$ , y  $w_2 = -0,5$ ; sesgo:  $b = 0,1$
- $Z = (0,8 \times 3) + (-0,5 \times 1) + 0,1 = 2,4 - 0,5 + 0,1 = \mathbf{2,0}$

Esa neurona tiene 3 parametros ( $w_1$ ,  $w_2$ ,  $b$ ). Un modelo de 8B tiene 8.000 millones de números como esos. Por eso el tamaño en parámetros se usa como medida de escala: se correlaciona fuertemente con la capacidad del modelo, y también define cuánta memoria necesitas (7B en FP16 ocupan unos 14 GB).

$$z = Wx + b, y$$

Es la formula de lo que hace una capa de neuronas: tomas las entradas, las pondera y les suma un desplazamiento.

- **X:** vector de entrada (los datos que entran).
- **W:** matriz de pesos (caunto importa cada entrada)
- **B:** vector de sesgos (el desplazamiento)
- **Z:** el resultado antes de la función de activación.

#### Control de tamaños (capa con 2 entradas y 1 neurona):

$$[1 \times 2] * [2 \times 1] + [1 \times 1] = [1 \times 1]$$

#### Con el ejemplo de antes (números inventados):

- $W = [0,8 \ -0,5]$ ,  $x = [3, 1]^T$ ,  $b = 0,1$
- $z = (0,8 \cdot 3) + (-0,5 \cdot 1) + 0,1 = 2,4 - 0,5 + 0,1 = \mathbf{2,0}$

#### Con una capa mas grande (la de 512 → 512 que vimos):

- $W$  es  $[512 \times 512]$ ,  $x$  es  $[512 \times 1]$ ,  $b$  es  $[512 \times 1]$ , así que  $z$  es  $[512 \times 1]$ .
- Parámetros:  $512 \times 512$  (en  $W$ ) +  $512$  (en  $b$ ) = **262.656**.

Los parámetros son justamente todo lo que hay en  $W$  y  $b$ . después,  $z$  pasa por una **función de activación** que decide que sale de la neurona.

Es la fórmula de lo que hace una capa de neuronas: toma las entradas, las pondera y les suma un desplazamiento.

$x$ : vector de entrada (los datos que entran).

En estas matemáticas, la “neurona” no está representada por una sola letra, sino que es la **unidad computacional completa** que agrupa esos elementos para tomar una decisión.

Para entender dónde está la neurona, hay que mirar cómo se reparten los datos dentro de las matrices y vectores:

- **Sus conexiones (Pesos en  $W$ ):** Una sola neurona es dueña de **una fila completa** de la matriz  $W$ . Esa fila contiene la importancia de **esa neurona específica** le da a cada uno de los datos de entrada.
- **Su sensibilidad (Sesgo  $b$ ):** Una sola neurona es dueña de un **solo número** dentro del vector  $b$ . Es su umbral o ajuste personal.
- **Su cálculo interno (Resultado  $z$ ):** El resultado matemático de esa neurona es **un solo número** dentro del vector  $z$ .

#### Viéndolo en tus ejemplos:

En tu primer ejemplo (Capa con 2 entradas y 1 neurona), **todo el cálculo representa a una sola neurona**. Esa única neurona tiene los pesos **[0,8 -0,5]**, tiene el sesgo **0,1**, junta las entradas **[3, 1]**, y calcula que su valor interno  $z$  es **2,0**. Después de esto, la neurona pasará ese **2,0** por su función de activación para emitir su “disparo” final.

En tu segundo ejemplo (Capa de 512 entradas a 512 salidas), **ahí hay 512 neuronas trabajando en paralelo**.

- La matriz  $W$  de  $[512 \times 512]$  significa que hay 512 neuronas, y cada una tiene 512 pesos.
- El vector de  $b$   $[512 \times 1]$  significa que cada una de las 512 neuronas tiene su propio sesgo individual.
- El vector  $z$  de  $[512 \times 1]$  contiene los 512 resultados individuales (uno por cada neurona) antes de pasar por la función de activación.

En resumen, **los parámetros** (los valores dentro de  $W$  y  $b$ ) son el “cerebro” de la neurona, las **entradas** ( $x$  son lo que la neurona está “mirando”, y la neurona en sí misma es el mecanismo que toma  $x$ , lo multiplica por  $W$ , le suma  $b$ , y decide qué señal enviar hacia adelante).

## Que sería la función de activación ReLU

ReLU es una función de activación, pero no la única. Es la mas simple: **ReLU (z) = max (0, z)**, o sea, si z es negativo sale 0 y si es positivo pasa igual.

**Ejemplo con el z anterior:**

- $z = 2,0 \rightarrow \text{ReLU}(2,0) = \mathbf{2,0}$
- $z = -1,5 \rightarrow \text{ReLU}(-1,5) = \mathbf{0}$

Qué se usa hoy en los LLMs, según lo que sé (no lo verifiqué ahora):

- Reds clásicas: ReLU
- GPT-2 y GP3-3: GELU, una versión mas suave de ReLU
- Llama y varios modelos recientes: SwiGLU, una variate con compuestas

La idea es la misma en todas: meter un no linealidad despues de  **$Wx + b$** . Sin ella, apilar muchas capas equivaldría a una sola.

1.14.5 Mas con menos: el caso Gemma + Variantes de tamaño: nano, mini, completo

Regla general: más parámetros, modelo más inteligente, porque absorbe más datos de entrenamiento. Pero el profesor destaca que hoy se condensa mejor la información.

|                         | GPT-2 (2019) | Gemma 3 270 M (2025)                                                                      |
|-------------------------|--------------|-------------------------------------------------------------------------------------------|
| Parametros              | 1,5B         | ~270 M                                                                                    |
| Desglose                | -            | -170 M de embeddings<br>(vocabulario de 262.144 tokens) +<br>-100M en bloques transformer |
| Tokens de entrenamiento | -            | 6 blones                                                                                  |

Cuentas.  
1,5 B / 270 M ≈ 5,6 veces menos parámetros que GPT-2.  
6 T tokens / 270 M parámetros ≈ 22.222 tokens por parámetro, unas 1.100 veces más que el 20:1 de Chinchilla.

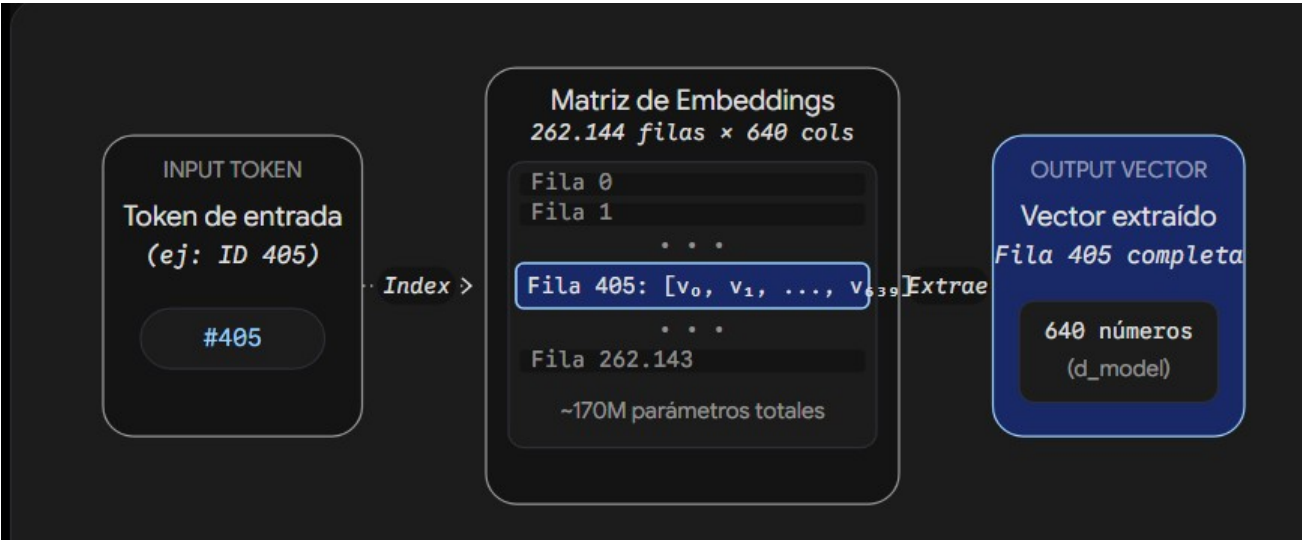
Que Gemma supere a GPT-2 lo dice el profesor. Mi interpretación del porqué: mejores datos, mejor arquitectura y muchísimo más entrenamiento por parámetro.

Los 170 millones representan el **total de números (celdas) que** hay dentro de una enorme matriz matemática, que funciona como el “diccionario” de la IA.

- **Las Filas (Los Tokens):** El modelo tiene un vocabulario de **262.144** palabras o fragmentos de palabras. Cada fila de la tabla corresponde a un token diferente.
- Las columnas (Las Dimensiones): cada fila tiene una cantidad fija de columnas (probablemente **640**, basada en las matemáticas del modelo). Estas columnas almacenan números que representan el “significado” o las características de esa palabra.
- **El Calculo de los Parametros:** Si multiplicas las filas por las columnas (262.144 × 640), te da aproximadamente 167,7 millones de parámetros, que es de donde salen los “~170 M” de embeddings.

Selección de Row Vector en Embedding

Búsqueda por índice (lookup) del token en la matriz de parámetros



¿Por qué Gemma gasta tanto en esto?

Cuando escribes una palabra, la red neuronal no entiendes letras. Lo primero que hace es ir a su tabla de embeddings, buscar la fila de esa palabra, y sacar sus 640 numeros. Ese vector de números es el “embedding”.

En este modelo particular ( Gemma 3 270M), ocurre algo muy peculiar con el diseño: el modelo tiene un vocabulario gigantesco (mas de 260.000 palabras para poder hablar muchos idiomas). Por lo tanto, gasta la mayor parte de su “cerebro” (170 millones de parámetros) solo en aprender este diccionario. Le quedan solo 100 millones de parámetros para hacer el razonamiento real en las capas siguientes.

Variantes de tamaño: nano, mini, completo

| Familia   | Chico    | Mediano | Grande   | Parametros    |
|-----------|----------|---------|----------|---------------|
| Gpt5      | Nano     | Mini    | Completo | No divulgados |
| Cloude    | Haiku    | Sonnet  | Opus     | No divulgados |
| Gemma 3   | 270M, 1B | 4B, 12B | 27B      | Públicos      |
| Llama 3.x | 3B, 8B   | -       | 70B      | Públicos      |

El precio refleja el costo computacional de calcular todos esos parámetros. Que los modelos cerrados sean tres tamaños distintos es una interpretación general, no un dato confirmado.

1.14.6 Escalado en tiempo de entrenamiento y Chinchilla – “sobrecalentamiento”

Cuantos textos (datos) debe leer un modelo según su tamaño para ser eficiente, y cuanta memoria RAM necesitas para usarlo en tu computadora.

1.14.6.1: La Ley de Chinchilla (El equilibrio entre tamaño y lectura)

Conceptos base:

- Parámetros: la "capacidad del cerebro" del modelo.
- Tokens: los trozos de texto que lee al entrenar, como los libros que estudia.
- Chinchilla: estudio de DeepMind (2022) que encontró que, con un presupuesto de cómputo fijo, lo óptimo es unos 20 tokens por cada parámetro.

**Analogía:** un estudiante con mucha capacidad que leyó pocos libros rinde menos que uno con menos capacidad que leyó muchos.

La tabla, fila por fila:

| Modelo     | Cuenta               | Qué pasó                                        |
|------------|----------------------|-------------------------------------------------|
| Gopher     | 300 B / 280 B ≈ 1,07 | Cerebro enorme, casi sin leer. Desperdicio      |
| GPT-3      | 300 B / 175 B ≈ 1,7  | Lo mismo: grande y poco entrenado               |
| Chinchilla | 1,4 T / 70 B = 20    | Más chico pero bien entrenado. Le ganó a Gopher |
| Llama 3 8B | 15 T / 8 B ≈ 1.875   | Se pasó de lectura a propósito                  |

Por qué Llama 3 8B se "sobrentrena": entrenar cuesta una sola vez, pero usar el modelo cuesta cada vez que alguien lo corre. Si lo entrenas de más, queda chico (barato de usar) y aun así rinde bien.

Las calculadoras del widget:

- Chinchilla: 8 B parámetros × 20 = 160 B tokens óptimos. Llama 3 usó más de 15 T, o sea unas 94 veces más: 15.000 / 160 ≈ 94.
  - RAM: 8 B parámetros × 2 bytes (FP16) = 16 GB solo para guardar los pesos.
-

Escalado en tiempo de entrenamiento y Chinchilla

1. Tres conceptos que no hay que mezclar

| Concepto                | Qué es                                                                                                                              | Ejemplo                           |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Parámetros              | Números internos (pesos y sesgos) que el modelo ajusta al entrenar. Miden el <i>tamaño</i> del modelo.                              | <b>Llama 3 8B: 8.000 millones</b> |
| Tokens del vocabulario  | Lista fija de trozos de texto que el modelo conoce. Cada uno tiene 1 embedding (un vector), y esa tabla es parte de los parámetros. | <b>Gemma 3: 262.144 tokens</b>    |
| Tokens de entrenamiento | Cantidad total de texto que el modelo lee para aprender. Se repiten tokens del mismo vocabulario millones de veces.                 | <b>Gemma 3 270M: 6 billones</b>   |

El "1 token = 1 embedding" es cierto, pero es la forma en que un token entra al modelo. No tiene nada que ver con la regla de Chinchilla.

2. Qué dice Chinchilla

Chinchilla (DeepMind, 2022) encontró que, con un cómputo de entrenamiento fijo, **parámetros y datos deben crecer juntos**. La referencia es de unos **20 tokens por parámetro**:

$$\text{tokens de entrenamiento} \div \text{parámetros} = 20$$

3. Cómo NO se interpreta

- El 20 no significa que cada parámetro reciba o procese 20 tokens. Lo que pasa en realidad:
- Cada token que entra pasa por (casi) todos los parámetros del modelo y los ajusta un poquito. En modelos de mezcla de expertos, solo por una parte.
  - Cada parámetro termina siendo ajustado por todos los tokens, no solo por 20.
  - El 20 es **una proporción total**: "el modelo completo recibe 20 veces más tokens que parámetros tiene".

4. Por qué se divide

Dividir sirve para comparar modelos de distinto tamaño con un solo número (datos inventados):

| Modelo | Datos | Parámetros | Datos ÷ parámetros | Diagnóstico            |
|--------|-------|------------|--------------------|------------------------|
| A      | 40    | 2          | 20                 | Justo según Chinchilla |
| B      | 400   | 20         | 20                 | Justo según Chinchilla |
| C      | 40    | 20         | 2                  | Le falta lectura       |

A y B tienen tamaños distintos, pero la misma proporción. Es como comparar autos por km/litro: 300 km con 20 litros y 600 km con 40 litros rinden igual (15 km/litro).

5. Ejemplo mínimo paso a paso

**Modelo de 2 parámetros (w y b) que calcula salida = w·entrada + b. Le muestras 40 ejemplos.**

1. Cada ejemplo pasa por w y por b.
2. El modelo se equivoca un poco y se corrigen w y b.
3. Ejemplos ÷ parámetros =  $40 \div 2 = 20$ .

## 6. Cuentas para practicar

- **Conozco los parámetros, quiero los tokens ( $\times 20$ ):** 8 B  $\rightarrow$  160 B tokens. 9 B  $\rightarrow$  180 B. 70 B  $\rightarrow$  1,4 T.
- **Conozco los tokens, quiero el modelo ( $\div 20$ ):** 1 T  $\div 20 = 50$  B parámetros.
- **Diagnóstico:** 5.000 tokens y 50 parámetros dan  $5.000 \div 50 = 100$ , que es 5 veces más que 20, así que está sobreentrenado.

## 7. Modelos reales

| Modelo       | Parámetros | Tokens      | Tokens por parámetro                         |
|--------------|------------|-------------|----------------------------------------------|
| Gopher       | 280 B      | 300 B       | $\approx 1,07$ (subentrenado)                |
| GPT-3        | 175 B      | 300 B       | $\approx 1,7$                                |
| Chinchilla   | 70 B       | 1,4 T       | 20 (referencia)                              |
| Llama 3 8B   | 8 B        | más de 15 T | $\approx 1.875$ (sobreentrenado a propósito) |
| Gemma 3 270M | 270 M      | 6 T         | $\approx 22.222$                             |

- Chinchilla (70 B) superó a Gopher (280 B) usando el mismo cómputo de entrenamiento.
- Entrenar cuesta una sola vez, pero usar el modelo cuesta cada vez que alguien lo corre. Por eso hoy se sobreentrenan modelos chicos a propósito: quedan baratos de usar y rinden bien.

## 8. RAM solo para los pesos

RAM  $\approx$  parámetros  $\times$  bytes por parámetro

| Modelo        | Precisión      | Cuenta                   | RAM      |
|---------------|----------------|--------------------------|----------|
| Llama 3.1 8B  | FP16 (2 bytes) | $8 \text{ B} \times 2$   | 16 GB    |
| DeepSeek 671B | FP16 (2 bytes) | $671 \text{ B} \times 2$ | 1.342 GB |

**Esto no incluye el contexto (KV cache) ni el sistema, así que en la práctica necesitas algo más.**

## 9. Resumen

- Parámetros = tamaño del modelo. Tokens de entrenamiento = cuánto texto leyó.
- Chinchilla propone unos 20 tokens por parámetro, como proporción total y no como reparto.
- Gopher y GPT-3 estaban subentrenados; Llama 3 y Gemma se sobreentrenan a propósito para ser baratos de usar.



### 1.14.6.2: El "Sobreentrenamiento" actual (El caso de Llama 3 8B)

Hoy en día, laboratorios como Meta (creadores de Llama) rompen la regla de Chinchilla a propósito.

Llama 3 8B es un modelo "pequeño" (8.000 millones de parámetros), pero lo forzaron a leer la brutalidad de 15 billones de tokens (ratio 1.875 a 1). ¿Por qué lo hacen? Entrenarlo costó carísimo, pero como el modelo resultante es pequeño, millones de personas pueden descargarlo y correrlo gratis y rápido en laptops comunes. El sobreentrenamiento hace que un modelo pequeño sea excepcionalmente listo.

Es una decisión de **dónde gastar la plata**: pagar más una sola vez (entrenar) para pagar menos todas las veces (usar).

#### Dos costos distintos:

| Costo             | Cuándo se paga                        | Cuántas veces     |
|-------------------|---------------------------------------|-------------------|
| Entrenar          | Al crear el modelo                    | 1 vez             |
| Usar (inferencia) | Cada vez que alguien le pregunta algo | Millones de veces |

**Qué optimiza Chinchilla:** el costo de entrenar. Responde "con X de presupuesto de entrenamiento, ¿qué combinación de tamaño y datos da el mejor modelo?". No dice nada sobre cuánto cuesta usarlo después. Por eso "romperla" no es un error: Meta optimiza otra cosa.

#### Ejemplo con números inventados (solo para visualizar):

|                             | Modelo A (grande)       | Modelo B (chico, sobreentrenado) |
|-----------------------------|-------------------------|----------------------------------|
| Parámetros                  | 70 B                    | 8 B                              |
| Costo de entrenar (una vez) | 100                     | 130                              |
| Costo por uso               | 70                      | 8                                |
| Costo total con 1.000 usos  | $100 + 70.000 = 70.100$ | $130 + 8.000 = 8.130$            |

B costó más de entrenar (130 vs 100), pero como se usa muchísimo, termina saliendo ~9 veces más barato. Y si le diste mucha lectura, un modelo chico puede rendir parecido a uno más grande.

**Analogía:** un estudiante con poca capacidad que estudió el triple rinde bien en el examen, y además "pesa" poco para llevarlo a todos lados.

Dos matices sobre el párrafo que pegaste, porque exagera un poco:

- "Excepcionalmente listo" es un adjetivo: el modelo de 8 B mejora con más lectura, pero no alcanza a los modelos mucho más grandes en todo.
- "Laptops comunes": los pesos de 8 B en FP16 ocupan 16 GB, así que en un laptop normal normalmente se usa una versión comprimida (cuantizada,  $\sim 4 \text{ bits} \approx 4 \text{ GB}$ ).

#### Entonces, ¿por qué leer de más?

Un modelo de 8 B es chico, y eso lo limita. Pero si lo dejas leer muchísimo más, aprovecha mejor esa capacidad y rinde mucho mejor de lo que rendiría con poca lectura. No llega al nivel de los modelos gigantes, pero se acerca bastante.

#### Y por qué importa que sea chico:

- Un modelo de 70 B puede ser más listo, pero necesita mucha memoria (muy caro correrlo).
- Uno de 8 B ocupa 16 GB en FP16, o unos 4 GB comprimido, y corre en equipos normales.

**La decisión de Meta, en una línea:**

"Pagamos más una vez para que el modelo chico quede lo más capaz posible, **porque así mucha gente lo puede correr barato.**"

**El beneficio de que lo usen: un modelo "bastante listo" que cualquiera puede correr vale más que uno "muy listo" que casi nadie puede correr.**

### 1.14.6.3: La RAM y los formatos (FP16 – INT8 – 4bits) – cada parámetro es un numero guardado en la memoria

que son FP16, INT8, 4 bits

Son tres formas de guardar un numero en el computador. Cada una usa una cantidad distinta de **bits**.

**Un bit** es lo mínimo que se guarda: un 0 o un 1. Con mas bits puedes distinguir mas valores diferentes, y la cantidad es 2 elevado al numero de bits.

| Formato       | Que significa                               | Bits | Valores posibles                                 |
|---------------|---------------------------------------------|------|--------------------------------------------------|
| <b>FP16</b>   | <b>Float Point 16: numero con decimales</b> | 16   | $2^{16} = 65.536$ combinaciones                  |
| <b>INT8</b>   | Integer 8: número entero                    | 8    | $2^8 = 256$ valores (por ejemplo, de -128 a 127) |
| <b>4 bits</b> | Entero de 4 bits                            | 4    | $2^4 = 16$ valores                               |

- **FP16** guarda decimales, por ejemplo 0,8731
- **INT8 y 4bits** guardan solo enteros, así que para representar un parametro decimal hay que “traducirlo” con una escala. Eso es cuantizar.

**Ejemplo de cuantización (simplificado, número inventado):** un parámetro vale 0,8731 y sus valores van entre -1 y 1.

- **INT8:**  $0,8731 \times 127 = 110,9$ , que se redondea a **111**. Para recuperarlo:  $111 / 127 \approx \mathbf{0,874}$ . Error muy chico.
- **4 bits:** con 16 valores (-8 a 7) usamos la escala de 7:  $0,8731 \times 7 = 6,1$ , que se redondea a **6**. Para recuperarlo:  $6 / 7 \approx \mathbf{0,857}$ . Error mayor.

Eso es lo que significa "pierde un poquito de calidad": al recuperar el número, queda un poco distinto del original. Mientras menos bits, menos valores posibles y más error.

En la práctica los métodos de cuantización son más elaborados que esto (por ejemplo, usan una escala por grupo de parámetros), pero la idea es la misma.

**Por eso ocupan distinto espacio:** 16 bits son 2 bytes, 8 bits son 1 byte y 4 bits son 0,5 bytes, y de ahí salen los 16 GB, 8 GB y 4 GB del modelo de 8 B.

**La idea:** Quieres guardar el numero 0,8731, pero solo puedes anotar “en que marca de la regla cae”

**Regla de INT8:** Va de -1 a 1 y tiene 255 marcas (mas o menos). Como hay muchas marcas, están muy juntas.

- 0,8731 cae casi justo en la marca numero 111 (de 127).
- Anotas “111”.
- Al recuperarlo, la marca 111 vale 0,874.
- Original: 0,8731 → recuperado: 0,874. Casi igual.

**Regla de 4 bits:** va de -1 a 1, pero solo tiene unas 15 marcas. Como hay pocas, están muy separadas.

- 0,8731 cae entre la marca 6 (0,857) y la marca 7 (1,0).
- Te quedas con la más cercana: la 6.
- Al recuperarlo, la marca 6 vale 0,857

- Original: 0,8731 → recuperado: 0,857. Se perdió un poco mas.

**Los cálculos de mi mensaje anterior son solo cómo se encuentra la marca:**

- INT8:  $0,8731 \times 127 = 110,9$ , que se redondea a la marca 111. Para volver:  $111 \div 127 \approx 0,874$ .
- 4 bits:  $0,8731 \times 7 = 6,1$ , que se redondea a la marca 6. Para volver:  $6 \div 7 \approx 0,857$ .

Multiplicas por 127 (o por 7) para convertir el decimal en “numero de marca”. Al dividir por lo mismo, vuelves al decimal.

**Qué se aprende de esto:**

| Formato | Marcas | Error en el ejemplo |
|---------|--------|---------------------|
| INT8    | muchas | 0,0009 (muy chico)  |
| 4 bits  | pocas  | 0,0161 (más grande) |

**Mientras menos bits, menos marcas y más redondeo.** Por eso un modelo en 4 bits ocupa menos espacio, pero pierde un poco de calidad.

## Sobre el parámetro

entonces, parametro al final es?

Un parámetro es **un número que el modelo aprendió durante el entrenamiento y que guarda en memoria.** Nada más.

**Con todo lo que vimos:**

**1.15 Se aprende:** empieza al azar y el entrenamiento lo ajusta poco a poco (por ejemplo 0,8731).

**1.16 Se usa en la cuenta  $z = Wx + b$ :** Los pesos (W) multiplican las entradas y los sesgos (b) desplazan el resultado.

**1.17 Ocupa espacio:** cada uno se guarda con 16, 8 o 4 bits, y por eso un modelo de 8 B pesa 16, 8 o 4 GB.

**1.18 Mide el tamaño del modelo:** “8B” significa que hay 8.000 millones de esos números.

**1.19 Se relaciona con los datos:** Chinchilla dice que conviene entrenar con unos 20 tokens por cada parámetro.

**Ejemplo con una neurona (números inventados):** entradas 3 y 1, pesos 0,8 y -0,5, sesgo 0,1.

$$z = (0,8 \times 3) + (-0,5 \times 1) + 0,1 = 2,0$$

Los 3 numeros que el modelo aprendió (0,8, -0,5 y 0,1) son los parámetros. Un LLM es lo mismo, pero con miles de millones de números como esos.

Cada parámetro es un numero guardado en la memoria, y puedes guardarlo con mas o menos precisión.

**Mientras menos precisión, menos espacio ocupa.**

**Un byte son 8 bits.** Los formatos son solo cuantos bits usas por parámetros.

- FP16: 16 bits = 2 bytes
- INT8: 8 bits = 1 byte
- 4 bits = 0,5 bytes

**Analogía con redondear (números inventados):** un parámetro vale 0,8731.

- Guardado con mucha precisión: 0,8731
- Guardado mas redondeado: 0,87
- Guardado muy redondeado: 0,9

Ocupa menos espacio, pero pierdes un poco de exactitud. A eso se le llama **cuantización**: redondear los parámetros para que ocupen menos.

**La cuenta para un modelo de 8 B (8.000.000.000 parámetros):**

| Formato | Cuenta                                   | Memoria |
|---------|------------------------------------------|---------|
| FP16    | $8.000.000.000 \times 2 \text{ bytes}$   | 16 GB   |
| INT8    | $8.000.000.000 \times 1 \text{ byte}$    | 8 GB    |
| 4 bits  | $8.000.000.000 \times 0,5 \text{ bytes}$ | 4 GB    |

Formula: **Memoria = parámetros x bytes por parámetro**

- La memoria que importa suele ser la de la **tarjeta grafica (VRAM)** si la usas. Si corres el modelo en el procesador, es la RAM normal.
- **Esos números son solo para los pesos.** Falta memoria extra para el contexto de la conversación y el sistema, así que en la practica necesitas algo mas.

El modelo es una lista de 8.000 millones de números. Cada número se guarda en el computador y ocupa espacio. Tú eliges **cuánto espacio le das a cada número**:

- **Espacio grande (FP16):** Cada numero ocupa 2 bytes. Es la calidad alta.
- **Espacio mediano (INT8):** Cada numero ocupa 1 byte. Calidad media.
- **Espacio chico (4 bits):** Cada numero ocupa 0,5 bytes. Calidad mas baja, pero ocupa poco.

**La cuenta es solo una multiplicación:**

- FP16:  $8.000.000.000 \times 2 = 16.000.000.000 \text{ bytes} = \mathbf{16 \text{ GB}}$
- INT8:  $8.000.000.000 \times 1 = 8.000.000.000 \text{ bytes} = \mathbf{8 \text{ GB}}$
- 4 bits:  $8.000.000.000 \times 0,5 = 4.000.000.000 \text{ bytes} = \mathbf{4 \text{ GB}}$

Es el **mismo modelo**, con los mismos 8.000 millones de números. Solo cambia cuánto espacio ocupa cada uno, igual que la foto en calidad alta o comprimida.

**Para que sirve:** Si tu computador solo tiene 8 GB libres, el modelo en calidad alta (16GB) no te cabe, pero comprimido a 4 GB si. Lo comprimido pierde un poco de calidad, pero puedes usarlo.

super breve, osea que FP16 por cada numero te lo guarda en 16 numeros?

No, en **16 bits**, no en 16 números. Un bit es una casilla que solo puede valer 0 o 1.

Cada parámetro se guarda como una fila de 16 casillas de ceros y unos, por ejemplo **0011010110010110**. Esa fila representa **un solo número** (como 0,8731).

- FP16: 16 casillas por número (2 bytes).
- INT8: 8 casillas por número (1 byte).
- 4 bits: 4 casillas por número (0,5 bytes).

**Mas casillas permiten mas combinaciones de ceros y unos, o sea mas valores posibles y más precisión.**

## Otro ejemplo

**Paso 1: cada parámetro es un número que hay que guardar.**

Los 10 números, por ejemplo: 0,87 / -0,5 / 0,1 / 0,33 y así hasta completar 10.

**Paso 2: cada número ocupa un espacio fijo, según el formato.**

- FP16: 2 bytes cada uno
- INT8: 1 byte cada uno
- 4 bits: 0,5 bytes cada uno

**Paso 3: multiplicas cantidad de números × espacio de cada uno.**

- FP16:  $10 \times 2 = 20$  bytes
- INT8:  $10 \times 1 = 10$  bytes
- 4 bits:  $10 \times 0,5 = 5$  bytes

Eso es toda la fórmula: **memoria = parámetros × bytes por parámetro.**

**Paso 4: lo mismo, pero con 8.000 millones de números.**

- FP16:  $8.000.000.000 \times 2 = 16.000.000.000$  bytes
- INT8:  $8.000.000.000 \times 1 = 8.000.000.000$  bytes
- 4 bits:  $8.000.000.000 \times 0,5 = 4.000.000.000$  bytes

**Paso 5: pasar bytes a GB.**

1 GB son 1.000.000.000 bytes (mil millones), así que:

- $16.000.000.000 \div 1.000.000.000 = 16$  GB
- $8.000.000.000 \div 1.000.000.000 = 8$  GB
- $4.000.000.000 \div 1.000.000.000 = 4$  GB

**Paso 6: para qué sirve.**

Es el mismo modelo, con los mismos 8.000 millones de números. Si tu computador tiene 8 GB libres:

- 16 GB (FP16) → no cabe
- 8 GB (INT8) → justo, casi sin margen (y falta espacio para otras cosas)
- 4 GB (4 bits) → cabe bien

pero cual es el que tiene mayor calidad?

**FP16 tiene la mayor calidad**, después INT8 y al final 4 bits.

| Formato | Espacio por parámetro    | Calidad             |
|---------|--------------------------|---------------------|
| FP16    | 2 bytes (el más grande)  | La más alta         |
| INT8    | 1 byte                   | Casi igual que FP16 |
| 4 bits  | 0,5 bytes (el más chico) | La más baja         |

**Regla simple:** más espacio por parámetro significa más precisión, y más precisión significa mejor calidad. Es como la foto: la de 10 MB tiene mejor calidad que la comprimida de 3 MB.

**Pero en la práctica:**

- La diferencia entre FP16 e INT8 suele ser muy chica.
- 4 bits pierde algo más, aunque muchas veces sigue siendo usable.
- Lo que pierde depende del modelo y del método de cuantización, y eso no lo verifiqué ahora, así que tómallo como una regla general.

**Entonces la decisión es un intercambio:** si tienes memoria de sobra, usas FP16. Si tienes poca, bajas a INT8 o a 4 bits y aceptas perder un poco de calidad.

como se veria esa misma tabla en numero?

Para sacar números de "calidad" usé el ejemplo del parámetro 0,8731, midiendo cuánto se aleja el valor recuperado del original.

| Formato | Bits | Valores posibles | Bytes por parámetro | Memoria (8 B) | Valor recuperado | Error     |
|---------|------|------------------|---------------------|---------------|------------------|-----------|
| FP16    | 16   | 65.536           | 2                   | 16 GB         | ≈ 0,87305        | ≈ 0,00005 |
| INT8    | 8    | 256              | 1                   | 8 GB          | ≈ 0,874          | ≈ 0,0009  |
| 4 bits  | 4    | 16               | 0,5                 | 4 GB          | ≈ 0,857          | ≈ 0,016   |

**Cómo leerla:**

- **Error = |original – recuperado|.** Mientras más chico, más calidad.

- INT8:  $|0,8731 - 0,8740| \approx 0,0009$ .
- 4 bits:  $|0,8731 - 0,8571| \approx 0,0160$ .
- FP16: es unas 18 veces más preciso que INT8 en este ejemplo ( $0,0009 \div 0,00005$ ), y INT8 unas 17 veces más preciso que 4 bits ( $0,016 \div 0,0009$ ).

### Ojo con dos cosas:

- El error es de **un solo número de ejemplo**, y no del modelo completo. Cuánta calidad pierde un modelo real depende del método de cuantización, y eso no lo verifiqué.
- El valor de FP16 lo calculé con su precisión interna ( $1.788/2.048 \approx 0,87305$ ), así que es aproximado.

La tendencia sí es clara: menos bits significan menos valores posibles, más error por parámetro y menos memoria.

### 1.14.6.4 Escalado en tiempo de inferencia

Inferencia es usar el modelo ya entrenado. Ahí hay trucos para sacarle más rendimiento **sin reentrenarlo**.

| Técnica            | Qué hace                                                                                                                       | Época            |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------|
| Razonamiento       | Le pides generar tokens que describan cómo va a responder antes de responder. Hasta sirve repetir "espera" para que piense más | El último año    |
| Más contexto (RAG) | Metes más información en la secuencia de entrada, como precios de entradas de todo el mundo, y el modelo se apoya en eso       | Hace unos 2 años |

Ejemplo con datos reales:  $17 \times 24$ .

Directo: el modelo suelta un número de una.

Con razonamiento:  $17 \times 20 = 340$ ;  $17 \times 4 = 68$ ;  $340 + 68 = 408$ . Más tokens generados, mejor resultado.

### Dos ejes independientes (ortogonales): puedes usar ambos

Dos ejes independientes (ortogonales): puedes usar ambos

Entrenamiento bajo, inferencia baja  
Modelo chico, respuesta directa

Entrenamiento alto, inferencia baja  
Modelo grande, respuesta directa

Entrenamiento bajo, inferencia alta  
Modelo chico con razonamiento y RAG

Entrenamiento alto, inferencia alta  
Modelo grande con razonamiento y RAG



**"Inferencia alta"** significa que le das mas trabajo al modelo en el momento de usarlo, sin cambiarlo ni reentrenarlo. Es un nombre que usé yo para el eje; en la transcripción se habla de **"escalado en tiempo de inferencia"**.

**Inferencia baja:** respondes de una, sin ayuda extra

**Inferencia alta:** haces que el modelo gaste más cálculo antes de responder. Hay dos formas, las que mencionó el profesor:

1. **Razonamiento:** Le pides que primero genere tokens explicando como va a resolver el problema, y recién despues responda.
2. **Mas contexto (RAG):** Le pegas información extra en la pregunta, y el modelo se apoya en ella.

**Ejemplo con  $17 \times 24$ :**

- Inferencia baja: el modelo responde un número directo, y puede equivocarse.
- Inferencia alta:  $17 \times 20 = 340$ ,  $17 \times 4 = 68$ ,  $340 + 68 = 408$ . Generó más tokens, pero llegó mejor al resultado.

**Cuesta más, igual que el entrenamiento:** más tokens generados significan más tiempo y más cómputo por cada respuesta.

**Cómo se lee la tabla que pegaste:**

- **Entrenamiento** = qué tan grande es el modelo y cuánto leyó (se decide una vez, al crearlo).
- **Inferencia** = cuanto esfuerzo le pones cada vez que lo usas.

Son ejes independientes: un modelo chico con razonamiento y RAG (entrenamiento bajo, inferencia alta) puede rendir mejor que uno grande respondiendo directo en algunas tareas. Esa última frase es una idea general de la transcripción, no un resultado que haya verificado.

1.14.6.5 Mezcla de expertos (MoE) + Resumen

Los modelos grandes suelen contener muchos submodelos "expertos". Para cada token se activa solo una parte, así que el cómputo depende de los parámetros activos, pero la memoria depende de los totales.

| Modelo           | Totales | Activos | Expertos |
|------------------|---------|---------|----------|
| Llama 4 Scout    | 109 B   | 17 B    | 16       |
| Llama 4 Maverick | 400 B   | 17 B    | 128      |
| Llama 4 Behemoth | ~2 T    | 288 B   | 16       |
| DeepSeek         | 671 B   | ~37 B*  | —        |
| GPT-4 Rumor      | ~1,8 T  | ~280 B  | 16       |

Maverick: 17 B / 400 B = 4,25 % de los parámetros trabaja por token. Cómputo de un modelo de 17 B, pero guardar los pesos en FP16 cuesta 400 B × 2 bytes = 800 GB.

\* Dato de mi conocimiento previo, no lo verifiqué en esta búsqueda. Las cifras de GPT-4 vienen de filtraciones.

Piensa en un hospital con 8 especialistas, donde un recepcionista (el "router") decide a cuáles 2 mandar cada caso.

Modelo de juguete (números inventados):

- 8 expertos de 10 B parámetros cada uno.
- Parámetros totales: 8 × 10 B = **80 B**.
- Por cada token, el router elige 2 expertos: 2 × 10 B = **20 B activos**.

Que cuesta cada cosa:

| Recurso             | Depende de         | Cuenta                                |
|---------------------|--------------------|---------------------------------------|
| Computo (velocidad) | Parámetros activos | Como un modelo de 20 B                |
| Memoria             | Parámetros totales | 80 B x 2 buyes (FP16) = <b>160 GB</b> |

Los 8 expertos tienen que estar cargados en memoria, aunque en cada token solo trabajen 2.

Un token pasando por el modelo:

1. Entra el token "casa".
2. El router puntúa a los 8 expertos y elige los 2 mejores (por ejemplo, el 3 y el 7).
3. Solo esos 2 hacen la cuenta.
4. Los otros 6 descansan para ese token.
5. El siguiente token puede usar otros 2 expertos distintos.

Con un caso real, Llama 4 Maverick:

- 128 expertos, 400 B totales, 17B activos.
- 17 ÷ 400 = 4,25% del modelo trabaja por token.
- Cómputo de un modelo de 17B, pero guardarlo en FP16 cuesta 400 B x 2 = **800 GB**.

Resumen

|                           |                                                                            |
|---------------------------|----------------------------------------------------------------------------|
| Idea                      | En una línea                                                               |
| Parámetro                 | Número que la red aprende; hoy hay miles de millones o billones            |
| Más parámetros            | En general más inteligente, pero se puede hacer más con menos (Gemma 270M) |
| Variantes                 | Distintos tamaños con distinto costo computacional                         |
| Escalado de entrenamiento | Más parámetros y más datos; Chinchilla propone ~20 tokens por parámetro    |
| Escalado de inferencia    | Razonamiento y más contexto (RAG) al momento de usar el modelo             |
| Escala log                | Cada marca es 10 veces mayor                                               |
| MoE                       | Muchos expertos; solo se activan algunos por pregunta                      |

|                           |                                                                            |
|---------------------------|----------------------------------------------------------------------------|
| Idea                      | En una línea                                                               |
| Parametro                 | Numero que la red aprende; hoy hay miles de millones o billones            |
| Mas parámetros            | En general mas inteligente, pero se puede hacer mas con menos (Gemma 270M) |
| Variantes                 | Distintos tamaños con distinto costo computacional                         |
| Escalado de entrenamiento | Mas parámetros y mas datos; chinllina propone -20 tokens por parametro     |
| Escalado de inferencia    | Razonamiento y mas contexto (RAG) al momento de usar el modelo             |
| Escala log                | Cada marca es 10 veces mayor                                               |
| MoE                       | Muchos expertos; solo se activan algunos por pregunta                      |

1.15 DIA 4 ¿Qué es un token? – carácter vs palabra vs token – Las raíces de palabras – Token, ID y vector no son lo mismo – el tokenizador de GPT

### Introducing Tokens

In the early days, neural networks were trained at the character level  
Predict the next character in this sequence  
Small vocab, but expects too much from the network

Then neural networks were trained off words  
Predict the next word in this sequence  
Much easier to learn from, but leads to enormous vocabs with rare words omitted

The breakthrough was to work with chunks of words, called 'tokens'  
A middle ground: manageable vocab, and useful information for the neural network  
In addition, elegantly handles word stems

From <https://platform.openai.com/tokenizer>

### GPT's Tokenizer

An important sentence for my class of AI engineers

Clear Show example

|        |            |
|--------|------------|
| Tokens | Characters |
| 9      | 50         |

An important sentence for my class of AI engineers

For common words, 1 word maps to 1 token

Observe how the break between words is part of the token

1. Idea central

Un **token** es un trozo de texto que el modelo trata como una unidad: puede ser una palabra, un fragmento de palabra o incluso dos palabras muy comunes juntas. El modelo tiene una lista fija de esos trozos (su **vocabulario**), y todo texto se convierte en una secuencia de ellos.

2. Cómo se llegó a los tokens (diapositiva 1)

| Etapas                    | Qué predice                   | Vocabulario    | Problema o ventaja                                                                                   |
|---------------------------|-------------------------------|----------------|------------------------------------------------------------------------------------------------------|
| Primeros días: caracteres | El siguiente carácter         | Pequeño (~100) | <b>Problema</b> Le exige demasiado a la red: armar palabras y asociarlas a significado               |
| Después: palabras         | La siguiente palabra          | Enorme         | <b>Problema</b> Más fácil de aprender, pero hay que omitir palabras raras (nombres propios, lugares) |
| El avance: tokens         | El siguiente trozo de palabra | Manejable      | <b>Ventaja</b> Término medio, información útil para la red y maneja bien las raíces de palabras      |

El profesor aclara que no hay nada fundamental que obligue a usar tokens: podría hacerse con caracteres o con diccionarios gigantes. Simplemente este compromiso resultó eficiente, rápido y aprende conceptos rápido.

| Etapa                            | Que predice                   | Vocabulario  | Problema o ventaja                                                                                                             |
|----------------------------------|-------------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Primeros días: caracteres</b> | El siguiente carácter         | Chico (~100) | Ocupa poca memoria y es eficiente, pero le exige demasiado a la red: armar palabras y asociarlas a significado                 |
| <b>Despues: palabras</b>         | La siguiente palabra          | Enorme       | Mas fácil de aprender, pero hay tantas palabras (y nombres propios) que hubo que omitir las raras, y el modelo no las entendía |
| <b>El avance: tokens</b>         | El siguiente trozo de palabra | Manejable    | Termino medio: información útil para la red y maneja bien las raíces de palabras                                               |

### Ejemplos:

Uso la frase de la diapositiva, "An important sentence for my class of AI engineers", y una palabra rara.

#### Ejemplo 1: la misma frase en los tres enfoques

| Enfoque    | Cómo se parte                                                                        | Pasos de la secuencia | Vocabulario |
|------------|--------------------------------------------------------------------------------------|-----------------------|-------------|
| Caracteres | A, n, (espacio), i, m, p, o, r, t, a, n, t...                                        | 50                    | ~100        |
| Palabras   | An, important, sentence, for, my, class, of, AI, engineers                           | 9                     | 170.000+    |
| Tokens     | "An", " important", " sentence", " for", " my", " class", " of", " AI", " engineers" | 9                     | ~100.000    |

Los 50 y los 9 salen de la diapositiva. En esta frase, palabras y tokens dan lo mismo porque todas son comunes.

#### Ejemplo 2: el problema de caracteres

Para escribir "important", la red tiene que predecir letra por letra: después de "impor" viene "t", y así sigue. Además tiene que aprender por sí sola que esas 9 letras juntas significan algo. Por eso decía el profesor que se le exigía demasiado a la red.

#### Ejemplo 3: el problema de palabras (ilustrativo, no verificado)

Imagina un vocabulario de palabras que no incluye "Arenas". La frase "Vivo en Punta Arenas" quedaría como "Vivo en Punta [desconocida]". El modelo no entiende esa parte. Y como hay tantos nombres propios y lugares, el vocabulario tendría que ser gigante o dejar muchas palabras afuera.

#### Ejemplo 4: cómo lo resuelven los tokens (ilustrativo)

Con tokens, "Arenas" no queda afuera: se parte en trozos conocidos, por ejemplo "Aren" + "as", y el modelo trabaja con eso. En el peor caso, se parte hasta llegar a letras sueltas.

#### Ejemplo 5: raíces de palabras (inventado)

| Palabra | En enfoque de palabras | En enfoque de tokens |
|---------|------------------------|----------------------|
| jugar   | 1 entrada propia       | jug + ar             |
| jugando | 1 entrada propia       | jug + ando           |
| jugador | 1 entrada propia       | jug + ador           |

Con palabras necesitas 3 entradas independientes, y el modelo no sabe que se parecen. Con tokens reutilizas "jug", y se nota que comparten significado.

Los ejemplos 3, 4 y 5 son para visualizar la idea. Un tokenizador real puede partir distinto, y eso no lo verifiqué.

### Diferencia entre palabra y token

La confusión es lógica: en la frase de la diapositiva dan exactamente lo mismo, porque todas son palabras comunes. La diferencia solo aparece con **palabras raras o poco comunes**.

#### La diferencia en una línea:

- **Enfoque de palabras:** la unidad es siempre la palabra entera. Si la palabra no está en la lista, no existe para el modelo.
- **Enfoque de tokens:** la unidad es un trozo. A veces el trozo es una palabra entera, pero si la palabra es rara se parte **en trozos más chicos que sí están en la lista**.

#### Analogía con piezas de LEGO:

- Palabras = piezas que ya vienen armadas (una casa, un auto, un árbol). Si te piden algo que no viene armado, no puedes hacerlo.
- Tokens = piezas sueltas (ladrillos, ventanas, ruedas). Con ellas armas casi cualquier cosa.

#### Ejemplo con una palabra rara (ilustrativo, no verificado):

Supón que ambos sistemas conocen "gato", pero ninguno tiene "gatuno" como entrada propia.

| Palabra | Enfoque de palabras                      | Enfoque de tokens                |
|---------|------------------------------------------|----------------------------------|
| gato    | gato                                     | gato                             |
| gatuno  | <b>desconocida</b> (no está en la lista) | gat + uno (trozos que sí conoce) |

- Con palabras, "gatuno" se pierde y el modelo no entiende nada de esa parte.
- Con tokens, el modelo igual la procesa, y además ve el trozo "gat" que ya conoce.

**Por qué importa:** en un idioma hay cientos de miles de palabras, más nombres propios y palabras nuevas todos los días. Con palabras tendrías que elegir entre una lista gigante o dejar muchas afuera. Con tokens usas una lista de tamaño manejable (por ejemplo ~100.000) y aun así cubres casi todo, porque lo raro se arma con trozos.

**Resumen:** en palabras comunes palabra y token son lo mismo, y en palabras raras el **token se parte para no quedarse sin poder representarlas**.

### 3. Por qué el vocabulario de tokens se puede limitar

Como los tokens son trozos, puedes fijar un tamaño de vocabulario y aun así representar cualquier texto. Si una palabra no está, se parte en trozos más chicos, **y en el peor caso se llega hasta el nivel de carácter. Así no queda ninguna palabra sin poder representarse.**

Ejemplo (ilustrativo, no verificado): si "Punta Arenas" no estuviera como token, podría partirse en trozos como "Pun" + "ta" + "Aren" + "as".

Tamaños reales de vocabulario (verificados en fuentes):

| Modelo o enfoque | Vocabulario             |
|------------------|-------------------------|
| Nivel carácter   | ~100                    |
| Nivel palabra    | 170.000+ solo en inglés |
| Llama 2          | 32.000                  |
| GPT-2            | 50.257                  |
| GPT-4            | ~100.000                |
| Llama 3          | ~128.000                |
| Gemma 3          | 262.144                 |

### 4. Las raíces de palabras

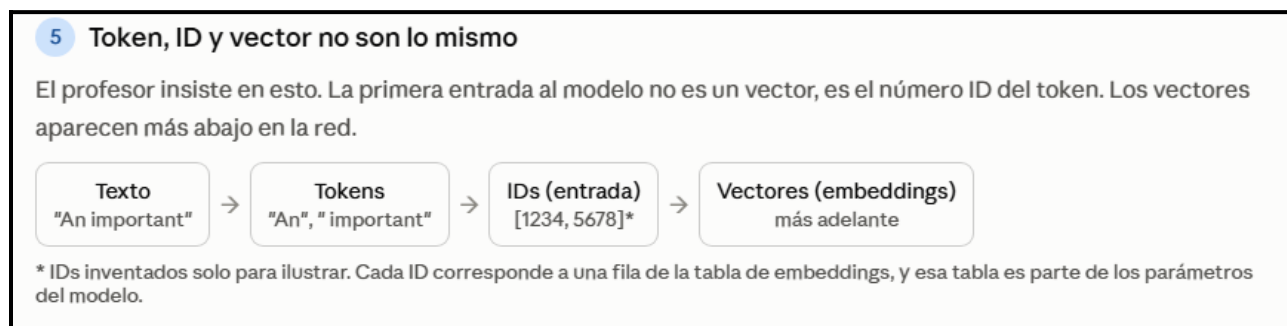
Un trozo de caracteres (la raíz) aparece seguido de distintas terminaciones, que solo modifican su significado. Con tokens, el modelo reutiliza la raíz.

Ejemplo (inventado, un tokenizador real puede partir distinto): "jugar" = jug + ar, "jugando" = jug + ando, "jugador" = jug + ador. Así no necesitas una entrada independiente por cada palabra.

### 5. Token, ID y vector no son lo mismo

La **primera entrada** al modelo es el **ID del token** (un numero que dice cual token es). Los **vectores** (embeddings) aparecen mas abajo en la red. Cada token del vocabulario tiene una fila en la tabla de embeddings, y esa tabla es parte de los parámetros del modelo.

Flujo: texto → tokens → IDs (entrada) → vectores (mas adelante).



### 6. El tokenizador de GPT (diapositiva 2)

En [platform.openai.com/tokenizer](https://platform.openai.com/tokenizer) puedes pegar texto y ver cómo se parte. La frase de la diapositiva:

**"An important sentence for my class of AI engineers"**

| Dato                 | Valor                |
|----------------------|----------------------|
| Tokens               | 9                    |
| Caracteres           | 50                   |
| Caracteres por token | $50 / 9 \approx 5,6$ |

Cuenta de caracteres:  $2 + 9 + 8 + 3 + 2 + 5 + 2 + 2 + 9 = 42$  letras, más 8 espacios = 50. A nivel carácter serían 50 pasos; con tokens son 9, casi 5,6 veces menos.

### 1.15.1 Resumen

1. Token = trozo de texto, con un vocabulario fijo.
2. Es el término medio entre caracteres (vocabulario chico, pero la red hace demasiado) y palabras (vocabulario gigante).
3. Si una palabra no está, se parte en trozos hasta llegar, si hace falta, a caracteres.
4. El espacio previo forma parte del token.
5. Lo que entra al modelo es el ID del token, no un vector.



## 1.16 DIA 4: Tokens en la practica: El tokenizador de GPT

Son tres ejemplos del tokenizador de OpenAI ([platform.openai.com/tokenizer](https://platform.openai.com/tokenizer)) y una regla empírica para estimar tokens.

### 1.16.1 Frase 1: palabras comunes

From <https://platform.openai.com/tokenizer>

# GPT's Tokenizer



For common words, 1 word maps to 1 token

Observe how the break between words is part of the token

Clear Show example

Tokens 9 Characters 50

An important sentence for my class of AI engineers

"An important sentence for my class of AI engineers" da 9 tokens y 50 caracteres.

Como todas las palabras son muy comunes, **cada palabra es un token**. Dos puntos que destaca el profesor:

- **Selector de modelo:** La interfaz deja elegir al modelo, porque cada uno puede tokenizar un poco distinto. No es importante: son decisiones tomadas en el entrenamiento y no cambian mucho. Si investigas tokens, elige el que más te interese.
- **El espacio forma parte del token.** " important" (con espacio al inicio) marca que empieza una palabra nueva. Es información que el modelo recibe junto con la palabra.

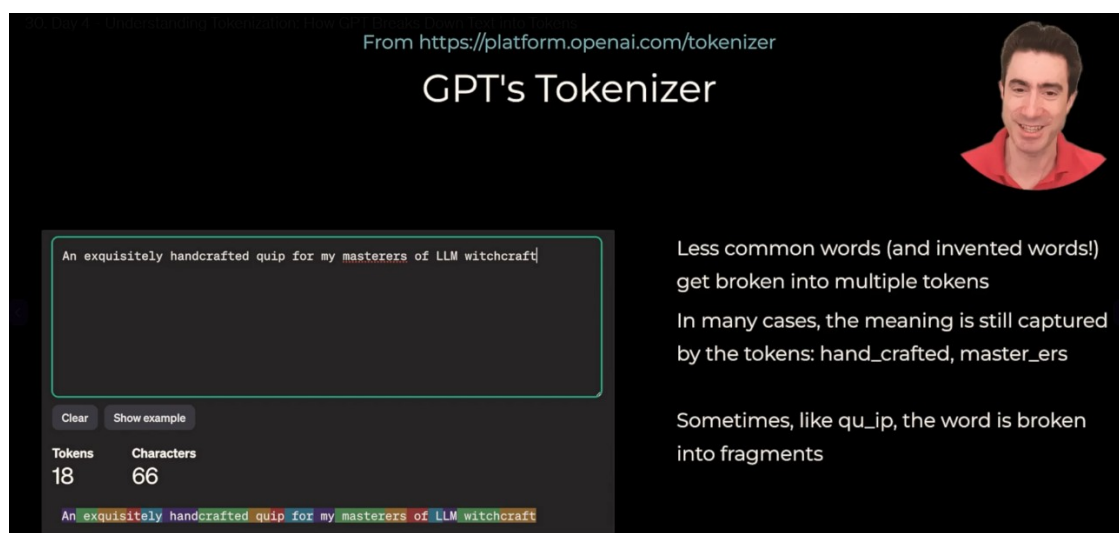
**Diferencia clave:** hay tokens que son **inicio de palabra** (con espacio delante) y tokens que son **fragmento** (mitad o final de una palabra, sin espacio).

Ejemplo del profesor: "unimportant" no contendría el token " important". Sería "un" + "important" (sin espacio), una versión distinta del token.

## 1.16.2 Frase 2: palabras poco comunes

From <https://platform.openai.com/tokenizer>

### GPT's Tokenizer



Less common words (and invented words!) get broken into multiple tokens

In many cases, the meaning is still captured by the tokens: hand\_crafted, master\_ers

Sometimes, like qu\_ip, the word is broken into fragments

Clear Show example

Tokens 18 Characters 66

An exquisitely handcrafted quip for my masterers of LLM witchcraft

"An exquisitely handcrafted quip for my masterers of LLM witchcraft" da 18 tokens y 66 caracteres.

| Palabra               | Cómo se parte                   | Tokens     |
|-----------------------|---------------------------------|------------|
| An, for, my, of       | Sin partir                      | 1 cada una |
| exquisitely           | Varios fragmentos (no es común) | 4*         |
| handcrafted           | hand + crafted                  | 2          |
| quip                  | qu + ip                         | 2          |
| masterers (inventada) | master + ers                    | 2          |
| LLM                   | L + LM                          | 2          |
| witchcraft            | witch + craft                   | 2          |

\* Lo deduje:  $18 - (4 + 2+2+2+2+2) = 4$ . El profesor solo dice que tiene "muchos fragmentos".

Por qué se parte así:

- "crafted" no lleva espacio delante: representa algo que termina en "crafted", y la red lo reconoce cada vez que lo ve.
- "master" es el verbo *to master* y "ers" es alguien que hace algo, así que el significado se conserva aunque la palabra sea inventada.
- "LLM" se partió porque cuando se creó el tokenizador no era un término común.
- No es que la red lo necesite así. Lo más probable es que **entrene más rápido y capte el significado más rápido** si las piezas ya se parecen a cómo se construyen las palabras.

Comparación de proporciones:

| Frase            | Caracteres | Tokens | Caracteres por token  |
|------------------|------------|--------|-----------------------|
| 1 (comunes)      | 50         | 9      | $50 / 9 \approx 5,6$  |
| 2 (poco comunes) | 66         | 18     | $66 / 18 \approx 3,7$ |

1.16.3 Frase 3: Los números + pregunta sobre si modelos mas capaces los tokens disminuyen

From <https://platform.openai.com/tokenizer>

### GPT's Tokenizer



Ed Donner's fave number is 3.141592653589793...|

Clear

Show example

Tokens

17

Characters

47

Ed Donner's fave number is 3.141592653589793...

See how numbers are treated - this may explain why earlier GPTs struggled with math with more than 3 digits

**Rule-of-thumb: in typical English writing:**

- 1 token is ~4 characters
- 1 token is ~0.75 words
- So 1,000 tokens is ~750 words

The collected works of Shakespeare are ~900,000 words or 1.2M tokens

Obviously the token count is higher for math, scientific terms and code

"Ed Donner's fave number is 3.141592653589793..." da 17 tokens y 47 caracteres (47 / 17 ≈ 2,8 caracteres por token).

El numero se parte en grupos de tres dígitos: 141 | 592 | 653 | 589 | 793. El profesor dice que hay una ficha para cada número posible de tres cifras, o sea unos mil tokens del vocabulario son solo números de 3 dígitos.

**Verificado en fuentes:** Un paper de 2024 confirma que el tokenizador de GPT-3.5 y GPT-4 tiene tokens para todos los números de 1, 2 y 3 dígitos y parte los números largos en grupos de 3 de izquierda a derecha. Pero no todos los modelos lo hacen igual: Llama 1 y 2 y PaLM parten dígito por dígito, y Llama 3 usa grupos de tres.

**La historia que cuenta:** Los primeros GPT resolvían bien cuentas de 3 dígitos, pero con 4 se tambaleaban y daban respuestas casi al azar. Bromeaba con que, cuando las máquinas tomaran el poder, bastaría plantearles una cuenta de 4 cifras. Hoy las resuelven bien.

**Su explicación:** un número de 4 dígitos ya no es un solo token, sino dos tokens combinados, y eso lo hacía más difícil de aprender. El mismo paper respalda que la forma de tokenizar los números influye en la aritmética, aunque no digo que sea la única causa.

| a día de hoy, como son los tokens para los numeros?                                                                                                                                       |                                                                                                                      |                           |        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------|--------|
| No hay un estándar único: <b>depende del tokenizador de cada modelo</b> . Hay dos estrategias, y las dos se usan hoy.                                                                     |                                                                                                                      |                           |        |
| Estrategia                                                                                                                                                                                | Modelos (según fuentes)                                                                                              | Ejemplo con 1234567       | Tokens |
| Grupos de hasta 3 dígitos, de izquierda a derecha                                                                                                                                         | GPT-3.5 y GPT-4, Llama 3 en adelante. Según un patrón técnico del tokenizador, también GPT-4o/GPT-5 y DeepSeek V3/R1 | 123   456   7             | 3      |
| Dígito por dígito                                                                                                                                                                         | Llama 1 y 2, Gemma, Mistral, Qwen 2                                                                                  | 1   2   3   4   5   6   7 | 7      |
| <b>Ventajas y desventajas:</b> <ul style="list-style-type: none"><li>- <b>Grupos de 3:</b> secuencias mas cortas. Un numero de 12 dígitos son 4 tokens. Pero los limites de los</li></ul> |                                                                                                                      |                           |        |

grupos dependen de cuantos dígitos tiene el número. En 1234567 el ultimo grupo queda con un solo digito, y eso desalinea las unidades, decenas y centenas al sumar.

- **Dígito por dígito:** más limpio para la matemática, porque cada dígito es un token con su posición. Pero ese mismo número de 12 dígitos son 12 tokens, hasta unas 3 veces más tokens en textos con muchos números.

**Con el número de la diapositiva** (3.141592653589793): en grupos de 3 queda "3", ".", "141", "592", "653", "589", "793". Dígito por dígito serían 16 tokens solo para el número.

**Lo que se está investigando:** partir los grupos de derecha a izquierda, para que queden alineados por unidades, podría mejorar la aritmética. Un estudio con Gemma y Mistral (dígito a dígito) y Llama 3 (grupos de 3) también muestra que la forma de tokenizar cambia cómo los modelos arman el resultado de una suma.

para los tokens para los numeros, cual es mejor para hacer calculos matematicos, por dígito o grupos de 3?

Para cálculos, **por dígito** es el mejor según la evidencia, aunque con una condición.

**Lo que muestran los estudios:**

- Un experimento de Hugging Face entrenemo modelos iguales cambiando solo como se tokenizan los números. El resultado: dígito por dígito rindió mejor en todas las operaciones.
- Entre los grupos de 3, **de derecha a izquierda** empate o supero a izquierda a derecha en todas áreas. Pero la diferencia fue menor que la que hay entre dígito por dígito y el resto.
- Otro estudio con GPT-3.5 y GPT-4 vio que usar comas para forzar el grupo de derecha a izquierda ("1,234,567") mejoró mucho la aritmética.

**Por que pasa:** al sumar a mano alineas unidades con unidades. Con grupos de 3 de izquierda a derecha, eso se desalinea.

Ejemplo con 1234567 (ilustrativo):

| Estrategia                       | Partición                 | ¿Alinea unidades?                   |
|----------------------------------|---------------------------|-------------------------------------|
| Por dígito                       | 1   2   3   4   5   6   7 | Sí, cada dígito tiene su lugar      |
| Grupos de 3, izquierda a derecha | 123   456   7             | No, el "7" queda solo y desalineado |
| Grupos de 3, derecha a izquierda | 1   234   567             | Sí, 567 son las unidades            |

**La condición:** el dígito por dígito tiene un costo. Un número de 12 dígitos son 12 tokens y no 4, así que las secuencias son más largas y el texto con muchos números sale más caro.

**Matrices:**

- Un estudio de 2025 encontró que, aun con distintas tokenizaciones, los modelos suman con un "vistazo" de un solo dígito hacia adelante, y fallan donde los acarreos se encadenan. La tokenización ayuda, pero no lo arregla todo.
- En el estudio con GPT, la diferencia entre direcciones **disminuye cuando el modelo es más grande**, como si los modelos grandes compensaran el mal alineado.

**En resumen:** para matemática pura, dígito por dígito. Si usas un modelo con grupos de 3, escribir los números con comas puede ayudar.

duda, a medida que salen nuevos modelos, las palabras en su conversión en token van disminuyendo o aumentando? cual es la tendencia?

**La tendencia es que disminuyen.** Los tokenizadores nuevos tienen vocabularios mas grandes, así que el mismo texto se parte en menos tokens.

| Modelo                 | Vocabulario aproximado       |
|------------------------|------------------------------|
| Llama 2                | 32.000                       |
| GPT-2                  | 50.257                       |
| GPT-4 (cl100k)         | ~100.000                     |
| Llama 3 y 4            | ~128.000                     |
| GPT-4o y GPT-5 (o200k) | ~200.000                     |
| Gemma                  | 256.000 (262.144 en Gemma 3) |

Entre Llama 2 y Gemma 3 el vocabulario creció unas 8 veces ( $262.144 / 32.000 \approx 8,2$ ).

**Por qué baja la cantidad de tokens:** con mas entradas en la lista, mas palabras y trozos comunes son un solo token y se parten menos. Una fuente indica que el vocabulario de GPT-4 codifica mas información por token que el de Llama 2.

**Dónde se nota más:** en otros idiomas y en código. Según una fuente, el tokenizador o200k redujo unos 30% los tokens en texto no inglés. Ejemplo ilustrativo: si un texto en chino daba 300 tokens, bajaría a unos  $300 \times 0,7 = 210$ . En inglés la mejora debería ser menor, pero esto último es mi suposición y no lo verifiqué.

**El costo de un vocabulario grande:** la tabla de embeddings crece, y eso suma parámetros. En Gemma 3 270M, unos 170 M de los 270 M son embeddings, o sea  $\approx 63\%$  del modelo ( $170 / 270$ ).

#### Dos matices:

- Cada modelo tokeniza distinto, así que el mismo texto puede costar distinto tokens según el modelo. Si vas a estimar costos, usa el tokenizador del modelo que usarás.
- Que el texto se convierta en menos tokens no significa que uses menos tokens en total. Los modelos de razonamiento generan muchos tokens antes de responder, como vimos en la transcripción anterior.

Una fuente de 2026 dice que Llama 4 y DeepSeek V4 se estandarizaron en vocabularios de 128K a 200K.

#### 1.16.4 Regla empírica (ingles común) – cuanto esta valorizado el token

| Equivalencia                          | Valor                                             |
|---------------------------------------|---------------------------------------------------|
| <b>1 token</b>                        | $\approx 4$ caracteres                            |
| <b>1 token</b>                        | $\approx 0,75$ palabras                           |
| <b>1.000 tokens</b>                   | $\approx 750$ palabras                            |
| <b>Obras completas de Shakespeare</b> | $\sim 900.000$ palabras $\approx 1,2$ M de tokens |

##### Cuentas rápidas:

- Shakespeare:  $900.000 / 0,75 = 1.200.000$  tokens ✓.
- Un texto de 3.000 palabras:  $3.000 / 0,75 = \mathbf{4.000}$  tokens.
- Un texto de 2.000 caracteres:  $2.000 / 4 = \mathbf{500}$  tokens.

**Por qué importa:** los precios se cotizan **por millón de tokens**. Las obras de Shakespeare son un poco más de 1 millón de tokens, una referencia para imaginarse cuánto es. Si un precio fuera \$1 por millón de tokens (inventado), leerlo entero costaría unos \$1,2.

Es una regla para **estimar cuantos tokens tiene un texto** sin tokenizarlo (vale para ingles común).

- **1 token  $\approx 4$  caracteres.** Un texto de 2.000 caracteres son unos  $2000 / 4 = \mathbf{500}$  tokens
- **1 token  $\approx 0,75$  palabras.** Un texto de 3.000 palabras son unos  $3.000 / 0,75 = \mathbf{4.000}$  tokens
- **Shakespeare:** sus obras tienen  $\sim 900.000$  palabras, o sea  $900.000 / 0,75 = \mathbf{1,2}$  millones de tokens.

**Para que sirve:** Los modelos cobran por millón de tokens. Con la regla calculas cuanto costaría tu texto.

Ejemplo con un precio inventado de \$1 por millón: leer todo Shakespeare costaría unos \$1,2.

1 por millon es 1\$ por token? super breve

No. Es **\$1 por cada millón de tokens**, o sea \$0,000001 por token ( $1 / 1.000.000$ ).

a día de hoy cuanto esta valorizado el token?

El token no tiene un precio único: se cobra **por millón de tokens**, y la salida cuesta mas que la entrada (unas 5 veces). Estos son precios de lista en dólares, de fuentes de septiembre y octubre de 2026:

| Modelo                | Entrada (por 1 M) | Salida (por 1 M) |
|-----------------------|-------------------|------------------|
| Claude Fable 5        | \$10              | \$50             |
| Claude Opus 5.5       | \$4               | \$20             |
| Claude Sonnet 5 / 5.5 | \$2               | \$10             |
| Claude Haiku 4.5      | \$1               | \$5              |
| Gemini 3.1 Pro        | \$2               | \$12             |
| Gemini 3.5 Flash-Lite | \$0,30            | \$2,50           |
| GPT-5.6 Luna          | \$0,20            | \$1,20           |

| Modelo                       | Entrada (por 1 M) | Salida ( por 1 M) |
|------------------------------|-------------------|-------------------|
| <b>Claude Fable 5</b>        | \$10              | \$50              |
| <b>Claude Opus 5.5</b>       | \$4               | \$20              |
| <b>Claude Sonnet 5 / 5.5</b> | \$2               | \$10              |
| <b>Claude Haiku 4.5</b>      | \$1               | \$5               |
| <b>Gemini 3.1 Pro</b>        | \$2               | \$12              |
| <b>Gemini 3.5 Flash-Lite</b> | \$0,30            | \$2,50            |
| <b>GPT-5.6 Luna</b>          | \$0,20            | \$1,20            |

un ejemplo con ese de un input a un output

Ejemplo con Claude Opus 5.5: **\$4 por millón de entrada y \$20 por millón de salida.**

**La consulta (números inventados):**

- Tu envías un texto de 750 palabras. Con la regla de 0,75 palabras por token son  $750 / 0,75 = \mathbf{1.000}$  **tokens de entrada.**
- El modelo responde con 300 palabras:  $300 / 0,75 = \mathbf{400}$  **tokens de salida.**

**Cuenta paso a paso:**

| Parte        | Tokens       | Cuenta                       | Costo          |
|--------------|--------------|------------------------------|----------------|
| Entrada      | 1.000        | $1.000 \times 4 / 1.000.000$ | \$0,004        |
| Salida       | 400          | $400 \times 20 / 1.000.000$  | \$0,008        |
| <b>Total</b> | <b>1.400</b> |                              | <b>\$0,012</b> |

| Parte          | Tokens       | Cuenta                       | Costo          |
|----------------|--------------|------------------------------|----------------|
| <b>Entrada</b> | <b>1.000</b> | $1.000 \times 4 / 1.000.000$ | <b>\$0,004</b> |
| <b>Salida</b>  | <b>400</b>   | $400 \times 20 / 1.000.000$  | <b>\$0,008</b> |
| <b>Total</b>   | <b>1.400</b> |                              | <b>\$0,012</b> |

**Dos cosas que se ven:**

- La salida es solo el 29% de los tokens ( $400 / 1.400$ ), pero cuesta el 67% del total ( $0,008 / 0,012$ ).
- Si hicieras 1.000 consultas así, serían  $1.000 \times \$0,012 = \mathbf{\$12}$ .

### 1.16.5 ¿Cuándo la regla no aplica + por que escribir codigo ocupa mas tokens?

- La regla es para **ingles común**.
- Con **matemática, términos científicos o código**, el uso de tokens subo mucho: nombres de variables oscuros y código denso pueden acercarse a 1 token por carácter, aunque siempre algo mejor que eso.
- Con otros idiomas también cambia: según una fuente, en el tokenizador de GPT-4, 100 palabras en inglés son ~130 tokens y el mismo texto en chino ~300. No verifiqué el caso del español.

El profesor recomienda probar código en el tokenizador para tener una buena idea de cuántos tokens ocupa cada tipo de texto.

#### por que escribir codigo ocupa mas tokens? explica dando un ejemplo

Porque el código está lleno de cosas que el vocabulario no tiene como un solo token: nombres abreviados o inventados, símbolos y números. El profesor lo dice así: los nombres de variables pueden ser oscuros y el código denso, así que se acerca a **1 token por carácter**, aunque siempre un poco mejor que eso.

**Ejemplo (partición inventada para visualizar, no es la salida real de un tokenizador):**

|                      | Texto en inglés                 | Código                                                          |
|----------------------|---------------------------------|-----------------------------------------------------------------|
| Contenido            | Calculate the total price       | t1l_prc=calc(o["it"])*1.19                                      |
| Caracteres           | 25                              | 26                                                              |
| Tokens               | 4: Calculate, the, total, price | ~17: tt, l, _, pr, c, =, calc, (, o, [", it, "], ), *, 1, ., 19 |
| Caracteres por token | $25 / 4 \approx 6$              | $26 / 17 \approx 1,5$                                           |

El texto y el código tienen casi la misma cantidad de caracteres, pero el código usa unas 4 veces más tokens ( $17 / 4 \approx 4,25$ ).

**Por qué pasa:**

- **Nombres abreviados:** "t1l\_prc" no es una palabra común, así que se parte en fragmentos, igual que "quip" se partía en "qu" + "ip" en la diapositiva.
- **Símbolos:** ( ) [ ] " = \* suelen ser tokens propios, uno por símbolo.
- **Números:** "1.19" se parte en "1", "." y "19", como el número pi de la otra diapositiva.

**Qué hacer:** si quieres saber cuántos tokens tiene tu código, pégalo en el tokenizador de OpenAI. La regla de 4 caracteres por token no sirve para código.



1.16.6 Resumen

| Idea                        | En una línea                                           |
|-----------------------------|--------------------------------------------------------|
| Palabras comunes            | 1 palabra = 1 token                                    |
| Espacio                     | Va dentro del token (marca inicio de palabra)          |
| Palabras raras o inventadas | Se parten en fragmentos con significado                |
| Números                     | Grupos de 3 dígitos (en GPT-3.5/4)                     |
| Regla empírica              | 1 token $\approx$ 4 caracteres $\approx$ 0,75 palabras |
| Texto técnico o código      | Más tokens por texto                                   |

| Idea                               | En una línea                                           |
|------------------------------------|--------------------------------------------------------|
| Palabras comunes                   | 1 palabra = 1 token                                    |
| <b>Espacio</b>                     | Va dentro del token (marca inicio de palabra)          |
| <b>Palabras raras o inventadas</b> | Se parten en fragmentos con significado                |
| <b>Números</b>                     | Grupos de 3 digitos (en GPT-3.5/4)                     |
| <b>Regla empírica</b>              | 1 token $\approx$ 4 caracteres $\approx$ 0,75 palabras |
| <b>Texto técnico o código</b>      | Más tokens por texto                                   |

## 1.17 DIA4: La Ilusión de memoria

```
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv(override=True)

openai = OpenAI(
    api_key=os.getenv("GOOGLE_API_KEY"),
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)

messages = [
    {"role": "system", "content": "You are a helpful assistant"},
    {"role": "user", "content": "Hi! I'm Ed!"}
]

response = openai.chat.completions.create(
    model="gemini-3.5-flash", # Usa el modelo vigente
    messages=messages
)
response.choices[0].message.content
```

[16] ✓ 2.1s

... "Hi Ed! It's great to meet you. How can I help you today?"

OK let's now ask a follow-up question

```
messages = [
    {"role": "system", "content": "You are a helpful assistant"},
    {"role": "user", "content": "What's my name?"}
]
```

[18] ✓ 0.0s

```
> v
response = openai.chat.completions.create(
    ... model="gemini-3.5-flash", # Usa el modelo vigente
    ... messages=messages
)
response.choices[0].message.content
```

[19] ✓ 4.4s

... "I don't know your name yet! Since I don't have access to your personal information, you'll have to tell me. \n\nWhat should I call you?"

## La ilusión de memoria

### 1. Qué muestra la imagen

Una lista llamada messages con 4 mensajes, y la respuesta del modelo: *"Your name is Ed! How can I help you today, Ed?"*

| Rol       | Contenido                          | Quién lo escribe                                                  |
|-----------|------------------------------------|-------------------------------------------------------------------|
| system    | You are a helpful assistant        | Tú, como instrucción general                                      |
| user      | Hi! I'm Ed!                        | Tú                                                                |
| assistant | Hi Ed! How can I assist you today? | La respuesta anterior del modelo, que <b>tú copias a la lista</b> |
| user      | What's my name?                    | Tú                                                                |

```
messages = [  
  {"role": "system", "content": "You are a helpful assistant"},  
  {"role": "user", "content": "Hi! I'm Ed!"},  
  {"role": "assistant", "content": "Hi Ed! How can I assist you today?"},  
  {"role": "user", "content": "What's my name?"}  
]
```

### 2. El problema: el modelo no recuerda nada

Cada llamada al modelo es **sin estado (stateless)**: empieza de cero, como si nunca te hubiera visto.

| Llamada           | Qué le mandas               | Respuesta          |
|-------------------|-----------------------------|--------------------|
| 1                 | "Hi! I'm Ed!"               | "Hi Ed!..."        |
| 2 (sin historial) | Solo "What's my name?"      | "No sé tu nombre"  |
| 2 (con historial) | Los 4 mensajes de la imagen | "Your name is Ed!" |

```
response = openai.chat.completions.create(  
    model="gemini-3.5-flash", # Usa el modelo vigente  
    messages=messages  
)  
response.choices[0].message.content
```

... "Your name is Ed! How can I help you today?"

La segunda llamada sin historial es el error que muestra el profesor: ya dijiste tu nombre, pero esa llamada no lo sabe.

### 3. La solución: reenviar toda la conversación

En cada llamada mandas **la conversación completa hasta ahora**, no solo el ultimo mensaje. Eso es lo que hace la lista de la imagen.

## 4. Por qué funciona

El modelo solo predice los siguientes tokens de la secuencia que recibe. Si la secuencia contiene "I'm Ed" y después "What's my name?", lo más probable que siga es "Ed". Si no contiene tu nombre, lo más probable es "no sé".

## 5. El costo

Como reenvías todo, **pagas de nuevo por toda la conversación en cada mensaje**. El profesor dice que es lo que queremos: el modelo calcula mirando todo el contexto, y ese cálculo cuesta.

### 1.17.1 ¿Qué es el rol assistant?

Son los **mensajes anteriores del modelo**. Hay tres roles:

| Rol       | Que es                                         | Quien lo escribe                                   |
|-----------|------------------------------------------------|----------------------------------------------------|
| System    | Intrduccion general ("eres un asistente útil") | Tu, el programador                                 |
| User      | Lo que dice la persona                         | El usuario                                         |
| Assistant | Lo que el modelo respondió antes               | El modelo, pero <b>tu lo guardas y lo reenvías</b> |

El modelo no guarda nada entre llamadas, así que para que "sepa" lo que dijo, tu pegas su respuesta anterior en la lista con el rol **assistant**:

```
python

respuesta = response.choices[0].message.content
messages.append({"role": "assistant", "content": respuesta})
messages.append({"role": "user", "content": "What's my name?"})
```

Sin ese mensaje, el modelo vería una conversación donde solo habla el usuario.

1.17.2 El costo de tokens de system, user y model, ejemplo detallado – el costo de entrada crece al cuadrado con historial, crece lineal sin historial

Supuestos (números inventados):  
Sistema = 10 tokens, cada mensaje tuyo = 20, cada respuesta del modelo = 40.  
Precios de Cloude Opus 5.5: \$4 por millón de entrada y \$20 por millón de salida.

Qué contiene la entrada en cada turno:

| Turno | Lo que envías                 | Cuenta            | Tokens de entrada |
|-------|-------------------------------|-------------------|-------------------|
| 1     | sistema + tú1                 | 10 + 20           | 30                |
| 2     | sistema + tú1 + modelo1 + tú2 | 10 + 20 + 40 + 20 | 90                |
| 3     | lo anterior + modelo2 + tú3   | 90 + 40 + 20      | 150               |
| 4     | lo anterior + modelo3 + tú4   | 150 + 40 + 20     | 210               |

| Turno | Lo que envías                 | Cuenta            | Tokens de entrada |
|-------|-------------------------------|-------------------|-------------------|
| 1     | sistema + tú1                 | 10 + 20           | 30                |
| 2     | sistema + tú1 + modelo1 + tú2 | 10 + 20 + 40 + 20 | 90                |
| 3     | lo anterior + modelo2 + tú3   | 90 + 40 + 20      | 150               |
| 4     | lo anterior + modelo3 + tú4   | 150 + 40 + 20     | 210               |

Cada turno repite todo lo anterior y suma lo nuevo.

Costo de cada turno:

| Turno | Entrada (tokens × 4 / 1.000.000) | Salida (40 × 20 / 1.000.000) | Total     |
|-------|----------------------------------|------------------------------|-----------|
| 1     | 30 → \$0,00012                   | \$0,0008                     | \$0,00092 |
| 2     | 90 → \$0,00036                   | \$0,0008                     | \$0,00116 |
| 3     | 150 → \$0,00060                  | \$0,0008                     | \$0,00140 |
| 4     | 210 → \$0,00084                  | \$0,0008                     | \$0,00164 |
| Suma  | 480 → \$0,00192                  | \$0,0032                     | \$0,00512 |

| Turno | Entrada (tokens × 4 / 1.000.000) | Salida (40 × 20 / 1.000.000) | Total     |
|-------|----------------------------------|------------------------------|-----------|
| 1     | 30 → \$0,00012                   | \$0,0008                     | \$0,00092 |
| 2     | 90 → \$0,00036                   | \$0,0008                     | \$0,00116 |
| 3     | 150 → \$0,00060                  | \$0,0008                     | \$0,00140 |
| 4     | 210 → \$0,00084                  | \$0,0008                     | \$0,00164 |
| Suma  | 480 → \$0,00192                  | \$0,0032                     | \$0,00512 |

La salida cuesta lo mismo en cada turno (\$0,0008). Lo que crece es la entrada.

**Comparación sin historial** (solo el último mensaje, 30 tokens cada vez):

- Entrada:  $4 \times 30 = 120$  tokens  $\rightarrow$  \$0,00048.
- Con historial son 480 tokens, **4 veces mas** en entrada.
- Pero sin historial el modelo no sabría tu nombre.

### 3. Por qué conviene saberlo

La entrada total para N mensajes sale como  $30 \times N^2$ , con estos números inventados:

| Mensajes  | Entrada con historial    | Sin historial ( $30 \times N$ ) | Cuántas veces mas |
|-----------|--------------------------|---------------------------------|-------------------|
| <b>4</b>  | $30 \times 16 = 480$     | 120                             | 4                 |
| <b>10</b> | $30 \times 100 = 3.000$  | 300                             | 10                |
| <b>20</b> | $30 \times 400 = 12.000$ | 600                             | 20                |

El costo de entrada **crece con el cuadrado** del numero de mensajes: si duplicas la conversación, la entrada se cuadriplica. Por eso las conversaciones largas se encarecen rápido.

Una fuente de precios de 2026 indica que los proveedores cobran bastante menos por la parte repetida (caché): \$0,20 por millón en vez de \$4 para Opus 5.5. No verifiqué cómo funciona en cada proveedor, pero es la forma habitual de abaratar esto.

#### 1.17.3 Resumen – los 5 puntos del resumen

1. Cada llamada es sin estado.
2. Se pasa la conversación completa cada vez.
3. Eso da la ilusión de memoria.
4. Es un truco, un efecto de mandar todo el historial.
5. El modelo solo predice los siguientes tokens de la secuencia.

1. Cada llamada es sin estado
2. Se pasa la conversión completa cada vez
3. Eso da la ilusión de memoria
4. Es un truco, un efecto de mandar todo el historial
5. El modelo solo predice los siguientes tokens de la secuencia

## 1.18 DIA4: Ventaja de contexto y costos de API

### 1.18.1 Ventaja de contexto (diapo 1) - Por qué la salida también ocupa ventana – para que importa la ventaja

### Context Window

Max number of tokens that the model can consider when generating the next token

Includes the original input prompt, subsequent conversation, the latest input prompt and almost all the output prompt

It governs how well the model can remember references, content and context

**Particularly important for multi-shot prompting where the prompt includes examples, or for long conversations**

**Or questions on the complete works of Shakespeare!**



Es el **máximo de tokens** que el modelo puede considerar al general el siguiente token. Si intentas meter más, falla (el error dice algo como “mas grande que mi ventaja de contexto”).

**Qué ocupa espacio en la ventana (no es solo tu último mensaje):**

| Parte                                 | Ejemplo                                     |
|---------------------------------------|---------------------------------------------|
| Prompt original (system)              | "Eres un asistente útil"                    |
| La conversación anterior              | "Hola, soy Ed" + "Hola Ed, ¿cómo te ayudo?" |
| Tu último mensaje                     | "¿Cómo me llamo?"                           |
| Los tokens que el modelo va generando | "Tu", "nombre", "es", "Ed"                  |

La diapositiva dice "casi toda la salida" y el profesor aclara que incluye hasta el último token generado.

## 2. Por qué la salida también ocupa ventana

El modelo genera **un token a la vez**, y cada token nuevo se agrega a la entrada antes de generar el siguiente:

| Paso | Entrada que recibe                           | Genera   |
|------|----------------------------------------------|----------|
| 1    | Conversación completa (supongamos 40 tokens) | "Tu"     |
| 2    | 40 + "Tu" = 41                               | "nombre" |
| 3    | 42                                           | "es"     |
| 4    | 43                                           | "Ed"     |

Al final la secuencia tiene 44 tokens, y todo eso debe caber.

El profesor simplifica: en la practica los modelos reutilizan cálculos (KV cache) para no repetir todo en cada paso. Es conocimiento mío, no del video.

Ejemplo de falla (ventana inventada de 8.000 tokens):

- Historial: 7.500 + tu pregunta: 200 = 7.700 de entrada.
- Si el modelo quiere responder con 500 tokens: 7.700 + 500 = 8.200 > 8.000. **No cabe.**

ejemepo reutilizando lo de cacke como seria?

Idea: El modelo guarda los cálculos de cada token (sus “keys y values”, dos vectores) y, al generar el siguiente, solo calcula el token nuevo y reutiliza lo guardado.

Con el ejemplo de "Tu nombre es Ed"

Entrada de 40 tokens (número inventado). El modelo genera 4 tokens.

| Paso  | Genera   | Sin caché (procesa todo) | Con caché                                     |
|-------|----------|--------------------------|-----------------------------------------------|
| 1     | "Tu"     | 40 tokens                | 40 (llenado inicial, llamado <i>prefill</i> ) |
| 2     | "nombre" | 41                       | 1 ("Tu")                                      |
| 3     | "es"     | 42                       | 1 ("nombre")                                  |
| 4     | "Ed"     | 43                       | 1 ("es")                                      |
| Total |          | 166                      | 43                                            |

| Paso  | Genera   | Sin cache (procesa todo) | Con cache                                     |
|-------|----------|--------------------------|-----------------------------------------------|
| 1     | "Tu"     | 40 tokens                | 40 (llenado inicial, llamado <i>prefill</i> ) |
| 2     | "nombre" | 41                       | 1 ("Tu")                                      |
| 3     | "es"     | 42                       | 1 ("nombre")                                  |
| 4     | "Ed"     | 43                       | 1 ("es")                                      |
| Total |          | 166                      | 43                                            |

Cuentas: sin caché 40 + 41 + 42 + 43 = 166; con caché 40 + 1 + 1 + 1 = 43. Son unas 3,9 veces menos trabajo (166 / 43).

Que se guarda: Las keys y values de cada token, en capa capa del modelo. No se guardan las queries.

Cada token genera 3 cosas:

| Nombre           | Qué es                       | Analogía                                  |
|------------------|------------------------------|-------------------------------------------|
| Query (consulta) | Lo que el token quiere saber | La pregunta que le haces al bibliotecario |
| Key (clave)      | Lo que el token ofrece       | La etiqueta del lomo del libro            |
| Value (valor)    | El contenido del token       | El contenido del libro                    |

Cómo se usan:

- 1.El token nuevo hace su pregunta (query)
2. Se compara con las etiquetas (keys) de todos los tokens anteriores.
3. Donde la etiqueta calza, toma el contenido (value)



**Ejemplo (simplificado):** el contexto es "Mi nombre es Ed. ¿Cómo me llamo?".

- El token nuevo pregunta: "¿Qué se dijo antes sobre el nombre?"
- La etiqueta de "Ed" calza con esa pregunta
- Toma el contenido de "Ed", y así el modelo puede responder "Ed".

**Por qué se guardan keys y values y no queries:**

- Las **keys y values** de los tokens viejos nunca cambian, y cada token nuevo las va a necesitar. Se guardan para no recalcularlas.
- Las **queries** viejas ya cumplieron su trabajo: esos tokens ya hicieron su pregunta. Solo el token nuevo necesita preguntar, así que no hace falta guardarlas.

**En una línea:** El modelo guarda las etiquetas y el contenido de lo que ya leyó, y en cada paso solo calculo lo del token nuevo.

### Lo que NO cambia

- **La ventana de contexto sigue igual:** al final hay 44 tokens y todos deben caber. El ejemplo de falla ( $8.200 > 8.000$ ) sigue siendo falla.
- **La cache ocupa memoria:** Crece con cada token, y en conversaciones largas puede pesar mas que los propios pesos del modelo. Por eso antes dije que los 16GB de pesos no incluyen el contexto.

### Relación con la "cache" del precio

El profesor menciona que pagas menos si repites la misma entrada. Esa es una versión de lo mismo: el proveedor reutiliza lo calculado para el inicio repetido del prompt, como el system prompt o el historial.

Ejemplo con precios de antes: Opus 5.5 cobra \$0,20 por millón de entrada en caché contra \$4 normal. Mi lectura de cómo se conectan es una inferencia mía; no verifiqué cómo lo implementa cada proveedor.

### Aclaración sobre la analogía del profesor

Él dice que el modelo "recibe todo de nuevo" en cada paso. Conceptualmente es correcto, porque cada token se mira contra todos los anteriores, pero en la práctica los cálculos viejos se reutilizan.

## 3. Para qué importa la ventana

- **Multi-shot prompting:** poner varios ejemplos de pregunta y respuesta en el prompt.
- **Conversaciones larga:** cada mensaje se reenvía completo (lo vimos con la ilusión de memoria)
- **RAG:** meter documentos en el prompt, como los precios de pasajes de todo el mundo.
- **Preguntas sobre un libro completo.** El profesor menciona que quienes usan Claude Code notan qué pasa cuando se llena la ventana.

## 4. El caso Shakespeare

Sus obras son ~1,2 M de tokens. Cuantas veces cabria en cada ventana:

| Ventana                      | Cuenta          | Resultado                               |
|------------------------------|-----------------|-----------------------------------------|
| 200.000 (Claude)             | 1,2 M / 200.000 | 6 veces más grande, no cabe             |
| 400.000 (GPT-5)              | 1,2 M / 400.000 | 3 veces más grande, no cabe             |
| 1.000.000 (Gemini 2.5 Flash) | 1,2 M / 1 M     | 1,2 veces, <b>tampoco cabe completo</b> |

| Ventana                             | Cuenta          | Resultado                               |
|-------------------------------------|-----------------|-----------------------------------------|
| <b>200.000 (Claude)</b>             | 1,2 M / 200.000 | 6 veces más grande, no cabe             |
| <b>400.000 (GPT-5)</b>              | 1,2 M / 400.000 | 3 veces más grande, no cabe             |
| <b>1.000.000 (Gemini 2.5 Flash)</b> | 1,2 M / 1 M     | 1,2 veces, <b>tampoco cabe completo</b> |

Por eso el profesor dice "casi": con 1 M faltan unos 200.000 tokens. Corrijo la frase "necesitarías un millón": serían unos 1,2 M.

1.18.2 Costos de API (diapositiva 2)

### API costs



Chat interfaces typically have Pro a monthly subscription.  
Rate limited, but no per-usage charge.  
APIs typically have no subscription, but charge per API call  
The cost is based on the number of input tokens and the number of output tokens

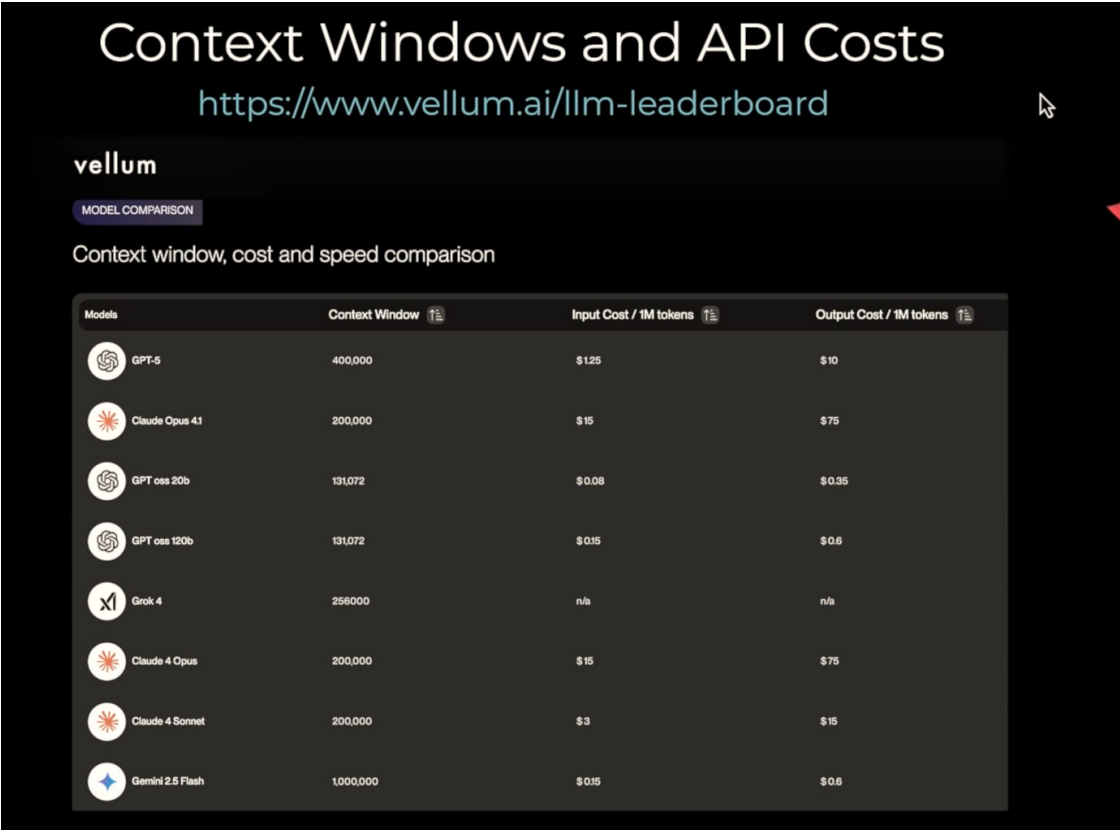
|          | Chat (ChatGPT, etc.)                                 | API                              |
|----------|------------------------------------------------------|----------------------------------|
| Pago     | Suscripción mensual (de \$20 a \$200 según el nivel) | Sin suscripción, pagas por uso   |
| Límite   | Limitado por tasa (rate limit)                       | Sin tope fijo, pagas lo que uses |
| Para qué | Usar el producto                                     | Construir tu propio producto     |

**El costo depende de:** tokens de entrada + tokens de salida. Se cobra porque cada llamada gasta cómputo (electricidad), y el profesor dice que parte del precio ayuda a recuperar los más de \$100 millones que costó entrenar el modelo.

Dos trampas:

1. **La entrada incluye todo:** el historial completo, la memoria falsa y lo que metas con RAG. El profesor dice que es justo, porque ese cálculo es lo que hace que el modelo responda bien. Podrías mandar solo el último mensaje, pero los resultados serían malos.
2. **La salida incluye el razonamiento.** En los modelos de razonamiento, esos tokens se pagan aunque no lo veas (en GPT ocurre “entre bastidores”). El costo se vuelve menos predecible.

1.18.3 La tabla de Vellum (diapositiva 3)



| Modelo           | Ventana   | Entrada (por 1 M) | Salida (por 1 M) |
|------------------|-----------|-------------------|------------------|
| GPT-5            | 400.000   | \$1,25            | \$10             |
| Claude Opus 4.1  | 200.000   | \$15              | \$75             |
| GPT-OSS 20B      | 131.072   | \$0,08            | \$0,35           |
| GPT-OSS 120B     | 131.072   | \$0,15            | \$0,60           |
| Grok 4           | 256.000   | n/a               | n/a              |
| Claude 4 Opus    | 200.000   | \$15              | \$75             |
| Claude 4 Sonnet  | 200.000   | \$3               | \$15             |
| Gemini 2.5 Flash | 1.000.000 | \$0,15            | \$0,60           |

Hoy (octubre de 2026), según fuentes que consulté:

| Modelo         | Ventana   | Entrada | Salida |
|----------------|-----------|---------|--------|
| Claude Opus 5  | 1.000.000 | \$5     | \$25   |
| GPT-5.6 Sol    | 1.050.000 | \$5     | \$30   |
| Gemini 3.1 Pro | 1.000.000 | \$2     | \$12   |

#### 1.18.4 Las cuentas del profesor + Cache

- **"10 dólares por millón de tokens de salida"** (GPT-5): generar todo Shakespeare, 1,2 M de tokens, cuesta  $1,2 \times \$10 = \$12$ . Él dice "unos 10 dólares", aproximado.
- **GPT-5 nano** (\$0,05 de entrada y \$0,40 de salida por millón): Shakespeare en salida cuesta  $1,2 \times 0,40 = \$0,48$ , menos de un dólar, como dice.
- **Un token de entrada en nano:**  $0,05 / 1.000.000 = \$0,00000005$ . Una cifra demasiado pequeña para decirla, como bromea.
- **"Hola, me llamo Ed"** (~10 tokens) con GPT-5:  $10 \times 1,25 / 1.000.000 = \$0,0000125$ . Por eso dice que las llamadas individuales son baratísimas y que lo que se encarece es construir sistemas grandes o bucles de agentes que consumen muchos tokens.

#### 8. Caché

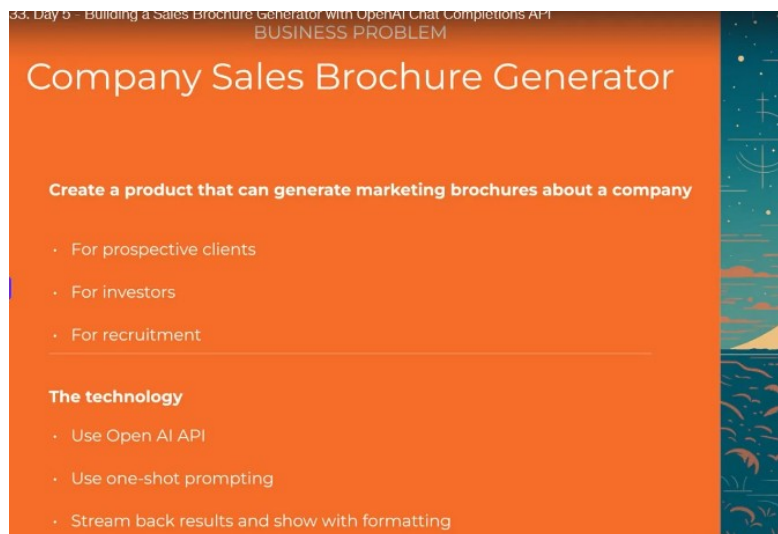
Si mandas la misma entrada dos veces en pocos minutos, pagas menos. En GPT es automático, y con Claude depende (hay que activarlo). Ejemplo con precios de 2026 que vi antes: Opus 5.5 cobra \$0,20 por millón de entrada en caché contra \$4 normal, o sea  $0,20 / 4 = 5\%$  del precio. Lo verás con más detalle más adelante, según dijo.

## 1.19: DIA-5: generador de folletos con la API de chat completions

### 1. Qué viene hoy

| Concepto             | Qué es                                                                                                                                               |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chat Completions API | La forma de llamar al modelo con una lista de mensajes ( <code>system</code> , <code>user</code> , <code>assistant</code> )                          |
| One-shot prompting   | "Un nombre elegante para dar un ejemplo": pones en el prompt un ejemplo de buena respuesta, para subir la probabilidad de que el modelo responda así |
| Streaming            | Los resultados llegan de a poco, con el efecto máquina de escribir                                                                                   |
| Markdown y JSON      | Formatos de salida: Markdown para mostrar, JSON para que tu código lo procese                                                                        |

#### 1.19.1: El problema de negocio (diapositiva)



**Company Sales Brochure Generator:** Un producto que genera folletos de marketing de una empresa para clientes potenciales, inversores y reclutamiento.

#### Flujo:

| Paso | Que pasa                                                    | ¿Usa LLM?                     |
|------|-------------------------------------------------------------|-------------------------------|
| 1    | Recibe el nombre de empresa y URL                           | No                            |
| 2    | Extrae todos los enlaces de la página                       | No (código con BeautifulSoup) |
| 3    | Decide cuáles enlaces son relevantes y los devuelve en JSON | <b>Sí, llamada 1</b>          |
| 4    | Descarga el contenido de esas páginas                       | No                            |
| 5    | Redacta el folleto con todo ese contenido                   | <b>Sí, llamada 2</b>          |

Son **dos llamadas al LLM encadenadas**: el resultado de la primera alimenta la segunda. El notebook lo llama el primer ejemplo de patrón *agentic*.

### 1.19.2: "¿Y por qué no pedírselo a ChatGPT?" – El código, paso a paso – “one-shot prompting” – “Structured Outputs (una mirada a esa técnica que se vera en la semana 8)”

El profesor reconoce que se puede, pero responde con dos ideas:

- ChatGPT es un producto que construyeron ingenieros de IA sobre GPT. Hoy tu actúas como ese ingeniero, enfocado en una tarea concreta.
- Los productos de IA verticales son así: usa **Duolingo**, y detrás hay llamadas a un modelo con prompts bien trabajados. Paga el plan premium por comodidad, aunque podría recrearlo.

Aunque se pueda recrear en ChatGPT, la lógica de negocio alrededor sigue teniendo valor. A esto le dice *GPT wrapper*. El lo ve como punto de partida, y en las 8 semanas se va a algo mas a medida.

## 4. El código, paso a paso

**Scraper (no es de IA):** `fetch_website_links(url)` devuelve todos los enlaces de una pagina. Usa BeautifulSoup y el profesor dice que no hace falta entenderlo. Reconoce que es ineficiente (descarga la página dos veces) y lo deja simple para no distraer.

Los dos problemas de esa lista:

```
[{"#wp--skip-link-target",
  "https://edwarddonner.com/avatar/",
  "https://edwarddonner.com/curriculum/",
  "https://edwarddonner.com/president/",
  "https://edwarddonner.com/connect-four/",
  "https://edwarddonner.com/outsmart/",
  "https://edwarddonner.com/about-me-and-about-nebula/",
  "https://edwarddonner.com/posts/",
  "https://edwarddonner.com/",
  "https://news.ycombinator.com",
  "https://nebula.io/?utm_source=ed&utm_medium=referral",
  "https://www.prnewswire.com/news-releases/wynden-stark-group-acquires-nyc-venture-backed-tech-startup-unt...",
  "https://edwarddonner.com/curriculum/",
  "https://edwarddonner.com/avatar/",
  "https://edwarddonner.com/2026/02/17/ai-coder-vibe-coder-to-agentic-engineer/",
  "https://edwarddonner.com/2026/02/17/ai-coder-vibe-coder-to-agentic-engineer/",
  "https://edwarddonner.com/2026/01/04/ai-builder-with-n8n-create-agents-and-voice-agents/",
  "https://edwarddonner.com/2026/01/04/ai-builder-with-n8n-create-agents-and-voice-agents/",
  "https://edwarddonner.com/2025/09/15/ai-in-production-een-ai-and-agentic-ai-on-aws-at-scale/"}]
```

| Problema                 | Ejemplo real de tu notebook                                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Hay enlaces irrelevantes | <code>news.ycombinator.com</code> , <code>prnewswire.com</code> , <code>mailto:...</code> , <code>#wp--skip-link--target</code> |
| Pueden ser relativos     | <code>/about</code> en vez de <code>https://empresa.com/about</code>                                                            |

**Por que se usa un LLM:** convertir relativos a absolutos es factible, pero decidir si un enlace es relevante requiere entender lo que representa. El LinkedIn del dueño sí sirve y Hacker News no. El profesor dice que hace unos 4 años esto no se podía hacer, y hoy es una llamada. La misma técnica sirve para analizar currículos o reseñas de restaurantes.

**El prompt con ejemplo (one-shot):** el `link_system_prompt` de tu notebook incluye este JSON de ejemplo:

```
link_system_prompt = """
Se te proporciona una lista de enlaces encontrados en una página web.
Puedes decidir cuáles de los enlaces serían más relevantes para incluir en un folleto sobre la
empresa,
como enlaces a una página Acerca de, página de la empresa, o páginas de Carreras/Empleos.
Debes responder en JSON como en este ejemplo:

{
  "links": [
    {
      "type": "página acerca de", "url": "https://full.url/ges/here/about",
      "type": "página de carreras", "url": "https://another.full.url/careers"
    }
  ]
}

"""
```

Ese ejemplo es el “one shot”: le muestra al modelo la forma exacta de la respuesta. Con `response_format={"type": "json_object"}` se pide además que devuelva JSON valido, y `json.loads(result)` lo convierte en un diccionario de Python.

**Relativo a absoluto: es una regla fija.** Si estás en `https://edwarddonner.com` y el enlace es `/about`, solo

pegas las dos partes: <https://edwarddonner.com/about>. Un código normal lo hace sin entender nada.

**Decidir si es relevante: requiere entender qué es cada enlace.** No hay una regla simple que sirva para todas las páginas. Con tu lista:

| Enlace                                                                                                        | ¿Relevante para el folleto? | Por qué                                          |
|---------------------------------------------------------------------------------------------------------------|-----------------------------|--------------------------------------------------|
| <a href="https://edwarddonner.com/about-me-and-about-nebula/">edwarddonner.com/about-me-and-about-nebula/</a> | Si                          | Habla de quién es la empresa                     |
| <a href="https://linkedin.com/in/eddonner/">linkedin.com/in/eddonner/</a>                                     | Sí                          | Esta fuera del sitio, pero describe a la persona |
| <a href="https://news.ycombinator.com">news.ycombinator.com</a>                                               | No                          | Es Hacker News, no tiene que ver con la empresa  |
| <a href="mailto:hello@...">mailto:hello@...</a>                                                               | No                          | Es un correo, no una página                      |

Aunque **linkedin.com** y **news.ycombinator.com** son dos sitios externos, uno sirve y el otro no. Para un código sería muy difícil escribir esa regla, y un LLM lo decide leyendo cada enlace con su contexto.



1.19.3: Lo que sigue en tu notebook (el video todavía no llega ahí) + Costo aproximado de un folleto

| Función                           | Que hace                                                                                          |
|-----------------------------------|---------------------------------------------------------------------------------------------------|
| fetch_page_and_all_relevant_links | Descarga la página principal y cada enlace elegido, y los junta en un texto                       |
| get_brochure_user_prompt          | Arma el prompt final y lo corta en 5.000 caracteres                                               |
| create_brochure                   | Segunda llamada al modelo, muestra el resultado en Markdown                                       |
| stream_brochure                   | Igual, con stream=True: junta los trozos (chunk.choices[0].delta.content) y actualiza la pantalla |

Costo aproximado de un folleto ejemplo

La formula

Costo = tokens x precio por millón / 1.000.000  
Precios usados (revísalos antes de confiar en ellos): **\$1,50** por millón de tokens de entrada y **9\$** por millón de salida.

Llamada 1: elegir los enlaces relevantes

| Parte                                     | Tokens | Cuenta                       | Costo           |
|-------------------------------------------|--------|------------------------------|-----------------|
| Entrada (prompt + lista de enlaces)       | 800    | $800 \times 1,5 / 1.000.000$ | \$0,0012        |
| Salida (el JSON con los enlaces elegidos) | 300    | $300 \times 9 / 1.000.000$   | \$0,0027        |
| Subtotal                                  |        |                              | <b>\$0,0039</b> |

Llamada 2: redactar el folleto

| Parte                                                         | Tokens | Cuenta                         | Costo             |
|---------------------------------------------------------------|--------|--------------------------------|-------------------|
| Entrada (el texto de las paginas, cortado a 5.000 caracteres) | 1.250  | $1.250 \times 1,5 / 1.000.000$ | \$0,001875        |
| Salida (el folleto)                                           | 500    | $500 \times 9 / 1.000.000$     | \$0,0045          |
| Subtotal                                                      |        |                                | <b>\$0,006375</b> |

Total

$\$0,0039 + \$0,006375 = \$0,010275$ , o sea **un poco más de 1 centavo de dólar** por folleto.

De dónde salen los tamaños

- **1.250 tokens** son 5.000 caracteres / 4 (la regla de 4 caracteres por token).
- **800, 300 y 500** los inventé para tener un ejemplo. Los reales dependen de cuántos enlaces tenga el sitio y de qué tan largo sea el folleto.

Qué se ve en la tabla

- La salida es más cara por token (\$9 contra \$1,50), por eso en la llamada 2 la salida cuesta más que la entrada aunque tenga menos tokens (500 contra 1.250).
- Si esto fuera un producto con 1.000 folletos al día, serían  $1.000 \times \$0,0103 \approx$  **\$10,30 al día**.

Una fuente indica que en Gemini los tokens de razonamiento (*thinking*) se cobran como salida, así que el costo real podría ser mayor que este.

1.20: El código del día 5, celda por celda

Mapa general

| Celda                                                                       | Qué hace                         | ¿Llama al LLM?    |
|-----------------------------------------------------------------------------|----------------------------------|-------------------|
| Imports                                                                     | Carga las librerías              | No                |
| Configuración                                                               | Lee la API key y crea el cliente | No                |
| <code>fetch_website_links</code>                                            | Saca los enlaces de una página   | No                |
| <code>link_system_prompt</code> y <code>get_links_user_prompt</code>        | Arman el prompt                  | No                |
| <code>select_relevant_links</code>                                          | Elige los enlaces relevantes     | Sí, llamada 1     |
| <code>fetch_page_and_all_relevant_links</code>                              | Descarga las páginas elegidas    | Usa la llamada 1  |
| <code>brochure_system_prompt</code> y <code>get_brochure_user_prompt</code> | Arman el prompt del folleto      | Usan la llamada 1 |
| <code>create_brochure</code> / <code>stream_brochure</code>                 | Redactan el folleto              | Sí, llamada 2     |

1.20.1: Imports

| <pre>python  import os, json from dotenv import load_dotenv from IPython.display import Markdown, display, update_display from scraper import fetch_website_links, fetch_website_contents from openai import OpenAI</pre> |                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Import                                                                                                                                                                                                                    | Para qué                                                                                |
| <code>os</code>                                                                                                                                                                                                           | Leer variables de entorno                                                               |
| <code>json</code>                                                                                                                                                                                                         | Convertir texto JSON a diccionario                                                      |
| <code>load_dotenv</code>                                                                                                                                                                                                  | Cargar tu archivo <code>.env</code>                                                     |
| <code>Markdown, display, update_display</code>                                                                                                                                                                            | Mostrar y actualizar texto formateado en el notebook                                    |
| <code>scraper</code>                                                                                                                                                                                                      | Archivo local del curso con las dos funciones que descargan páginas (usa BeautifulSoup) |
| <code>OpenAI</code>                                                                                                                                                                                                       | El cliente que hace las llamadas HTTP al modelo                                         |

## 1.20.2: Configuración

```
python

load_dotenv(override=True)
api_key = os.getenv('GOOGLE_API_KEY')
```

- **load\_dotenv(override=True)** lee el **.env** y sobrescribe variables de entorno que ya existieran con el mismo nombre.
- **os.getenv('GOOGLE\_API\_KEY')** devuelve la clave, o **None** si no existe.

```
python

if api_key and api_key.startswith(('AIz', 'AQ.')) and len(api_key) > 10:
```

Es solo una validación: la clave existe, empieza con algunos de esos dos prefijos (**startswith** acepta una tupla) y tiene mas de 10 caracteres.

```
load_dotenv(override=True)
api_key = os.getenv('GOOGLE_API_KEY')

if api_key and api_key.startswith(('AIz', 'AQ.')) and len(api_key) > 10:
    print("La API key se ve bien hasta ahora")
else:
    print("¿Podría haber un problema con tu API key? ¡Por favor visita el notebook de solución de problemas!")

MODEL = 'gemini-3.1-flash-lite'
gemini = OpenAI(
    api_key=api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)
```

- **MODEL** es una constante: el nombre del modelo, definido en un solo lugar.
- **Gemini** es el cliente. Usa la librería de OpenAI pero con **base\_url** de Google, porque la API de Gemini es compatible con ella (lo verifiqué en la documentación).

## 1.20.3: Enlaces de la página

```
links = fetch_website_links("https://edwarddonner.com")
links
```

```
[ '#wp--skip-link--target',
  'https://edwarddonner.com/avatar/',
  'https://edwarddonner.com/curriculum/',
  'https://edwarddonner.com/proficient/',
  'https://edwarddonner.com/connect-four/',
  'https://edwarddonner.com/outsmart/',
  'https://edwarddonner.com/about-me-and-about-nebula/',
  'https://edwarddonner.com/posts/',
  'https://edwarddonner.com/',
  'https://news.ycombinator.com',
  'https://nebula.io/?utm_source=ed&utm_medium=referral',
  'https://www.prnewswire.com/news-releases/wynden-stark-group-acquires-nyc-venture-backed-tech-startup-unta',
  'https://edwarddonner.com/curriculum/',
  'https://edwarddonner.com/avatar/',
  'https://edwarddonner.com/2026/02/17/ai-coder-vibe-coder-to-agentic-engineer/',
  'https://edwarddonner.com/2026/02/17/ai-coder-vibe-coder-to-agentic-engineer/' ]
```

Devuelve una lista de texto. En tu salida hay **33 entradas**, pero solo **20 distintas** (por ejemplo **/curriculum/** aparece 3 veces).

#### 1.20.4: El prompt del sistema (one-shot)

```
link_system_prompt = """
Se te proporciona una lista de enlaces encontrados en una página web.
Puedes decidir cuáles de los enlaces serían más relevantes para incluir en un folleto sobre la
empresa,
como enlaces a una página Acerca de, página de la empresa, o páginas de Carreras/Empleos.
Debes responder en JSON como en este ejemplo:

{
  "links": [
    {"type": "página acerca de", "url": "https://full.url/goes/here/about"},
    {"type": "página de carreras", "url": "https://another.full.url/careers"}
  ]
}
"""
```

El JSON de ejemplo es el **one-shot prompting**: le muestras la forma exacta de la respuesta.

#### 1.20.5: El prompt del usuario

```
def get_links_user_prompt(url):
    user_prompt = f"""
Aquí está la lista de enlaces en el sitio web {url} -
Por favor, decide cuáles de estos son enlaces web relevantes para un folleto sobre la empresa,
responde con la URL https completa en formato JSON.
No incluyas Términos de Servicio, Privacidad, enlaces de correo electrónico.

Enlaces (algunos podrían ser enlaces relativos):

"""
    links = fetch_website_links(url)
    user_prompt += "\n".join(links)
    return user_prompt
```

[7] ✓ 0.0s

Python

- El `f"""..."""` inserta la variable `{url}` dentro del texto.
- `"\n".join(links)` pega todos los enlaces en un solo texto, uno por línea.
- Le dice al modelo que no incluya términos de servicio, privacidad ni correos.

### 1.20.6: La llamada 1: select\_relevant\_links

```
def select_relevant_links(url):
    response = gemini.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": link_system_prompt},
            {"role": "user", "content": get_links_user_prompt(url)}
        ],
        response_format={"type": "json_object"}
    )
    result = response.choices[0].message.content
    links = json.loads(result)
    return links
```

| Linea                                                | Que hace                                             |
|------------------------------------------------------|------------------------------------------------------|
| <code>messages=[...]</code>                          | La lista de roles que vimos en la ilusión de memoria |
| <code>response_format={"type": "json_object"}</code> | Le exige a la API devolver JSON valido               |
| <code>response.choices[0]</code>                     | La primera respuesta posible (hay una sola)          |
| <code>.message.content</code>                        | El texto de la respuesta                             |
| <code>json.loads(result)</code>                      | Pasa de texto a diccionario de Python                |

El notebook define la función dos veces. La segunda agrega dos **print** (modelo usado y cuántos enlaces encontró con `len(links['links'])`) y reemplaza a la primera.

Salida:

```
select_relevant_links("https://edwarddonner.com")
[9] ✓ 7.0s Pyti
... {'links': [{'type': 'página acerca de',
  'url': 'https://edwarddonner.com/about-me-and-about-nebula/'},
  {'type': 'página de currículum',
  'url': 'https://edwarddonner.com/curriculum/'},
  {'type': 'perfil profesional',
  'url': 'https://www.linkedin.com/in/eddonner/'}]}
```

Definición redefinida:

```
def select_relevant_links(url):
    print(f"Seleccionando enlaces relevantes para {url} llamando a {MODEL}")
    response = gemini.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": link_system_prompt},
            {"role": "user", "content": get_links_user_prompt(url)}
        ],
        response_format={"type": "json_object"}
    )
    result = response.choices[0].message.content
    links = json.loads(result)
    print(f"Encontrados {len(links['links'])} enlaces relevantes")
    return links
```

[10] ✓ 0.0s

## Salidas:

```
> select_relevant_links("https://edwarddonner.com")
11] ✓ 6.0s Python

.. Seleccionando enlaces relevantes para https://edwarddonner.com llamando a gemini-3.1-flash-lite
Encontrados 4 enlaces relevantes

.. {'links': [{'type': 'página acerca de',
  'url': 'https://edwarddonner.com/about-me-and-about-nebula/'},
  {'type': 'página de currículum',
  'url': 'https://edwarddonner.com/curriculum/'},
  {'type': 'página de inicio', 'url': 'https://edwarddonner.com/'},
  {'type': 'perfil profesional',
  'url': 'https://www.linkedin.com/in/eddonner/'}]}
```

```
> select_relevant_links("https://huggingface.co")
12] ✓ 2.5s

.. Seleccionando enlaces relevantes para https://huggingface.co llamando a gemini-3.1-flash-lite
Encontrados 6 enlaces relevantes

.. {'links': [{'type': 'página acerca de',
  'url': 'https://huggingface.co/huggingface'},
  {'type': 'página de empresa', 'url': 'https://huggingface.co/enterprise'},
  {'type': 'página de carreras',
  'url': 'https://apply.workable.com/huggingface/'},
  {'type': 'página de marca', 'url': 'https://huggingface.co/brand'},
  {'type': 'página de blog', 'url': 'https://huggingface.co/blog'},
  {'type': 'página de documentación', 'url': 'https://huggingface.co/docs'}]}
```

### 1.20.7: Juntar el contenido: `fetch_page_and_all_relevant_links`

```
def fetch_page_and_all_relevant_links(url):
    contents = fetch_website_contents(url)
    relevant_links = select_relevant_links(url)
    result = f"## Landing Page:\n\n{contents}\n\n## Relevant Links:\n"
    for link in relevant_links['links']:
        result += f"\n\n## Link: {link['type']}\n"
        result += fetch_website_contents(link['url'])
    return result

print(fetch_page_and_all_relevant_links("https://huggingface.co"))
```

Descarga la página principal, pide los enlaces relevantes y recorre cada uno con un **for** sumando su contenido a un texto grande con títulos en Markdown.

Esta función **junta en un solo texto** el contenido de la página principal y el de las páginas relevantes. Uso un ejemplo con `url = "https://edwarddonner.com"`. Los textos de las páginas son inventados.

#### Paso 1: descargar la página principal

```
python
contents = fetch_website_contents(url)
```

Guarda el texto de la página principal en **contents**. Por ejemplo: **"Edward Donner. Cursos de IA..."**.

#### Paso 2: pedirle al LLM los enlaces relevantes

```
python
relevant_links = select_relevant_links(url)
```

Esto es la **llamada 1 al LLM**. Devuelve un diccionario (ilustrativo):

```
{
  "links": [
    {
      "type": "página acerca de",
      "url": "https://edwarddonner.com/about-me-and-about-nebula/"
    },
    {
      "type": "cursos",
      "url": "https://edwarddonner.com/curriculum/"
    }
  ]
}
```

#### Paso 3: crear el texto base

```
python
result = f"## Landing Page:\n\n{contents}\n\n## Relevant Links:\n"
```

Arma un texto con un título en Markdown y pega **contents**. El **\n** es un salto de línea. En este punto **result** vale:

```
## Landing Page:

Edward Donner. Cursos de IA...

## Relevant Links:
```

#### Paso 4: recorrer los enlaces

```
python
for link in relevant_links['links']:
```

- **relevant\_links["links"]** es la lista con los 2 enlaces.
- El **for** repite lo que sigue una vez por enlace, y en cada vuelta **link** es un diccionario distinto.

#### Paso 5: agregar el título del enlace

```
python
result += f"\n\n### Link: {link['type']}\n"
```

- **+=** agrega al final de **result**, sin reemplazar lo anterior
- **link["type"]** es el tipo, por ejemplo "página acerca de".
- Agrega **### Link: página acerca de** en el texto.

#### Paso 6: agregar el contenido de ese enlace

```
python
result += fetch_website_contents(link["url"])
```

#### Resultado después de las 2 vueltas

```
## Landing Page:

Edward Donner. Cursos de IA...
## Relevant Links:

### Link: página acerca de
Sobre mí y Nebula...

### Link: cursos
Curriculum del curso...
```

#### Paso 7: devolverlo

```
python
return result
```

Entrega todo el texto, que luego se pega en el prompt del folleto.

#### Qué conviene notar

| Cosa                                 | Detalle                                                                                                    |
|--------------------------------------|------------------------------------------------------------------------------------------------------------|
| Cantidad de descargas                | 1 (principal) + 1 por cada enlace elegido, o sea 3 en el ejemplo                                           |
| Llamadas al LLM                      | 1 (dentro de <code>select_relevant_links</code> )                                                          |
| Si el LLM devuelve enlaces repetidos | Cada repetido se descarga otra vez                                                                         |
| Tamaño                               | Cuanto más enlaces, más texto. Después se corta a 5.000 caracteres, y eso puede borrar los últimos enlaces |



### 1.20.8: El folleto + ¿que es un folleto?

#### breve, que es un folleto?

Un folleto (en ingles *brochure*) es un documento corto que presenta una empresa o un producto de forma resumida y atractiva, para convencer a alguien de contratarla, invertir o postular.

En el ejercicio, el modelo lee el sitio web y escribe un texto así, en Markdown. Por ejemplo, para una empresa podría incluir:

- Que hace la empresa
- Quienes son sus clientes
- Como es su cultura
- Que puestos de trabajo tiene

Es como el "resumen de ventas" de la empresa en una página.

**brochure\_system\_prompt:** le dice al modelo que es un asistente que crea un folleto corto, en Markdown sin bloques de código, con cultura, clientes y empleos. Hay una versión humorística comentada con #, para mostrar lo fácil que es cambiar el tono.

```
brochure_system_prompt = """
Eres un asistente que analiza el contenido de varias páginas relevantes del sitio web de una empresa
y crea un folleto corto sobre la empresa para clientes potenciales, inversores y reclutas.
Responde en markdown sin bloques de código.
Incluye detalles de la cultura empresarial, clientes y carreras/empleos si tienes la información.
"""

# O descomenta las líneas de abajo para un folleto más humorístico - esto demuestra lo fácil que es
# incorporar 'tono':

# brochure_system_prompt = """
# Eres un asistente que analiza el contenido de varias páginas relevantes del sitio web de una
# empresa
# y crea un folleto corto, humorístico, entretenido y ingenioso sobre la empresa para clientes
# potenciales, inversores y reclutas.
# Responde en markdown sin bloques de código.
# Incluye detalles de la cultura empresarial, clientes y carreras/empleos si tienes la información.
# """
```

#### get\_brochure\_user\_prompt:

```
def get_brochure_user_prompt(company_name, url):
    user_prompt = f"""
    Estás mirando una empresa llamada: {company_name}
    Aquí está el contenido de su página de inicio y otras páginas relevantes;
    usa esta información para construir un folleto corto de la empresa en markdown sin bloques de código.
    \n\n
    """
    user_prompt += fetch_page_and_all_relevant_links(url)
    user_prompt = user_prompt[:5_000] # Truncar si hay más de 5.000 caracteres
    return user_prompt
```

- **[ :5\_000 ]** se queda con los **primeros 5.000 caracteres** (el guion bajo es solo separador visual). 5.000 / 4 ≈ 1.250 tokens, muy poco frente a la ventana de 1 M.

- Corta por el final, así que se pierden las últimas páginas. Si el folleto sale incompleto, sube ese número.

**Salida:**

```
get_brochure_user_prompt("HuggingFace", "https://huggingface.co")
[20] ✓ 6.3s Python

... Seleccionando enlaces relevantes para https://huggingface.co llamando a gemini-3.1-flash-lite
Encontrados 5 enlaces relevantes

... "\nEstás mirando una empresa llamada: HuggingFace\nAquí está el contenido de su página de inicio y otras pa

... AI Apps\nor\nBrowse 2M+ models\nTrending on\nthis week\nModels\nCloudflare/clef\nUpdated\n3 days ago\n•\nn4.2
```

### 1.20.9: La llamada 2

**Créate\_brochure:** igual que la llamada 1, sin **response\_format**. Usa **display(Markdown(result))** para mostrar el texto con formato.

| Línea                                      | Qué hace                                                                                             |
|--------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>stream=True</code>                   | La API envía trozos a medida que genera                                                              |
| <code>display(..., display_id=True)</code> | Crea un espacio en pantalla que se puede reescribir                                                  |
| <code>for chunk in stream</code>           | Recibe cada trozo                                                                                    |
| <code>delta.content or ''</code>           | Puede venir <code>None</code> (por ejemplo en el último trozo); el <code>or ''</code> evita un error |
| <code>response +=</code>                   | Va acumulando el texto                                                                               |
| <code>update_display</code>                | Redibuja el espacio con el texto acumulado: el efecto máquina de escribir                            |

```
def create_brochure(company_name, url):
    response = gemini.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": brochure_system_prompt},
            {"role": "user", "content": get_brochure_user_prompt(company_name, url)}
        ],
    )
    result = response.choices[0].message.content
    display(Markdown(result))
```

Python

```
create_brochure("HuggingFace", "https://huggingface.co")
```

✓ 15.8s Python

Seleccionando enlaces relevantes para <https://huggingface.co> llamando a gemini-3.1-flash-lite  
Encontrados 7 enlaces relevantes

## Hugging Face: El motor de la colaboración en Inteligencia Artificial

### Sobre la empresa

Hugging Face es la plataforma líder mundial para la comunidad de aprendizaje automático (Machine Learning). Definida como "el hogar del Machine Learning", la misión de la empresa es democratizar la IA mediante la creación de un ecosistema abierto, colaborativo y accesible donde investigadores, desarrolladores y empresas pueden crear, descubrir y compartir soluciones tecnológicas de vanguardia.

### ¿Qué ofrecemos?

La plataforma sirve como un centro neurálgico donde la comunidad interactúa a través de:

- **Modelos:** Acceso a más de 2 millones de modelos pre-entrenados para diversas tareas.
- **Datasets:** Acceso a más de 500,000 conjuntos de datos para el entrenamiento y ajuste de modelos.
- **Spaces:** Un entorno interactivo para alojar, explorar y demostrar aplicaciones de IA de forma rápida.
- **Soluciones Enterprise:** Herramientas diseñadas para equipos profesionales, incluyendo soporte técnico, puntos de conexión de inferencia (Inference Endpoints) y almacenamiento seguro (Buckets).

### Cultura y Comunidad

Hugging Face se distingue por ser una plataforma impulsada por la comunidad. La cultura de la empresa se basa en la **apertura y el intercambio**. Fomentamos un entorno donde la innovación ocurre de manera colectiva, facilitando que personas de todo el mundo colaboren en proyectos desde el desarrollo técnico hasta la resolución de problemas complejos. Si te apasiona el código abierto y el futuro de la inteligencia artificial, este es el lugar donde el trabajo colaborativo define el estándar de la industria.

### Clientes y Usuarios

Nuestra comunidad es vasta y diversa:

- **Investigadores y Científicos de Datos:** Que utilizan la plataforma para publicar sus descubrimientos y acceder a los últimos avances.
- **Desarrolladores:** Que integran modelos y datasets en sus propias aplicaciones.
- **Empresas:** Desde startups hasta grandes corporaciones que utilizan los planes "Team & Enterprise" para escalar sus infraestructuras de IA, garantizar la seguridad de sus datos y contar con soporte especializado de nivel experto.

### Carreras y Talento

En Hugging Face, buscamos personas apasionadas por el impacto global de la IA. Trabajar aquí significa formar parte de una infraestructura esencial que está dando forma al futuro de la tecnología. Buscamos talento con visión técnica, espíritu colaborativo y compromiso con el ecosistema de código abierto. Si buscas un lugar donde tu trabajo llegue a millones de desarrolladores y acelere el progreso de la IA, te invitamos a ser parte de nuestra comunidad.

---

¿Quieres saber más? Explora nuestra plataforma en [huggingface.co](https://huggingface.co), únete a nuestro Discord, participa en nuestros foros o revisa nuestro GitHub para empezar a construir el futuro de la IA hoy mismo.

🔗 [Página](#) 🔗 [Código](#) 🔗 [Modelos](#)

## stream\_brochure:

```
def stream_brochure(company_name, url):  
    stream = gemini.chat.completions.create(  
        model=MODEL,  
        messages=[  
            {"role": "system", "content": brochure_system_prompt},  
            {"role": "user", "content": get_brochure_user_prompt(company_name, url)}  
        ],  
        stream=True  
    )  
    response = ""  
    display_handle = display(Markdown(""), display_id=True)  
    for chunk in stream:  
        response += chunk.choices[0].delta.content or ""  
        update_display(Markdown(response), display_id=display_handle.display_id)
```

✓ 0.0s Python

```
def stream_brochure(company_name, url):
    stream = gemini.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": brochure_system_prompt},
            {"role": "user", "content": get_brochure_user_prompt(company_name, url)}
        ],
        stream=True
    )

    response = ""
    display_handle = display(Markdown(""), display_id=True)
    for chunk in stream:
        response += chunk.choices[0].delta.content or ''
        update_display(Markdown(response), display_id=display_handle.display_id)
```

En anterior:

```
result = response.choices[0].message.content
display(Markdown(result))
```

Ese código muestra el folleto **a medida que el modelo lo escribe**, como una máquina de escribir, en vez de esperar a que termine.

### Línea por línea

```
python

response = ""
```

Un texto vacío donde se va acumulando lo que llega.

```
python

display_handle = display(Markdown(""), display_id=True)
```

Crea un **espacio en blanco en pantalla** y guarda un identificador (**display\_id**) para poder reescribirlo despues.

```
python

for chunk in stream:
```

**Stream** entrega el texto en **trozos (chunk)**. El ciclo se repite una vez por cada trozo que llega.

```
python

response += chunk.choices[0].delta.content or ''
```

- **Chunk.choices[0].delta.content** es el pedazo de texto nuevo, por ejemplo **"Hugging"**.
- Or ' ': en algunos trozos (como el ultimo) el contenido viene vacío (**None**). El or ' ' lo cambia por texto vacío para evitar un error.
- += lo agrega al final de **response**.

```
python

update_display(Markdown(response), display_id=display_handle.display_id)
```

**Redibuja** el espacio en pantalla con todo el texto acumulado hasta ahora, usando el identificador guardado.

Ejemplo (trozos inventados)

| Vuelta | Trozo que llega | response vale         | Lo que ves en pantalla |
|--------|-----------------|-----------------------|------------------------|
| 1      | "Hugging"       | "Hugging"             | Hugging                |
| 2      | " Face"         | "Hugging Face"        | Hugging Face           |
| 3      | " es"           | "Hugging Face es"     | Hugging Face es        |
| 4      | " una"          | "Hugging Face es una" | Hugging Face es una    |

Cada vez que llega un trozo, la pantalla se reescribe con el texto completo. Eso da la ilusión de que alguien está escribiendo en vivo.

Con stream=True o sin él

|          | Sin stream (create_brochure)                   | Con stream=True (stream_brochure) |
|----------|------------------------------------------------|-----------------------------------|
| Que pasa | Esperas hasta que termina y aparece todo junto | Aparece de a poco                 |
| Costo    | Igual                                          | Igual                             |

1.20.10: Un detalle importante de costo + Costo con tu modelo

get\_brochure\_user\_prompt llama a fetch\_page\_and\_all\_relevant\_links, que llama a select\_relevant\_links. Es decir, cada vez que ejecutas create\_brochure o stream\_brochure, se repite la llamada 1, mas las descargas. Incluso las celdas con print(...) hacen eso.

11. Costo con tu modelo

gemini-3.1-flash-lite cuesta \$0,25 de entrada y \$1,50 de salida por millón (varias fuentes de 2026; una lista \$0,125 y \$0,75, así que revísalo). Con los mismos tamaños inventados de antes:

| Llamada | Entrada                                       | Salida                                   | Costo       |
|---------|-----------------------------------------------|------------------------------------------|-------------|
| 1       | $800 \times 0,25 / 1.000.000 = \$0,0002$      | $300 \times 1,5 / 1.000.000 = \$0,00045$ | \$0,00065   |
| 2       | $1.250 \times 0,25 / 1.000.000 = \$0,0003125$ | $500 \times 1,5 / 1.000.000 = \$0,00075$ | \$0,0010625 |
| Total   |                                               |                                          | ≈ \$0,0017  |

Es unas 6 veces más barato que con Gemini 3.5 Flash (\$0,0103). Los tokens de razonamiento pueden sumar más.

## 1.21: Notas de video 34. Day 5 – building Json prompts and using OpenAI's chat completions API

### 1. Todo parte de la secuencia de entrada

El profesor dice que se empieza **pensando en el prompt**: qué secuencia de entrada hay que enviar para que los tokens más probables sean los que cumplen la tarea.

### 2. Por qué JSON y Markdown funcionan bien

Según él, los modelos se entrenan sobre todo con tres tipos de datos:

| Tipo de dato                      | Secuencia                                                                   |
|-----------------------------------|-----------------------------------------------------------------------------|
| Lenguaje natural (inglés y otros) | Entienden y escriben texto                                                  |
| Markdown (sitios web convertidos) | "Les encanta" generarlo y lo conocen muy bien                               |
| JSON                              | Si les mandas información estructurada en JSON, la reconocen y la entienden |

oye si, markdown con html en qu se relaciona?

**Markdown es una forma simple de escribir texto con formato, que se convierte en HTML.** HTML es el lenguaje con el que se arman las páginas web, y Markdown está pensado como atajo para escribirlo más fácil.

El creador, John Gruber (2004), lo describe como una herramienta que convierte texto plano fácil de leer y escribir en HTML válido. Una frase que cita un artículo: "HTML es un formato de publicación; Markdown es un formato de escritura".

#### Ejemplo: el mismo contenido en los dos

| Markdown                     | HTML                                    |
|------------------------------|-----------------------------------------|
| # Título                     | <h1>Título</h1>                         |
| ## Subtítulo                 | <h2>Subtítulo</h2>                      |
| <b>**negrita**</b>           | <strong>negrita</strong>                |
| - Rápido                     | <ul><li>Rápido</li></ul>                |
| [sitio](https://ejemplo.com) | <a href="https://ejemplo.com">sitio</a> |

El resultado en pantalla es el mismo, pero Markdown se lee bien aunque no lo conviertas, y HTML con tantas etiquetas es más difícil de leer.

#### Cómo se relacionan

- **Markdown cubre solo una parte pequeña de lo que hace HTML.** No sirve para todo, y no lo reemplaza.
- Un procesador (como Pandoc o Python-Markdown) convierte tu texto Markdown en HTML.
- Si necesitas algo que Markdown no tiene, puedes escribir HTML dentro del mismo documento, y Markdown lo entiende.

#### En tu notebook

Cuando haces `display(Markdown(response))`, Jupyter toma el texto en Markdown que escribió el modelo (por ejemplo `## Cultura` y `**clientes**`) y lo muestra con formato: ahí ocurre esa conversión a lo visual.

### 3. JSON versus describir el formato con palabras

Antes se intentaba decir "responde con esto en el punto 1, con esto en el punto 2". Él lo llama "ridículo": es mejor dar la estructura en JSON. Además comenta que sin LLM sería difícil programar algo como "un título (tipo) para cada enlace".

### 4. One-shot versus multi-shot

| Técnica    | Qué das en el prompt                                                                          |
|------------|-----------------------------------------------------------------------------------------------|
| One-shot   | Un ejemplo de respuesta                                                                       |
| Multi-shot | Varios ejemplos (por ejemplo, pregunta, respuesta buena, otra pregunta, otra respuesta buena) |

**Por qué funciona:** el modelo solo genera los tokens más probables a partir de la entrada. Con un ejemplo en la entrada, lo más probable pasa a ser justo lo que queremos.

### 5. También se pueden dar ejemplos de lo que NO hacer

La línea "No incluyas términos de servicio, privacidad ni correos" es lo que él llama "casi lo contrario" del one-shot: ejemplos de malas respuestas. La puso porque el modelo los estaba incluyendo.

### 6. Lo más importante del curso: experimentar e iterar

Para él, ser buen ingeniero de LLMs es ser buen científico de datos: **probar, mirar el resultado y afinar el prompt.**

Ejemplo de su flujo: el modelo incluye una política de privacidad como relevante, entonces agregas "no incluyas políticas de privacidad", vuelves a probar, y si hace falta agregas ejemplos.

### 7. Diseño: constante contra función

| Prompt                                  | Cómo está en el código | Por qué                   |
|-----------------------------------------|------------------------|---------------------------|
| <code>link_system_prompt</code>         | Variable (constante)   | Siempre es el mismo texto |
| <code>get_links_user_prompt(url)</code> | Función                | Cambia con cada URL       |



### 1.21.1: Restricción en tiempo de inferencia (lo más interesante) - *Structured Outputs* (para ayudar a seguir un esquema)

Cuando ejecuto `get_links_user_prompt`, aclaro que “aquí no hay IA de por medio”: solo esta “horneando” un texto que después se enviara al modelo.

```
def select_relevant_links(url):
    print(f"Seleccionando enlaces relevantes para {url} llamando a {MODEL}")
    response = gemini.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": link_system_prompt},
            {"role": "user", "content": get_links_user_prompt(url)}
        ],
        response_format={"type": "json_object"}
    )
    result = response.choices[0].message.content
    links = json.loads(result)
    print(f"Encontrados {len(links['links'])} enlaces relevantes")
    return links
```

El profesor pide taparse los oídos a quienes ya lo saben. La idea:

1. El modelo no elige un token directamente: calcula una **probabilidad para cada token posible** del vocabulario.
2. Después se elige uno (el más probable o por muestreo).
3. Al pedir JSON con `response_format`, se puede **impedir elegir tokens que romperían el JSON**.
4. El modelo calcula lo que quiera, pero solo se pueden elegir los tokens que mantienen un JSON correcto.

**Lo verifiqué:** esto se llama *constrained decoding*. Las fuentes dicen que el motor pone los valores de los tokens inválidos en menos infinito antes de elegir, y que el vocabulario típico es de 32 K a 128 K tokens (coincide con lo que vimos).

**Ejemplo con números inventados:** después de `{"links": [`, solo son válidos `{` o `]`.

| Token                         | Probabilidad del modelo | Después de la máscara   |
|-------------------------------|-------------------------|-------------------------|
| {                             | 0,70                    | $0,70 / 0,80 = 87,5 \%$ |
| Sure (texto suelto, inválido) | 0,20                    | 0 %                     |
| ]                             | 0,10                    | $0,10 / 0,80 = 12,5 \%$ |

**Dos matices, de las fuentes:**

- El modo `json_object` solo garantiza JSON válido. **No garantiza que tenga tus claves.** Eso lo garantizan *Structured Outputs* con esquema (lo veras en la semana 8). Por eso el ejemplo del prompt sigue siendo útil: es lo que lleva al modelo a usar links, type y url.
- **JSON válido no es JSON correcto:** garantiza la forma, no que lo escrito sea verdad.

## La idea en simple

El modelo escribe **un token a la vez**. Antes de cada token, calcula qué tan probable es cada opción de su vocabulario. Lo normal es que elija entre todas.

Con el **modo JSON**, antes de elegir se tachan las opciones que romperían el JSON, y solo se elige entre las que quedan.

## Ejemplo paso a paso (probabilidades inventadas)

El modelo ya escribió esto:

```
{"links": [
```

Ahora debe elegir el siguiente token. En un JSON correcto, aquí solo puede venir { (abrir un enlace) o ] (cerrar la lista).

### Paso 1: el modelo calcula probabilidades para todo su vocabulario. Mostramos solo tres opciones:

| Token                                 | Probabilidad del modelo | ¿Mantiene el JSON válido? |
|---------------------------------------|-------------------------|---------------------------|
| {                                     | 70 %                    | Sí                        |
| Sure (como en "Sure, aquí tienes...") | 20 %                    | No                        |
| ]                                     | 10 %                    | Sí                        |

### Paso 2: se tacha lo inválido. Sure queda en 0 %.

### Paso 3: se reparte lo que queda. Quedan 70 % + 10 % = 80 %, así que:

- } :  $70 / 80 = 87,5\%$
- ] :  $10 / 80 = 12,5\%$

### Paso 4: se elige entre esas dos. Sure ya no puede salir, aunque el modelo "quisiera".

Se repite en cada token, hasta terminar el JSON.

## Qué se logra

|                                                             | Sin restricción | Con restricción   |
|-------------------------------------------------------------|-----------------|-------------------|
| El modelo puede escribir "Claro, aquí esta": antes del JSON | Si              | No                |
| Puede olvidar cerrar una llave                              | Si              | No                |
| json.loads(result) puede fallar                             | Si              | Muy poco probable |

Eso importa en tu código: **json.loads(result)** falla si el texto no es JSON válido.

## Lo que hay que recordar

- Es un truco **del sistema que ejecuta el modelo**, no del modelo: el no entiende JSON "técnicamente", como dice el profesor. El sistema solo le tacha las opciones prohibidas.
- Solo asegura que el JSON este bien escrito. No asegura que tenga tus llaves (**links, type, url**), ni que lo que dice sea verdad. Para las claves, el ejemplo del prompt ayuda, y en la semana 8 verás los *Structured Outputs*, que obligan a seguir un esquema.

2. Week 2 – Build a Multi-Modal Chatbot: LLMs, Gradio UI, and Agents

2.1 Day 1 – Testing GPT-5 – Escalado en tiempo de entrenamiento vs encalado en tiempo de inferencia – temperature

2.1.1 Escalar en entrenamiento contra escalar en inferencia, en vivo

Escalado en tiempo de entrenamiento vs. Inferencia

```
easy_puzzle = [{"role": "user", "content": "Lanzas 2 monedas. Una de ellas es cara. ¿Cuál es la probabilidad de que la otra sea cruz? Responde solo con la probabilidad."}]
```

La respuesta es 2/3.  
Hay 4 combinaciones posibles al lanzar dos monedas:

| Moneda 1 | Moneda 2 | ¿Se cumple "una es cara"? |
|----------|----------|---------------------------|
| Cara     | Cara     | Sí                        |
| Cara     | Cruz     | Sí                        |
| Cruz     | Cara     | Sí                        |
| Cruz     | Cruz     | No (se descarta)          |

Quedan 3 casos posibles, y en 2 de ellos la otra moneda es cruz:  $2/3 \approx 66,7\%$ .

Si pidieras "responde solo con la probabilidad", la salida correcta es 2/3. Un modelo que contesta 1/2 o 1/3 se equivocó.

Si el enunciado dijera "la primera moneda es cara", la respuesta sería 1/2, porque ya sabrías cuál es.

El profesor probó el mismo acertijo con tres configuraciones de GPT-5:

```
response = openai.chat.completions.create(model="gpt-5-nano", messages=easy_puzzle, reasoning_effort="minimal")
display(Markdown(response.choices[0].message.content))

response = openai.chat.completions.create(model="gpt-5-nano", messages=easy_puzzle, reasoning_effort="low")
display(Markdown(response.choices[0].message.content))

response = openai.chat.completions.create(model="gpt-5-mini", messages=easy_puzzle, reasoning_effort="minimal")
display(Markdown(response.choices[0].message.content))
```

| Modelo     | reasoning_effort | Resultado  |
|------------|------------------|------------|
| gpt-5-nano | minimal          | 1/3 (mal)  |
| gpt-5-nano | low              | 2/3 (bien) |
| gpt-5-mini | minimal          | 2/3 (bien) |

- Subir el **razonamiento** (inferencia) o subir el **tamaño** (entrenamiento) arregla la respuesta, y cada uno por separado.
- **reasoning effort** es un parametro nuevo. En GPT-5 acepta minimal, low, médium y high. Las versiones 5.1 o posteriores agregan **none**, y las 5.4 o posteriores **xhigh** (verificado en fuentes).
- Los modelos GPT-5 con razonamiento **rechazan temperatura** cuando el esfuerzo no es **none**.
- Ojo: **fue una sola corrida por celda**. Las respuestas pueden variar, y el profesor mismo dice que a veces da respuestas distintas.

### Qué es temperatura

Es un parámetro que controla que **tan predecible o variado** es el texto que genera el modelo. Se relaciona con lo que vimos de la restricción en inferencia: el modelo calcula una probabilidad para cada token posible y luego se elige uno.

- **Temperatura baja (cerca de 0)**: casi siempre elige el token mas probable. Respuestas mas estables y repetibles.
- **Temperatura alta**: da mas oportunidad a tokens menos probables. Respuestas mas variadas y creativas.

**Ejemplo (probabilidades inventadas)**: después de "El cielo es", el modelo da estas opciones:

| Token  | Probabilidad |
|--------|--------------|
| azul   | 70 %         |
| gris   | 20 %         |
| nuboso | 10 %         |

Con temperatura baja casi siempre saldría "azul". Con temperatura alta, "gris" y "nuboso" saldrían más seguido.

### Qué pasa con GPT-5

Si le mandas temperature=0.2 a un modelo GPT-5, la API responde un **error 400**: "Only the default (1) value is supported". Solo acepta el valor 1. También rechaza top\_p, y según una fuente, max\_tokens (hay que usar max\_completion\_tokens).

Lo mismo pasa con las familias o1, o3 y o4.

**Cómo evitarlo**: no pases temperature en las llamadas a estos modelos. El notebook del curso no la usa en ninguna llamada, así que no te va a fallar.

### Por qué ocurre

Las fuentes que encontré **no dan una razón oficial**. Lo que dicen los usuarios en el foro de OpenAI es que:

- Estos modelos tienen un razonamiento interno, y OpenAI no permite modificar los parámetros de muestreo en ellos.
- Al parecer, no se espera que se vuelva a permitir.

breve, y como se veria con la configuracion de temepature

Con un modelo que **sí** acepta temperatura (GPT-4.1, no GPT-5):

```
python
response = openai.chat.completions.create(
    model="gpt-4.1-mini",
    messages=easy_puzzle,
    temperature=0.2
)
```

Qué pasaría según el valor:

| temperature | Comportamiento                   |
|-------------|----------------------------------|
| 0           | Casi siempre la misma respuesta  |
| 0,2         | Poca variación                   |
| 1 (normal)  | Variación equilibrada            |
| 1,5-2       | Muy variado, a veces incoherente |

Con **GPT-5**, si agregas esa misma línea, la API devuelve un error y no responde nada:

```
openai.BadRequestError: Unsupported value: 'temperature' does not support 0.2 with this model. Only the default (1) value is supported.
```

Para usar GPT-5 sin error, simplemente no pasas temperature:

```
python
response = openai.chat.completions.create(
    model="gpt-5-nano",
    messages=easy_puzzle,
    reasoning_effort="low"
)
```

Diferencia entre razonamiento y tamaño

Analogía: un estudiante en un examen

- Tamaño del modelo: qué tan capaz es el estudiante. Uno de 3.º básico contra uno de la universidad.
- Razonamiento: cuánto tiempo y cuánto papel borrador le dejas para pensar antes de responder.

Un estudiante más capaz puede acertar de una. Uno menos capaz, con papel borrador, también puede llegar a la respuesta.

Comparación

|                  | Tamaño                                      | Razonamiento                                          |
|------------------|---------------------------------------------|-------------------------------------------------------|
| Nombre técnico   | Escalado en tiempo de <b>entrenamiento</b>  | Escalado en tiempo de <b>inferencia</b>               |
| Que cambia       | Mas parámetros y mas datos                  | El modelo genera tokens “pensando” antes de responder |
| Cuando se decide | Al crear el modelo, una sola vez            | Cada vez que lo usas                                  |
| Como lo eliges   | Eligiendo otro modelo (nano o mini)         | Con el parámetro <b>reasoning_effort</b>              |
| Quien lo paga    | El laboratorio, y tú en el precio por token | Tú, en cada llamada (más tokens de salida)            |

## En el ejemplo de las monedas

| Configuración | Qué cambió       | Resultado  |
|---------------|------------------|------------|
| nano, minimal | Nada             | 1/3 (mal)  |
| nano, low     | Más razonamiento | 2/3 (bien) |
| mini, minimal | Más tamaño       | 2/3 (bien) |

Cada línea mejora una sola cosa respecto de la primera, y ambas arreglaron la respuesta.

## Lo que cuesta cada una

Los tokens de razonamiento se cobran como salida, aunque no los veas. Ejemplo con números inventados y el precio de salida de nano que dijo el profesor (\$0,40 por millón):

- minimal: 20 tokens de salida →  $20 \times 0,40 / 1.000.000 = \$0,000008$
- low: 300 tokens de salida →  $300 \times 0,40 / 1.000.000 = \mathbf{\$0,00012}$

Razonar sale unas 15 veces más caro en ese ejemplo (300 / 20), y también tarda más.

## Se pueden combinar

Son ejes independientes. Un modelo grande con mucho razonamiento es lo más potente y lo más caro. Uno chico sin razonamiento es lo más barato y rápido, pero falla más en acertijos.

**Cuándo conviene cada uno:** razonamiento para problemas con varios pasos (lógica, matemática), y un modelo más grande cuando el modelo chico no tiene la capacidad, aunque piense.

## 2.1.2 El acertijo de las monedas (2/3) - Un solo cliente para muchos proveedores

La clase OpenAI sirve para otros proveedores cambiando base\_url:

```
Generate + Code + Markdown | Run All | Restart | ... | llm-engineering (3.14.x) (Python 3.14.-1)
# Un enviótorio deigado alrededor de llamadas a endpoints HTTP

openai = OpenAI(api_key=openai_api_key)

# Para Gemini, DeepSeek y Groq, podemos usar el cliente de Python de OpenAI
# Porque Google y DeepSeek tienen endpoints compatibles con OpenAI
# Y OpenAI te permite cambiar el base_url

anthropic_url = "https://api.anthropic.com/v1/"
gemini_url = "https://generativelanguage.googleapis.com/v1beta/openai/"
deepseek_url = "https://api.deepseek.com"
groq_url = "https://api.groq.com/openai/v1"
grok_url = "https://api.x.ai/v1"
openrouter_url = "https://openrouter.ai/api/v1"
ollama_url = "http://localhost:11434/v1"

anthropic = OpenAI(api_key=anthropic_api_key, base_url=anthropic_url)
gemini = OpenAI(api_key=google_api_key, base_url=gemini_url)
deepseek = OpenAI(api_key=deepseek_api_key, base_url=deepseek_url)
groq = OpenAI(api_key=groq_api_key, base_url=groq_url)
grok = OpenAI(api_key=grok_api_key, base_url=grok_url)
openrouter = OpenAI(base_url=openrouter_url, api_key=openrouter_api_key)
ollama = OpenAI(api_key="ollama", base_url=ollama_url)
```

2.1.2 Groq vs Grok

|                   | Groq (con Q)                                                                                                                                                 | Grok (con K)                                                     |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Qué es            | Empresa de hardware e inferencia en la nube                                                                                                                  | Familia de modelos de lenguaje                                   |
| Fundada / Lanzado | 2016                                                                                                                                                         | Noviembre de 2023                                                |
| Fundador          | Jonathan Ross (ex Google, lideró el proyecto TPU) y Douglas Wightman                                                                                         | Elon Musk, a través de xAI                                       |
| Qué ofrece        | Un chip propio, el <b>LPU</b> (Language Processing Unit), y la plataforma <b>GroqCloud</b> , donde corre modelos de otros (llama, GPT-OSS) a mucha velocidad | Sus propios modelos (Grok 4, Grok 4.6, etc.), accesibles por API |
| Analogía          | Es la "autopista": el hardware que hace correr modelos rápido                                                                                                | Es el "auto": el modelo que compite con GPT o Claude             |

**El origen del conflicto de nombres:** Groq registró la marca en 2016, siete años antes de que existiera xAI. Cuando Musk lanzó Grok en noviembre de 2023, el CEO de Groq publicó una carta pública pidiéndole que cambiara el nombre. Nunca lo cambió.

**Por qué se confunden tanto:** se pronuncian casi igual, ambos nombres vienen de la palabra en inglés "grok" (entender algo profundamente), de la novela de ciencia ficción *Stranger in a Strange Land* (1961), y ambos son parte del mismo mundo de la IA.

**Regla simple para no confundirte:** si lees sobre un chatbot, un modelo o "X" (la red social de Musk) → es **Grok**. Si lees sobre velocidad de inferencia, un chip o correr modelos abiertos barato → es **Groq**.

2.2 Día 1: Router (enrutador) – Capa de abstracción

**Router (enrutador):** Un servicio externo al que le mandas la petición, y el decide a que proveedor llamar. Esta “en la nube”, no en tu computador. Ejemplo: **OpenRouter**

**Capa de abstracción:** Código que corre en **tu propio computador**. Tu le pides algo con una forma de llamar siempre igual, y ese código decide internamente como hablarle a cada proveedor. Ejemplo: **LangChain**, **LiteLLM**.

La diferencia clave

|               | Router                                                                         | Capa de abstracción                                                                                |
|---------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Donde corre   | En un servidor externo (de terceros)                                           | En tu propio computador                                                                            |
| Que necesitas | Una sola clave de API (la del router)                                          | Las claves de cada proveedor que uses                                                              |
| Analogia      | Un <b>conserje de hotel</b> que llama al taxi, al restaurante o al tour por ti | Un <b>traductor universal</b> que tú llevas encima y que sabe hablarle a cada persona en su idioma |

Ejemplo con el código que tienes

```
[ ] response = openrouter.chat.completions.create(model="z-ai/glm-4.5",
messages=tell_a_joke)
display(Markdown(response.choices[0].message.content))
Python
```

Aquí tú le hablas a **OpenRouter** (el router), y es OpenRouter quien internamente llama a GLM 4.5, un modelo chino de la startup z.ai. Tú nunca tuviste que crear una cuenta en z.ai ni conseguir su clave.

Por qué sirve un router

- Pagas una sola vez por adelantado (o usas capas gratis), en vez de manejar cuentas y claves con cada proveedor.
- Con un solo cliente (**openrouter**) accedes a muchísimos modelos distintos, cambiando solo el nombre: **"proveedor/modelo"**.

Por qué sirve una capa de abstracción

- Si mañana quieres cambiar de OpenAI a Anthropic, cambias poco código, porque la forma de llamar es siempre la misma.
- Corre en tu máquina, así que no dependes de que el servicio del router esté funcionando.

**En una línea:** las dos resuelven el mismo problema (no tener que aprender la forma particular de cada proveedor), pero el router lo resuelve fuera de tu código, y la capa de abstracción lo resuelve adentro.



```
response = openrouter.chat.completions.create(model="z-ai/glm-4.5",
messages=tell_a_joke)

de ese que sería capa de abstraccion?
```

Ninguna parte de esa línea es "capa de abstracción": esa línea usa un **router**.

La confusión es entendible porque se ven parecidas. **La diferencia está en qué hay detrás de ese openrouter**, no en cómo se escribe el código.

Cómo distinguirlos

```
python

response = openrouter.chat.completions.create(model="z-ai/glm-4.5", messages=tell_a_joke)
```

Aquí openrouter es un cliente OpenAI(...) apuntando a base\_url="https://openrouter.ai/api/v1". Cuando ejecutas esto, tu petición **sale de tu computador** y viaja por internet hasta el servidor de OpenRouter. Ese servidor (que no es tuyo) decide llamar a GLM 4.5. Por eso es un router: el que enruta la petición está afuera.

Cómo se vería con una capa de abstracción (LiteLLM, del mismo notebook)

```
python

from litellm import completion
response = completion(model="openai/gpt-4.1", messages=tell_a_joke)
```

Aquí **completion** es una funcion de una librería **instalada en tu computador**. Cuando la ejecutas, **LiteLLM mismo** (código corriendo en tu maquina) decide como armar la llamada a OpenAI y la hace directo, sin pasar por un servidor intermediario de tercero.

La diferencia real

|                                     | openrouter.chat.completions.create(...) | completion(...) de LiteLLM          |
|-------------------------------------|-----------------------------------------|-------------------------------------|
| Quien decide a que proveedor llamar | El servidor de OpenRouter (externo)     | La librería en tu propio computador |
| Tu petición pasa por un tercero     | Sí, por OpenRouter                      | No, va directo al proveedor         |
| Necesitas la clave de GLM/z.ai      | No, solo la de OpenRouter               | Si llamaras a GLM con LiteLLM, sí   |

**En una línea:** si tu código le habla a un servidor externo que reenvía la petición, es un router. Si tu código mismo arma y envía la petición al proveedor final, es una capa de abstracción.

## 2.3 día 1: LangChain, LiteLLM y caché de prompts

### 2.3.1. LangChain (capa de abstracción o framework): la capa de abstracción más famosa

El profesor es honesto sobre su opinión: dice ser un poco "negacionista" de LangChain. Reconoce partes que le gustan, pero en general le parece **pesada** (mucho que aprender). Promete enseñarla igual, sobre todo en la semana 5.

Cómo se ve:

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-5-mini")
response = llm.invoke(tell_a_joke)

display(Markdown(response.content))
```

✓ 11.0s

python

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-5-mini")
response = llm.invoke(tell_a_joke)
```

---

LangChain es una capa de abstracción (framework) para construir aplicaciones con LLMs. Te da piezas ya armadas para tareas comunes, así no tienes que escribir todo desde cero.

#### La idea central

En vez de llamar directamente a la API de cada proveedor, usas las "piezas" de LangChain, que por detrás saben hablarle a OpenAI, Anthropic, Gemini y otros.

```
python

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-5-mini")
response = llm.invoke(tell_a_joke)
```

- **ChatOpenAI(...)** crea un objeto que representa al modelo.
- **.invoke(...)** es el método para llamarlo (en vez de **chat.completions.create**).

Por qué existe

Construir una aplicación de IA real necesita más que una sola llamada al modelo: encadenar varios pasos, conectar documentos, dar memoria a una conversación, usar herramientas (como buscar en internet). LangChain trae piezas para todo eso, ya armadas.

Comparación con lo que ya viste

|                      | Librería de OpenAI directa          | LangChain                                                           |
|----------------------|-------------------------------------|---------------------------------------------------------------------|
| Cómo llamas          | openai.chat.completions.create(...) | llm.invoke(...)                                                     |
| Que trae             | Solo llamada básica                 | Piezas para conversaciones, documentos, memoria, herramientas, etc. |
| Curva de aprendizaje | Baja                                | Alta                                                                |

Para qué sirve en la práctica

Es muy usada en la industria para construir chatbots, asistentes que buscan en tus documentos (RAG) o agentes que usan herramientas. Por eso el profesor dice que, aunque a él no le encante, vale la pena conocerla, porque es probable que la encuentres en un trabajo.

capa de abstraccion es sinonimo de framework?

Sí, en este contexto son sinónimos. El profesor lo dice explícitamente al inicio de la transcripción: "capas de abstracción, también llamadas frameworks".

Por qué se usan los dos nombres

- “Capa de abstracción” describe **que hace**: esconde los detalles de cada proveedor y te da una forma uniforme de llamarlos a todos.
- “Framework” describe **que es**: una librería o conjunto de herramientas con una estructura propia que tu usas para construir tu aplicación.

Es como llamar "vehículo" o "auto" a la misma cosa: distinta palabra, mismo objeto.

Ejemplos que ya viste

| Nombre que le dan   | A qué se refiere   |
|---------------------|--------------------|
| Framework LLM       | LangChain, LiteLLM |
| Capa de abstracción | LangChain, LiteLLM |

Ambos términos apuntan a las mismas dos herramientas del notebook.

Dónde entra el router

Ojo, que no es sinónimo de **router**. Eso sí es otra cosa:

|             | Capa de abstracción / Framework | Router                 |
|-------------|---------------------------------|------------------------|
| Ejemplos    | LangChain, LiteLLM              | OpenRouter             |
| Dónde corre | En tu computador                | En un servidor externo |

|                    |                                             |                                         |
|--------------------|---------------------------------------------|-----------------------------------------|
| Ejemplos ya vistos | ChatOpenAI(...),<br>litellm.completion(...) | openrouter.chat.completions.create(...) |
|--------------------|---------------------------------------------|-----------------------------------------|

**En una línea:** "capa de abstracción" = "framework", y los dos son distintos de "router".

### 2.3.2. LiteLLM (capa de abstracción o framework): la capa de abstracción simple

LiteLLM es un framework (capa de abstracción) para LLMs, pensado para ser súper simple. El profesor dice que es su favorito y lo usa mucho.

#### La idea central

Te da una sola función para llamar a cualquier modelo de cualquier proveedor. En vez de aprender la forma particular de cada proveedor, usas siempre la misma.

```
from litellm import completion
response = completion(model="gemini/gemini-3.1-flash-lite", messages=tell_a_joke)
reply = response.choices[0].message.content
display(Markdown(reply))
```

- **Completion(...)** es la función que lo hace todo.
- Se parece a la forma de OpenAI, pero la función se llama **completion** en vez de **chat.completions.create**.
- Te devuelve lo mismo que devuelve OpenAI: **response.choices[0].message.content**.

#### Lo que lo hace especial: el formato del modelo

"proveedor/nombre-del-modelo"



| Si quieres usar | Escribes                       |
|-----------------|--------------------------------|
| Gemini          | "gemini/gemini-3.1-flash-lite" |
| OpenAI          | "openai/gpt-4.1"               |
| AWS Bedrock     | "bedrock/..."                  |
| Azure           | "azure/..."                    |
| Google Vertex   | "vertex_ai/..."                |

Cambias solo esa palabra y el resto del código **no cambia**. Así es fácil probar modelos distintos sin reescribir nada.

#### Su función extra: medir costos

Lo que lo hace realmente útil, según el profesor, es que te dice automáticamente cuánto gastaste en cada llamada:

```
python
print(f"Input tokens: {response.usage.prompt_tokens}")
print(f"Output tokens: {response.usage.completion_tokens}")
print(f"Total cost: {response._hidden_params['response_cost']*100:.4f} cents")
```

#### Comparación con LangChain

|                                         | LangChain                                           | LiteLLM                                        |
|-----------------------------------------|-----------------------------------------------------|------------------------------------------------|
| Tamaño/peso                             | Pesado, mucho que aprender                          | Ligero, muy simple                             |
| Para qué sirve además de llamar modelos | Memoria, documentos, herramientas, cadenas de pasos | Básicamente solo llamar modelos y medir costos |
| Opinión del profesor                    | No le encanta, pero la usa                          | Es su favorito                                 |

## Por qué sirve en la práctica

El profesor cuenta que lo ha usado en **sistemas de producción reales**, para llevar la cuenta de cuanto gasta cada cliente en llamadas de modelos, y comparar ese gasto contra los ingresos: control de costos a nivel de negocio.

---

Lo describe como el extremo opuesto a LangChain: **muy ligera**, con una interfaz simple para cualquier modelo.

```
from litellm import completion  Import "litellm" could not be resolved
response = completion(model="gemini/gemini-3.1-flash-lite", messages=tell_a_joke)
reply = response.choices[0].message.content
display(Markdown(reply))
```

Python

**El formato del modelo es proveedor/nombre-del-modelo.** Eso es lo que permite apuntar a Gemini, AWS Bedrock, Azure o Google Vertex solo cambiando ese texto, sin cambiar el resto del código.

### 2.3.3. Medir tokens y costo con LiteLLM

```
tell_a_joke = [
    {"role": "user", "content": "Cuéntame un chiste para un estudiante en el viaje para convertirse en experto en Ingeniería de LLM"},
]

print(f"Input tokens: {response.usage.prompt_tokens}")
print(f"Output tokens: {response.usage.completion_tokens}")
print(f"Total tokens: {response.usage.total_tokens}")
print(f"Total cost: {response._hidden_params["response_cost"]*100:.4f} cents")
```

1] ✓ 0.0s Python

Input tokens: 22  
Output tokens: 387  
Total tokens: 409  
Total cost: 0.0586 cents

En tu salida real: 22 tokens de entrada, 387 de salida, costo **0,0586 centavos** (es decir, \$0,000586).

El profesor insiste en una aclaración: cuando dice "\$0,023" **no** significa 2 centavos y tres décimas. Significa **2 diezmilésimas de dólar**, o sea casi nada. Esto conecta con lo que vimos antes: el costo por llamada individual es mínimo; lo que importa es la economía a gran escala.

```
print(f"Total cost: {response._hidden_params["response_cost"]*100:.4f} cents")
```

#### Paso 1: response.\_hidden\_params

Es un **diccionario** (datos guardados con clave y valor) que viene dentro de la respuesta de LiteLLM, con información extra que no es parte de la respuesta normal del modelo.

#### Paso 2: ["response\_cost"]

Saca del diccionario el valor guardado bajo la clave **"response\_cost"**. Es un número, el costo en **dólares** de esa llamada (por ejemplo, **0.00058553**).

#### Paso 3: \* 100

Multiplica por 100 para pasar de **dólares a centavos**. Dólares × 100 = centavos.

0.00058553 × 100 = 0.058553

#### Paso 4: :.4f

Es formato de impresión dentro de un f-string: muestra el número con **4 decimales**.

0.058553 → 0.0586

#### Tabla resumen

| Parte                                                 | Qué hace                 | Resultado del ejemplo      |
|-------------------------------------------------------|--------------------------|----------------------------|
| <code>response._hidden_params["response_cost"]</code> | Saca el costo en dólares | 0,00058553                 |
| <code>* 100</code>                                    | Pasa a centavos          | 0,058553                   |
| <code>:.4f</code>                                     | Redondea a 4 decimales   | 0,0586                     |
| Texto final                                           | Arma el mensaje          | "Total cost: 0.0586 cents" |



### 2.3.4: El experimento de Hamley (cache de prompts)

Son 4 llamadas seguidas, y cada una enseña algo distinto.

```
with open("hamlet.txt", "r", encoding="utf-8") as f:
    hamlet = f.read()

loc = hamlet.find("Speak, man")
print(hamlet[loc:loc+100])
```

Python

#### Línea 1-2: abrir y leer el archivo

```
python

with open("hamlet.txt", "r", encoding="utf-8") as f:
    hamlet = f.read()
```

| Parte                                                  | Qué hace                                                                                                                                                     |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>open("hamlet.txt", "r", encoding="utf-8")</code> | Abre el archivo <code>hamlet.txt</code> en modo lectura (" <code>r</code> "), con codificación UTF-8 (para que acentos y caracteres especiales se lean bien) |
| <code>with ... as f</code>                             | Abre el archivo, lo deja disponible como <code>f</code> , y lo <b>cierra automáticamente</b> al terminar el bloque, aunque haya un error                     |
| <code>f.read()</code>                                  | Lee <b>todo</b> el contenido del archivo de una vez, como un solo texto                                                                                      |
| <code>hamlet = ...</code>                              | Guarda ese texto completo en la variable <code>hamlet</code>                                                                                                 |

Después de esto, `hamlet` contiene el texto completo de la obra, como un string gigante (miles de caracteres).

#### Línea 3: buscar una frase

```
python

loc = hamlet.find("Speak, man")
```

- `.find("Speak, man")` busca esa frase exacta dentro del texto.
- Devuelve la **posición** (el índice del carácter) donde empieza esa frase.
- Si no la encontrara, devolvería -1.

**Ejemplo con un texto chico (inventado):** si `texto = "Hola, Speak, man, adiós"`, entonces `texto.find("Speak, man")` devuelve 6, porque "Speak" empieza en la posición 6 (contando desde 0).

#### Línea 4: mostrar un fragmento

```
python

print(hamlet[loc:loc+100])
```

- `hamlet[loc:loc+100]` es **slicing**: toma el texto desde la posición `loc` hasta `loc+100`, o sea, los **100 caracteres siguientes** a partir de donde empieza "Speak, man".

- print(...) lo muestra en pantalla.

### El resultado en tu notebook

```
Speak, man.  
Laer. Where is my father?  
King. Dead.  
Queen. But not by him!  
King. Let him deman
```

Es exactamente ese fragmento del guion de Hamlet, cortado justo a los 100 caracteres (por eso la última línea queda a medias: "Let him deman...").

### 2.3.4.1: Llamada 1: sin contexto

```
question = [{"role": "user", "content": "En Hamlet, cuando Laertes pregunta '¿Dónde está mi padre?'  
¿cuál es la respuesta?"}]
```

```
response = completion(model="gemini/gemini-3.1-flash-lite", messages=question)  
display(Markdown(response.choices[0].message.content))
```

**En el video:** el modelo inventa una respuesta ("Bendito el padre que tiene un hijo que sabe") que suena segura, pero está mal. El profesor lo marca como **alucinación confiada**: cuando el modelo no sabe algo, tiende a da la respuesta incorrecta con la misma confianza que si supiera, porque esa es la secuencia de tokens que le parece más probable.

**En tu notebook:** la respuesta salió **correcta** ("Dead", de Claudio). Es otra diferencia entre la corrida del profesor y la tuya; el mismo prompt puede dar resultados distintos en distintas ejecuciones.

Cuando Laertes entra enreducido en el castillo preguntando: ¿Dónde

«Ha muerto».

Inmediatamente después, Gertrudis (la reina) interviene rápidamente para aclarar: «**Pero no a manos de él**» (refiriéndose a que no fue el rey quien lo mató), tratando de calmar la ira de Laertes antes de que lo acuse directamente.

```
print(f"Input tokens: {response.usage.prompt_tokens}")  
print(f"Output tokens: {response.usage.completion_tokens}")  
print(f"Total tokens: {response.usage.total_tokens}")  
print(f"Total cost: {response._hidden_params["response_cost"]*100:.4f} cents")
```

```
... Input tokens: 22  
Output tokens: 97  
Total tokens: 119  
Total cost: 0.0151 cents
```

|             | Tokens entrada | Tokens salida | Costo           |
|-------------|----------------|---------------|-----------------|
| Video       | 19             | 103           | \$0,00004       |
| Tu notebook | 22             | 97            | 0,0151 centavos |

### 2.3.4.2: Llamada 2: con el texto completo de Hamlet

```
question[0]["content"] += "\n\nPara el contexto, aquí está el texto completo de Hamlet:\n\n"+hamlet

response = completion(model="gemini/gemini-3.1-flash-lite", messages=question)
display(Markdown(response.choices[0].message.content))
```

Ahora sí responde bien, con cita exacta y acto/escena. El costo sube mucho porque entra todo Hamlet como contexto.

Salida:

```
En la obra de William Shakespeare, la escena a la que haces referencia ocurre en el Acto IV, Escena V.

Cuando Laertes, enfurecido por la muerte de su padre Polonio, irrumpe en el castillo y grita "¿Dónde está mi padre?" (Where is my father?), la respuesta del Rey Claudio es corta y directa:

King. Dead. (Muerto.)

Inmediatamente después, la Reina Gertrudis añade: "But not by him!" (¡Pero no por él!), tratando de aclarar que el Rey no es el responsable de la muerte de Polonio, para evitar que Laertes se vuelva contra él.

print(f"Input tokens: {response.usage.prompt_tokens}")
print(f"Output tokens: {response.usage.completion_tokens}")
print(f"Cached tokens: {response.usage.prompt_tokens_details.cached_tokens}")
print(f"Total cost: {response._hidden_params["response_cost"]*100:.4f} cents")

Input tokens: 53212
Output tokens: 148
Cached tokens: None
Total cost: 1.3525 cents
```

|             | Tokens entrada | Tokens salida | Costo                  |
|-------------|----------------|---------------|------------------------|
| Video       | —              | —             | medio centavo          |
| Tu notebook | <b>53.212</b>  | 148           | <b>1,3525 centavos</b> |

**Cached tokens: None** (o 0): es la primera vez que se manda ese texto, así que no hay nada en caché todavía.

```
question = [{"role": "user", "content": "En Hamlet, cuando Laertes pregunta '¿Dónde está mi padre?'  
¿cuál es la respuesta?"}]

question[0]["content"] += "\n\nPara el contexto, aquí está el texto completo de Hamlet:\n\n"+hamlet
```

El `question = [...]` crea la variable **question** desde cero: una lista con un diccionario adentro. Es la primera vez que existe.

El `question[0]["content"] += ...` No crea nada nuevo. Entra a la lista que ya existe y le agrega texto al final

del “content” que ya tenía.

- **Question[0]** -> el primer (y único) diccionario de la lista.
- **["content"]** -> el valor de esa clave, que hoy es la pregunta sobre Laertes.
- **+=** → le pega más texto al final, sin borrar lo que ya había.

## Antes y después

**Antes (question[0]["content"]):**

En Hamlet, cuando Laertes pregunta '¿Dónde está mi padre?' ¿cuál es la respuesta?

**Después (question[0]["content"]):**

En Hamlet, cuando Laertes pregunta '¿Dónde está mi padre?' ¿cuál es la respuesta? 

Para el contexto, aquí está el texto completo de Hamlet:

[...todo el texto de hamlet.txt...]

**En una tabla**

|                                                             | Qué hace                                           | Resultado                                                                     |
|-------------------------------------------------------------|----------------------------------------------------|-------------------------------------------------------------------------------|
| <code>question = [...]</code>                               | Crea la lista con la pregunta corta                | <code>question</code> existe por primera vez                                  |
| <code>question[0]</code><br><code>["content"] += ...</code> | Agrega el texto de Hamlet al final de esa pregunta | <code>question</code> sigue siendo la misma lista, pero con más texto adentro |

Por eso en el notebook, la llamada 2 (sin el +=) usa la pregunta corta, y la llamada 3 en adelante (después del +=) ya manda la pregunta junto con todo Hamlet pegado.

### 2.3.4.3: Llamada 3: la misma pregunta, otra vez

```
response = completion(model="gemini/gemini-3.1-flash-lite", messages=question)
display(Markdown(response.choices[0].message.content))
```

Python

En la tragedia de *Hamlet* de William Shakespeare, cuando Laertes pregunta "¿Dónde está mi padre?" en el **Acto IV, escena V**, la respuesta del **Rey Claudio** es simplemente:

**"Dead."** (Muerto).

Posteriormente, la Reina Gertrudis añade inmediatamente: **"But not by him!"** (¡Pero no por él!), refiriéndose a que Hamlet no fue el asesino (aunque en ese punto de la historia, Laertes aún no sabe que fue Hamlet quien mató a Polonio, y el Rey intenta desviar la culpa).

```
print(f"Input tokens: {response.usage.prompt_tokens}")
print(f"Output tokens: {response.usage.completion_tokens}")
print(f"Cached tokens: {response.usage.prompt_tokens_details.cached_tokens}")
print(f"Total cost: {response._hidden_params["response_cost"]*100:.4f} cents")
```

Input tokens: 53212  
Output tokens: 124  
Cached tokens: 49124  
Total cost: 0.2436 cents

Antes:

```
Input tokens: 53212
Output tokens: 148
Cached tokens: None
Total cost: 1.3525 cents
```

Mismos 53.212 tokens de entrada, pero ahora:

```
Cached tokens: 49124
Total cost: 0.2436 cents
```

$1,3525 / 0,2436 \approx 5,6$  veces más barato, muy cerca de las "unas cinco veces menos" que dice el profesor.

Por qué pasó

El sistema detectó que ya habías mandado ese mismo texto hace poco, y reutilizó el cálculo guardado (lo que vimos antes como KV cache), cobrando mucho menos por esa parte repetida.

### 2.3.5 La letra chica del cache, por proveedor: IMPORTANTE – para sacarle el máximo provecho al rendimiento

| Proveedor | Cómo funciona                                                                                                                                                             |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OpenAI    | Automático. Solo funciona si el <b>inicio</b> del prompt coincide exactamente. Contenido estático al principio, lo variable (fecha, usuario) al final. 4 veces más barato |
| Anthropic | Hay que decirle explícitamente qué cachear. Pagas 25 % más para "cebar" el caché, y después 10 veces menos al reutilizarlo                                                |
| Gemini    | Soporta modo implícito (automático) y explícito                                                                                                                           |

Habla de **cómo cada proveedor cobra menos cuando le mandas texto repetido**, y que la forma de activarlo cambia según el proveedor.

#### La idea general (ya la vimos con Hamlet)

Si le mandas el mismo texto largo dos veces seguidas, el proveedor puede reutilizar calculo que ya hizo antes, en vez de recalcular todo de cero. Eso te cobra mucho menos la segunda vez.

#### OpenAI: automático, pero con una regla estricta

- "Los aciertos de caché solo son posibles para coincidencias exactas de **prefijo**"

Significa que el inicio del prompt debe ser **idéntico, carácter por carácter**, hasta el punto donde quieres que se corte el cache.

| Orden del prompt                                       | ¿Activa cache?                      |
|--------------------------------------------------------|-------------------------------------|
| Instrucciones fijas al inicio + dato variable al final | Si                                  |
| Dato variable al inicio + instrucciones fijas al final | No, porque el inicio cambia siempre |

**Costo:** lo cacheado sale **4 veces más barato**.

**Ejemplo con la fecha (el error típico que mencionó el profesor):**

- Mal: "**Hoy es 6 de octubre. Eres un asistente útil...**" → el inicio cambia cada día, nunca hay cache.
- Bien: "**Eres un asistente útil... Hoy es 6 de octubre**" → el inicio queda fijo, si hay cache.

#### Anthropic: manual, con un costo inicial

- **No es automático.** Tienes que decirle explícitamente a Claude que parte quieres cachear.
- **Pagas 25% mas** la primera vez, para "preparar" (cebar) el cache.
- **Pagas 10 veces menos** las veces siguientes, al reutilizarlo.

**Ejemplo con números inventados:** si el prompt normal cuesta \$1:

- Primera llamada (cebando caché):  $\$1 \times 1,25 = \$1,25$
- Sigüientes llamadas (reutilizando):  $\$1 / 10 = \$0,10$

Si lo usas muchas veces, sale mucho más barato a pesar del costo inicial.

#### Gemini: las dos formas

- **Implícito:** automático, como OpenAI.
- **Explícito:** tú decides qué cachear, como Anthropic.

El notebook no entra en detalle y solo deja el enlace a la documentación.

## Por qué importa esta comparación

| Proveedor | ¿Automático?          | ¿Costo extra inicial? | Ahorro        |
|-----------|-----------------------|-----------------------|---------------|
| OpenAI    | Sí                    | No                    | 4×            |
| Anthropic | No, hay que indicarlo | Sí, +25%              | 10×           |
| Gemini    | Ambos modos           | Depende del modo      | No especifica |

Esto conecta directo con el experimento de Hamlet que vimos: ahí usaron Gemini, y el caché se activó solo (sin configurarlo a mano), lo que calza con que Gemini también soporta el modo implícito.

## Ejemplo:

El error clásico que menciona el profesor: si pones la fecha de hoy al inicio del prompt, nunca vas a tener caché, porque el inicio cambia cada día. La fecha debe ir al final.

## Mal: la fecha al inicio

```
python

system_prompt = f"""
Hoy es {fecha_actual}.
Eres un asistente útil que responde preguntas sobre literatura clásica.
Responde siempre en español, con citas exactas cuando sea posible.
Sé conciso y directo.
"""
```

Qué pasa cada día:

| Día          | Prompt generado (inicio) |
|--------------|--------------------------|
| 6 de octubre | "Hoy es 6 de octubre..." |
| 7 de octubre | "Hoy es 7 de octubre..." |

El inicio **cambia todos los días**, así que OpenAI nunca encuentra una coincidencia de prefijo con lo que mandaste antes. **Nunca hay cache**, aunque el resto del prompt (las instrucciones, que son largas) sea siempre igual.

pero el modelo siempre por cada fecha leería "Eres un asistente útil que responde preguntas sobre literatura clásica. Responde siempre en español, con citas exactas cuando sea posible. Sé conciso y directo."

y eso no estaría aun así en cache si es lo mismo?

Buena observación, pero no. **El cache de OpenAI no busca "texto parecido en algún lado"; busca "los**

mismos caracteres, desde la posición 0".

## Por qué no sirve, aunque el resto sea idéntico

```
"Hoy es 6 de octubre.\nEres un asistente útil..."
```

```
"Hoy es 7 de octubre.\nEres un asistente útil..."
```



El sistema de cache compara **carácter por carácter, empezando desde el principio**:

| Posición | Prompt del día 6 | Prompt del día 7 | ¿Coinciden? |
|----------|------------------|------------------|-------------|
| 1        | H                | H                | Sí          |
| 2        | o                | o                | Sí          |
| ...      | ...              | ...              | Sí          |
| 9        | 6                | 7                | No          |

En el carácter donde esta el numero del día, **ya se rompe la coincidencia**. Y como el cache exige la coincidencia de **prefijo** (desde el inicio, sin saltos), todo lo que viene después – aunque sea idéntico – ya no cuenta como “mismo prefijo”.

## La analogía

Es como fotocopiar un documento: si la primera línea es distinta, el sistema no se pone a buscar "a ver si el resto del documento se parece a otro que ya tengo". Simplemente dice "este documento es nuevo" y lo procesa entero de nuevo.

## Qué sí necesitarías para que funcione como tú imaginas

Un sistema que **entienda el significado** y encuentre que “las instrucciones de literatura” son las mismas aunque la fecha cambie. Eso no es cache, es otra cosa (**mas parecido a lo que hace un modelo al razonar**). **El cache de prefijo es mecánico y literal**: no entiende nada, solo compara texto carácter por carácter desde el inicio.

## Por eso la solución es tan simple

Mover la fecha al final no es un truco raro: es aprovechar que, si la parte que cambia queda al final, **todo lo de antes sí es una coincidencia de prefijo perfecta**, y esa parte sí se cachea.

## Bien: la fecha al final

```
python
```

```
system_prompt = f"""
Eres un asistente útil que responde preguntas sobre literatura clásica.
Responde siempre en español, con citas exactas cuando sea posible.
Sé conciso y directo.

Hoy es {fecha_actual}.
"""
```



Qué pasa cada día:



| Día          | Prompt generado (inicio)               |
|--------------|----------------------------------------|
| 6 de octubre | "Eres un asistente útil..."            |
| 7 de octubre | "Eres un asistente útil..." (idéntico) |

El inicio es **siempre el mismo**, solo cambia el final. OpenAI detecta que ese prefijo largo ya lo vio antes, y te cobra 4 veces menos por esa parte, todos los días.

## Con el ejemplo de Hamlet que vimos

Es la misma lógica, pero llevada al extremo:

| Orden                                      | Que pasa                                                                                  |
|--------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Pregunta + luego el texto de Hamlet</b> | Mal: el inicio cambia según la pregunta                                                   |
| <b>Texto de Hamlet + luego la pregunta</b> | Bien: el inicio (Hamlet, 53.000 tokens) siempre es igual, y solo la pregunta final cambia |

Por eso, si quisieras aprovechar el caché haciendo varias preguntas distintas sobre Hamlet, deberías armar el prompt como:

```
python
contenido = hamlet + "\n\nPregunta: " + pregunta_del_usuario
```

y no al revés, porque así Hamlet queda fijo al inicio en todas las llamadas, y solo la pregunta cambia al final.

## 2.3.6 Conversación entre dos chatbots: paso a paso del código

### 2.3.6.1 La configuración inicial

Y ahora para un poco de diversión - ¡una conversación adversarial entre Chatbots!

Ya estás familiarizado con los prompts organizados en listas como:

```
[
  {"role": "system", "content": "mensaje del sistema aquí"},
  {"role": "user", "content": "prompt del usuario aquí"}
]
```

De hecho, esta estructura se puede usar para reflejar un historial de conversación más largo:

```
[
  {"role": "system", "content": "mensaje del sistema aquí"},
  {"role": "user", "content": "primer prompt del usuario aquí"},
  {"role": "assistant", "content": "la respuesta del asistente"},
  {"role": "user", "content": "el nuevo prompt del usuario"},
]
```

De hecho, esta estructura se puede usar para reflejar un historial de conversación más largo:

```
gpt_model = "gpt-4.1-mini"
claude_model = "claude-haiku-4-5"

gpt_system = "Eres un chatbot que es muy argumentativo; \
desacuerdas con cualquier cosa en la conversación y cuestionas todo, de una manera sarcástica."

claude_system = "Eres un chatbot muy educado y cortés. Intentas estar de acuerdo con \
todo lo que dice la otra persona, o encontrar terreno común. Si la otra persona es argumentativa, \
intentas calmarla y seguir charlando."

gpt_messages = ["Hola allá"]
claude_messages = ["Hola"]
```

- **Gpt\_system** y **claude\_system** son los “guiones de personalidad” de cada modelo.
- **Gpt\_messages** y **claude\_messages** son dos listas **separadas**, con lo que ha dicho cada uno hasta ahora. Cada una arranca con un saludo inicial.
- El \ al final de línea solo permite seguir escribiendo el texto en la línea de abajo (no corta el string).

### 2.3.6.2 La funcion call\_gpt(): paso a paso

```
def call_gpt():
    messages = [{"role": "system", "content": gpt_system}]
    for gpt, claude in zip(gpt_messages, claude_messages):
        messages.append({"role": "assistant", "content": gpt})
        messages.append({"role": "user", "content": claude})
    response = openai.chat.completions.create(model=gpt_model, messages=messages)
    return response.choices[0].message.content
```

**Paso 1:** messages = [{"role": "system", "content": gpt\_system}] → arranca la lista con la personalidad de GPT.

**Paso 2:** zip(gpt\_messages, claude\_messages) → empareja ambas listas, turno por turno.

**Paso 3:** dentro del for, por cada turno:

- Lo que dijo GPT antes → se guarda como **“assistant”** (porque, desde el punto de vista de GPT, el mismo es el asistente).
- Lo que dijo Claude antes → se guarda como **“user”** (porque, desde el punto de vista de GPT, Claude es “el usuario” que le habla).

**Paso 4:** openai.chat.completions.create(...) → llama al modelo con toda esa lista armada.

**Paso 5:** return response.choices[0].message.content → devuelve solo el texto de la respuesta.

**Ejemplo con el primer estado (gpt\_messages = ["Hola allá"], claude\_messages = ["Hola"])**

**zip** empareja ("Hola allá", "Hola"), una sola vuelta:

```
python

messages = [
    {"role": "system", "content": gpt_system},
    {"role": "assistant", "content": "Hola allá"},
    {"role": "user", "content": "Hola"}
]
```

Esto le dice a GPT: "tú ya dijiste 'Hola allá', y el usuario te respondió 'Hola'; ahora sigue tú". Por eso GPT respondió algo sarcástico tipo "¿No te molestaste en saludarme? Patético."

---

```
gpt_system = "Eres un chatbot que es muy argumentativo; \
desacuerdas con cualquier cosa en la conversación y cuestionas todo, de una manera sarcástica."
```

```
gpt_messages = ["Hola allá"]
claude_messages = ["Hola"]
```

```
def call_gpt():
    messages = [{"role": "system", "content": gpt_system}]
    for gpt, claude in zip(gpt_messages, claude_messages):
        messages.append({"role": "assistant", "content": gpt})
        messages.append({"role": "user", "content": claude})
    response = openai.chat.completions.create(model=gpt_model, messages=messages)
    return response.choices[0].message.content
```

```
call_gpt()
```

## call\_gpt(), paso a paso

### Línea 1

```
python

messages = [{"role": "system", "content": gpt_system}]
```

Crea la lista **messages** con un solo elemento: el mensaje de sistema, que define la personalidad de GPT (argumentativo, sarcástico).

### Línea 2: recorre el historial

```
python

for gpt, claude in zip(gpt_messages, claude_messages):
```

**Zip(gpt\_messages, Claude\_messages)** empareja las dos listas, posición por posición. En cada vuelta:

- **Gpt** = un mensaje que dijo GPT antes
- **Claude** = el mensaje con el que Claude le respondió, en ese mismo turno.

### Líneas 3-4: agrega el par, con roles invertidos

```
python

messages.append({"role": "assistant", "content": gpt})
messages.append({"role": "user", "content": claude})
```

- Lo que dijo **GPT** → se guarda como **"assistant"** (porque desde el punto de vista de GPT, el mismo es el asistente).
- Lo que dijo **Claude** → se guarda como "user" (porque desde el punto de vista de GPT, Claude es "el usuario" que le habla).

## Ejemplo con el estado inicial del notebook

`gpt_messages = ["Hola allá"], claude_messages = ["Hola"]`. Listas de tamaño 1, así que zip da **una sola vuelta**.

`messages` queda así:

```
python
[
    {"role": "system", "content": gpt_system},
    {"role": "assistant", "content": "Hola allá"},
    {"role": "user", "content": "Hola"}
]
```

Son 3 elementos. Esto es lo que se **manda** al modelo.

#### Línea 5: llama al modelo

```
python
response = openai.chat.completions.create(model=gpt_model, messages=messages)
```

Manda esos 3 mensajes. El modelo lee todo y genera una respuesta nueva.

#### Línea 6: devuelve solo el texto

```
python
return response.choices[0].message.content
```

- `Response.choices[0]` → la respuesta generada
- `.message.content` → solo el texto de esa respuesta.

Esa salida siempre viene marcada como **role: “assistant”** por la API (eso lo decide OpenAI, no tu), y es distinta de los 3 mensajes que mandaste.

**Ejemplo de lo que devuelve, en este caso:**

```
"¿No te molestaste en saludarme? Patético. Esfuérzate más."
```

### Resumen visual

| Paso | Línea                                            | Qué hace                                                                               |
|------|--------------------------------------------------|----------------------------------------------------------------------------------------|
| 1    | <code>messages = [...]</code>                    | Pone la personalidad de GPT                                                            |
| 2    | <code>for gpt, claude in zip(...)</code>         | Recorre el historial turno por turno                                                   |
| 3-4  | <code>.append(...)</code> × 2                    | Agrega cada turno, con GPT como <code>assistant</code> y Claude como <code>user</code> |
| 5    | <code>openai.chat.completions.create(...)</code> | Manda todo y el modelo genera una respuesta nueva                                      |
| 6    | <code>return ...content</code>                   | Devuelve solo el texto de esa respuesta nueva                                          |

### 2.3.6.3: La función `call_claude()`: la misma idea, invertida

```
gpt_messages = ["Hola allá"]
claude_messages = ["Hola"]
```

```
claude_system = "Eres un chatbot muy educado y cortés. Intentas estar de acuerdo con \
todo lo que dice la otra persona, o encontrar terreno común. Si la otra persona es argumentativa, \
intentas calmarla y seguir charlando."
```

```
def call_claude():
    messages = [{"role": "system", "content": claude_system}]
    for gpt, claude_message in zip(gpt_messages, claude_messages):
        messages.append({"role": "user", "content": gpt})
        messages.append({"role": "assistant", "content": claude_message})
    messages.append({"role": "user", "content": gpt_messages[-1]})
    response = anthropic.chat.completions.create(model=claude_model, messages=messages)
    return response.choices[0].message.content
```

### `call_claude()`, paso a paso

#### Línea 1

```
python
messages = [{"role": "system", "content": claude_system}]
```

Crea la lista con la personalidad de Claude (educado, busca puntos en común).

#### Línea 2: recorre el historial

```
python
for gpt, claude_message in zip(gpt_messages, claude_messages):
```

Empareja las dos listas turno por turno. **gpt** = lo que dijo GPT en ese turno; **claude\_message** = lo que Claude respondió en ese mismo turno.

#### Líneas 3-4: agrega el par, con roles invertidos (al revés que en `call_gpt()`)

```
python
messages.append({"role": "user", "content": gpt})
messages.append({"role": "assistant", "content": claude_message})
```

- Lo que dijo **GPT** → **"user"** (desde la vista de Claude, GPT es quien le habla).
- Lo que dijo **Claude** → **"assistant"** (Claude se ve a si mismo como el asistente).

## Ejemplo con el estado inicial

`gpt_messages = ["Hola allá"], claude_messages = ["Hola"]`. `zip` da una vuelta.

Después de las líneas 3-4, `messages` lleva:

```
python
[
    {"role": "system", "content": claude_system},
    {"role": "user", "content": "Hola allá"},
    {"role": "assistant", "content": "Hola"}
]
```

**Línea 5: agrega el último mensaje de GPT, suelto**

```
python
messages.append({"role": "user", "content": gpt_messages[-1]})
```

`gpt_messages[-1]` es el último elemento de esa lista. Se agrega **de nuevo**, como un turno extra al final.

**Por qué hace falta:** en el bucle del notebook, siempre se llama primero `call_gpt()` y después `call_claude()`. Cuando le toca a Claude, GPT ya dijo algo nuevo que Claude todavía no vio. En ese momento `gpt_messages` tiene un **elemento mas** que `Claude_messages`, así que `zip` deja ese ultimo mensaje de GPT afuera (porque `zip` solo cubre hasta donde ambas listas alcanzan). Esta línea lo agrega aparte, para que Claude lo vea y pueda responderle.

**Con el estado inicial, antes de cualquier ronda del bucle**, `gpt_messages[-1]` es "Hola allá" (el único elemento), así que se repite:

```
python
[
    {"role": "system", "content": claude_system},
    {"role": "user", "content": "Hola allá"},
    {"role": "assistant", "content": "Hola"},
    {"role": "user", "content": "Hola allá"}
]
```

**Pero en el flujo real del bucle**, cuando se llama `call_claude()` justo después de que `call_gpt()` generó una respuesta nueva (por ejemplo "¿No te molestaste en saludarme?"), esa respuesta ya se agregó a `gpt_messages` antes de llamar a `call_claude()`. Entonces `gpt_messages = ["Hola allá", "¿No te molestaste en saludarme?"]` y `claude_messages = ["Hola"]` (todavía). Ahí `zip` solo empareja la primera posición, y la línea 5 agrega el mensaje nuevo de GPT que todavía no tiene respuesta:

```
python

[
    {"role": "system", "content": claude_system},
    {"role": "user", "content": "Hola allá"},
    {"role": "assistant", "content": "Hola"},
    {"role": "user", "content": "¿No te molestaste en saludarme?"}
]
```

Ahora Claude ve la conversación completa, terminando justo en lo último que GPT dijo, listo para responder.

**Líneas 6-7: llama al modelo y devuelve el texto**

```
python

response = anthropic.chat.completions.create(model=claude_model, messages=messages)
return response.choices[0].message.content
```

Manda esos mensajes a Claude, y devuelve solo el texto de la respuesta nueva (que la API siempre marca como **assistant**, sin que tú lo decidas)

**Comparación rápida con call\_gpt()**

|                        | Call_gpt()                                                             | Call_claude()                                                        |
|------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------|
| GPT se guarda como     | Assistant                                                              | User                                                                 |
| Claude se guarda como  | User                                                                   | Assistant                                                            |
| Linea extra al final   | No tiene                                                               | Sí: agrega el último mensaje de GPT suelto                           |
| Por qué la línea extra | No hace falta, porque cuando le toca a GPT las listas ya están parejas | Hace falta, porque cuando le toca a Claude, GPT ya habló una vez más |



#### 2.3.6.4: "Bucle de la conversación (5 rondas GPT ↔ Claude)" + Puntos de vista de gpt y claude + crecer de forma cuadrativa

```
gpt_messages = ["Hi there"]
claude_messages = ["Hi"]

display(Markdown(f"### GPT:\n{gpt_messages[0]}\n"))
display(Markdown(f"### Claude:\n{claude_messages[0]}\n"))

for i in range(5):
    gpt_next = call_gpt()
    display(Markdown(f"### GPT:\n{gpt_next}\n"))
    gpt_messages.append(gpt_next)

    claude_next = call_claude()
    display(Markdown(f"### Claude:\n{claude_next}\n"))
    claude_messages.append(claude_next)
```

de nuevo, pero en un bucle de 10 por ejemplo

Con un bucle de 10 rondas, cada ronda agrega un mensaje de GPT y uno de Claude. Al final hay 1 mensaje inicial + 10 de GPT + 10 de Claude = 21 mensajes en cada lista.

#### Las listas después del bucle completo

```
python

gpt_messages = [
    "Hi there",          # inicial
    "respuesta 1 de GPT",
    "respuesta 2 de GPT",
    ...
    "respuesta 10 de GPT"
] # 11 elementos

claude_messages = [
    "Hi",                # inicial
    "respuesta 1 de Claude",
    "respuesta 2 de Claude",
    ...
    "respuesta 10 de Claude"
] # 11 elementos
```

#### Después el punto de vista de GPT (dentro de call\_gpt() en la última ronda)

Cuando se llama **call\_gpt()** en la ronda 10, las listas tienen **10 elementos cada una** (antes de que GPT hable esta vez), así que **zip** da 10 vueltas parejas.

```
python
messages = [
    {"role": "system", "content": gpt_system},
    {"role": "assistant", "content": "Hi there"},
    {"role": "user", "content": "Hi"},
    {"role": "assistant", "content": "respuesta 1 de GPT"},
    {"role": "user", "content": "respuesta 1 de Claude"},
    {"role": "assistant", "content": "respuesta 2 de GPT"},
    {"role": "user", "content": "respuesta 2 de Claude"},
    ...
    {"role": "assistant", "content": "respuesta 9 de GPT"},
    {"role": "user", "content": "respuesta 9 de Claude"}
]
```

### Desde el punto de vista de Claude (dentro de call\_claude()) en esa misma ronda 10)

En ese momento, GPT **ya dijo** su respuesta 10 (se ejecutó primero y se agregó a gpt\_messages), pero Claude **todavía no respondió a eso**. Entonces gpt\_messages tiene 11, claude\_messages tiene 10. zip da solo 10 vueltas parejas, y la última línea rescata el mensaje suelto de GPT:

```
python
messages = [
    {"role": "system", "content": claude_system},
    {"role": "user", "content": "Hi there"},
    {"role": "assistant", "content": "Hi"},
    {"role": "user", "content": "respuesta 1 de GPT"},
    {"role": "assistant", "content": "respuesta 1 de Claude"},
    {"role": "user", "content": "respuesta 2 de GPT"},
    {"role": "assistant", "content": "respuesta 2 de Claude"},
    ...
    {"role": "user", "content": "respuesta 9 de GPT"},
    {"role": "assistant", "content": "respuesta 9 de Claude"},
    {"role": "user", "content": "respuesta 10 de GPT"}
]
```

**22 mensajes:** 1 system + 10 pares (user, assistant) + 1 mensaje suelto de GPT al final (la línea gpt\_messages[-1]).

### Tabla resumen, por ronda

| Ronda | Tamaño gpt_messages al llamar call_gpt() | Tamaño claude_messages al llamar call_gpt() | Tamaño gpt_messages al llamar call_claude() | Tamaño claude_messages al llamar call_claude() |
|-------|------------------------------------------|---------------------------------------------|---------------------------------------------|------------------------------------------------|
| 1     | 1                                        | 1                                           | 2                                           | 1                                              |
| 2     | 2                                        | 2                                           | 3                                           | 2                                              |
| ...   | ...                                      | ...                                         | ...                                         | ...                                            |
| 10    | 10                                       | 10                                          | 11                                          | 10                                             |

**El patrón se repite igual en cada ronda:** cuando le toca a GPT, las listas están parejas. Cuando le toca a Claude, gpt\_messages siempre tiene uno más, por eso la línea extra se necesita en todas las rondas, no solo en la primera.

por ejemplo cuando esta en el bucle 2 se vuelve como a repetir todo? y en el bucle 3 tambien?

**Sí, exacto.** En cada vuelta del for, `call_gpt()` y `call_claude()` **reconstruyen messages desde cero**, con toda la conversación acumulada hasta ese momento. No siguen desde donde quedaron; arman la lista entera otra vez.

### Por qué pasa esto

Mira el código de `call_gpt()`:

```
def call_gpt():
    messages = [{"role": "system", "content": gpt_system}] # ← empieza vacía cada vez
    for gpt, claude in zip(gpt_messages, claude_messages):
        messages.append(...)
    ...
```

Cada vez que se **llama a `call_gpt()`**, la línea `messages = [...]` crea una lista **nueva**, y el **for** interno la rellena recorriendo **todo** lo que hay en `gpt_messages` y `Claude_messages` hasta ese punto. Esto es justo lo que vimos con “la ilusión de memoria”: el modelo no recuerda nada entre llamadas, así que hay que mandarle siempre el historial completo de nuevo.

### Cómo crece en cada ronda del `for i in range(5)`

| Ronda | Que se manda a <code>call_gpt()</code> | Que se manda a <code>call_claude()</code> |
|-------|----------------------------------------|-------------------------------------------|
| 1     | system + 1 par (inicial)               | system + 1 par + 1 mensaje suelto         |
| 2     | system + 2 pares                       | system + 2 pares + 1 mensaje suelto       |
| 3     | system + 3 pares                       | system + 3 pares + 1 mensaje suelto       |
| 4     | system + 4 pares                       | system + 4 pares + 1 mensaje suelto       |
| 5     | system + 5 pares                       | system + 5 pares + 1 mensaje suelto       |

### Qué significa “se repite todo”

Sí: en la ronda 3, por ejemplo, se le manda a GPT **toda** la conversación de las rondas 1 y 2 otra vez, más lo nuevo. No es que el modelo “siga leyendo” desde donde iba; cada llamada es una conversación completa, reenviada entera.

**Consecuencia de costo:** como vimos antes con la ilusión de memoria, esto significa que la cantidad de tokens que se manda crece en cada ronda, porque siempre se reenvía todo lo anterior más lo nuevo.

y eso sube de manera cuadrática porque tiene que volver a pasar todo otra vez

**Si, exactamente.** Es la misma idea que vimos antes con la ilusión de memoria, aplicada aquí.

Por qué es cuadrático

En cada ronda, se manda **toda** la conversación acumulada hasta ese punto. Si cada mensaje pesa aproximadamente lo mismo (tokens inventados, digamos ~30 tokens por mensaje), la entrada de cada ronda crece así:

| Ronda | Mensajes en el historial | Tokens de entrada (aprox.) |
|-------|--------------------------|----------------------------|
| 1     | 3                        | 90                         |
| 2     | 5                        | 150                        |
| 3     | 7                        | 210                        |
| 4     | 9                        | 270                        |
| 5     | 11                       | 330                        |

Es la cuenta de **cuántos tokens se mandan como entrada en cada ronda**, porque el historial crece en cada vuelta.

Columna "Mensajes en el historial"

Cuenta cuantos mensajes hay en **messages** (system + lo acumulado) cada vez que se llama a **call\_gpt()**. Con el system prompt + 1 par nuevo por ronda:

| Ronda | Qué contiene                 | Cantidad de mensajes |
|-------|------------------------------|----------------------|
| 1     | system + 1 par (GPT, Claude) | 1 + 2 = 3            |
| 2     | system + 2 pares             | 1 + 4 = 5            |
| 3     | system + 3 pares             | 1 + 6 = 7            |
| 4     | system + 4 pares             | 1 + 8 = 9            |
| 5     | system + 5 pares             | 1 + 10 = 11          |

Cada ronda agrega 2 mensajes más que la anterior (1 de GPT, 1 de Claude).

Columna "Tokens de entrada (aprox.)"

Es un supuesto inventado: **cada mensaje pesa ~30 tokens**. Multiplcias la cantidad de mensajes por 30.

| Ronda | Mensajes | Cuenta  | Tokens |
|-------|----------|---------|--------|
| 1     | 3        | 3 × 30  | 90     |
| 2     | 5        | 5 × 30  | 150    |
| 3     | 7        | 7 × 30  | 210    |
| 4     | 9        | 9 × 30  | 270    |
| 5     | 11       | 11 × 30 | 330    |

Qué muestra la tabla

Que el tamaño de lo que se manda **no es fijo**: crece ronda tras ronda, porque cada llamada reenvía todo el

historial anterior más lo nuevo. En la ronda 5 se manda casi 4 veces más que en la ronda 1 ( $330 / 90 \approx 3,7$ ).

### Para qué sirvió esta tabla

Para mostrar que, si sumas los tokens de **todas** las rondas ( $90+150+210+270+330 = 1.050$ ), ese total crece más rápido que si simplemente fueran 5 rondas de tamaño fijo. Esa suma creciente es lo que lleva al crecimiento cuadrático que vimos después.

**Sumando los tokens de entrada de las 5 rondas:**  $90+150+210+270+330 = 1.050$  tokens, solo de entrada, solo para `call_gpt()` (y lo mismo le pasa a `call_claude()`, aparte).

### La fórmula

Si cada mensaje pesa  $m$  tokens y hay  $N$  rondas, la suma total de tokens de entrada es proporcional a:

$$m \times (1 + 2 + 3 + \dots + N) = m \times N(N+1)/2$$



Esa suma crece con **N al cuadrado** ( $N^2$ ), no en línea recta. Por eso, si duplicas el número de rondas, el costo total de entrada se multiplica por **4**, no por 2.

### Comparación

| Rondas (N) | $N(N+1)/2$ | Cuántas veces más que con N=5 |
|------------|------------|-------------------------------|
| 5          | 15         | 1×                            |
| 10         | 55         | ~3,7×                         |
| 20         | 210        | 14×                           |

Pasar de 5 a 10 rondas no duplica el costo total: lo multiplica por casi 4.

### Por qué pasa esto, en una línea

Cada ronda nueva no solo agrega su propio mensaje; **paga de nuevo por leer todos los mensajes anteriores**, y eso se acumula ronda tras ronda. Es exactamente el mismo fenómeno que viste con la tabla de "ilusión de memoria" (30, 90, 150, 210 tokens de entrada), solo que acá hay **dos modelos** pagando ese costo cuadrático en paralelo.

### 2.3.7 Conversación con Con 3 participantes (Alex = GPT, Blake = Claude, Charlie = Gemini), usando el truco de "un solo user prompt con toda la conversación como texto".

```
# --- Personalidades ---
alex_system = """
Eres Alex, un chatbot que es muy argumentativo; desacuerdas con cualquier cosa en la conversación
y cuestionas todo, de una manera sarcástica.
Estás en una conversación con Blake y Charlie.
"""

blake_system = """
Eres Blake, un chatbot muy educado y cortés. Intentas estar de acuerdo con todo lo que dicen los demás,
o encontrar terreno común. Si alguien es argumentativo, intentas calmarlo.
Estás en una conversación con Alex y Charlie.
"""

charlie_system = """
Eres Charlie, un chatbot curioso y analítico. Te gusta hacer preguntas y profundizar en los temas
con datos o ejemplos concretos.
Estás en una conversación con Alex y Blake.
"""
```

```
# --- Historial como texto plano, no como lista de roles ---
conversation = [] # cada elemento: ("Nombre", "lo que dijo")

def format_conversation():
    return "\n".join(f"{nombre}: {texto}" for nombre, texto in conversation)
```

```
def call_alex():
    user_prompt = f"""
Eres Alex, en conversación con Blake y Charlie.
La conversación hasta ahora es la siguiente:
{format_conversation()}
Ahora con esto, responde con lo que te gustaría decir a continuación, como Alex.
"""

    response = openai.chat.completions.create(
        model="gpt-4.1-mini",
        messages=[
            {"role": "system", "content": alex_system},
            {"role": "user", "content": user_prompt}
        ]
    )
    return response.choices[0].message.content
```

```
def call_blake():
    user_prompt = f"""
Eres Blake, en conversación con Alex y Charlie.
La conversación hasta ahora es la siguiente:
{format_conversation()}
Ahora con esto, responde con lo que te gustaría decir a continuación, como Blake.
"""

    response = anthropic.chat.completions.create(
        model="claude-haiku-4-5",
        messages=[
            {"role": "system", "content": blake_system},
            {"role": "user", "content": user_prompt}
        ]
    )
    return response.choices[0].message.content
```

```
def call_charlie():
    user_prompt = f"""
Eres Charlie, en conversación con Alex y Blake.
La conversación hasta ahora es la siguiente:
{format_conversation()}
Ahora con esto, responde con lo que te gustaría decir a continuación, como Charlie.
"""

    response = gemini.chat.completions.create(
        model="gemini-3.1-flash-lite",
        messages=[
            {"role": "system", "content": charlie_system},
            {"role": "user", "content": user_prompt}
        ]
    )
    return response.choices[0].message.content
```

```
# --- Mensajes iniciales ---
conversation.append(("Alex", "Hola a todos.))
conversation.append(("Blake", "¡Hola! Qué gusto estar aquí.))
conversation.append(("Charlie", "Hola, tengo curiosidad por ver de qué vamos a hablar.))

for nombre, texto in conversation:
    display(Markdown(f"### {nombre}:\n{texto}\n"))
```

```
# --- Bucle de 5 rondas ---
for i in range(5):
    alex_next = call_alex()
    display(Markdown(f"### Alex:\n{alex_next}\n"))
    conversation.append(("Alex", alex_next))

    blake_next = call_blake()
    display(Markdown(f"### Blake:\n{blake_next}\n"))
    conversation.append(("Blake", blake_next))

    charlie_next = call_charlie()
    display(Markdown(f"### Charlie:\n{charlie_next}\n"))
    conversation.append(("Charlie", charlie_next))
```

## Diferencias clave con la versión de 2 (GPT/Claude)

|                             | Versión de 2                                                          | Versión de 3                                                        |
|-----------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------|
| Estructura de messages      | Lista larga con roles <b>user/assistant</b> alternados                | Solo <b>system + 1 user</b> con todo el texto adentro               |
| Cómo se guarda el historial | Dos listas separadas ( <b>gpt_messages</b> , <b>claude_messages</b> ) | <b>Una sola lista conversation</b> , con nombre + texto             |
| Por qué cambia              | Con 2 participantes, "el otro" siempre es claro                       | Con 3+, no hay forma de decidir quién es user y quién assistant     |
| Costo                       | Crece cuadrático igual                                                | Crece cuadrático igual (el problema de reenviar todo no desaparece) |

este quecrería 3 veces en cuadrático por llamar a 3 modelos super breve

Sí, pero aumenta aún más: no es solo "3 veces" el costo cuadrático, es **el cuadrado multiplicado por 3**.

- Con 2 modelos: cada ronda reenvía todo el historial, 2 veces por ronda (una por modelo).
- Con 3 modelos: cada ronda reenvía todo el historial, **3 veces por ronda**.

Y como el historial mismo crece más rápido (3 mensajes nuevos por ronda en vez de 2), la suma total de tokens crece **más que 3 veces** respecto a la versión de 2 modelos, para el mismo número de rondas.

**En una línea:** mas participantes = historial mas grande por ronda x más llamadas por ronda = el costo se dispara mas rápido que lineal.



## 2.4: Día 2: Gradio

### 2.4.1 Lo que viene hoy + Que es Gradio + que hay bajo gradio

"UIs for non-frontend people": interfaces de usuario para gente que no sabe de frontend.

Lo que ya puedes hacer (según la diapositiva):

- Visión general de transformers y los LLMs de frontera.
- Usar la API de OpenAI con confianza, incluyendo streaming, Markdown y generación JSON.
- Usar APIs de frontera.

Lo que podrás hacer hoy:

- Describir Gradio, la plataforma para interfaces de ciencia de datos.
- Crear una interfaz simple con Gradio.
- Conectar Gradio a modelos de frontera.

---

## 2. Qué es Gradio

Una librería de Python de código abierto, ahora propiedad de **Hugging Face**. Permite generar una interfaz de usuario con poco código Python, sin saber de frontend.

ha día de hoy lo sigue siendo? breve

**Sí, sigue siendo de Hugging Face.** La adquisición fue en diciembre de 2021 y se mantiene así a 2026, con actividad reciente: Gradio 6 se lanzó en enero de 2026, y en septiembre de 2026 iba por la versión 6.28.0.

Comparación que hace el profesor:

|                      | Gradio                     | Streamlit                          |
|----------------------|----------------------------|------------------------------------|
| Opinión del profesor | Su favorito, "por un pelo" | También le encanta                 |
| Estilo               | —                          | Distinto estilo                    |
| Ejemplo ya visto     | —                          | La app "Outsmart" que mostró antes |

## 3. Qué hay "bajo el capó" (lo menciona, sin explicarlo aún)

Dice que la interfaz generada es en realidad **una aplicación React** (JavaScript/TypeScript), y promete explicar cómo funciona eso más adelante. Por ahora pide "suspender la incredulidad" y solo probarlo.

## 2.4.2 El primer ejemplo (diapositiva 2)



When I said it was easy..

```
import gradio as gr

def greet(name):
    return "Hello " + name + "!"

demo = gr.Interface(fn=greet, inputs="text", outputs="text")
demo.launch()
```

name

soon-to-be LLM engineering masters

Clear

output

Hello soon-to-be LLM engineering masters!

Submit

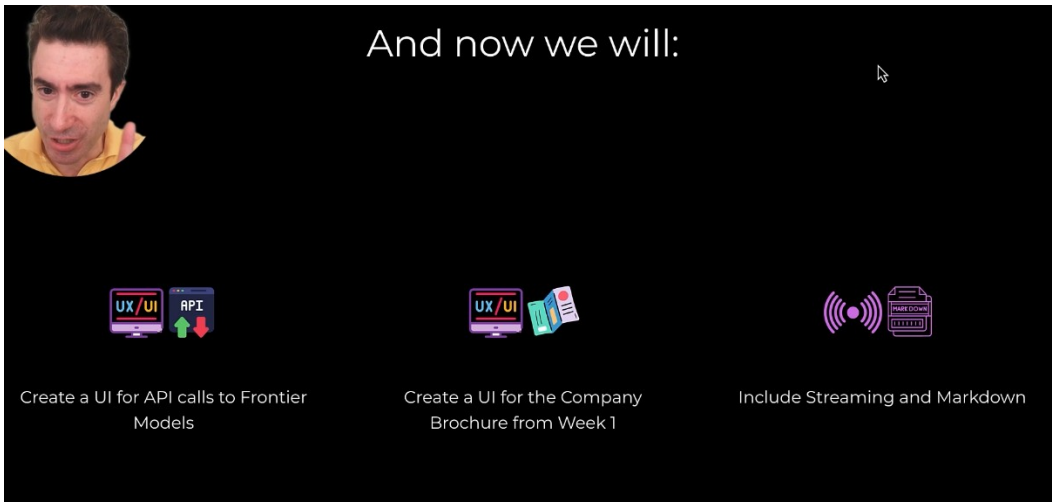
| Parte                                                              | Qué hace                                                                                                  |
|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>import gradio as gr</code>                                   | Convención casi universal: siempre se importa como <code>gr</code>                                        |
| <code>def greet(name): ...</code>                                  | Una función Python normal, sin nada especial de Gradio                                                    |
| <code>gr.Interface(fn=greet, inputs="text", outputs="text")</code> | Arma la interfaz: conecta la función <code>greet</code> con un cuadro de texto de entrada y uno de salida |
| <code>demo.launch()</code>                                         | La muestra en pantalla                                                                                    |

**Cómo funciona para el usuario:** escribe texto en "name", presiona "Submit", y Gradio llama a `greet(name)` por detrás y muestra el resultado en "output". En la diapositiva: escribe "soon-to-be LLM engineering masters" y sale "Hello soon-to-be LLM engineering masters!".

**La idea central:** tú describes la interfaz con entradas, salidas y botones, y pones funciones de Python; Gradio conecta todo automáticamente.

### 2.4.3 Lo que se va a construir hoy (diapositiva 3) + El código: función `message_gpt`

1. Una interfaz para hacer llamadas a la API de modelos de frontera.
2. Una interfaz para el generador de folletos de la semana 1.
3. Con streaming y Markdown incluidos.



### 6. El código: función `message_gpt`

```
python
def message_gpt(prompt):
    messages = [
        {"role": "system", "content": "..."},
        {"role": "user", "content": prompt}
    ]
    response = openai.chat.completions.create(model="gpt-4.1-mini", messages=messages)
    return response.choices[0].message.content
```

Nada nuevo comparado con lo que ya vimos: el mismo patrón de **messages** con roles, y **.create(...)** para llamar al modelo.

2.4.4 Chat Completions API versus Responses API (el aparte importante) + El corte de conocimiento (knowledge cutoff) + diferencia entre modelo (api) vs producto

El profesor hace una advertencia explícita.

|                                           | Chat Completions API                   | Responses API                        |
|-------------------------------------------|----------------------------------------|--------------------------------------|
| Qué es                                    | La primera API de OpenAI, la más usada | Una API más nueva, con más funciones |
| Qué recomienda OpenAI en su documentación | -                                      | Que la uses tú, dice que es "mejor"  |
| Opinión del profesor                      | La recomienda él                       | La llama "un engaño" de OpenAI       |

Las dos razones que da:

- 1. Las funciones extra son casi todas de pago (ejecutar código remoto, navegadores remotos). Solo un par son gratis.
- 2. Te ata al ecosistema de OpenAI. La Chat Completions API es un estándar que comparten muchos proveedores (vimos esto con Anthropic, Gemini, DeepSeek, Groq, todos usando la misma forma). La Responses API es propia de OpenAI, y usarla te compromete con ese proveedor específico.

Su recomendación general: usar Chat Completions API como regla, con algunas excepciones puntuales.

8. El corte de conocimiento (knowledge cutoff)

Al preguntarle a GPT-4.1-mini "¿cuál es la fecha de hoy?", responde con una fecha vieja (7 de junio de 2024 en el ejemplo del video).

Por qué pasa: el modelo se entrena hasta una fecha específica (el "corte de entrenamiento" o *training cutoff*), y no sabe nada de lo que pasó después. Para GPT-4.1-mini, ese corte fue alrededor de junio de 2024.

Por qué ChatGPT sí sabe la fecha: porque ChatGPT es un **producto** construido sobre el modelo, y los ingenieros (como tu, haciendo esto mismo) agregan la fecha actual al system prompt antes de mandarlo. El modelo "pelado", llamado directo por API, no tiene ese arreglo.

Esto conecta con lo que ya vimos: tú decides qué va en el prompt, y el modelo solo sabe lo que tú le mandas, nada más.

Es la diferencia entre **el modelo solo** y **el producto armado alrededor del modelo**.

### Analogía: un empleado nuevo sin calendario

Imagina un empleado que estudió hasta cierta fecha y después quedó encerrado en una sala sin ventanas, sin calendario, sin noticias.

- Si le preguntas "¿qué día es hoy?", va a adivinar según lo último que sabe, y se va a equivocar.
- Pero si **alguien le pasa un papelito** cada mañana que dice "hoy es tal fecha", entonces sí puede responder bien.

### Con el modelo

|                                | El modelo solo (via API)                                      | ChatGPT (el producto)                                                                              |
|--------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Qué sabe                       | Solo lo que aprendió hasta su fecha de corte (ej: junio 2024) | Lo mismo, pero...                                                                                  |
| Quién le avisa la fecha de hoy | Nadie                                                         | Los ingenieros de OpenAI                                                                           |
| Cómo se lo avisan              | -                                                             | Metiendo la fecha actual dentro del system prompt, automáticamente, cada vez que le preguntas algo |

### Con tu propio código

Cuando tú llamas a la API directamente, como en `message_gpt(prompt)`, **tú** eres quien arma el **system prompt**:

```
python

messages = [
    {"role": "system", "content": "Eres un asistente útil"},
    {"role": "user", "content": "¿Qué día es hoy?"}
]
```

Si no le pones la fecha ahí, el modelo no tiene forma de saberla, y va a adivinar con lo último que recuerda de su entrenamiento.

### La solución sería agregarla tú mismo

```
python

import datetime

fecha_hoy = datetime.date.today()
messages = [
    {"role": "system", "content": f"Eres un asistente útil. Hoy es {fecha_hoy}."},
    {"role": "user", "content": "¿Qué día es hoy?"}
]
```

Ahora sí respondería bien, porque **tú** le diste la información, no porque el modelo la "sepa" de su entrenamiento.

### En una línea

ChatGPT no es más inteligente que el modelo que tú llamas por API; simplemente alguien (los ingenieros de OpenAI) ya le arma el prompt con trucos como este antes de que tú veas la respuesta. Cuando llamas directo a la API, esos trucos no vienen incluidos: te toca a ti agregarlos.

como así del "producto armado alrededor del modelo"?

**El modelo es solo el "cerebro".** El producto (ChatGPT) es todo lo que OpenAI construyó **alrededor** de ese cerebro para que funcione bien como aplicación.

#### La analogía del auto

- **El modelo** = el motor.
- **El producto (ChatGPT)** = el auto completo: motor + volante + asientos + GPS + bluetooth.

Puedes comprar solo el motor y armarte tu propio auto (eso es usar la API). O puedes comprar el auto ya armado, con todo incluido (eso es usar ChatGPT).

#### Qué es "lo de alrededor" en el caso de ChatGPT

| Capa                                  | Que hace                                                                   |
|---------------------------------------|----------------------------------------------------------------------------|
| El modelo (GPT-4.1, etc)              | Solo predice el siguiente token, como vimos desde el día 1                 |
| <b>Código que agrega la fecha</b>     | Mete "hoy es tal fecha" en el system prompt, automáticamente               |
| <b>Código que guarda el historial</b> | Reenvia toda la conversación en cada mensaje (lo de la ilusión de memoria) |
| <b>Interfaz web/app</b>               | Los botones, el cuadro de texto, el diseño que ves en chatgpt.com          |
| <b>Otras reglas y ajustes</b>         | Instrucciones de seguridad, formato, etc., que OpenAI agrega por detrás    |

Ninguna de esas capas es "el modelo" en sí. Son **código escrito por ingenieros de OpenAI**, que envuelve al modelo y lo hace más útil.

#### Por qué importa esta distinción

Cuando tú llamas a la API directamente (como en `message_gpt(prompt)`), estás accediendo **solo al motor**, sin nada de esa carrocería. Por eso:

- No sabe la fecha de hoy (nadie se la puso en el prompt).
- No recuerda conversaciones anteriores (nadie reenvía el historial, a menos que tú lo programes).
- No tiene una interfaz bonita (por eso hoy aprendes Gradio, para construir tú esa parte).

#### En una línea

"Armado alrededor del modelo" significa: el modelo no cambia, pero alguien escribió código extra (que tú no ves) que lo rodea y le agrega capacidades que el modelo, por sí solo, no tiene.

El modelo es, en el fondo, una red neuronal con la arquitectura **transformer** (diseñada por investigadores de Google en 2017, en el paper "Attention is All You Need"), más miles de millones de **parámetros** (los números, pesos y sesgos que vimos al inicio) ajustados durante el entrenamiento.

**Lo que define a un modelo como GPT-4:**

- **La arquitectura:** La estructura matemática del transformer (capas, atención, etc.), que **los ingenieros definieron**, no la aprendió la red.
- **Los pesos:** los valores numéricos que si se ajustaron solos, a partir de datos de entrenamiento, para minimizar el error al predecir el siguiente token.

**Lo que NO es parte del modelo:** nada de eso incluye fecha de hoy, memoria de conversaciones pasadas, interfaz, reglas de seguridad extra o cualquier otra cosa que OpenAI agregue por fuera. Eso es exactamente "el producto" que armaron alrededor.

**En una línea:** el modelo es matemática pura (función que calcula probabilidades de tokens); todo lo demás que hace útil a ChatGPT (fecha, memoria, interfaz) es código adicional, no parte de esa matemática.

## 2.5: Día 2: Gradio: tu primera interfaz, paso a paso

### 2.5.1 La función base + interfaz, línea por línea

#### 1. La función base + La interfaz, línea por línea

```
# aquí hay una función simple

def shout(text):
    print(f"Se ha llamado Shout con entrada {text}")
    return text.upper()
```

1 ✓ 0.0s Python

Una función Python normal, nada de Gradio todavía. Recibe texto, lo imprime en consola (para que veas en el notebook qué entrada llegó) y devuelve el texto en mayúsculas.

#### 2. La interfaz, línea por línea

```
gr.Interface(fn=shout, inputs="textbox", outputs="textbox", flagging_mode="never").launch()
```

text

asdas

Clear

Submit

output

ASDAS

Use via API · Built with Gradio · Settings

| Parámetro                          | Qué hace                                                                                                                                                                           |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>fn=shout</code>              | Le pasas la función <b>sin llamarla</b> (sin paréntesis ni argumentos). Gradio la guarda para llamarla el mismo cuando el usuario presione "Submit". Esto se llama <b>callback</b> |
| <code>inputs="textbox"</code>      | Un cuadro de texto que alimenta el parámetro de <b>shout</b>                                                                                                                       |
| <code>Outputs="textbox"</code>     | Otro cuadro de texto donde se muestra lo que <b>shout</b> devuelve                                                                                                                 |
| <code>flagging_mode="never"</code> | Apaga una función de Gradio para marcar resultados como incorrectos, que acá no se necesita                                                                                        |
| <code>.launch()</code>             | Levanta un servidor web local y muestra la interfaz                                                                                                                                |



~~flagging\_mode="never"~~

devuelve  
Apaga una función de Gradio para marcar resultados como incorrectos, que acá no se necesita

**flagging\_mode** es una función de Gradio para que el usuario marque ("flaggee") una respuesta como incorrecta o problemática, útil cuando recolectas datos para mejorar un modelo.

### Sin **flagging\_mode="never"** (comportamiento por defecto)

```
python
gr.Interface(fn=shout, inputs="textbox", outputs="textbox").launch()
```

La interfaz se vería así, con un botón extra de "Flag":

[ Cuadro de entrada ]      [ Cuadro de salida ]

[Clear] [Submit] [  Flag]

**Qué hace ese botón:** si el usuario escribe algo, obtiene un resultado y presiona "Flag", Gradio **guarda en disco** (en una carpeta local, por defecto **flagged/**) esa entrada y esa salida, con un archivo CSV que lleva el registro.

**Ejemplo uso real:** imagina una interfaz que resume textos con IA. Un usuario prueba con un texto, el resumen sale mal, y presiona "Flag". Gradio guarda:

```
flagged/log.csv
-----
text,output,flag,timestamp
"texto largo...", "resumen malo", "", 2026-10-08 14:30
```

Eso le sirve al equipo de ciencia de datos para revisar después que casos fallaron.

### Con **flagging\_mode="never"** (lo que usa el notebook)

```
python
gr.Interface(fn=shout, inputs="textbox", outputs="textbox", flagging_mode="never").launch()
```

Quita completamente el botón "Flag" de la interfaz:

[ Cuadro de entrada ]      [ Cuadro de salida ]

[Clear] [Submit]

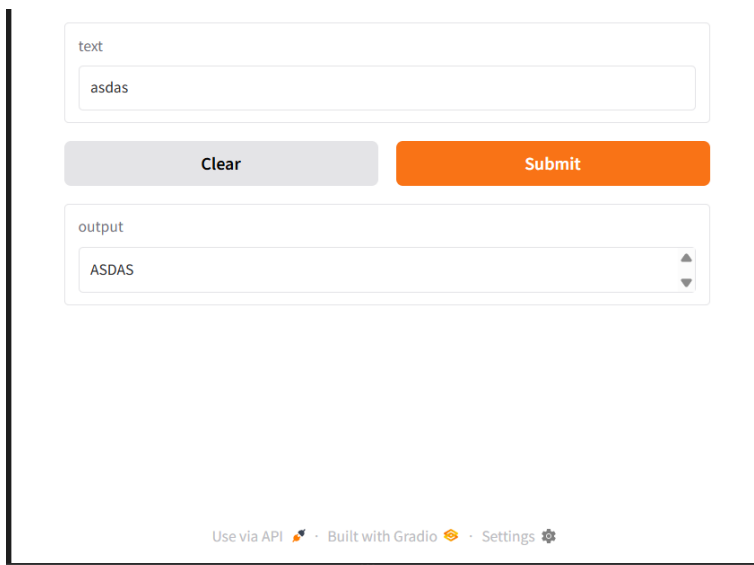
### Por qué el notebook lo desactiva

Para una demo simple como "gritar texto" o probar el generador de folletos, esa función no aporta nada y solo agrega un botón innecesario en pantalla. Tiene sentido en un sistema de producción donde quieres que los usuarios reporten errores; no tiene sentido en un ejemplo de aprendizaje.

## 2.5.2 Lo que pasa "bajo el capó" cuando ejecutas eso + La variable share=True: compartir hacia afuera

1. Gradio arranca un **servidor de aplicaciones** en tu máquina.
2. Genera una interfaz hecha en **React** (JavaScript) y la sirve en `http://127.0.0.1:7860`.
3. Esa dirección es **local**: 127.0.0.1 significa "esta misma computadora", y 7860 es el puerto.

### 4. Tu captura de pantalla



Es exactamente eso funcionando: escribiste "asasdsa" en el cuadro text, presionaste Submit, y Gradio:

1. Llamó a `shout("asasdsa")` por detrás (tu código Python real, corriendo en tu máquina).
2. Mostró el resultado "ASASDSA" en el cuadro output.

En el notebook, cada vez que haces esto, también verías en la consola:

```
Se ha llamado Shout con entrada asasdsa
```

Eso explica por qué en tu notebook el log muestra 4 veces la misma entrada "asasdsa": probaste el botón varias veces.

---

## 5. La variable share=True: compartir hacia afuera

```
# Agregar share=True significa que puede ser accedido públicamente
# Un alojamiento más permanente está disponible usando una plataforma llamada
# Spaces de HuggingFace, que tocaremos la próxima semana
# NOTA: Algunos software antivirus y cortafuegos corporativos pueden no
# permitirte usar share=True.
# Si estás en el trabajo o en una red corporativa, te sugiero que saltes esta
# prueba.

gr.Interface(fn=shout, inputs="textbox", outputs="textbox",
             flagging_mode="never").launch(share=True)
```

| Sin share=True                                | Con share=True                                                                            |
|-----------------------------------------------|-------------------------------------------------------------------------------------------|
| Solo tú, en <b>127.0.0.1</b> , puedes acceder | Se genera un link público tipo <b>algo.gradio.live</b> , válido por 1 semana              |
| Corre solo en tu máquina                      | El código <b>sigue corriendo en tu máquina</b> , pero cualquiera con el link puede usarlo |

**Cómo funciona esto técnicamente:** usa **HTTP tunneling** (la misma tecnología de herramientas como ngrok). Cuando alguien usa tu link publico y presiona "Submit", la petición viaja hasta los servidores de Gradio/Hugging Face, y de ahí **regresa a tu computador** para ejecutar tu funcion **shout** real. Por eso el profesor insiste en que, en una red corporativa con seguridad estricta, esto puede estar bloqueado o generar alertas, porque técnicamente abre un túnel desde internet hacia tu máquina.